# Frankenstein Transformer: Unified Encoder-Decoder Library, CLI, and Research-Grounded Design Notes

Erick F. Merino M.
This work is not affiliated
erickfmm@gmail.com

October 2026

**Abstract**

Transformer architectures dominate sequence modeling across natural language processing, computer vision, and computational biology. Recent work has diversified attention beyond dense softmax—recurrent and retentive blocks, sparse patterns, gated mechanisms, latent projections, memory-augmented layers, and fast-weight rules—while optimizers have expanded beyond AdamW with variance-reduction, memory-efficient, schedule-free, second-order, and low-rank methods. Comparing these ideas under consistent conditions remains costly: each variant must be implemented, wired into a training pipeline, and maintained, which hinders reproducibility. We propose Frankenstein Transformer, a configuration-driven CLI toolkit (`frankenstein-transformer`) that unifies model definition, training, deployment, and inference behind one strict YAML schema. The toolkit covers forty-three sequence mixers across eight families and twenty-three optimizer families with per-parameter-group hyperparameter control, and supports encoder, decoder, and Vision Transformer model classes for masked language modeling, causal language modeling, sentence embeddings, and image tasks. The contribution of this software is a validated experimentation contract: invalid configurations fail before training, every architecture and optimizer shares one operational interface, and the same document drives training, quantized deployment and export, and a schema-generated web builder. We describe the architecture taxonomy, optimizer routing, deployment paths, and integration surface, and defer mathematical and implementation detail to the appendices.

## Contents

**1 Introduction** **9**
1.1 Motivation and Problem Statement . . . 9
1.2 Proposal and Contributions . . . 9
1.3 Reading Guide . . . 9

**2 Related Work** **10**
2.1 Base Deep Learning Frameworks . . . 10
2.2 Out-of-the-Box Training Frameworks . . . 10
2.3 Low-Code and Efficiency-Oriented Tools . . . 11
2.4 Configuration-Driven Experimentation . . . 11

**3 System Design and Architecture** **12**
3.1 Configuration-Centric Architecture . . . 12
3.2 Schema Scope and Validation Rules . . . 13
3.3 Configuration Schema . . . 13
3.4 Training Tasks . . . 14

3.5 Training Safety and Runtime Semantics . . . 14
3.6 Normalization Variants . . . 14
3.7 Residual-Stream Architectures . . . 14

**4 Architecture Taxonomy and Implementation** **14**
4.1 Attention and Sequence-Mixer Families . . . 14
4.2 Dense and Recurrent Families . . . 15
4.3 Latent and Low-Rank Attention Family . . . 16
4.4 Sparse Attention Extensions . . . 16
4.5 Gated and Fast-Weight Attention . . . 16

**5 Optimizer Families and Training Dynamics** **17**

**6 Quantization and Deployment** **17**

**7 SBERT Downstream Tasks** **18**

**8 No-Code Integration: The `dashai-frankenstein` Plugin** **19**
8.1 Registered Components . . . 19
8.2 Passthrough Configuration Schema . . . 20
8.3 Status . . . 20

**9 Discussion** **20**
9.1 Schema-Driven Design Trade-offs . . . 21
9.2 Architectural Coverage and Gaps . . . 21
9.3 Optimizer Landscape Fragmentation . . . 21
9.4 Deployment and Production Considerations . . . 21
9.5 Integration and Extensibility Challenges . . . 22

**10 Conclusion** **22**
10.1 Limitations and Future Directions . . . 23

**Bibliography** **23**

**Appendices**

**1 Conceptual Introduction—Transformers and Attention for Beginners** **32**
1.1 What is a Transformer? . . . 32
1.2 The Attention Mechanism: An Analogy . . . 32
1.3 Practical Example Step-by-Step . . . 33
1.3.1 Context 1: “The cat eats fish” . . . 33
1.3.2 Context 2: “The restaurant eats into profit margins” . . . 33
1.4 How Attention Works Mathematically (Simple Version) . . . 33
1.4.1 Step 1: Queries, Keys, and Values . . . 33
1.4.2 Step 2: Compatibility . . . 33
1.4.3 Step 3: Focus . . . 33
1.4.4 Step 4: Combine . . . 34
1.5 Multiple Attention Heads: Multiple Perspectives . . . 34
1.6 Stacking Layers . . . 34
1.7 Why Does It Matter? . . . 34
1.8 Practical Challenges . . . 34
1.8.1 Computational Complexity . . . 35
1.8.2 Memory Requirements . . . 35

1.8.3 Device Efficiency . . . 35
1.9 Modern Solutions . . . 35
1.10 Key Takeaways . . . 35

**2 Configuration Schema Structure** **36**
2.1 Top-Level Structure . . . 36
2.2 Model Configuration . . . 36
2.2.1 Layer Pattern Enumeration . . . 41
2.3 Training Configuration . . . 43
2.4 Tokenizer Configuration . . . 45
2.5 Optimizer Configuration . . . 46
2.5.1 Shared Per-Group Suffixes . . . 46
2.5.2 Optimizer-Specific Global Suffixes . . . 47
2.6 SBERT Configuration . . . 47
2.7 Validation Rules . . . 48

**3 Comprehensive Summary Tables** **49**
3.1 Sequence Mixer Summary . . . 49
3.2 Optimizer Summary . . . 52
3.3 Normalization Variants . . . 53
3.4 Residual Connections . . . 53
3.5 Activation Functions . . . 54
3.6 Embedding Variants . . . 55

**4 Optimizer Families** **56**
4.1 The Evolution of Optimization in Neural Networks . . . 56
4.2 Standard Baseline and Adaptive Optimizers . . . 56
4.3 Advanced Momentum and Variance Reduction (2024–2025) . . . 62
4.4 Large-Batch, Memory-Efficient, and Parameter-Free Optimizers . . . 71
4.5 Second-Order, Geometric, and Orthogonality Optimizers . . . 83

**5 Dense, Recurrent, and Memory-Augmented Transformers** **93**
5.1 Dense Attention Baselines: Standard and Sigmoid . . . 93
5.1.1 Standard Softmax Attention . . . 93
5.1.2 Sigmoid Attention . . . 94
5.1.3 Grouped-Query Attention (GQA) . . . 94
5.2 Recurrent and Retentive Architectures . . . 95
5.2.1 Retentive Networks (RetNet) . . . 95
5.2.2 Mamba: Selective State-Space Models . . . 96
5.3 Continuous-Depth Transformers: ODE Integration . . . 97
5.3.1 ODE Transformer . . . 97
5.4 Test-Time Memory: Titans . . . 97
5.5 Pattern-Driven Mixer Dispatch . . . 98

**6 Latent and Low-Rank Attention Families** **98**
6.1 Multi-Head Latent Attention (MLA) . . . 99
6.1.1 Mathematical Formulation . . . 99
6.1.2 Algorithmic Pseudocode . . . 99
6.1.3 Key Characteristics . . . 100
6.2 Group-Query Latent Attention (GQLA) . . . 100
6.2.1 Mathematical Formulation . . . 100
6.2.2 Algorithmic Pseudocode . . . 101

6.2.3 Key Characteristics . . . 101
6.3 Multi-Head Low-Rank Attention (MLRA) . . . 101
6.3.1 Mathematical Formulation . . . 101
6.3.2 Algorithmic Pseudocode . . . 102
6.3.3 Key Characteristics . . . 102
6.4 Tucker Attention . . . 102
6.4.1 Mathematical Formulation . . . 102
6.4.2 Algorithmic Pseudocode . . . 103
6.4.3 Key Characteristics . . . 103
6.5 Interleaved Head Attention (IHA) . . . 103
6.5.1 Mathematical Formulation . . . 103
6.5.2 Algorithmic Pseudocode . . . 103
6.5.3 Key Characteristics . . . 104
6.6 Grouped-head laTenT Attention (GTA) . . . 104
6.6.1 Mathematical Formulation . . . 104
6.6.2 Algorithmic Pseudocode . . . 104
6.6.3 Key Characteristics . . . 105
6.7 Multi-head Temporal Latent Attention (MTLA) . . . 105
6.7.1 Mathematical Formulation . . . 105
6.7.2 Algorithmic Pseudocode . . . 105
6.7.3 Key Characteristics . . . 106
6.8 Architectural Comparison and Synthesis . . . 106
6.9 Compressed Convolutional Attention (CCA) . . . 106
6.9.1 Mathematical Formulation . . . 106
6.9.2 Algorithmic Pseudocode . . . 107
6.9.3 Key Characteristics . . . 107
6.10 Compressed Convolutional Grouped Query Attention (CCGQA) . . . 107
6.10.1 Mathematical Formulation . . . 108
6.10.2 Key Characteristics . . . 108
6.11 Gaussian Mixture Attention (GMA) . . . 108
6.11.1 Mathematical Formulation . . . 109
6.11.2 Algorithmic Pseudocode . . . 109
6.11.3 Key Characteristics . . . 110

**7 Comprehensive Sparse Attention Mechanisms 110**
7.1 Executive Summary . . . 110
7.2 Sparse Transformer: Factorized Strided and Fixed Patterns . . . 110
7.2.1 Mathematical Formulation . . . 110
7.2.2 Algorithmic Pseudocode . . . 111
7.2.3 Key Characteristics . . . 111
7.3 Longformer: Sliding Window, Dilation, and Global Tokens . . . 112
7.3.1 Mathematical Formulation . . . 112
7.3.2 Algorithmic Pseudocode . . . 112
7.3.3 Key Characteristics . . . 113
7.4 SWA with Attention Sinks: Sliding Window Plus a Sink Prefix . . . 113
7.4.1 Mathematical Formulation . . . 113
7.4.2 Algorithmic Pseudocode . . . 114
7.4.3 Key Characteristics . . . 114
7.5 BigBird: Random, Local, and Global Sparse Graph . . . 114
7.5.1 Mathematical Formulation . . . 115
7.5.2 Theoretical Guarantee . . . 115

7.5.3 Algorithmic Pseudocode . . . 115
7.5.4 Key Characteristics . . . 116
7.6 SparseK Attention: Differentiable Top-$k$ Selection . . . 116
7.6.1 Mathematical Formulation . . . 116
7.6.2 Algorithmic Pseudocode . . . 117
7.6.3 Key Characteristics . . . 117
7.7 NSA (Native Sparse Attention): Hardware-Aligned Hierarchical Branches . . . 117
7.7.1 Mathematical Formulation . . . 117
7.7.2 Algorithmic Pseudocode . . . 118
7.7.3 Key Characteristics . . . 118
7.8 FASA: Frequency-Aware Sparse Attention . . . 119
7.8.1 Mathematical Formulation . . . 119
7.8.2 Algorithmic Pseudocode . . . 120
7.8.3 Key Characteristics . . . 120
7.9 SpargeAttn: Two-Stage Block-Level Filtering . . . 121
7.9.1 Mathematical Formulation . . . 121
7.9.2 Algorithmic Pseudocode . . . 122
7.9.3 Key Characteristics . . . 122
7.10 MSA (MiniMax Sparse Attention): Blockwise Sparse Attention with Lightweight Index Branch . . . 123
7.10.1 Mathematical Formulation . . . 123
7.10.2 Algorithmic Pseudocode . . . 124
7.10.3 Key Characteristics . . . 124
7.11 SparDA (Sparse Decoupled Attention): Forecast-Guided Lookahead Block Selection 124
7.11.1 Mathematical Formulation . . . 124
7.11.2 Algorithmic Pseudocode . . . 125
7.11.3 Key Characteristics . . . 125
7.12 Design Space and Selection Criteria . . . 126

**8 Gated Attention Families—Complete Literature Analysis 126**
8.1 Executive Summary: Gating for Memory Control . . . 126
8.2 1. Gated Linear Attention (GLA) . . . 127
8.2.1 Mathematical Foundation . . . 127
8.2.2 Hardware-Efficient Training . . . 128
8.2.3 Strengths and Limitations . . . 128
8.3 2. DeltaNet: Error-Correcting Linear Attention . . . 129
8.3.1 Mathematical Foundation . . . 129
8.3.2 Efficient Parallel Training . . . 129
8.3.3 Strengths and Limitations . . . 130
8.4 3. Gated DeltaNet: Synthesis of Gating and Error Correction . . . 130
8.4.1 Mathematical Foundation . . . 131
8.4.2 Empirical Performance and Trade-offs . . . 131
8.5 4. HGRN2: Hierarchical Gating with Outer-Product Expansion . . . 132
8.5.1 Mathematical Foundation . . . 132
8.5.2 Scaling and Empirical Results . . . 132
8.6 5. Forgetting Transformer (FoX): Gating in Softmax Logit Space . . . 133
8.6.1 Mathematical Foundation . . . 133
8.6.2 Integration with FlashAttention . . . 134
8.6.3 Strengths and Limitations . . . 134
8.7 6. Gated Attention (Post-SDPA Sigmoid Gating) . . . 134
8.7.1 Mathematical Foundation . . . 135

8.7.2 Key Findings . . . 135
8.8 7. Gated DeltaNet-2: Decoupled Channel-wise Erase and Write . . . 136
8.8.1 Mathematical Foundation . . . 136
8.8.2 Empirical Performance and Trade-offs . . . 137
8.9 8. Kimi Delta Attention (KDA): Channel-wise Decay Gating for the Delta Rule . 138
8.9.1 Mathematical Foundation . . . 138
8.9.2 Empirical Performance and Trade-offs . . . 139
8.10 Unified Gating Principle . . . 139
8.11 Implementation and Practical Guidance . . . 140
8.11.1 When to Use Each Architecture . . . 140
8.11.2 Hardware Considerations . . . 140

**9 Normalization, Quantization, Depth Routing, and Runtime Algorithms 140**
9.1 Normalization Variants: LayerNorm, RMSNorm, pRMSNorm, Dynamic Tanh, Dynamic Erf, and FlashNorm . . . 141
9.1.1 LayerNorm (Baseline) . . . 141
9.1.2 RMSNorm . . . 141
9.1.3 Partial RMSNorm (pRMSNorm) . . . 141
9.1.4 Dynamic Tanh (DyT) . . . 141
9.1.5 Dynamic Erf (Derf) . . . 142
9.1.6 FlashNorm (Weightless RMSNorm) . . . 142
9.1.7 Comparison . . . 143
9.2 BitNet Ternary Quantization Path . . . 144
9.3 Mixture-of-Depths (MoD) Token Routing . . . 144
9.3.1 Configuration . . . 144
9.4 Engram Conditional Memory . . . 145
9.4.1 Configuration . . . 145
9.5 Factorized Embeddings and Embedding Convolution . . . 145
9.5.1 Factorized Embeddings . . . 145
9.5.2 Embedding Convolution . . . 146
9.6 Training-Step Stability Controls . . . 146
9.7 SBERT Inference Mode Router . . . 146

**10 Activation Functions 147**
10.1 Classical / Sigmoid–Tanh Family (12 activations) . . . 147
10.2 Rectified Family (12 activations) . . . 148
10.3 Exponential / ELU Family (12 activations) . . . 149
10.4 Swish and Parametric Activations . . . 149
10.5 Rational Activation Functions (RAF) . . . 150
10.6 Gated FFN Variants: SwiGLU, GEGLU, ReGLU . . . 151
10.7 Configuration Schema . . . 151
10.8 Selection Guide . . . 152

**11 Manifold-Constrained Hyper-Connections (mHC) 152**
11.1 Motivation: The Identity-Mapping Property . . . 152
11.2 Formulation . . . 153
11.3 Properties of the Manifold Constraint . . . 153
11.4 Implementation in Frankenstein Transformer . . . 154
11.5 Reported Results . . . 155
11.6 Hyperloop: Loop-Level Hyper-Connections on Looped Transformers . . . 155

**12 Attention Residuals (AttnRes) 156**
12.1 Motivation: From a Fixed Sum to Depth-Wise Attention . . . 156
12.2 Four Strategies . . . 157
12.2.1 Standard Residual (`"standard"`) . . . 157
12.2.2 No Residual (`"none"`, experimental) . . . 157
12.2.3 Full Attention Residuals (`"full_attn"`, paper § 3.1) . . . 157
12.2.4 Block Attention Residuals (`"block_attn"`, paper § 3.2) . . . 158
12.3 Implementation in Frankenstein Transformer . . . 158
12.4 Integration with mHC and Mixture-of-Depths . . . 159
12.5 Reported Results . . . 159

**13 Vision Transformer: Patch Prediction, Classification, and Segmentation 160**
13.1 Patch Embedding and Positional Encoding . . . 160
13.2 Masked Patch Prediction (Autosupervised) . . . 160
13.3 Image Classification . . . 161
13.4 Image Segmentation . . . 161
13.4.1 Per-Pixel Head (`pixel`) . . . 161
13.4.2 Encoder-only Mask Transformer (`eomt`) . . . 161
13.5 Reuse of Existing Infrastructure . . . 162
13.6 PyTorch Pseudocode . . . 162

**14 Model-Wide Positional Encodings 163**
14.1 Architecture: One Shared Module, Per-Mixer Opt-Out . . . 163
14.2 Injection Styles . . . 164
14.3 Mathematical Formulations . . . 164
14.3.1 RoPE — Rotary Position Embedding . . . 164
14.3.2 HoPE — Hyperbolic Rotary Positional Encoding . . . 164
14.3.3 NoPE — No Positional Encoding . . . 164
14.3.4 ALiBi — Attention with Linear Biases . . . 165
14.3.5 BAM — Bayesian Attention Mechanism . . . 165
14.3.6 PaPE — Parabolic Position Encoding . . . 165
14.3.7 Sinusoidal Absolute . . . 166
14.3.8 Sinusoidal Rotary . . . 166
14.3.9 Learned Absolute . . . 166
14.4 Per-Mixer `use_pe` Flags . . . 166
14.5 Vision Transformer Unification . . . 166
14.6 Legacy Compatibility . . . 167
14.7 Architectural Profile: Positional Encodings . . . 167

**15 SSOG: Separable Sum of Gaussians Attention 168**
15.1 Mathematical Formulation . . . 168
15.2 Algorithmic Pseudocode . . . 169
15.3 Key Characteristics . . . 169

**16 Fast-Weight Attention for Continual Learning 169**
16.1 Preliminaries: Alignment, Normalization, and Stability . . . 169
16.2 The Regression Family (Falcon-1/2/3) . . . 170
16.2.1 Falcon-1: Scalar NLMS Regression . . . 170
16.2.2 Falcon-2: Per-Column NLMS Regression . . . 171
16.2.3 Falcon-3: Sliding-Window Minibatch Regression . . . 171
16.3 The Inner-Product Family (Falcon-1A/2A/3A) . . . 171
16.3.1 Falcon-1A and Falcon-2A . . . 172

16.3.2 Falcon-3A: Sliding-Window Inner-Product . . . 172
16.4 Configuration Interface . . . 172
16.5 Published Results . . . 173
16.6 Relation to Prior Work . . . 173
16.6.1 When to Use Each Variant . . . 174

**17 KV Cache and Incremental Decoding** **174**
17.1 Motivation and Scope . . . 174
17.2 Cache Architecture . . . 175
17.2.1 Central Manager and Layer Identity . . . 175
17.2.2 Typed Slots . . . 176
17.2.3 Capability Registry . . . 176
17.2.4 Forward Plumbing and Fallback Semantics . . . 176
17.3 Correctness Contracts . . . 176
17.3.1 Post-RoPE Storage and the Absolute Schedule . . . 177
17.3.2 Additive Score Biases . . . 177
17.3.3 Length-Dependent Rescaling . . . 177
17.3.4 Causal Masking from Positions . . . 177
17.3.5 Absolute Positional Encodings . . . 177
17.3.6 Interlocks with Mixture-of-Depths, mHC, Hyperloop and AttnRes . . . 178
17.4 Incremental Recipes by Family . . . 178
17.4.1 Quadratic Softmax Family . . . 178
17.4.2 Latent Family . . . 178
17.4.3 Sparse and Window Families . . . 179
17.4.4 Recurrent and Linear State Family . . . 179
17.4.5 Auxiliary State and Documented Fallbacks . . . 180
17.5 KV-Cache Quantization . . . 180
17.5.1 Symmetric int8 with Asymmetric Axes . . . 181
17.5.2 Group-Wise int4 with a Protected Window . . . 181
17.5.3 Interface and Future Codecs . . . 181
17.6 Compression and Eviction . . . 182
17.7 Configuration and Deployment Interface . . . 182
17.8 Validation . . . 183
17.9 Relation to Prior Work . . . 183

# 1 Introduction

## 1.1 Motivation and Problem Statement

Transformer architectures are the dominant paradigm for sequence modeling in natural language processing [97], computer vision, and computational biology, with models such as BERT [21] and GPT establishing the standard for representation learning. The field has since diversified rapidly: recurrent and retentive blocks, sparse attention, gated mechanisms, latent projections, memory-augmented layers, fast-weight rules, and continuous-depth formulations each address a different limitation of dense attention, while the optimizer landscape has expanded beyond AdamW with variance-reduction, memory-efficient, schedule-free, second-order, and low-rank methods [16, 93, 35, 7, 82, 103, 73, 110, 87, 117, 19, 37, 98].

The practical consequence is a fragmented ecosystem. Comparing sequence mixers or optimizers under identical training conditions requires re-implementing each variant, wiring it into a training pipeline, and maintaining the result across framework changes. This effort hinders reproducibility, slows scientific progress, and raises the barrier to entry for new researchers.

## 1.2 Proposal and Contributions

We propose **Frankenstein Transformer** (`https://github.com/erickfmm/frankenstein-transformer`), a configuration-driven toolkit that treats architecture, optimizer, and training choices as a single validated document. The project provides:

1. **Schema-first design**: a strict YAML contract supporting forty-three sequence mixers across eight families and twenty-three optimizer families, enforced before training begins.
2. **Architecture-agnostic training**: one pipeline for dense baselines, recurrent and retentive blocks, sparse patterns, gated mechanisms, latent compression, conditional memory, fast-weight attention, and geometric-field attention, with encoder and decoder modes [120, 15, 104, 116, 76].
3. **Optimizer routing**: prefixed hyperparameter groups that control embeddings, normalization layers, attention blocks, and remaining parameters independently for each optimizer family [118].
4. **End-to-end workflows**: training (including sentence-embedding objectives), deployment and export (quantized, standard, HuggingFace Transformers, and BitNet GGUF formats), inference, and a schema-driven web builder inspired by SBERT-style workflows [83].
5. **Vision support**: a Vision Transformer model class covering masked patch prediction, classification, and segmentation under the same mixer and optimizer infrastructure.
6. **Validation tooling**: unit tests, YAML preset validation, and continuous integration across supported Python versions.

The command surface is `frankenstein-transformer`, with four subcommands: `train`, `deploy`, `infer`, and `web-server`. The last launches a Streamlit configuration builder that renders schema metadata as forms, validates input live, previews the resulting YAML, and generates the equivalent CLI command, serving both as a configuration aid and as documentation of the available design space.

## 1.3 Reading Guide

This document is organized as a technical reference:

1. **Prerequisites**: Appendix 1 introduces transformers and attention for readers new to the field.

2. **Configuration contract**: Section 3.1 describes the schema and Section 3.2 its validation rules.

3. **Architecture selection**: Section 4.1 compares sequence-mixer families; Appendices 5, 6, 7, and 8 collect the supporting literature, with normalization, BitNet, and conditional-memory details in Appendix 9 and manifold-constrained residuals in Appendix 11.

4. **Optimization**: Section 5 summarizes optimizer routing; Appendix 4 provides the full family analysis.

5. **Deployment and inference**: Sections 6 and 7 cover quantized deployment and sentence embeddings.

# 2 Related Work

This work sits at the intersection of deep learning frameworks, training tooling, and configuration-driven experimentation. While Sections 4.1 and 5 survey the architectural and optimization literature in depth, this section focuses on the software ecosystem for building, training, and configuring transformer models.

## 2.1 Base Deep Learning Frameworks

The modern transformer research ecosystem rests on several foundational frameworks that provide automatic differentiation, GPU acceleration, and high-level model-building APIs.

**PyTorch** [74] has become the dominant framework for transformer research. Its imperative programming model and dynamic computation graphs enable flexible debugging and rapid prototyping of novel architectures. The `torch.nn.Module` abstraction, automatic mixed-precision training via `torch.cuda.amp`, and the `torch.compile` JIT compiler form the backbone of most contemporary transformer implementations. Frankenstein Transformer is built entirely on PyTorch, leveraging its module system for composable architecture definitions.

**TensorFlow** [1] pioneered production-oriented deep learning with its static computation graph paradigm and the Keras high-level API. While its influence in transformer research has waned relative to PyTorch, TensorFlow's serving infrastructure (`TF Serving`, `TF Lite`) and the `tensorflow-text` library remain relevant for deployment scenarios.

**JAX** [11] offers a functional programming model built around composable transformations (`jit`, `vmap`, `grad`) and XLA compilation. Its design enables clean implementations of complex algorithms and has been adopted by several recent transformer libraries (e.g., Flax, Haiku). JAX's stateless design and automatic vectorization are particularly well-suited for research on novel attention mechanisms and optimization algorithms.

**scikit-learn** [75] provides classical machine learning algorithms and utilities (preprocessing, metrics, model selection) that serve as baselines and complementary tools in transformer workflows. While not designed for deep learning, its consistent API and extensive documentation make it a standard reference point for comparing transformer-based approaches against traditional methods.

## 2.2 Out-of-the-Box Training Frameworks

Several libraries provide complete training pipelines that abstract away the complexity of distributed training, data loading, and hyperparameter management.

**Hugging Face Transformers** [101] is the de facto standard for accessing, fine-tuning, and sharing pretrained transformer models. Its `Trainer` API, model hub with over 500,000 models, and tight integration with the `datasets` and `tokenizers` libraries have made it the primary entry point for transformer-based NLP. However, its architecture is optimized for fine-tuning existing

pretrained models rather than experimenting with novel sequence mixer designs or training from scratch with custom optimization strategies.

**Oumi** (https://github.com/oumi-ai/oumi, 2025) is an open-source end-to-end platform for training, fine-tuning, evaluating, and deploying foundation models. It provides a unified interface across multiple cloud providers and supports both supervised fine-tuning and preference optimization (DPO, RLHF). Oumi emphasizes production readiness with built-in monitoring, checkpointing, and deployment tooling.

**LLaMA Factory** (https://github.com/hiyouga/LLaMA-Factory, 2024) offers a unified framework for efficient fine-tuning of over 100 large language models. It provides both a web-based UI and a CLI, supporting LoRA, QLoRA, full-parameter fine-tuning, and various alignment techniques. Its focus is on adapting existing pretrained LLMs to downstream tasks rather than architectural experimentation.

**Axolotl** (https://github.com/axolotl-ai-cloud/axolotl, 2024) is a streamlined fine-tuning tool supporting LoRA, QLoRA, DeepSpeed, and FSDP. It emphasizes configuration-driven workflows via YAML files and has become popular in the open-source LLM fine-tuning community. Like LLaMA Factory, it targets fine-tuning of pretrained models rather than training novel architectures from scratch.

### 2.3 Low-Code and Efficiency-Oriented Tools

A growing category of tools prioritizes accessibility and computational efficiency, lowering the barrier to transformer training.

**Unsloth** (https://github.com/unslothai/unsloth, 2024) provides low-code fine-tuning with 2–5× speedup and significantly reduced VRAM requirements through hand-written CUDA kernels and optimized Triton operations. It supports GGUF export for quantized deployment and integrates with the Hugging Face ecosystem. Unsloth's focus is on making fine-tuning faster and more memory-efficient, not on architectural exploration.

**dashAI** (https://github.com/DashAISoftware/dashAI, v0.9.7.post2, 2026) is an open-source, local-first desktop workbench for machine learning developed by a Chilean academic consortium (Universidad de Chile/FCFM, CENIA, IMFD, and ANID funding). Its interface is fully schema-driven: configuration forms are generated directly from Pydantic component schemas with no hand-written frontend code, and its catalog spans roughly 76 multitask models—tabular classifiers, NLP models, translation, open-weight LLMs, and text-to-image generation—unified behind a dozen typed base abstractions. Extensibility is central to its design: plugins are ordinary PyPI packages discovered through the `dashai.plugins` entry-points group and installable from the UI, a mechanism comprehensive enough that even dashAI's own generative capabilities (e.g., Stable Diffusion, Flux) are distributed as plugins. Where earlier releases targeted classical ML workflows, the current platform spans predictive and generative modeling across domains, making it a natural host for transformer components via its plugin system; Section 8 presents the `dashai-frankenstein` adapter built on this mechanism.

### 2.4 Configuration-Driven Experimentation

A common thread across the tools surveyed above is their focus on fine-tuning existing pretrained models. Hugging Face Transformers, LLaMA Factory, Axolotl, and Unsloth all assume a pretrained checkpoint as the starting point. Oumi supports training from scratch but targets standard architectures. None of these tools provide a systematic framework for experimenting with novel sequence mixer architectures, comparing diverse optimization algorithms under controlled conditions, or exploring the interaction between architectural choices and training dynamics.

**Frankenstein Transformer** fills this gap by providing a schema-first configuration system purpose-built for architecture experimentation. Its key differentiators include:

- **43 sequence mixers** spanning dense attention, recurrent/retentive blocks, sparse attention patterns, gated mechanisms, adaptive depth routing, and conditional memory layers, all selectable via a single YAML field.
- **23 optimizer families** with a prefixed hyperparameter routing system that enables per-parameter-group control across embeddings, normalization layers, attention blocks, and feed-forward networks.
- **Dual training modes** supporting both encoder-style masked language modeling (MLM) and decoder-style autoregressive (AR) next-token prediction, with specialized fine-tuning workflows for each.
- **Strict schema validation** via `additionalProperties: false` constraints that prevent configuration errors before training begins, ensuring reproducibility across experiments.
- **End-to-end workflows** spanning quantized deployment (ternary weight packing, INT8 activations) and sentence-embedding training inspired by SBERT [83].
- **Web-based configuration interface** providing schema-driven form rendering with inline documentation, real-time validation, and CLI command generation.

By decoupling architecture definition from training infrastructure through a validated configuration contract, Frankenstein Transformer enables systematic comparison of sequence mixer families and optimization strategies under identical training conditions—a capability not provided by any existing tool in the ecosystem.

# 3 System Design and Architecture

## 3.1 Configuration-Centric Architecture

Experimentation in this project is *schema first*. Rather than exposing loosely checked flags, model topology, optimizer family, training limits, and telemetry pass through a single validated configuration document, which reduces ambiguity when reproducing results and lets many architectures share one operational interface.

The authoritative contract is `src/schema.yaml`, a modular JSON Schema whose modules under `src/schema/` cover the model, model class, tokenizer, optimizer, SBERT, training, and dataset blocks. The top-level objects are `base_model`, `tokenizer`, `model_class`, `model`, `training`, `image`, `vision_dataset`, and `text_dataset`.

**Model class.** The `model_class` field selects among three variants:

- `frankenstein_text_encoder`: mixed-architecture encoder with the full range of sequence mixers, MoE, and advanced normalization, trained with masked language modeling.
- `frankenstein_text_decoder`: autoregressive causal decoder for next-token generation; the runtime forces decoder masking.
- `frankenstein_vit`: Vision Transformer over image patches for masked patch prediction, classification, and segmentation; the runtime forces encoder masking and the configuration requires the `image` and `vision_dataset` blocks.

**Training mode.** `model.dims.mode` controls masking: `encoder` uses bidirectional attention for masked language modeling, and `decoder` uses causal masking for autoregressive prediction.

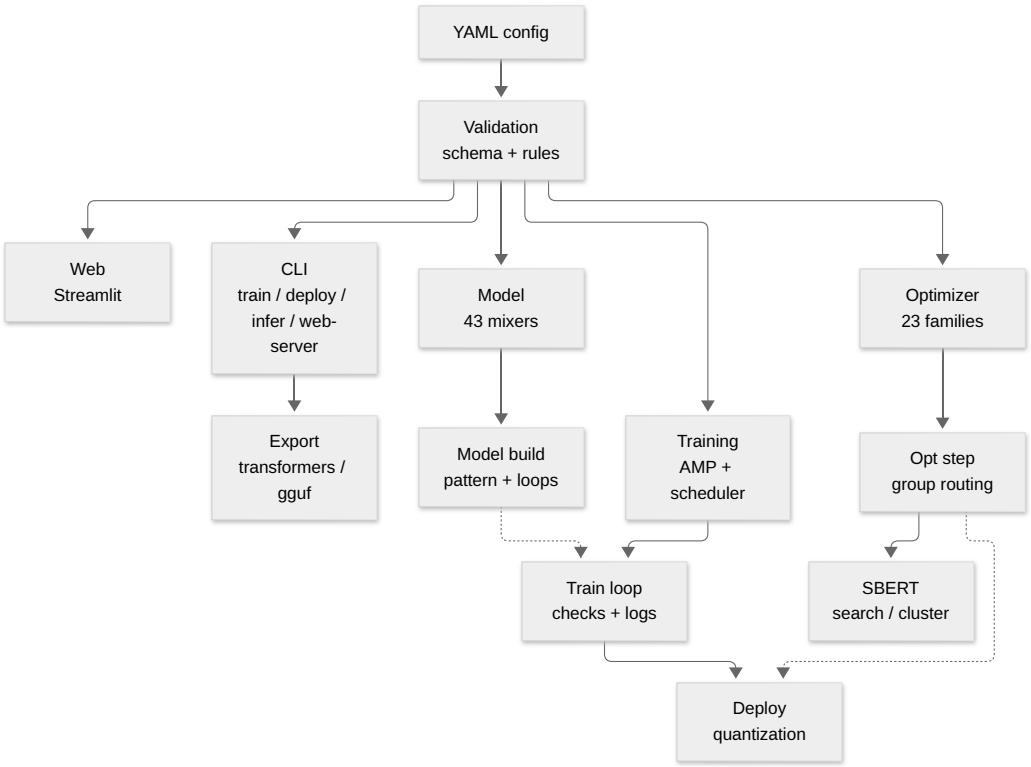



Figure 1: System architecture: configuration flows through validation, partitions into model, training, and optimizer components, executes at runtime, and produces deployment and SBERT artifacts.

**Mixer selection.** `model.dims.layer_pattern` is an ordered list of mixer identifiers drawn from the eight families described in Section 4; the schema fixes the accepted identifiers, and the complete enumeration appears in Figure 2 and Appendix 3.

**Optimizer selection.** `training.optimizer.optimizer_class` selects one of twenty-three optimizer families, summarized in Section 5 and detailed in Appendix 4.

### 3.2 Schema Scope and Validation Rules

Top-level and nested objects set `additionalProperties: false`, so unknown keys fail fast instead of being silently ignored. Optimizer parameters are further constrained by per-family prefix rules. Cross-component validation rejects incompatible combinations before training: hidden size must be divisible by the number of heads, the number of key–value heads must divide the number of heads, vocabulary size must match the tokenizer, a base model requires a task and (for MLM) a tokenizer, each task requires its matching optimizer block, BitNet router flags require BitNet to be enabled, and the decoder and vision classes force their respective masking modes.

### 3.3 Configuration Schema

The complete schema—fields, types, ranges, and validation rules—is documented in Appendix 2. Looped blocks are configured through `num_layers` and `num_loops`, giving `num_layers`×`num_loops` logical layers at fixed parameter count.

### 3.4 Training Tasks

`training.task` selects the objective: `mlm` (masked language modeling), `causal_lm` (autoregressive next-token prediction, decoder class only), `sbert` (sentence embeddings), `text_classification` (encoder class only), and the vision tasks `patch_prediction`, `classification`, and `segmentation` (Vision Transformer only). NLP tasks consume `text_dataset`; vision tasks consume `vision_dataset`. Sentence-embedding workflows are described in Section 7.

### 3.5 Training Safety and Runtime Semantics

The training step accumulates gradients over micro-batches, clips the global norm, checks for post-clip overflow, retries a bounded number of times on non-finite values, dispatches the configured scheduler, and rotates rolling and best checkpoints. The full pseudocode is deferred to Appendix 9.

### 3.6 Normalization Variants

Normalization is both a modeling choice and a schema compatibility question. The accepted `norm.type` values are `layer_norm`, `dynamic_tanh`, `derf`, `rms_norm`, `prms_norm` (partial RMSNorm), and `flash_norm` (weightless RMSNorm). Their formulations and comparison are detailed in Appendix 9.

### 3.7 Residual-Stream Architectures

Residual connections are exposed as a macro-design choice through the `model.residuals` block:

- **Standard**: the classical identity residual [40].
- **None**: no skip connection, retained only as an ablation probe.
- **Attention Residuals** (`full_attn` or `block_attn`): learned attention over depth replaces the fixed residual sum, either over all previous layers or over block-wise summaries [94].
- **Manifold-constrained hyper-connections** (`mhc.enabled`): an $n$-stream residual whose mixing matrix is constrained to the Birkhoff polytope [104].

Attention Residuals compose with mHC and Mixture-of-Depths; the formulations, wiring, and reported results are collected in Appendices 12 and 11.

## 4 Architecture Taxonomy and Implementation

### 4.1 Attention and Sequence-Mixer Families

The system implements forty-four sequence mixers organized into eight functional categories that reflect how each method balances expressivity, computational cost, and memory.

1. **Dense baselines** (3): standard softmax attention, sigmoid attention, and Grouped-Query Attention (GQA) provide full global routing at quadratic cost.
2. **Recurrent and retentive** (5): RetNet, Mamba, ODE-style blocks, and Titans maintain state for constant-time inference through recurrence, selective state spaces, continuous depth, or test-time memory.
3. **Sparse patterns** (10): Sparse Transformer, Longformer, SWA with attention sinks, BigBird, SparseK, NSA, SpargeAttn, FASA, MSA, and SparDA reduce quadratic cost through structured sparsity, token selection, or training-free pruning.

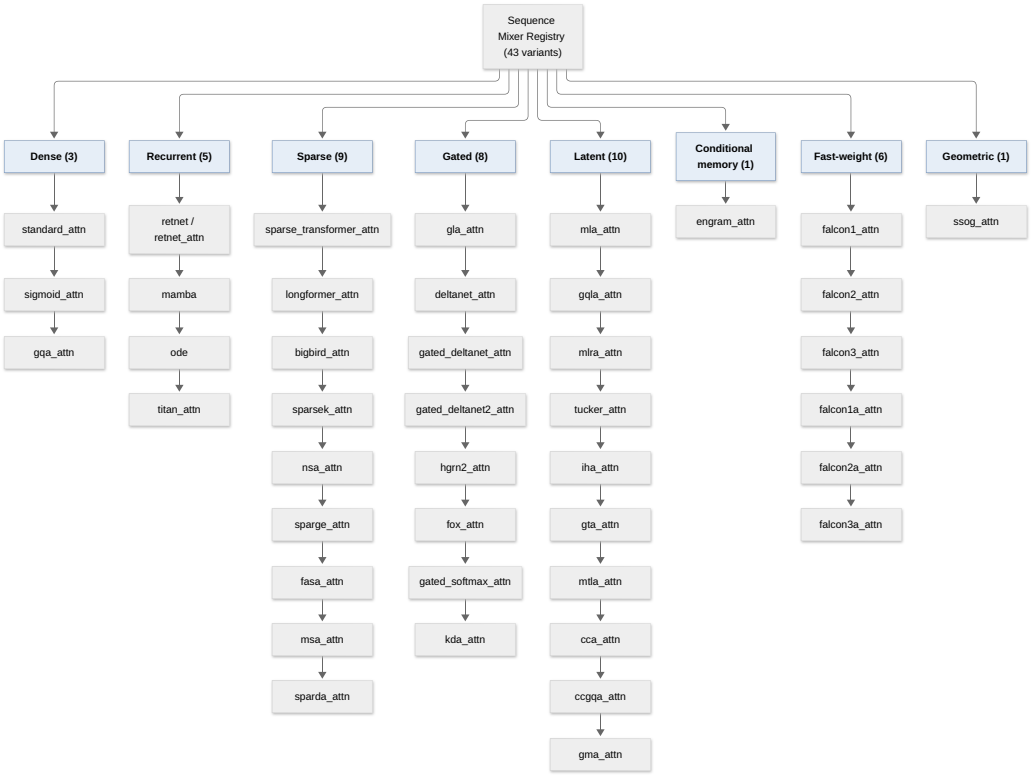



Figure 2: Taxonomy of the forty-three sequence mixers across eight functional categories. Dense baselines provide full global routing at quadratic cost; recurrent methods compress state for constant-time inference; sparse variants restrict the attention pattern; gated mechanisms modulate retention and forgetting; latent methods compress key–value representations; conditional memory adds test-time read/write lookups; fast-weight methods cast the state transition as an online learning rule; geometric methods encode structured priors.

4. **Gated mechanisms** (8): GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, FoX, Gated Softmax, and KDA introduce data-dependent control over retention, forgetting, and update strength.

5. **Latent compression** (10): MLA, GQLA, MLRA, Tucker, IHA, GTA, MTLA, CCA, CCGQA, and GMA compress key–value state into low-rank bottlenecks or route through a Gaussian-mixture latent memory.

6. **Conditional memory** (1): Engram provides test-time read/write memory through learned lookups.

7. **Fast-weight attention** (6): Falcon-1/2/3 and Falcon-1A/2A/3A treat the recurrent state transition as an online continual-learning rule [116].

8. **Geometric field** (1): SSOG applies a structured geometric prior over the attention matrix [76].

## 4.2 Dense and Recurrent Families

The dense baselines—standard softmax [97] and sigmoid attention [82]—and the recurrent family—RetNet [93], Mamba [35], ODE-style blocks [114], and Titans [7]—define the expressiveness-versus-efficiency trade-off. Formulations, parallel and recurrent forms, pseudocode, and a comparison table are collected in Appendix 5.

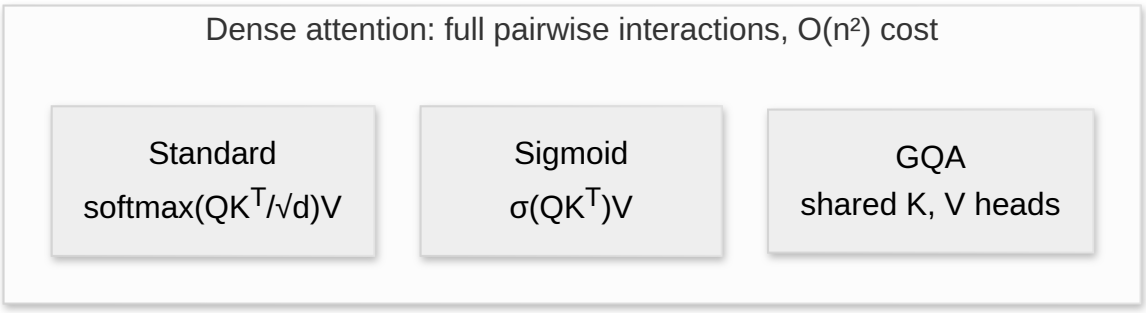



Figure 3: Dense attention baselines. Standard and sigmoid attention compute full pairwise interactions; Grouped-Query Attention shares key–value heads across query groups, reducing memory traffic while keeping full attention expressivity.

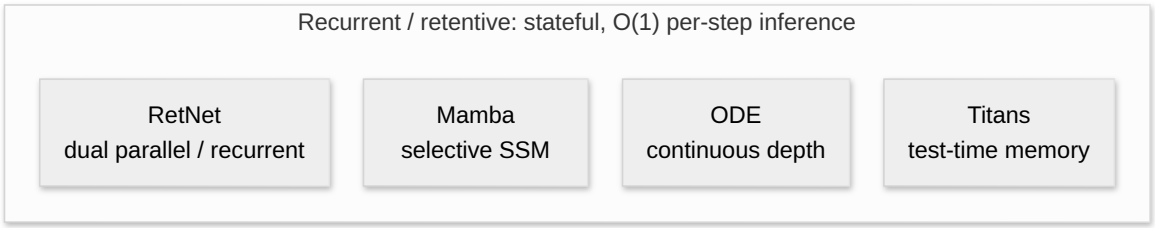



Figure 4: Recurrent and retentive family. RetNet offers parallel and recurrent forms, Mamba uses selective state-space parameters, ODE blocks model continuous depth, and Titans add test-time memory; all achieve constant-time per-step inference.

### 4.3 Latent and Low-Rank Attention Family

This family compresses per-token key–value state into a low-rank latent: MLA, GQLA, MLRA, Tucker, IHA, GTA, and MTLA unify grouped-query, latent, factorized, and temporal designs. CCA and CCGQA formulate attention in a compressed latent with decoupled grouping, while GMA replaces pairwise interaction with probabilistic routing through learned Gaussian-mixture components. Formulations, pseudocode, and a comparison table are collected in Appendix 6.

### 4.4 Sparse Attention Extensions

The registry includes ten sparse blocks [16, 8, 47, 111, 64, 109, 115, 100, 54, 30]. Two inference-time methods, `fasa_attn` and `sparge_attn`, apply their sparse heuristic only when the model is in evaluation mode: during training the enclosing hybrid layer substitutes standard dense attention over the same $\mathbf{q}/\mathbf{k}/\mathbf{v}/\mathbf{o}$ projections, so the trained weights are exactly the ones the sparse path reads at inference. `swa_attn` is also training-free but is a normal trainable mixer, so it runs its windowed pattern in both training and inference. Formulations, pseudocode, per-block characteristics, and a comparison table are collected in Appendix 7.

### 4.5 Gated and Fast-Weight Attention

The gated family contains eight blocks [105, 106, 107, 38, 79, 58, 80, 51]. Gating controls what information survives, either in recurrent state updates (GLA, DeltaNet variants, HGRN2, KDA) or in the full attention path (FoX, Gated Softmax). Formulations, pseudocode, and a comparison table are collected in Appendix 8.

Fast-weight attention recasts the recurrent update as an online continual-learning rule; the six Falcon variants [116] unify DeltaNet-style regression and linear-attention writes under read-

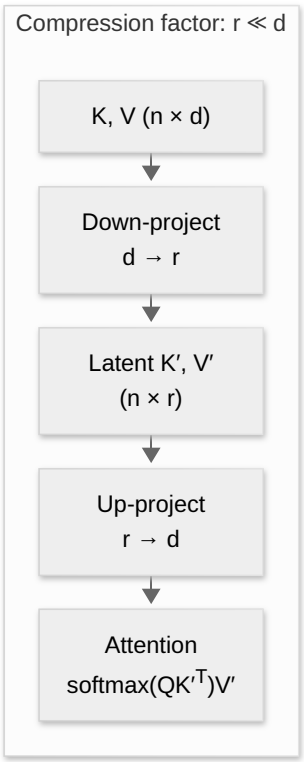



Figure 5: Latent and low-rank attention. Key–value pairs are projected into a low-rank latent space, where attention is computed at reduced cost, and projected back. Variants differ in factorization, head interleaving, temporal compression, cross-covariance formulation, and probabilistic routing.

after-write semantics with normalized step sizes and explicit forgetting. Each variant ships a recurrent reference scan and a chunk-parallel kernel. Formulations, configuration knobs, and published results are collected in Appendix 16.

## 5 Optimizer Families and Training Dynamics

Optimizing large transformer architectures is complicated by non-convex landscapes, saddle points, and heterogeneous parameter groups. The system supports twenty-three optimizer families in seven categories: classical baselines (SGD with momentum, AdamW, RAdam); momentum and variance reduction (Adan, ADOPT, AdEMAMix, MARS, Cautious AdamW); memory-efficient and low-rank methods (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO); schedule-free and parameter-free methods (Schedule-Free AdamW, Prodigy) with large-batch scaling (LAMB); curvature-aware and second-order methods (Sophia, Shampoo, SOAP); geometry-oriented methods (Muon, Turbo-Muon); and adaptivity-tunable optimizers (Anon). Algorithms, pseudocode, and memory and complexity comparisons are collected in Appendix 4.

## 6 Quantization and Deployment

The deploy stage converts a trained checkpoint into a compact artifact through ternary weight packing [99] and INT8 activation scaling, with size estimates aligned with lightweight deployment goals [85, 12]. The same subcommand emits standard or quantized artifacts, or exports

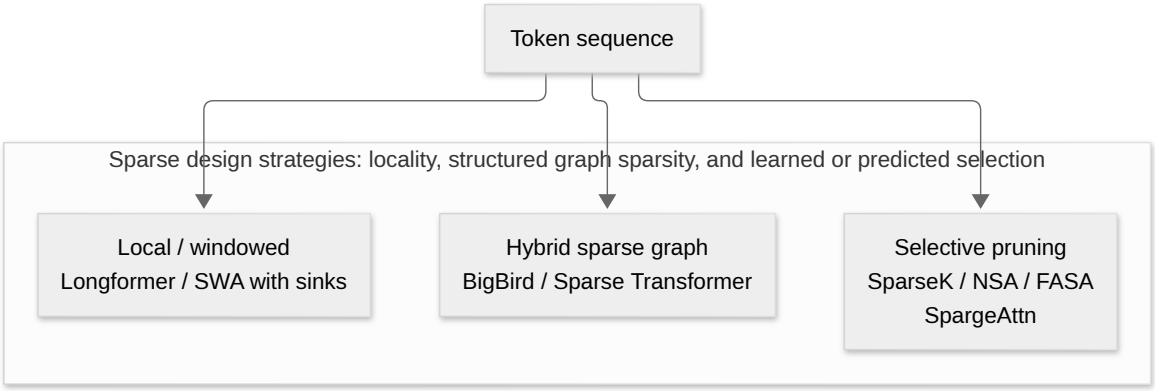



Figure 6: Sparse attention design choices. Methods reduce cost by restricting neighborhoods, constructing sparse graphs, or selecting only high-value tokens or blocks.

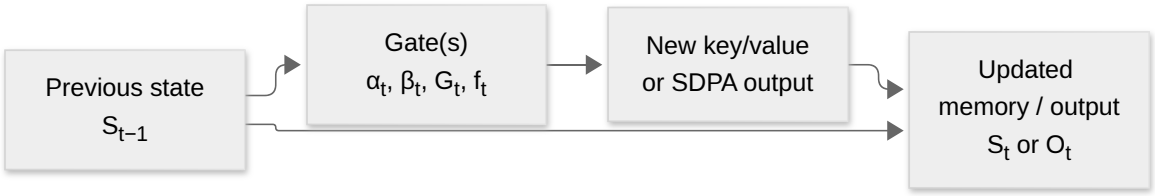



Figure 7: Generic gating template. A gate can decay existing memory, regulate write strength, or modulate dense attention outputs, depending on the block family.

HuggingFace Transformers and BitNet GGUF formats for external runtimes. The formulations and size estimates are deferred to Appendix 9.

## 7 SBERT Downstream Tasks

Sentence embedding is built on Siamese-style training [83]. For sentence pair $(s_1, s_2)$ with embeddings $(e_1, e_2)$:

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}$$

and regression-style cosine loss:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

with $y \in [-1, 1]$ in this pipeline.

Supported downstream modes:

- **Similarity**: pairwise score between two sentences.
- **Search**: top-$k$ nearest neighbors over a corpus.
- **Cluster**: grouping embeddings (e.g., k-means).
- **Encode**: persistent embedding export for later retrieval.

The pseudocode for the SBERT inference mode router—dispatching among similarity, search, cluster, and encode modes over a shared encoder—is deferred to Appendix 9.

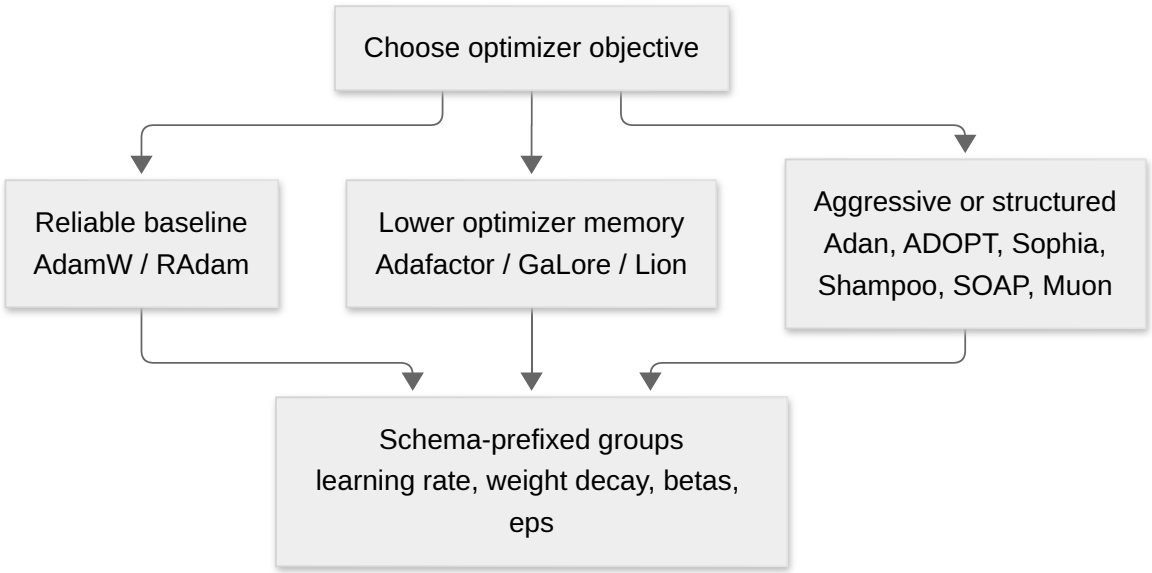



Figure 8: Optimizer selection in practice: the class determines the update rule, while the schema controls how hyperparameters apply across embeddings, normalization layers, attention blocks, and the remaining parameters.

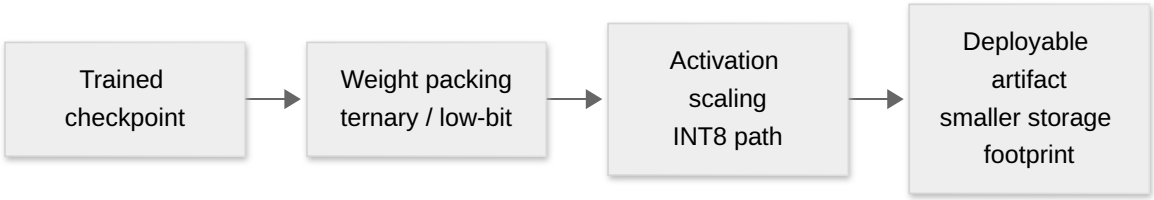



Figure 9: Deployment path from a trained checkpoint to a compact artifact. Quantization is a deploy-stage transformation rather than a separate model family.

# 8 No-Code Integration: The `dashai-frankenstein` Plugin

The CLI and web builder described in Section 3 serve researchers comfortable with YAML configuration files. A complementary audience—domain specialists, educators, and analysts who need a trained model but not a training script—is better served by a graphical environment. Frankenstein Transformer therefore ships `dashai-frankenstein`, an adapter that registers its model classes as components of **dashAI** (https://github.com/DashAISoftware/dashAI), an open-source, local-first machine-learning workbench developed by a Chilean academic consortium (Universidad de Chile/FCFM, CENIA, IMFD, and ANID funding). Both systems are schema-driven—dashAI renders forms from Pydantic schemas, Frankenstein validates YAML against a strict JSON Schema—so the integration leaves dashAI's backend untouched; the plugin is discovered through the standard `dashai.plugins` entry-points group documented in dashAI's plugin guide (https://docs.dash-ai.com/learn/guides/plugins/).

## 8.1 Registered Components

A single entry-points group registers five components:

- `FrankensteinMLMModel` (`BaseModel`, `TextClassificationTask`): encoder with a classification head over pooled output; `predict` returns per-class probabilities.

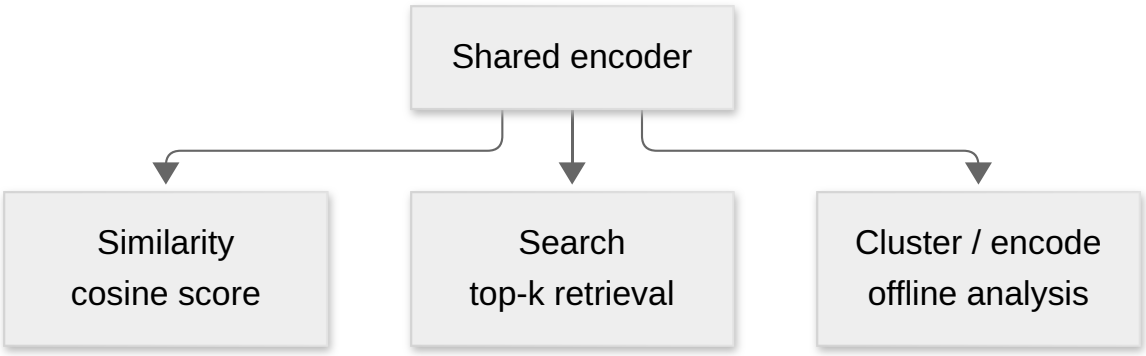



Figure 10: SBERT workflow reuse. A single encoder supports online pair scoring, corpus retrieval, clustering, and persistent embedding export.

- `FrankensteinDecoderModel` (`BaseGenerativeModel`, `TextToTextGenerationTask`): forces the decoder class and exposes autoregressive top-$k$ sampling.
- `FrankensteinViTClassifier` (`BaseModel`, `ImageClassificationTask`): uses the ViT classification head (Appendix 13).
- `FrankensteinViTSegmenter` (`BaseModel`, plugin `SegmentationTask`): uses the ViT segmentation head and returns per-pixel class maps.
- `SegmentationTask` (`BaseTask`): declares the image-in/mask-out contract that dashAI does not ship.

All share one thin facade over the engine API, so the plugin has a single heavy dependency.

### 8.2 Passthrough Configuration Schema

The Frankenstein JSON Schema remains the single source of truth instead of translating its fields into native dashAI forms. Each component exposes one `frankenstein_json` field holding a complete training configuration as single-line JSON. Before any training runs, the adapter parses the JSON, validates it against the resolved schema, and applies the config loader's cross-component checks; failures surface as a readable error tagged with the failing stage. Dataset adapters map dashAI datasets onto Frankenstein's expectations, inject the vocabulary size from the resolved tokenizer, and stream per-epoch metrics back to dashAI.

### 8.3 Status

The three implementation phases—classification MVP, generative decoder and ViT classifier, and ViT segmenter—are complete, but the package has not yet run inside a live dashAI instance and PyPI publication is pending. Known limitations include a direct classification loop rather than MLM pretraining, deferred native form fields for high-leverage knobs, and unconfirmed rendering of segmentation masks. The adapter deliberately leaves dashAI's core untouched, so future releases cannot break it through coupling.

## 9 Discussion

The design choices in Frankenstein Transformer reflect several engineering and research tensions in modern deep learning tooling.

### 9.1 Schema-Driven Design Trade-offs

The schema-first approach provides significant reproducibility benefits by enforcing explicit contracts and failing fast on invalid configurations. However, this approach also introduces rigidity: adding new architectures or optimizers requires schema extensions rather than loose command-line arguments. The prefixed hyperparameter system enables fine-grained control but increases configuration complexity for users accustomed to simpler interfaces.

The decision to enforce `additionalProperties: false` at all schema levels eliminates silent parameter swallowing that has plagued earlier configuration systems, but this strictness requires careful schema maintenance when extending system capabilities. Each new attention mechanism or optimizer variant must be properly integrated into the validation framework, including schema field definitions with appropriate types and constraints, prefixed hyperparameter mapping for optimizer-specific groups, default values aligned with research best practices, and documentation strings for web interface rendering.

### 9.2 Architectural Coverage and Gaps

The forty-three implemented mixers span the major research directions in sequence modeling, but certain gaps remain. The system lacks recent hybrid architectures such as Griffin [89] and Jamba [36], which combine gating with state-space models. MoE routing is implemented for FFN layers but not for attention computation, where recent work has shown benefits [56].

The sparse attention coverage is broad. The two inference-time methods (FASA, SpargeAttn) keep their training-free character in the sense that their sparsification rule is never learned; instead, the implementation trains the layer as standard dense attention over the same projections and activates the sparse selection/pruning path only at evaluation, so both mixers can be trained from scratch and their sparse behaviour is exactly the one described in the original papers.

The gated and fast-weight coverage spans the major categories, including the six Falcon variants [116]. Residual-stream macro-design is covered by Attention Residuals [94] and by the manifold-constrained $n$-stream residual (mHC) [104], whose Sinkhorn–Knopp projection onto the Birkhoff polytope restores the identity-mapping property lost by unconstrained hyper-connections; mHC is validated only at the level of the published 3B–27B MoE experiments and would benefit from independent reproductions at smaller scales (Appendix 11).

### 9.3 Optimizer Landscape Fragmentation

The support for twenty-three optimizer families across seven categories demonstrates comprehensiveness but also highlights the fragmented state of optimization research. Users face significant decision complexity when choosing among variance-reduction methods (Adan, MARS), memory-efficient variants (GaLore, Adafactor, APOLLO), and curvature-aware approaches (Sophia, Shampoo). The prefixed hyperparameter system, while powerful, requires understanding of which parameters are relevant for each optimizer class.

The implementation quality varies across optimizers: classical methods (AdamW, SGD with momentum) are highly optimized in PyTorch, while newer methods (Muon, Turbo-Muon, SOAP) may require custom implementations that affect numerical stability and performance characteristics.

### 9.4 Deployment and Production Considerations

The quantization pipeline demonstrates practical deployment concerns but makes specific engineering trade-offs. Ternary weight packing reduces storage to approximately 1.58 bits per parameter, but this aggressive compression may degrade performance, especially for smaller models where quantization error is more significant. The current implementation applies quantization uniformly across parameter types.

The SBERT workflows provide practical utility for semantic similarity and retrieval tasks, but implementation assumes standard pooling strategies (CLS token, mean pooling). Recent advances such as Matryoshka embeddings [70] and contrastive learning refinements [50] are not yet incorporated.

### 9.5 Integration and Extensibility Challenges

The current codebase structure, while functional, presents maintenance challenges as architecture and optimizer families expand. The dispatcher pattern for mixer selection and optimizer routing handles extensibility but risks becoming a "kitchen sink" of conditional logic. Future versions would benefit from plugin-based architectures where new mixers and optimizers could be registered declaratively rather than modifying core dispatch logic.

The web configuration interface provides significant usability improvements but introduces deployment complexity: running Streamlit alongside training jobs requires additional resources and infrastructure considerations that may not be appropriate for all environments, particularly HPC clusters without web access.

## 10 Conclusion

Frankenstein Transformer presents a unified, configuration-driven experimentation platform addressing three challenges in modern deep learning research: architectural fragmentation across dense attention, recurrent models, sparse patterns, gated mechanisms, latent compression, conditional memory, fast-weight memories, and geometric priors; optimizer landscape complexity spanning classical baselines, variance-reduction methods, memory-efficient variants, schedule-free approaches, curvature-aware algorithms, and geometry-oriented methods; and end-to-end deployment workflows spanning quantization and sentence embedding.

The system's primary contributions are:

1. **Schema-First Design**: A strict YAML-based configuration contract with validation and prefixed hyperparameter routing enabling reproducible experiments across forty-three sequence mixers and twenty-three optimizer families.

2. **Comprehensive Architecture Support**: Implementation spanning dense baselines, recurrent and retentive blocks, sparse attention, gated mechanisms, latent compression, conditional memory, fast-weight attention, and geometric-field attention, with encoder, decoder, and Vision Transformer model classes.

3. **Unified Optimizer Framework**: Prefixed hyperparameter groups enabling fine-grained control over embeddings, normalization layers, attention weights, and the remaining parameters across classical baselines (SGD+Momentum, AdamW, RAdam), variance reduction (Adan, ADOPT, AdEMAMix, MARS, Cautious), memory-efficient and low-rank methods (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), large-batch and schedule simplification (LAMB, Schedule-Free AdamW, Prodigy), curvature-aware (Sophia), second-order (Shampoo, SOAP), geometry-oriented (Muon, Turbo-Muon), and adaptivity-tunable (Anon) optimizers.

4. **End-to-End Workflows**: Integrated deployment pipeline supporting ternary weight packing and INT8 activation quantization; SBERT-inspired training and inference for semantic similarity, retrieval, and clustering tasks.

5. **Interactive Configuration**: Streamlit-based web interface providing schema-driven form generation, real-time validation, inline documentation, and CLI command synthesis.

This system enables rapid experimental iteration while maintaining reproducibility through strict configuration contracts. By consolidating diverse research contributions into a unified

toolkit, it lowers barriers to exploring novel architectures and optimization strategies, particularly for researchers who may lack resources to implement and validate each variant independently.

### 10.1 Limitations and Future Directions

Several limitations and promising directions for future work emerge from this system's design and implementation:

1. **Architecture Integration**: Recent hybrid architectures (Griffin, Jamba, Mamba-X) demonstrate benefits of combining multiple mechanisms into unified blocks. Future versions should integrate these architectures and explore systematic composition patterns.

2. **Advanced Quantization**: Current implementation uses uniform ternary packing across all parameters. Research on layer-wise, channel-wise, and importance-aware quantization suggests more sophisticated strategies could improve quality-efficiency trade-offs.

3. **Plugin-Based Extensibility**: The current dispatch pattern becomes increasingly complex with each new addition. A plugin architecture allowing declarative registration of new mixers, optimizers, and normalization methods would improve maintainability and reduce risk of bugs in core dispatch logic.

4. **Automated Hyperparameter Optimization**: The schema supports extensive hyperparameter spaces, but users must manually explore these spaces. Integration with Bayesian optimization, multi-armed bandit strategies, or gradient-based hyperparameter tuning could automate effective configuration discovery.

5. **Production Deployment**: The web interface improves usability but may not be appropriate for all deployment environments. Headless configuration modes, API-based configuration management, or improved CLI ergonomics could serve HPC and production workflows.

6. **Evaluation Benchmarking**: While the system enables training with diverse architectures, comprehensive benchmarking comparing performance across mixers and optimizers on standardized tasks would provide valuable guidance for configuration selection.

7. **Training Stability Guarantees**: Current implementation includes NaN/Inf guards and gradient clipping, but formal analysis of stability conditions for different mixer-optimizer combinations, particularly with looped blocks and aggressive quantization, remains open.

8. **Multimodal and Task-Specific Extensions**: The current design covers text and single-image tasks through the Vision Transformer class. Extensions to vision-language models, multimodal architectures, and task-specific fine-tuning workflows (e.g., instruction tuning, RLHF) would broaden applicability.

The research trajectory of sequence modeling continues toward hybrid approaches that combine the strengths of multiple paradigms—compression from recurrence, selectivity from attention, gating for memory management, and sparsity for efficiency. A unified experimentation platform like Frankenstein Transformer becomes increasingly valuable as this convergence accelerates, enabling researchers to explore this expanding design space with reproducible, well-engineered infrastructure.

## Bibliography

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur,

Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyansky, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL https://arxiv.org/abs/2305.13245.

[3] Anonymous authors. Anon: Extrapolating adaptivity beyond sgd and adam, 2026. URL https://arxiv.org/abs/2605.02317.

[4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL https://arxiv.org/abs/1607.06450.

[5] Jonathan T. Barron. Continuously differentiable exponential linear units. *arXiv preprint arXiv:1704.07483*, 2017.

[6] Mina Basirat and Peter M. Roth. The quest for a better ReLU: ELiSH and HardELiSH activation functions. *arXiv preprint arXiv:1905.10144*, 2019.

[7] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, 2025. URL https://arxiv.org/abs/2501.00663. Version Number: 1.

[8] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

[9] Arthur S. Bianchessi, Yasmin C. Aguirre, Rodrigo C. Barros, and Lucas S. Kupssinskü. Bayesian attention mechanism: A probabilistic framework for positional encoding and context length extrapolation, 2025. URL http://arxiv.org/abs/2505.22842.

[10] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning, 2025. URL https://arxiv.org/abs/2512.04632. Version Number: 1.

[11] James Bradbury, Roy Frostig, Peter Hawkins, Matthew J. Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/jax-ml/jax.

[12] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory, 2026. URL http://arxiv.org/abs/2502.10001.

[13] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, and Zhuang Liu. Stronger normalization-free transformers, 2025. URL http://arxiv.org/abs/2512.10938.

[14] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, 2023. URL https://arxiv.org/abs/2302.06675. Version Number: 4.

[15] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL https://arxiv.org/abs/2601.07372.

[16] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL https://arxiv.org/abs/1904.10509.

[17] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). *arXiv preprint arXiv:1511.07289*, 2016.

[18] Chang Dai, Hongyu Shan, Mingyang Song, and Di Liang. HoPE: Hyperbolic rotary positional encoding for stable long-range dependency modeling in large language models, 2025. URL http://arxiv.org/abs/2509.05218.

[19] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled, 2024. URL https://arxiv.org/abs/2405.15682. Version Number: 4.

[20] Keqi Deng and Philip C. Woodland. Multi-head temporal latent attention, 2025. URL https://arxiv.org/abs/2505.13544.

[21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding, 2019. URL http://arxiv.org/abs/1810.04805.

[22] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, 2021. URL http://arxiv.org/abs/2010.11929.

[23] Haojie Duanmu, Zhihang Yuan, Xiuhong Li, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. SKVQ: Sliding-window KV cache quantization for coding LLMs, 2024. URL https://arxiv.org/abs/2405.06219. arXiv:2405.06219.

[24] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 453:1–24, 2021. doi: 10.1016/j.neucom.2021.03.063.

[25] Sai Surya Duvvuri, Chanakya Ekbote, Rachit Bansal, Rishabh Tiwari, Devvrit Khatri, David Brandfonbrener, Paul Liang, Inderjit Dhillon, and Manzil Zaheer. Interleaved head attention, 2026. URL https://arxiv.org/abs/2602.21371.

[26] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 106:300–312, 2018. doi: 10.1016/j.neunet.2018.07.012.

[27] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. LayerSkip: Enabling early exit inference and self-speculative decoding, 2024. URL https://arxiv.org/abs/2404.16710. arXiv:2404.16710.

[28] Aiming Fang, Suhyune Lee, Nafise Sadat Moosavi, and Iryna Gurevych. Transformers with learnable activation functions. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1736–1746, 2023. doi: 10.48550/arXiv.2208.14111.

[29] Tomas Figliola, Nicholas Alonso, Rishi Iyer, Quentin Anthony, and Beren Millidge. Compressed convolutional attention: Efficient attention in a compressed latent space, 2025. URL https://arxiv.org/abs/2510.04476.

[30] Yaosheng Fu, Guangxuan Xiao, Xin Dong, Song Han, and Oreste Villa. Sparda: Sparse decoupled attention for efficient long-context llm inference, 2026. URL https://arxiv.org/abs/2606.04511.

[31] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 315–323, 2011.

[32] Luke B. Godfrey and Michael S. Gashler. A continuum among logarithmic, linear, and exponential functions, and its potential to improve generalization in neural networks. In *Proceedings of the International Conference on Knowledge Discovery and Information Retrieval (KDIR)*, 2015.

[33] Ian J. Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1319–1327, 2013.

[34] Nils Graef, Filip Makraduli, Andrew Wasielewski, and Matthew Clapp. FlashNorm: Fast normalization for transformers, 2026. URL http://arxiv.org/abs/2407.09577.

[35] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL https://arxiv.org/abs/2312.00752. Version Number: 2.

[36] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL https://arxiv.org/abs/2403.19887.

[37] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization, 2018. URL https://arxiv.org/abs/1802.09568. Version Number: 2.

[38] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated deltanet-2: Decoupling erase and write in linear attention, 2026. URL https://arxiv.org/abs/2605.22791.

[39] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[40] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016. doi: 10.1109/CVPR.2016.90. URL http://arxiv.org/abs/1512.03385.

[41] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked Autoencoders Are Scalable Vision Learners, 2022. URL http://arxiv.org/abs/2111.06377.

[42] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELU). In *arXiv preprint arXiv:1606.08415*, 2016.

[43] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization, 2024. URL https://arxiv.org/abs/2401.18079. arXiv:2401.18079.

[44] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1325, 2019.

[45] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. In *arXiv preprint arXiv:1704.04861*, 2017.

[46] Yongchao Huang and Hassan Raza. Gaussian mixture attention: Linear-time sequence mixing via probabilistic latent routing, 2026. URL https://arxiv.org/abs/2606.18283.

[47] Alexia Jolicoeur-Martineau, Rhea Sanjay Sukthanker, Pashmina Cameron, and Emy Gervais. Sliding window beats linear attention, 2026. URL https://arxiv.org/abs/2608.28444.

[48] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers, 2023. URL http://arxiv.org/abs/2305.19466.

[49] Daan Kerssies, Quentin Sartenaer, Fabio Ferraro, Gabriel Y. da Silva, Harkanwarpreet Maan, Matthieu Galon, Arsha Nagrani, Nathan Beeler, Bartlomiej Twardowski, Joost van de Weijer, Robert Dangovski, and Luca Berton. Your ViT is Secretly an Image Segmentation Model, 2025. URL http://arxiv.org/abs/2503.19108.

[50] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL https://arxiv.org/abs/2004.11362.

[51] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL https://arxiv.org/abs/2510.26692.

[52] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[53] Timon Klein, Jonas Kusch, Sebastian Sager, Stefan Schnake, and Steffen Schotthöfer. Tucker attention: A generalization of approximate attention mechanisms, 2026. URL https://arxiv.org/abs/2603.30033.

[54] Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. Minimax sparse attention, 2026. URL https://arxiv.org/abs/2606.13392.

[55] Johannes Lederer. Activation functions in artificial neural networks: A systematic overview. *arXiv preprint arXiv:2101.09957*, 2021. doi: 10.48550/arXiv.2101.09957.

[56] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL https://arxiv.org/abs/2103.16716.

[57] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code, 2024. URL https://arxiv.org/abs/2411.16085. Version Number: 4.

[58] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL https://arxiv.org/abs/2503.02130.

[59] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, 2023. URL https://arxiv.org/abs/2305.14342. Version Number: 4.

[60] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, 2019. URL https://arxiv.org/abs/1908.03265. Version Number: 4.

[61] Songtao Liu, Hongwu Peng, Zhiwei Zhang, Zhengyu Chen, and Yue Guo. Multi-head low-rank attention, 2026. URL https://arxiv.org/abs/2603.02188.

[62] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache, 2024. URL https://arxiv.org/abs/2402.02750. arXiv:2402.02750.

[63] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2017. URL https://arxiv.org/abs/1711.05101. Version Number: 3.

[64] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL https://arxiv.org/abs/2406.16747.

[65] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proc. ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.

[66] Sushant Mehta, Raj Dandekar, Rajat Dandekar, and Sreedath Panat. Latent multi-head attention for small language models, 2025. URL https://arxiv.org/abs/2506.09342.

[67] Fanxu Meng. Gqla: Group-query latent attention for hardware-adaptive large language model decoding, 2026. URL https://arxiv.org/abs/2605.15250.

[68] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner, 2023. URL https://arxiv.org/abs/2306.06101. Version Number: 4.

[69] Diganta Misra. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*, 2019.

[70] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL https://arxiv.org/abs/2205.13147.

[71] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[72] Christoffer Koo Øhrstrøm, Rafael I. Cabral Muchacho, Yifei Dong, Filippos Moumtzidellis, Ronja Güldenring, Florian T. Pokorny, and Lazaros Nalpantidis. Parabolic position encoding: Vision-centric, principled, extrapolatable, general, 2026. URL http://arxiv.org/abs/2602.01418.

[73] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older, 2024. URL https://arxiv.org/abs/2409.03137. Version Number: 2.

[74] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.

[75] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[76] Raphael Pisoni. A few gaussians is all you need: Ssog-attention that steers instead of scores, 2026. URL https://www.pisoni.ai/posts/ssog/. Reference implementation: https://github.com/4rtemi5/ssog.

[77] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.

[78] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation, 2022. URL http://arxiv.org/abs/2108.12409.

[79] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL https://arxiv.org/abs/2404.07904.

[80] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL https://arxiv.org/abs/2505.06708.

[81] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. In *arXiv preprint arXiv:1710.05941*, 2017.

[82] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention, 2024. URL https://arxiv.org/abs/2409.04431. Version Number: 2.

[83] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks, 2019. URL http://arxiv.org/abs/1908.10084.

[84] Susmita Roy, Souvik Manna, Subhash Bagui, et al. LiSHT: Non-parametric linearly scaled hyperbolic tangent activation function for neural networks. *arXiv preprint arXiv:1811.05870*, 2019.

[85] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications, 2026. URL http://arxiv.org/abs/2601.03290.

[86] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[87] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost, 2018. URL https://arxiv.org/abs/1804.04235. Version Number: 1.

[88] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon, 2025. URL https://arxiv.org/abs/2505.23737. Version Number: 1.

[89] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL https://arxiv.org/abs/2402.19427.

[90] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/hash/a58f4765a99bf77336cf034926677066-Abstract.html.

[91] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2024. URL http://arxiv.org/abs/2104.09864.

[92] Luoyang Sun, Cheng Deng, Jiwen Jiang, Xinjian Wu, Haifeng Zhang, Lei Chen, Lionel Ni, and Jun Wang. Gta: Grouped-head latent attention, 2025. URL https://arxiv.org/abs/2506.17286.

[93] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL https://arxiv.org/abs/2307.08621. Version Number: 4.

[94] Yutao Sun et al. Attention residuals. URL http://arxiv.org/abs/2603.15031.

[95] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any $\beta_2$ with the optimal rate, 2024. URL https://arxiv.org/abs/2411.02853. Version Number: 3.

[96] Ludovic Trottier, Philippe Giguère, and Brahim Chaib-draa. Parametric exponential linear unit for deep convolutional neural networks. In *Proceedings of the 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 207–214, 2017. doi: 10.1109/ICMLA.2017.0-133.

[97] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. URL https://arxiv.org/abs/1706.03762. Version Number: 7.

[98] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam, 2024. URL https://arxiv.org/abs/2409.11321. Version Number: 2.

[99] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models, 2023. URL http://arxiv.org/abs/2310.11453.

[100] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL https://arxiv.org/abs/2602.03152.

[101] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 38–45, 2020.

[102] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17453.

[103] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models, 2022. URL https://arxiv.org/abs/2208.06677. Version Number: 5.

[104] Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, Liang Zhao, Shangyan Zhou, Zhean Xu, Zhengyan Zhang, Wangding Zeng, Shengding Hu, Yuqing Wang, Jingyang Yuan, Lean Wang, and Wenfeng Liang. mhc: Manifold-constrained hyper-connections, 2025. URL https://arxiv.org/abs/2512.24880.

[105] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL https://arxiv.org/abs/2312.06635.

[106] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL https://arxiv.org/abs/2406.06484.

[107] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL https://arxiv.org/abs/2412.06464.

[108] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes, 2020. URL https://arxiv.org/abs/1904.00962. Version Number: 3.

[109] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL https://arxiv.org/abs/2502.11089.

[110] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models, 2024. URL https://arxiv.org/abs/2411.10438. Version Number: 4.

[111] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL https://arxiv.org/abs/2007.14062.

[112] Abbas Zeitoun, Lucas Torroba-Hennigen, and Yoon Kim. Hyperloop transformers, 2026. URL https://arxiv.org/abs/2604.21254.

[113] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization, 2019. URL http://arxiv.org/abs/1910.07467.

[114] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(16):14393–14401, 2021. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL https://ojs.aaai.org/index.php/AAAI/article/view/17692.

[115] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL https://arxiv.org/abs/2502.18137.

[116] Yifan Zhang, Steve Ta, Jasper Zhang, Jichen Feng, Shuzhen Li, Yongxin Zhang, Yifeng Liu, Huizhuo Yuan, Mengdi Wang, Quanquan Gu, and Andrew Chi-Chih Yao. Fast weight attention for continual learning, 2026. URL https://arxiv.org/abs/2608.27763.

[117] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection, 2024. URL https://arxiv.org/abs/2403.03507. Version Number: 2.

[118] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance, 2025. URL https://arxiv.org/abs/2412.05270.

[119] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization, 2025. URL http://arxiv.org/abs/2503.10622.

[120] Lianghui Zhu, Yuxin Fang, Bencheng Liao, Shijie Wang, Tianheng Cheng, Zilong Huang, Chen Chen, Lai Wei, Yutao Zeng, Ya Wang, Yi Lin, Yu Li, and Xinggang Wang. Mixture-of-depths attention, 2026. URL https://arxiv.org/abs/2603.15619.

# 1 Conceptual Introduction—Transformers and Attention for Beginners

This appendix provides an accessible introduction to the fundamental concepts of transformers and the attention mechanism, aimed at readers without deep prior knowledge in deep learning. Transformers are the engine behind modern artificial intelligence systems like ChatGPT, but their operation can be understood through real-world analogies.

## 1.1 What is a Transformer?

A transformer is a neural architecture that processes text or data sequences by interpreting the meaning of each word while considering all other words in context. Imagine you read a sentence:

*"The bank was full of people waiting. Mary went to the bank."*

What does "bank" mean in each case? In the first sentence, it refers to a financial institution (or a bench). In the second, it is less clear without context. A transformer solves this automatically by looking at all surrounding words. In the second sentence, seeing "Mary went", the model better understands that it probably is a riverbank (where she goes to swim) or a financial institution (where she goes to conduct a transaction).

This ability to understand the complete meaning of a word by considering the entire sentence is called *contextualization*, and it is the heart of how modern transformers work.

## 1.2 The Attention Mechanism: An Analogy

The attention mechanism is the "magic trick" that allows transformers to understand context. Think of it as participating in an important conversation in a noisy room:

1. **Lots of noise**: Someone is speaking and being very important to you.
2. **You ignore the rest**: Your brain automatically focuses attention on that person, ignoring other sounds.
3. **You understand better**: By concentrating, you catch every word clearly.

The neural attention mechanism works the same way: each word "asks" how important every other word is to understanding its meaning, then "focuses" its attention on the most relevant ones.

## 1.3 Practical Example Step-by-Step

Let's see how a transformer understands the word "eats" in two different contexts:

### 1.3.1 Context 1: "The cat eats fish"

The transformer, when processing "eats", internally asks:

- How relevant is "The"? Little (it's just an article). **Low attention**.
- How relevant is "cat"? Very relevant (it's the subject, who eats). **High attention**.
- How relevant is "fish"? Very relevant (it's what is eaten). **High attention**.

Thus, the mental representation of "eats" focuses primarily on "cat" and "fish".

### 1.3.2 Context 2: "The restaurant eats into profit margins"

Here the transformer asks in a different context:

- How relevant is "restaurant"? Very relevant (it's the subject). **High attention**.
- How relevant is "profit"? Very relevant (it's affected). **High attention**.
- How relevant is "margins"? Very relevant (it's what's being consumed). **High attention**.

The word "eats" gets a completely different representation because it "attends" to different words in different contexts.

## 1.4 How Attention Works Mathematically (Simple Version)

Behind the attention shift is mathematics. Here is the simplified version without too much technical detail:

### 1.4.1 Step 1: Queries, Keys, and Values

Each word in the sentence is transformed into three versions:

- **Query**: "What information do I need to know?"
- **Key**: "What information do I have?"
- **Value**: "Here is my important information."

Imagine in a library, each visitor is a "query", each book is a "key", and the content is the "value". The librarian (attention) matches queries with the most relevant keys to access the correct value.

### 1.4.2 Step 2: Compatibility

The system examines how *compatible* each query is with each key. Queries similar to keys get a *high* compatibility score. This is calculated by multiplying the query by the key (mathematically, the dot product).

### 1.4.3 Step 3: Focus

The compatibility scores are converted into "focus weights". High compatibilities mean "focus attention here", and low ones mean "ignore this". Mathematically, a function called `softmax` converts scores into percentages (like: 40% attention here, 35% here, 25% there).

### 1.4.4 Step 4: Combine

Finally, the system combines all values, weighted by the focus weights. If a word has 80% attention on the subject, 80% of the subject's information gets blended into that word's representation.

## 1.5 Multiple Attention Heads: Multiple Perspectives

A key feature is that transformers do not use a single attention but *multiple attention heads* in parallel. It's as if you had 8 people analyzing the sentence *simultaneously*, each paying attention to different aspects:

- **Person 1**: "I focus on subject-verb relationships."
- **Person 2**: "I search for the direct object."
- **Person 3**: "I track information about verb tense."
- **Person 4**: "I search for modifiers and adjectives."

Each "head" learns to focus on different language patterns. Together, they capture much richer understanding than a single head.

## 1.6 Stacking Layers

Transformers process information in *multiple layers* (usually 12 to 48 for practical models). Imagine editing an essay:

1. **First pass**: You fix spelling and basic grammar.
2. **Second pass**: You improve clarity and sentence structure.
3. **Third pass**: You ensure consistency and narrative flow.

Transformers work the same way: each layer refines text understanding. The first layer captures basic features (simple words, genders), while later layers understand complex concepts (word relationships, abstract meanings).

## 1.7 Why Does It Matter?

The attention mechanism was revolutionary because:

1. **Parallelism**: It finds context for *any word with any other word*, without processing them sequentially (unlike earlier systems). This makes it very fast.
2. **Flexibility**: It learns what patterns to look for automatically from data, without you needing to program rules manually.
3. **Scalability**: It works from small texts to contexts with millions of words.
4. **General Capabilities**: The same mechanism works for translation, summarization, question-answering, text generation, computer vision, and more.

## 1.8 Practical Challenges

Despite the extraordinary success of transformers, they present challenges:

### 1.8.1 Computational Complexity

If a sentence has 1,000 words, the standard attention mechanism must compare each word with all other 1,000 words. That's $1,000 \times 1,000 = 1,000,000$ comparisons. For long documents with millions of words, this becomes prohibitively expensive in time and memory.

### 1.8.2 Memory Requirements

Especially during inference (when using trained models), storing the entire attention matrix can consume enormous amounts of RAM or GPU memory, limiting the sequence lengths you can process.

### 1.8.3 Device Efficiency

Transformers were designed for powerful TPUs and GPUs. Running them on phones or embedded devices is challenging.

## 1.9 Modern Solutions

To solve these challenges, research has proposed many variants:

- **Sparse Attention**: Instead of comparing each word with ALL others, it compares only with a strategic subset (nearby neighbors, periodic patterns). Reduces complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$.
- **Recurrent Models**: Like Mamba, they incorporate aspects of old recurrent neural networks but with better modern efficiency.
- **Gated Attention**: Mechanisms that selectively learn what information to pass forward, reducing storage needs.
- **Quantization**: Use smaller numbers (integers instead of decimals) to reduce memory without losing too much precision.

These innovations are what this toolkit (Frankenstein) allows you to experiment with easily.

## 1.10 Key Takeaways

- A **transformer** is a neural network that understands language context very well.
- The **attention mechanism** allows each word to "focus" on which other words are relevant to understanding its meaning.
- Attention works by computing **compatibility** between each word (query) and all others (keys), then combining information based on that compatibility (values).
- **Multiple heads** of attention explore different patterns in parallel.
- **Multiple layers** progressively refine understanding.
- The main challenge is that standard attention has quadratic complexity, which is expensive for long texts.
- Many **modern solutions** (sparse attention, recurrent models, quantization) address these challenges, and this toolkit lets you explore all of them.

# 2 Configuration Schema Structure

The Frankenstein Transformer toolkit enforces a strict YAML configuration schema with `additionalProperties: false` at all levels. The schema is defined in `src/schema.yaml`, which references modular sub-schemas in `src/schema/`. Every configurable field must be declared in the schema; unrecognized keys are rejected at validation time. This appendix documents the complete schema structure, including all fields, their types, constraints, and cross-component validation rules.

## 2.1 Top-Level Structure

The root schema object has the following top-level properties. All are optional except `training`, which is required:

- `base_model` (string) — HuggingFace model identifier or local path for continual pretraining or SBERT fine-tuning. When set, `model_class` and `model` are not required (the architecture is inferred from the checkpoint).
- `tokenizer` (object) — Tokenizer configuration, available to both the `base_model` path and custom from-scratch models. Two modes are selected via `source`: `hf_repo` (default) loads a pretrained tokenizer from `name_or_path`; `train_from_dataset` trains a new tokenizer from the dataset text (see §2.4). Required when using `base_model` with the `mlm` task.
- `model_class` (enum) — Model architecture variant: `frankenstein_text_encoder` (mixed-architecture encoder), `frankenstein_text_decoder` (autoregressive causal decoder), or `frankenstein_vit` (vision transformer). Required when `base_model` is not set.
- `model` (object) — Model architecture parameters (see §2.2). Required when `base_model` is not set.
- `training` (object, **required**) — Training configuration (see §2.3).

The schema enforces a mutual-exclusivity rule: either `base_model` must be present, or both `model_class` and `model` must be present.

## 2.2 Model Configuration

The `model` object defines all architectural hyperparameters. Required fields are `dims.vocab_size`, `dims.hidden_size`, `dims.num_layers`, `dims.num_heads`, and `dims.layer_pattern`. Table 1 lists all model fields with their types and descriptions.

| Field | Type | Description |
|---|---|---|
| *model.dims.** | | |
| `dims.mode` | enum (encoder, decoder) | Attention masking mode. `encoder` = bidirectional; `decoder` = causal. |
| `dims.vocab_size` | int $\geq 1$ (required) | Number of unique tokens in the vocabulary. Must match the tokenizer's vocab size. |
| `dims.hidden_size` | int $\geq 1$ (required) | Hidden dimension. Must be divisible by `num_heads`. Typical: 512–2048. |
| `dims.num_layers` | int $\geq 1$ (required) | Number of physical transformer layers. Typical: 6–24. |
| `dims.num_loops` | int $\geq 1$ | Logical loop count. Each layer in `layer_pattern` runs `num_loops` times. Default: 1. |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| `dims.num_heads` | int $\geq 1$ (required) | Number of parallel attention heads. Must divide `hidden_size` evenly. |
| `dims.dropout` | float [0,1] | Global dropout rate. Typical: 0.1 (BERT standard). |
| `dims.layer_pattern` | array of enum (required) | Ordered sequence of layer types. Length must equal `num_layers`. 43 mixer codes available (see Table 2.2.1). |
| *model.norm.** | | |
| `norm.type` | enum | Normalization strategy: `layer_norm`, `dynamic_tanh`, `derf`, `rms_norm`, `prms_norm`, `flash_norm`. |
| `norm.partial_ratio` | float (0,1] | Fraction of hidden dims for pRMSNorm estimation. Default: 0.0625. |
| *model.embedding.factorized.** | | |
| `embedding.factorized.enabled` | bool | Enable embedding matrix factorization to reduce parameters. |
| `embedding.factorized.dim` | int $\geq 1$ | Intermediate dimension for factorized embeddings. Typical: 64–256. |
| *model.embedding.conv.** | | |
| `embedding.conv.enabled` | bool | Apply Conv1d over token embeddings to capture local n-gram patterns. |
| `embedding.conv.kernel` | int $\geq 1$ | Kernel size for embedding Conv1d. Typical: 3. |
| *model.attention.ode.** | | |
| `attention.ode.solver` | enum (rk4, euler) | Numerical integration method for ODE layers. Default: rk4. |
| `attention.ode.steps` | int $\geq 1$ | Integration steps between consecutive transformer layers. Default: 2. |
| *model.attention.retnet.** | | |
| `attention.retnet.heads` | int $\geq 1$ | Retention heads for `retnet`/`retnet_attn` layers. Must divide `hidden_size`. Default: 8. |
| *model.attention.mla.** | | |
| `attention.mla.latent_rank` | int $\geq 1$ | Rank of joint KV latent for MLA. Default: `hidden_size // 2`. |
| *model.attention.gqla.** | | |
| `attention.gqla.latent_rank` | int $\geq 1$ | Rank of joint KV latent for GQLA. Default: `hidden_size // 2`. |
| `attention.gqla.num_groups` | int $\geq 1$ | Number of GQA groups for GQLA decode path. Must divide `num_heads`. |
| `attention.gqla.decode_path` | enum (mqa_absorb, gqa) | Decoding path for GQLA. |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| *model.attention.mlra.** | | |
| attention.mlra.latent_rank | int ≥ 1 | Total latent rank for MLRA. Default: hidden_size // 2. |
| attention.mlra.num_latent_heads | int ≥ 1 | Number of disjoint latent sub-spaces for MLRA. Default: 4. |
| *model.attention.tucker.** | | |
| attention.tucker.query_rank | int ≥ 1 | Tucker rank for query weight tensor. Default: hidden_size. |
| attention.tucker.key_rank | int ≥ 1 | Tucker rank for key weight tensor. Default: hidden_size // 2. |
| attention.tucker.value_rank | int ≥ 1 | Tucker rank for value weight tensor. Default: hidden_size // 2. |
| *model.attention.iha.** | | |
| attention.iha.num_pseudo_heads | int ≥ 1 | Number of pseudo-heads per head for IHA. Default: num_heads. |
| *model.attention.gta.** | | |
| attention.gta.num_shared_groups | int ≥ 1 | Number of head groups sharing an attention map for GTA. Must divide num_heads. |
| attention.gta.value_latent_rank | int ≥ 1 | Rank of value-cache latent for GTA. Default: hidden_size // 2. |
| *model.attention.mtla.** | | |
| attention.mtla.latent_rank | int ≥ 1 | Latent rank for MTLA. Default: hidden_size // 2. |
| attention.mtla.merge_factor | int ≥ 1 | Number of consecutive KV entries merged per temporal slot. Default: 2. |
| attention.mtla.stride | int ≥ 1 | Stride between merged temporal slots. Default: mtla_merge_factor. |
| *model.attention.cca.** | | |
| attention.cca.latent_rank | int ≥ 1 | Latent width for CCA. Default: hidden_size // 4. Must be divisible by num_heads. |
| attention.cca.num_conv_layers | enum (0, 1, 2) | Number of convolution layers in CCA. Default: 2. |
| attention.cca.conv_kernel_seq | int ≥ 1 | Kernel size of depth-wise causal sequence conv. Default: 4. |
| attention.cca.conv_kernel_ch | int ≥ 1 | Kernel size of head-wise grouped channel conv. Default: 3. |
| attention.cca.qk_mean | bool | Enable q-k-mean bias in CCA. Default: true. |
| attention.cca.value_shift | bool | Enable value-shift in CCA. Default: true. |
| *model.attention.ccgqa.** | | |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| attention.ccgqa.query_latent_rank | int ≥ 1 | Query latent width for CCGQA. Default: hidden_size // 2. |
| attention.ccgqa.kv_latent_rank | int ≥ 1 | KV latent width for CCGQA. Default: hidden_size // 8. |
| attention.ccgqa.num_kv_heads | int ≥ 1 | Number of KV heads for CCGQA. Must divide num_heads. |
| attention.ccgqa.num_conv_layers | enum (0, 1, 2) | Number of convolution layers in CCGQA. Default: 2. |
| attention.ccgqa.conv_kernel_seq | int ≥ 1 | Kernel size of depth-wise causal sequence conv. Default: 4. |
| attention.ccgqa.conv_kernel_ch | int ≥ 1 | Kernel size of head-wise grouped channel conv. Default: 3. |
| attention.ccgqa.qk_mean | bool | Enable q-k-mean bias in CCGQA. Default: true. |
| attention.ccgqa.value_shift | bool | Enable value-shift in CCGQA. Default: true. |
| *model.attention.msa.** | | |
| attention.msa.block_size | int ≥ 1 | Block size for MiniMax Sparse Attention. Default: 128. |
| attention.msa.topk_blocks | int ≥ 1 | Number of blocks selected per GQA group per query. Default: 16. |
| attention.msa.index_dim | int ≥ 1 | Dimensionality of Index Branch heads. Default: 64. |
| attention.msa.kl_loss_weight | float ≥ 0 | Weight of KL alignment loss for MSA index branch. Default: 0.0. |
| *model.attention.sparda.** | | |
| attention.sparda.block_size | int ≥ 1 | KV block size for SparDA. Default: 128. |
| attention.sparda.topk_blocks | int ≥ 1 | Number of KV blocks selected per GQA group. Default: 16. |
| attention.sparda.forecast_dim | int ≥ 1 | Dimensionality of Forecast projection. Default: 64. |
| *model.attention.engram.** | | |
| attention.engram.max_ngram_size | int ≥ 2 | Maximum N-gram order for Engram memory module. |
| attention.engram.n_heads_per_ngram | int ≥ 1 | Number of hash heads per N-gram order. |
| attention.engram.embed_dim_per_head | int ≥ 1 | Embedding dimension per hash head. |
| attention.engram.kernel_size | int ≥ 1 | Causal depthwise conv kernel width for Engram. |
| attention.engram.seed | int | Random seed for N-gram hash multipliers. |
| *model.mhc.** | | |
| mhc.enabled | bool | Enable the manifold-constrained n-stream residual (mHC, arXiv:2512.24880). Default: false. |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| `mhc.expansion_rate` | int $\geq 1$ | Residual-stream expansion factor $n$. The stream becomes $n\times$ `hidden_size`. 1 recovers the identity mapping. Default: 4. |
| `mhc.sinkhorn_iters` | int $\geq 1$ | Sinkhorn-Knopp normalisation rounds constraining $H^{\text{res}}$ to be doubly stochastic. Default: 20. |
| `mhc.gating_init` | float $> 0$ | Initial value of the mHC gating scalars $\alpha$. Default: 0.01. |
| `mhc.checkpoint` | bool | Gradient checkpointing on mHC layers to mitigate the $\sim n\times$ activation-memory increase. Default: `false`. |
| `mhc.full_prec_under_bitnet` | bool | Keep the mHC coefficient projection full-precision under BitNet. Default: `true`. |
| *model.moe.** | | |
| `moe.enabled` | bool | Enable Mixture of Experts in FFN layers. Default: `true`. |
| `moe.num_experts` | int $\geq 1$ | Total number of expert networks. Default: 8. |
| `moe.top_k` | int $\geq 1$ | Experts activated per token. Must be $\leq$ `num_experts`. Default: 2. |
| *model.bitnet.** | | |
| `bitnet.enabled` | bool | Enable BitLinear ternary weight quantization. Default: `true`. |
| `bitnet.routers` | bool | Also quantize routing/scoring projections. Requires `bitnet.enabled=true`. Default: `false`. |
| `bitnet.conv` | bool | Quantize embedding Conv1d with BitConv1d. Requires `bitnet.enabled=true` and `embedding.conv.enabled=true`. Default: `false`. |
| *model.mixture_of_depths.** | | |
| `mixture_of_depths.enabled` | bool | Enable Mixture-of-Depths token routing. Default: `false`. |
| `mixture_of_depths.capacity_ratio` | float (0,1] | Fraction of tokens updated per layer. Default: 0.5. |
| `mixture_of_depths.router_aux_loss_weight` | float $\geq 0$ | Weight for the router auxiliary loss. Default: 0.0. |
| *model.ssmax.** | | |
| `ssmax.enabled` | bool | Enable Scalable Softmax (arXiv:2505.22842). Default: `false`. |
| `ssmax.s_init` | float $> 0$ | Initial per-head SSMax scalar. Default: 1.0. |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| *model.ffn.** | | |
| ffn.hidden_size | int ≥ 1 | Intermediate FFN dimension. Default: 2× hidden_size. |
| ffn.activation | enum | FFN non-linearity (43 values). Default: silu. |
| ffn.activation_config | object | Parameters for learnable/parametric activations. |
| *model.classification_head.** | | |
| classification_head.enabled | bool | Enable the NLP sequence-classification head. Default: false. |
| classification_head.num_labels | int or null | Number of classes. |
| classification_head.pooling_mode | enum (cls, gap) | Pooling for the classification head. Default: cls. |

Table 1: Complete model configuration fields. All fields are optional unless marked required.

Fields are grouped under hierarchical sub-objects (dims, norm, embedding, attention, moe, bitnet, mixture_of_depths, ssmax, ffn, classification_head, mhc, residuals); each sub-header row marks the prefix in effect for the rows that follow it. There are no flat feature keys at model.*.

### 2.2.1 Layer Pattern Enumeration

The model.dims.layer_pattern array accepts the following 43 mixer codes. Each element defines one physical layer in the model stack. Table 2 lists all available types with their training and inference support.

| Type | Description | Train | Infer |
|---|---|---|---|
| standard_attn | Classic multi-head self-attention (BERT-style) | ✓ | ✓ |
| sigmoid_attn | Sigmoid-gated attention | ✓ | ✓ |
| gated_softmax_attn | Gated softmax attention with post-SDPA sigmoid | ✓ | ✓ |
| titan_attn | Titan attention with positional encoding (HoPE/RoPE) | ✓ | ✓ |
| retnet / retnet_attn | Retention Network (RNN+transformer hybrid) | ✓ | ✓ |
| mamba | Selective state-space model (SSM) | ✓ | ✓ |
| ode | Continuous-depth ODE layer | ✓ | ✓ |
| gla_attn | Gated Linear Attention with multiplicative decay | ✓ | ✓ |
| deltanet_attn / gated_deltanet_attn | DeltaNet with corrective delta rule | ✓ | ✓ |

*continued on next page*

| Type | Description | Train | Infer |
|---|---|---|---|
| gated_deltanet2_attn | Gated DeltaNet-2 with decoupled channel-wise erase/write gates | ✓ | ✓ |
| hgrn2_attn | HGRN2 with hierarchical forget gates | ✓ | ✓ |
| fox_attn | Forgetting Transformer (FoX) with forget bias in logits | ✓ | ✓ |
| nsa_attn | Native Sparse Attention (three-branch) | ✓ | ✓ |
| engram_attn | N-gram conditional memory lookup | ✓ | ✓ |
| gqa_attn | Grouped-Query Attention with configurable KV heads | ✓ | ✓ |
| longformer_attn | Longformer: sliding windows + global tokens | ✓ | ✓ |
| bigbird_attn | BigBird: local + random + global attention | ✓ | ✓ |
| sparse_transformer_attn | Sparse Transformer: factorized patterns | ✓ | ✓ |
| sparsek_attn | SparseK: differentiable top-k KV selection | ✓ | ✓ |
| sparge_attn | SpargeAttn — two-stage block pruning; dense training, sparse inference | ✓ | ✓ |
| fasa_attn | FASA — frequency-aware token selection; dense training, sparse inference | ✓ | ✓ |
| msa_attn | MiniMax Sparse Attention — block-sparse on GQA | ✓ | ✓ |
| sparda_attn | SparDA — decoupled sparse attention with Forecast projection | ✓ | ✓ |
| kda_attn | Kimi Delta Attention — channel-wise decay + scalar write gate | ✓ | ✓ |
| falcon1_attn / falcon2_attn / falcon3_attn | Falcon regression family — RAW prefix-aligned NLMS delta rule | ✓ | ✓ |
| falcon1a_attn / falcon2a_attn / falcon3a_attn | Falcon inner-product family — shifted additive writes with ridge decay | ✓ | ✓ |
| mla_attn | Multi-Head Latent Attention + RoPE | ✓ | ✓ |
| gqla_attn | Group-Query Latent Attention — two decoding paths | ✓ | ✓ |
| mlra_attn | Multi-Head Low-Rank Attention — partitionable latent | ✓ | ✓ |

*continued on next page*

| Type | Description | Train | Infer |
|---|---|---|---|
| tucker_attn | Tucker Attention — generalised low-rank factorisation | ✓ | ✓ |
| iha_attn | Interleaved Head Attention — cross-head pseudo-heads | ✓ | ✓ |
| gta_attn | Grouped-head laTenT Attention — shared map + latent values | ✓ | ✓ |
| mtla_attn | Multi-head Temporal Latent Attention — temporal KV merge | ✓ | ✓ |
| cca_attn | Compressed Convolutional Attention — latent-space attention + convs | ✓ | ✓ |
| ccgqa_attn | Compressed Convolutional Grouped Query Attention — CCA + GQA in latent | ✓ | ✓ |
| gma_attn | Gaussian Mixture Attention — probabilistic latent routing, $\mathcal{O}(NK)$ linear-time | ✓ | ✓ |
| ssog_attn | Separable Sum of Gaussians — Gaussian field over relative position, near-linear $\mathcal{O}(RN\sqrt{N})$ | ✓ | ✓ |

Table 2: Complete `layer_pattern` enumeration. ✓ = supported, × = not supported. `fasa_attn` and `sparge_attn` train as standard dense attention over their own projections and apply their sparse heuristic only at inference.

### 2.3 Training Configuration

The `training` object controls the entire training pipeline. The only required field is `task`. Table 3 lists all training fields.

| Field | Type | Description |
|---|---|---|
| task | enum (mlm, sbert, causal_lm) (required) | Training objective. `mlm` = Masked Language Modeling; `sbert` = Sentence-BERT fine-tuning; `causal_lm` = autoregressive next-token prediction (requires `model_class='frankenstein_text_decoder'`). |
| num_epochs | int $\geq 1$ | Complete passes through the training dataset (MLM only). |
| batch_size | int $\geq 1$ | Examples per optimizer step (before gradient accumulation). |
| dataloader_workers | int $\geq 0$ | Parallel worker processes for data loading. |
| max_length | int $\geq 1$ | Maximum token sequence length. Typical: 128–2048. |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| mlm_probability | float [0,1] | Fraction of tokens masked for MLM prediction. Standard: 0.15. |
| max_samples | int $\geq 1$ | Total samples before stopping, independent of epochs. |
| dataset_batch_size | int $\geq 1$ | Internal batch size for streaming dataset. |
| num_workers | int $\geq 0$ | Parallel workers for dataset streaming pipeline. |
| cache_dir | string | Directory for caching processed datasets. |
| local_parquet_dir | string | Path to local parquet files for offline streaming. |
| prefer_local_cache | bool | Check local cache first before remote download. |
| stream_local_parquet | bool | Stream from local parquet when `local_parquet_dir` is set. |
| join_temp_data_context_window | int $\geq 0$ | If $> 0$, joins cached token chunks into sequences of this length. |
| join_temp_data_min_remainder_tokens | int $\geq 0$ | Minimum content tokens in last partial joined window. |
| use_amp | bool | Enable FP16 mixed precision. |
| gradient_accumulation_steps | int $\geq 1$ | Accumulate gradients over N mini-batches before optimizer step. |
| optimizer | object | Optimizer configuration (see §2.5). Required when `task=mlm` or `task=causal_lm`. |
| sbert | object | SBERT training configuration (see §2.6). Required when `task=sbert`. |
| scheduler_total_steps | int $\geq 1$ | Total optimizer steps for LR schedule. |
| scheduler_warmup_ratio | float [0,1] | Fraction of `scheduler_total_steps` for LR warmup. |
| scheduler_type | enum (cosine, constant, linear_warmup_then_constant) | LR decay strategy. |
| grad_clip_max_norm | float $\geq 0$ | Maximum allowed gradient norm. 0 = disabled. |
| inf_post_clip_threshold | float $\geq 0$ | Threshold for flagging post-clip gradient explosions. |
| max_nan_retries | int $\geq 0$ | Maximum retries on NaN/Inf gradients before stopping. |
| checkpoint_every_n_steps | int $\geq 1$ | Save rolling checkpoint every N steps. |
| max_rolling_checkpoints | int $\geq 1$ | Maximum rolling checkpoints kept. Oldest deleted when exceeded. |
| num_best_checkpoints | int $\geq 1$ | Number of best checkpoints kept permanently (by validation loss). |
| nan_check_interval | int $\geq 1$ | Steps between NaN/Inf checks. |
| log_gradient_stats | bool | Log per-layer gradient norms, min/max. |
| gradient_log_interval | int $\geq 1$ | Steps between gradient stat logs. |

*continued on next page*

| Field | Type | Description |
|---|---|---|
| csv_log_path | string | File path for step-level training metrics in CSV format. |
| csv_rotate_on_schema_change | bool | Rotate CSV file when schema changes. |
| gpu_metrics_backend | enum (nvml, none) | GPU telemetry backend. |
| nvml_device_index | int $\geq 0$ | GPU index to monitor (0 = first GPU). |
| enable_block_grad_norms | bool | Log gradient norms per block (embeddings, attention, FFN, etc.). |
| telemetry_log_interval | int $\geq 1$ | Steps between heavy telemetry logs. |
| gpu_temp_guard_enabled | bool | Strict GPU temperature monitoring. Requires gpu_metrics_backend=nvml. |
| switch_on_thermal | bool | Critical temp triggers GPU→CPU fallback, auto-resume on cooldown. |
| gpu_temp_pause_threshold_c | float | Temperature (°C) to pause training. |
| gpu_temp_resume_threshold_c | float | Temperature (°C) to resume training after pause. |
| gpu_temp_critical_threshold_c | float | Informational logging marker for severe thermal events. |
| gpu_temp_poll_interval_seconds | float | Seconds between temp checks during cooldown wait. |
| use_galore | bool | Enable Gradient Low-Rank Projection for memory-efficient training. |
| galore_rank | int $\geq 1$ | Low-rank projection dimension for GaLore. |
| galore_update_interval | int $\geq 1$ | Steps between projection matrix recomputation. |
| galore_scale | float $\geq 0$ | Scaling factor for projected gradients. |
| galore_max_dim | int $\geq 1$ | Maximum tensor dimension for GaLore projection. |
| hf_output_dir | string | Directory for saving final model via save_pretrained(). |

Table 3: Complete training configuration fields.

### 2.4 Tokenizer Configuration

The top-level `tokenizer` object is available to both training paths: the `base_model` route (continual pretraining / fine-tuning, where the tokenizer must match the pretrained checkpoint) and the custom from-scratch route (where it resolves the vocabulary before the embedding matrix is allocated). The `source` field selects between two modes:

- `hf_repo` (default) — loads a pretrained tokenizer from `name_or_path` (a HuggingFace repo id or local path) through `AutoTokenizer.from_pretrained()`. Complementary flags: `use_fast` (Rust-backed fast tokenizer, default `true`) and `trust_remote_code` (default `false`).
- `train_from_dataset` — trains a *new* tokenizer from the corpus text with the HuggingFace `tokenizers` library. The corpus is the top-level `text_dataset` block (HF hub id, optionally streamed, or a local parquet/json directory); when the config is driven by an embedding host such as dashAI, the host's run dataset can supply it instead. The resulting vocabulary size is injected into `model.dims.vocab_size` before model construction, so the embedding matrix always matches the tokenizer. In this mode `name_or_path` must be omitted.

Table 4 lists the `tokenizer.training` sub-object fields.

| Field | Type | Description |
|---|---|---|
| `algorithm` | enum (bpe, wordpiece, wordlevel, unigram) | Tokenizer training algorithm. `bpe` (Byte Pair Encoding, GPT-2 style, default); `wordpiece` (original BERT); `wordlevel` (word lookup table); `unigram` (subword language model, auto-pruning). |
| `vocab_size` | int $\geq 1$ | Target vocabulary size. Required for `bpe`/`wordpiece`/`wordlevel`; optional for `unigram`. |
| `min_frequency` | int $\geq 1$ | Minimum token frequency to keep (default 2). |
| `special_tokens` | array of string | Special tokens registered in the tokenizer (default `[PAD]`/`[UNK]`/`[CLS]`/`[SEP]`/`[MASK]`). |
| `save_dir` | string | Directory persisted via `save_pretrained()`. |
| `lowercase` | bool | Lowercase all text before training (default `false`). |
| `strip_accents` | bool | Strip diacritics before training (default `false`). |
| `pre_tokenizer` | enum (whitespace, bytelevel, bert) | Pre-tokenization strategy (default `whitespace`). |
| `max_token_length` | int $\geq 0$ | Per-token character cap; 0 disables the limit. |

Table 4: `tokenizer.training.*` fields (required when `source=train_from_dataset`).

## 2.5 Optimizer Configuration

The `training.optimizer` object has two fields:

- `optimizer_class` (enum, required) — One of 23 optimizer algorithms: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `anon`, `apollo`, `apollo_mini`, `q_apollo`.
- `parameters` (object) — Optimizer hyperparameters with prefixed keys in the format `<optimizer>-<group>_<param>`.

### 2.5.1 Shared Per-Group Suffixes

All optimizers support the following per-parameter-group suffixes, prefixed by the optimizer class name (e.g., `adamw-lr_embeddings`):

- **Learning rate groups:** `lr_embeddings`, `lr_norms`, `lr_attention`, `lr_other`
- **Weight decay groups:** `wd_embeddings`, `wd_norms`, `wd_attention`, `wd_other`
- **Beta groups:** `betas_embeddings`, `betas_norms`, `betas_attention`, `betas_other`
- **Epsilon groups:** `eps_embeddings`, `eps_norms`, `eps_attention`, `eps_other`

Note: parameters of ODE, RetNet, and Mamba mixers are routed into the `attention` group rather than having dedicated groups.

### 2.5.2 Optimizer-Specific Global Suffixes

Table 5 lists the additional global suffixes supported by each optimizer (prefixed by the optimizer name).

| Optimizer | Specific Global Suffixes |
|---|---|
| sgd_momentum | momentum, nesterov |
| adamw | (none — shared suffixes only) |
| adafactor | beta2_decay, clip_threshold, eps1, eps2 |
| galore_adamw | rank, update_proj_gap |
| prodigy | d_coef |
| lion | (none — shared suffixes only) |
| sophia | rho, update_k |
| muon | momentum, nesterov, ns_steps, ns_eps |
| turbo_muon | momentum, nesterov, ns_steps, ns_eps |
| radam | (none — shared suffixes only) |
| adan | (none — shared suffixes only) |
| adopt | (none — shared suffixes only) |
| ademamix | (none — shared suffixes only) |
| mars_adamw | (none — shared suffixes only) |
| cautious_adamw | cautious_clip |
| lamb | (none — shared suffixes only) |
| schedulefree_adamw | (none — shared suffixes only) |
| shampoo | (none — shared suffixes only) |
| soap | (none — shared suffixes only) |
| anon | gamma |
| apollo | rank, update_proj_gap, scale, scale_type, proj_type, scale_front, disable_nl |
| apollo_mini | update_proj_gap, scale, proj_type, scale_front, disable_nl |
| q_apollo | rank, update_proj_gap, scale, scale_type, proj_type, scale_front, disable_nl, quant_bits |

Table 5: Optimizer-specific global parameter suffixes. All keys are prefixed with the optimizer class name (e.g., `sgd_momentum-momentum`).

## 2.6 SBERT Configuration

The `training.sbert` object configures Sentence-BERT fine-tuning. It is required when `training.task=sbert`. Key fields include:

- `dataset_name` (string) — HuggingFace dataset ID or local path.
- `dataset_type` (enum) — `paired_similarity`, `triplets`, or `qa`.
- `columns` (object) — Optional column name overrides for dataset fields.
- `query_prefix` / `document_prefix` (string) — Text prepended to query/document fields during tokenization.
- `output_dir` (string) — Directory for SBERT artifacts.
- `batch_size` (int) — Examples per batch. Typical: 16–64.
- `gradient_accumulation_steps` (int) — Accumulation steps for SBERT.

- `grad_clip_max_norm` (float) — Gradient clipping threshold. Typical: 0.5–2.0.
- `num_epochs` (int) — Complete passes through SBERT dataset. Typical: 1–10.
- `warmup_steps` (int) — Steps for LR warmup.
- `evaluation_steps` (int) — Evaluate every N steps.
- `checkpoint_save_steps` (int) — Save rolling checkpoint every N steps.
- `resume_from_checkpoint` (string or null) — `auto` resumes from the latest checkpoint; `null` starts fresh.
- `learning_rate` (float) — Initial LR. Typical: $1 \times 10^{-5}$ to $1 \times 10^{-4}$.
- `max_train_samples` / `max_eval_samples` (int) — Limit training/evaluation samples.
- `max_length` (int) — Max token length for input texts. Typical: 128–512.
- `pooling_mode` (enum) — `mean`, `cls`, or `max`.
- `trust_remote_code` (bool) — Allow custom code from model repo.
- `use_amp` (bool) — Enable FP16 mixed precision.
- `resample_balanced` (bool) — Resample to balanced distribution over similarity scores.
- `standardize_scores` (bool) — Z-score normalize similarity scores.
- `resample_std` (float) — Target std dev when `standardize_scores=true`.
- `optimizer` (object) — Custom optimizer for SBERT (same classes as MLM training); constructor kwargs go under `optimizer.optimizer_kwargs`.
- `wandb_project` (string) — Weights & Biases project name for telemetry.

### 2.7 Validation Rules

The schema enforces the following cross-component validation rules, implemented both in the JSON Schema (`_conditional_rules.yaml`) and at runtime in the configuration loader:

1. `additionalProperties: false` is enforced at all nesting levels. Any unrecognized key in any object triggers a validation error.
2. `model.dims.hidden_size` must be divisible by `model.dims.num_heads`. The per-head dimension is `hidden_size/num_heads`.
3. `model.dims.num_kv_heads` must divide `model.dims.num_heads` exactly when Grouped-Query Attention is used.
4. `vocab_size` must match the tokenizer's vocabulary size. Mismatch causes token ID corruption and unrecoverable training failure.
5. When `base_model` is set, `training.task` is required. If `task=mlm`, `tokenizer.name_or_path` is also required — unless `tokenizer.source=train_from_dataset`, in which case the tokenizer is trained from the corpus and no name is needed.
6. `tokenizer.source=train_from_dataset` requires the `tokenizer.training` sub-object (with `vocab_size` for `bpe`/`wordpiece`/`wordlevel`) and is mutually exclusive with `tokenizer.name_or_path`. The trained vocabulary overrides `model.dims.vocab_size` before model construction.
7. `task: mlm` requires `training.optimizer`; `task: sbert` requires `training.sbert`; `task: causal_lm` requires `training.optimizer` and `model_class='frankenstein_text_decoder'`.
8. `bitnet.routers=true` requires `bitnet.enabled=true`. The flag has no effect otherwise.
9. `mhc.enabled=true` (mHC n-stream residual) and `mixture_of_depths.enabled=true` are mutually exclusive. Enabling both raises a `ValueError`: MoD token routing operates on a single $C$-dimensional stream, which conflicts with the n-stream residual.
10. `fasa_attn` and `sparge_attn` are inference-sparse layer types: they apply their sparse heuristic only in evaluation mode. During training the enclosing hybrid layer substitutes standard dense attention over the same **q**/**k**/**v**/**o** projections, so they can appear in `model.dims.layer_pattern` for training runs; the trained weights are the ones the sparse path reads at inference.

11. `model_class: frankenstein_text_decoder` forces `model.dims.mode: decoder` at runtime. Setting `model.dims.mode: encoder` with this class is overridden.
12. Either `base_model` must be present, or both `model_class` and `model` must be present. The schema rejects configurations that provide neither or an incomplete combination.

# 3 Comprehensive Summary Tables

This appendix provides a complete reference of all components implemented in the Frankenstein Transformer codebase: 43 sequence mixers across eight families, 23 optimizers, 6 normalization variants, 43 activation functions, and 4 embedding methods. Each table is self-contained and cross-references the bibliography.

## 3.1 Sequence Mixer Summary

Table 6 enumerates every sequence mixer supported by the `layer_pattern` schema field, organized by architectural family. Training and inference complexity are expressed in terms of sequence length $n$, hidden dimension $d$, and family-specific parameters.

| Name | Family | Training | Inference | Key Characteristic | Reference |
|---|---|---|---|---|---|
| | | | **Dense (3)** | | |
| standard_attn | Dense | $\mathcal{O}(n^2 d)$ | $\mathcal{O}(n)$ cache | Full global context; KV bottleneck at long sequences | [97] |
| sigmoid_attn | Dense | $\mathcal{O}(n^2 d)$ | $\mathcal{O}(n)$ cache | Element-wise gating; hardware-efficient replacement for softmax | [82] |
| gqa | Dense | $\mathcal{O}(n^2 d)$ | $\mathcal{O}(n)$ cache | Tunable KV compression; $G\times$ cache reduction via shared heads | [2] |
| | | | **Recurrent (5)** | | |
| retnet_attn | Recurrent | $\mathcal{O}(n^2 d)$ parallel | $\mathcal{O}(1)$ | Multi-scale decay; triple computation mode (parallel, recurrent, chunkwise) | [93] |
| mamba | Recurrent | $\mathcal{O}(nd)$ | $\mathcal{O}(1)$ | Selective state-space model; hardware-aware parallel scan | [35] |
| ode | Recurrent | $\mathcal{O}(k \cdot n^2 d)$ | $\mathcal{O}(kn)$ | Continuous-depth RK4 integration; adaptive step solver | [114] |
| titan_attn | Recurrent | $\mathcal{O}(c^2 + nf)$ | $\mathcal{O}(c^2 + 1)$ | Test-time memory adaptation; surprise-driven write dynamics | [7] |
| engram_attn | Memory | $\mathcal{O}(n \cdot N \cdot H)$ | $\mathcal{O}(1)$ lookup | Conditional N-gram memory; hash-based scalable retrieval | [15] |
| | | | **Sparse (10)** | | |
| sparse_transformer_attn | Sparse | $\mathcal{O}(n\sqrt{n} \cdot d)$ | $\mathcal{O}(n)$ | Factorized strided + fixed masks; $\sqrt{n}$ sparsity pattern | [16] |

*continued on next page*

| Name | Family | Training | Inference | Key Characteristic | Reference |
|---|---|---|---|---|---|
| longformer_attn | Sparse | $\mathcal{O}(n \cdot w \cdot d)$ | $\mathcal{O}(n)$ | Sliding window + dilation + global tokens; linear in $n$ | [8] |
| swa_attn | Sparse | $\mathcal{O}(n \cdot w \cdot d)$ | $\mathcal{O}(w)$ | Sliding window + attention sinks; training-free, fixed KV cache | [47] |
| bigbird_attn | Sparse | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | Random + local + global; Turing complete attention | [111] |
| sparsek_attn | Sparse | $\mathcal{O}(n \cdot d)$ | $\mathcal{O}(k)$ | Differentiable top-$k$ selection; learned sparsity mask | [64] |
| nsa_attn | Sparse | $\mathcal{O}((t/d + n_l + w)d)$ | $\mathcal{O}(n)$ | Hardware-aligned 3-branch design; token/neighbor/window | [109] |
| sparge_attn | Sparse | $\mathcal{O}(n^2 \cdot s \cdot d)$ | $\mathcal{O}(n)$ | Two-stage block filtering; dense training, sparse inference | [115] |
| fasa_attn | Sparse | $\mathcal{O}(t \cdot N_{\text{tip}} + N_{\text{fac}} \cdot d)$ | $\mathcal{O}(n)$ | RoPE frequency-aware sparse; dense training, sparse inference | [100] |
| msa_attn | Sparse | $\mathcal{O}(H_{\text{kv}} \cdot d_{\text{idx}} \cdot N^2 + H_q \cdot d_h \cdot N \cdot k \cdot B_k)$ | $\mathcal{O}(n)$ | Index-branch block sparse; deployed at 109B scale | [54] |
| sparda_attn | Sparse | $\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_b + H_q \cdot d_h \cdot N \cdot k \cdot B_k)$ | $\mathcal{O}(n)$ | Forecast-guided lookahead prefetch; predictive sparsity | [30] |
| **Gated (8)** | | | | | |
| gla_attn | Gated | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Data-dependent diagonal decay; linear attention with gating | [105] |
| deltanet_attn | Gated | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Delta rule; error-correcting linear attention | [106] |
| gated_deltanet_attn | Gated | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Decay + write gating synthesis; unified delta framework | [107] |
| gated_deltanet2_attn | Gated | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Decoupled channel-wise erase/write; improved gating | [38] |
| hgrn2_attn | Gated | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Hierarchical bounded forget gates; multi-scale recurrence | [79] |
| fox_attn | Gated | $\mathcal{O}(L^2d)$ | $\mathcal{O}(L)$ KV | Logit-space recency bias; FlashAttn compatible | [58] |
| gated_softmax_attn | Gated | $\mathcal{O}(L^2d)$ | $\mathcal{O}(L)$ KV | Post-SDPA sigmoid gating; attention-sink mitigation | [80] |
| kda_attn | Gated | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Channel-wise decay + scalar write; Kimi Linear design | [51] |
| **Fast-Weight (6)** | | | | | |
| falcon1_attn | Fast-weight | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Prefix-aligned NLMS delta rule; best LM perplexity of the family | [116] |
| falcon2_attn | Fast-weight | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Per-value-channel NLMS step sizes | [116] |

*continued on next page*

| Name | Family | Training | Inference | Key Characteristic | Reference |
|---|---|---|---|---|---|
| falcon3_attn | Fast-weight | $\mathcal{O}(Ld^2B)$ | $\mathcal{O}(d^2) + \mathcal{O}(B)$ tail | Sliding-window minibatch rehearsal with exact window statistics | [116] |
| falcon1a_attn | Fast-weight | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Shifted additive linear attention with ridge decay | [116] |
| falcon2a_attn | Fast-weight | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2)$ | Per-channel additive decays | [116] |
| falcon3a_attn | Fast-weight | $\mathcal{O}(Ld^2)$ | $\mathcal{O}(d^2) + \mathcal{O}(B)$ tail | Windowed additive; best length extrapolation | [116] |
| | | | **Latent (9)** | | |
| mla_attn | Latent | $\mathcal{O}(n^2d)$ | $\mathcal{O}(r_{\text{kv}} \cdot n)$ | KV latent of rank $r_{\text{kv}} \approx d/2$; DeepSeek-style | [66] |
| gqla_attn | Latent | $\mathcal{O}(n^2d)$ | $\mathcal{O}(r_{\text{kv}} \cdot n)$ | Hardware-adaptive dual decode paths; GQA + latent fusion | [67] |
| mlra_attn | Latent | $\mathcal{O}(n^2d)$ | $\mathcal{O}(r_{\text{kv}} \cdot n)$ | $L$ sub-heads; 4-way tensor-parallel compatible | [61] |
| tucker_attn | Latent | $\mathcal{O}(n^2d)$ | $\mathcal{O}(k_{\text{rank}} \cdot n)$ | Tucker factorization; unifies MHA/GQA/MLA | [53] |
| iha_attn | Latent | $\mathcal{O}(n^2H^2)$ | $\mathcal{O}(H \cdot n \cdot d_h)$ | Pseudo-heads; $\Theta(\sqrt{k})$ parameter scaling | [25] |
| gta_attn | Latent | $\mathcal{O}(n^2 \cdot G \cdot d_h)$ | $\mathcal{O}(r_v \cdot n)$ | Shared map + SiLU decoder; 62.5% FLOP reduction | [92] |
| mtla_attn | Latent | $\mathcal{O}(n^2d/m)$ | $\mathcal{O}(r_{\text{kv}} \cdot n/m)$ | Temporal merge; 5.3× decode speedup | [20] |
| cca_attn | Latent | $\mathcal{O}(n^2d/C)$ | $\mathcal{O}(\tilde{e} \cdot n)$ | Full attention in compressed latent; $C\times$ fewer FLOPs | [29] |
| ccgqa_attn | Latent | $\mathcal{O}(n^2d/C_1)$ | $\mathcal{O}(\tilde{e}_{\text{kv}} \cdot n)$ | CCA + decoupled GQA; best loss at 8× cache reduction | [29] |
| gma_attn | Latent | $\mathcal{O}(nKd_r+nKd_v)$ | $\mathcal{O}(K)$/step | Gaussian Mixture Attention; probabilistic latent routing through $K$ learned GMM components per head; linear-time for fixed $K$; causal via prefix cumsums | [46] |
| ssog_attn | Geometric field | $\mathcal{O}(R \cdot n\sqrt{n} \cdot d)$ | $\mathcal{O}(R \cdot n\sqrt{n} \cdot d)$/fwd | SSOG: separable Gaussian field over relative position; content steers (bounded $\mu/\sigma/\lambda$ residuals behind cold-started gates), never scores; encoder-only | [76] |

Table 6: Complete sequence mixer catalog. $n$ = sequence length, $d$ = hidden size, $w$ = window size, $r$ = low-rank dimension, $k$ = sparsity parameter, $C$ = compression ratio. Family-specific parameters are defined in the corresponding annex.

## 3.2 Optimizer Summary

Table 7 catalogs all 23 supported optimizers with their memory footprint, per-step computational cost, and key hyperparameters. State buffers are expressed as the number of optimizer-owned tensors per parameter; per-step cost focuses on the dominant term beyond gradient computation.

| Optimizer | Family | State Buffers | Per-Step Cost | Key Hyperparams | Reference |
|---|---|---|---|---|---|
| **Classical & Adaptive Baselines** | | | | | |
| SGD+ Momentum | Classical | 1 ($m$) | $\mathcal{O}(n)$ | lr, momentum, wd | [77] |
| AdamW | Adaptive | 2 ($m$, $v$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd | [63] |
| RAdam | Adaptive | 2 ($m$, $v$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd | [60] |
| **Variance Reduction & Advanced Momentum** | | | | | |
| Adan | Variance reduction | 3 ($m$, $v$, $s$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, $\beta_3$, eps, wd | [103] |
| ADOPT | Adam variant | 2 ($m$, $v$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd | [95] |
| AdEMAMix | Multi-EMA | 3 ($m_1$, $m_2$, $v$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, $\beta_3$, eps, wd | [73] |
| MARS | Variance-reduced | 3 ($m$, $v$, $z$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd, $\gamma$ | [110] |
| Cautious AdamW | Masked momentum | 2 ($m$, $v$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd | [57] |
| **Large-Batch, Memory-Efficient & Parameter-Free** | | | | | |
| LAMB | Layer-wise adaptive | 2 ($m$, $v$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd | [108] |
| Schedule-Free | Scheduler-free | 3 ($z$, $x$, $n$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, wd | [19] |
| Adafactor | Memory-efficient | 1–2 (row/col) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd | [87] |
| GaLore | Low-rank projection | 2 ($m$, $v$) + SVD | $\mathcal{O}(nr)$ | lr, rank $r$, $\beta_1$, $\beta_2$, eps, wd | [117] |
| Prodigy | Parameter-free | 3 ($m$, $v$, $d$) | $\mathcal{O}(n)$ | $\beta_1$, $\beta_2$, eps, wd, $d_0$ | [68] |
| Lion | Sign momentum | 1 ($m$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, wd | [14] |
| **Second-Order, Geometric & Orthogonality** | | | | | |
| Shampoo | Second-order | $2d$ ($L_i$, $R_i$) | $\mathcal{O}(n^{1+2/d})$ | lr, eps, wd | [37] |
| SOAP | Second-order | $2d+2$ ($m$, $v$) | $\mathcal{O}(n^{1+1/d})$ | lr, $\beta_1$, $\beta_2$, eps, wd | [98] |
| Sophia | Curvature-aware | 3 ($m$, $v$, $h$) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd, $k$ | [59] |
| Muon | Orthogonality | 2 ($m$, $v$) | $\mathcal{O}(n \cdot k)$ | lr, momentum, wd, NS steps $k$ | [88] |
| Turbo-Muon | Orthogonality | 2 ($m$, $v$) | $\mathcal{O}(n \cdot k)$ | lr, momentum, wd, NS steps $k$ | [10] |
| **Low-Rank Adaptive & Tunable Adaptivity** | | | | | |
| APOLLO | Low-rank adaptive | 2 low-rank + proj | $\mathcal{O}(nr)$ | lr, rank $r$, update gap, scale, betas, eps, wd | [118] |
| APOLLO-Mini | Low-rank adaptive | 2 rank-1 + proj | $\mathcal{O}(n)$ | lr, update gap, scale, betas, eps, wd | [118] |
| Q-APOLLO | Quantized low-rank | 2 quantized + proj | $\mathcal{O}(nr)$ | lr, rank $r$, update gap, scale, quant bits, betas, eps, wd | [118] |
| Anon | Adaptivity-tunable | 3 ($m$, $v$, fixed_lr) | $\mathcal{O}(n)$ | lr, $\beta_1$, $\beta_2$, eps, wd, $\gamma$ | [3] |

Table 7: Complete optimizer catalog. $n$ = number of parameters, $d$ = tensor dimensionality, $r$ = low-rank dimension, $k$ = Newton–Schulz iteration count. State buffers lists the number of optimizer state tensors maintained per parameter group.

### 3.3 Normalization Variants

Table 8 summarizes the six normalization methods available through the `norm_type` schema field, ranging from classical statistics-based approaches to normalization-free bounded transforms and weightless RMSNorm with fused norm-then-project execution.

| Method | Formula | Stats Needed | Reference |
|---|---|---|---|
| LayerNorm | $\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$ | mean + variance | [4] |
| RMSNorm | $\gamma_i x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$ | RMS only | [113] |
| pRMSNorm | $\gamma_i x_i / \sqrt{\frac{1}{k}\sum_{j \le k} x_j^2 + \epsilon}$ | Partial RMS ($p\,\%$) | [113] |
| Dynamic Tanh (DyT) | $\tanh(\alpha x)$ | none | [119] |
| Dynamic Erf (Derf) | $\mathrm{erf}(\alpha x + s)$ | none | [13] |
| FlashNorm | $x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$ | RMS only (weightless) | [34] |

Table 8: Normalization variants. $d$ = hidden dimension, $k = \lceil d \cdot p \rceil$ for partial ratio $p$, $\alpha$ and $s$ are learnable parameters. FlashNorm is weightless (no learnable parameters); the per-dim scale is folded into the subsequent linear layer per Prop. 1 of [34].

### 3.4 Residual Connections

Table 9 compares the residual-connection schemes available in the codebase: the standard identity residual, the four Attention Residuals (AttnRes) strategies [94], and the manifold-constrained n-stream residual (mHC) [104]. $L$ is the logical depth, $C$ the hidden dimension, $n$ the stream expansion factor, $t_{\max}$ the Sinkhorn–Knopp iteration count, $\alpha$ the mHC gating scalars, and $\mathbf{w}_l$ the per-layer AttnRes pseudo-query (initialized to zero). HC (unconstrained Hyper-Connections) is listed for reference but is *not* exposed in this codebase.

| Scheme | Stream Update | Constraint | Reference |
|---|---|---|---|
| Standard residual | $x_{l+1} = x_l + F(x_l, W_l)$ | Identity mapping (by construction) | [40] |
| No residual (experimental) | $x_{l+1} = F(x_l, W_l)$ | No skip path | — |
| Full AttnRes | $x_{l+1} = \sum_{i=0}^{l} \mathrm{softmax}_i(\mathbf{w}_l^\top \mathrm{RMSNorm}(v_i))\, v_i$ | $\mathbf{w}_l$ learned, zero-init; RMSNorm on keys; $L \cdot C$ parameters | [94] |
| Block AttnRes | $x_{l+1} = \sum_{m=0}^{n} \mathrm{softmax}_m(\mathbf{w}_l^\top \mathrm{RMSNorm}(v_m))\, v_m$ | $\mathbf{w}_l$ learned, zero-init; RMSNorm on keys; $N$ block reps + partial sum; $L \cdot C$ parameters | [94] |
| Hyper-Connections (HC) | $x_{l+1} = H_l^{\mathrm{res}} x_l + (H_l^{\mathrm{post}})^\top F(H_l^{\mathrm{pre}} x_l, W_l)$ | Unconstrained | [104] |
| mHC (this codebase) | same update as HC | $H_l^{\mathrm{res}}$ doubly stochastic via Sinkhorn–Knopp ($t_{\max} = 20$), $H^{\mathrm{pre}} = \sigma(\cdot)$, $H^{\mathrm{post}} = 2\sigma(\cdot)$, $\alpha$-init 0.01 | [104] |

*continued on next page*

| Scheme | Stream Update | Constraint | Reference |
|---|---|---|---|
| Hyperloop (this codebase) | $y_t^{(l+1)} = H_l^{\text{res}} y_t^{(l)} + H_l^{\text{post}} \otimes (F(H_l^{\text{pre}} y_t^{(l)}) + e_l)$ | Loop-level HC on a middle-cycle looped middle block; per-loop $\{W_l, b_l, \alpha_l, e_l\}$; copy/avg expansion; $H^{\text{res}}$ diagonal (default), Sinkhorn or identity | [112] |

Table 9: Residual-connection schemes. $x_l \in \mathbb{R}^C$ (standard / AttnRes / no residual) or $\mathbb{R}^{n \times C}$ (mHC) is the stream at layer $l$, $F$ the layer function, $H^{\text{pre}}, H^{\text{post}}, H^{\text{res}}$ the learned mHC mappings. AttnRes strategies are controlled by `model.residuals.type`; mHC by `model.mhc`. Full details in the mHC and AttnRes appendices.

## 3.5 Activation Functions

Table 10 catalogs the 43 feed-forward activation functions available through the `ffn_activation` schema field (40 elementwise plus 3 gated-FFN variants), grouped by family. "Learn." marks activations with trainable parameters; the full formulas are in Annex 8.

| Name | Family | Formula | Learn. | Reference |
|---|---|---|---|---|
| **Classical / Sigmoid–Tanh (12)** | | | | |
| silu / swish | Classical | $x\,\sigma(\beta x)$ | no / $\beta$ | [81] |
| gelu | Classical | $x\,\Phi(x)$ | no | [42] |
| gelu_tanh | Classical | $\frac{x}{2}\left(1+\tanh\sqrt{\frac{2}{\pi}}(x+0.044715x^3)\right)$ | no | [42] |
| relu | Classical | $\max(0, x)$ | no | [71] |
| sigmoid | Classical | $1/(1+e^{-x})$ | no | [55] |
| tanh | Classical | $(e^x - e^{-x})/(e^x + e^{-x})$ | no | [55] |
| arctan | Classical | $\arctan(x)$ | no | [55] |
| softsign / elliott | Classical | $x/(1+\lvert x\rvert)$ | no | [55] |
| identity | Classical | $x$ | no | (internal) |
| softplus | Classical | $\log(1+e^x)$ | no | [31] |
| mish | Classical | $x\tanh(\log(1+e^x))$ | no | [69] |
| **Rectified family (12)** | | | | |
| leaky_relu | Rectified | $\max(0,x) + a\min(0,x)$, $a$=0.01 | no | [65] |
| relu6 | Rectified | $\min(\max(0,x), 6)$ | no | [44] |
| hardswish | Rectified | $x\,\text{ReLU6}(x+3)/6$ | no | [44] |
| prelu | Rectified | $\max(0,x) + \mathbf{p} \odot \min(0,x)$ | **yes** | [39] |
| abs_relu | Rectified | $\max(0,x) - \max(0,-x)$ | no | [24] |
| nl_relu | Rectified | $\beta\log(1+\max(0,x))$ | no | [24] |
| brelu | Rectified | $\min(\max(0,x), t)$ | no | [24] |
| vrelu | Rectified | $\lvert x\rvert$ | no | [24] |
| hexpo | Rectified | $a\max(0,x) - c\max(0,-x)$ | no | [24] |
| ptanh | Rectified | $\max(0,\tanh(x))$ | no | [24] |
| dis_relu | Rectified | $\max(0, x-\delta)$ | no | [24] |
| lisht | Rectified | $x\tanh(x)$ | no | [84] |
| **Exponential / ELU family (12)** | | | | |
| elu | Exponential | $x$ if $x \geq 0$ else $\alpha(e^x - 1)$ | no | [17] |
| selu | Exponential | $\lambda\,\text{ELU}_{\alpha'}(x)$ | no | [52] |
| celu | Exponential | $x$ if $x \geq 0$ else $\alpha(e^{x/\alpha} - 1)$ | no | [5] |
| pelu | Exponential | $\frac{\alpha}{\beta}x$ / $\alpha(e^{x/\beta} - 1)$ | **yes** | [24] |
| mpelu | Exponential | ELU + learnable $\alpha, \beta$ | **yes** | [24] |
| felu / eelu / pdelu / preu | Exponential | ELU variants w/ learnable params | **yes** | [24] |
| softexp | Exponential | exp/linear/log interpolation | **yes** | [24] |
| elish / hardelish | Exponential | Swish $\cup$ ELU gated branches | no | [6] |
| **Learnable / Adaptive (4)** | | | | |
| swish_trainable | Learnable | $x\,\sigma(\beta x)$, $\beta$ trainable | **yes** | [81] |
| maxout | Learnable | $\max_i(W_i x + b_i)$ | **yes** | [33] |
| raf | Learnable | $P(x)/Q(x)$ Padé(5, 4), version A | **yes** | [28] |

*continued on next page*

| Name | Family | Formula | Learn. | Reference |
|---|---|---|---|---|
| | | **Gated FFN variants (3)** | | |
| swiglu | Gated | $\mathrm{SiLU}(xW_g) \odot (xW_u)\, W_d$ | **yes** | [86] |
| geglu | Gated | $\mathrm{GELU}(xW_g) \odot (xW_u)\, W_d$ | **yes** | [86] |
| reglu | Gated | $\mathrm{ReLU}(xW_g) \odot (xW_u)\, W_d$ | **yes** | [86] |

Table 10: Activation function catalog. "Learn." indicates the presence of trainable parameters. RAF = Rational Activation Function (learnable Padé ratio). $\sigma$ is the logistic sigmoid.

### 3.6 Embedding Variants

Table 11 catalogs the positional encodings and structural embedding optimizations supported by the codebase. The 12 positional encodings are configured via the model-wide `positional_encoding` field (see Annex 12 for full formulations); the two structural optimizations are controlled by their own flags.

| Method | Description | Reference |
|---|---|---|
| | **Positional Encodings (12)** | |
| RoPE | Rotary Position Embedding; frequency-based rotation of Q/K, encoding relative position via inner-product geometry | [91] |
| HoPE | Hyperbolic Rotary Positional Encoding; Lorentz-inspired rotation using hyperbolic functions; monotonic attention decay with distance; RoPE is a special case | [18] |
| NoPE | No Positional Encoding; model learns position implicitly from causal mask; outperforms explicit PE on length generalization | [48] |
| ALiBi | Attention with Linear Biases; static distance-proportional penalty on attention scores; enables length extrapolation | [78] |
| BAM | Bayesian Attention Mechanism; Generalized-Gaussian relative-position bias with learnable per-head shape ($\beta$) and scale ($\alpha$); recovers ALiBi at $\beta = 1$; SSMax rescale via `use_ssmax` | [9] |
| PaPE | Parabolic Position Encoding; parabolic basis functions concatenated to Q/K; vision-centric, $n$-D, extrapolatable | [72] |
| PaPE-Efficient | Pure-PyTorch PaPE; no custom Triton kernels, same math | [72] |
| PaPE-RI | Rotation-invariant PaPE (PaPE-RI) | [72] |
| Sinusoidal Absolute | Fixed sinusoidal added to embeddings (original Transformer) | [97] |
| Sinusoidal Rotary | Sinusoidal frequencies as a rotation matrix on Q/K | [97] |
| Learned Absolute | Learnable parameter matrix added to embeddings | [97] |
| None | Null module; no positional information injected | — |
| | **Structural Embedding Optimizations (2)** | |
| Factorized Embeddings | Decomposes $V \times H$ embedding matrix into $V \times R$ and $R \times H$ factors; reduces parameters from $\mathcal{O}(VH)$ to $\mathcal{O}(VR + RH)$; controlled by `factorized_embedding_dim` | (internal) |
| Embedding Convolution | Depthwise Conv1d applied over the embedding stream before the first transformer block; provides lightweight local-context filtering; controlled by `embedding_conv_kernel` | (internal) |

Table 11: Embedding variants. $V$ = vocabulary size, $H$ = hidden size, $R$ = factorized intermediate dimension. The first 12 rows are positional encodings (Annex 12); the last two are structural embedding optimizations.

# 4 Optimizer Families

## 4.1 The Evolution of Optimization in Neural Networks

The optimizer survey frames transformer optimization as a response to three structural pressures: non-convex loss landscapes, severe curvature heterogeneity across parameter blocks, and the memory cost of storing optimizer state for very large models. The report argues that the field has diverged into several trajectories: adaptive first-order baselines, variance-reduction methods, memory-efficient methods, structured second-order preconditioners, schedule-free methods, and orthogonality-oriented updates.

## 4.2 Standard Baseline and Adaptive Optimizers

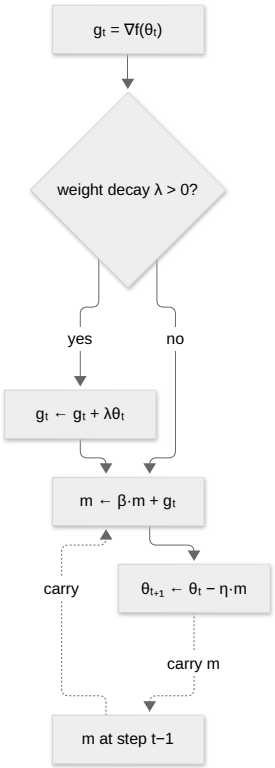



Figure 11: Step flow of SGD with momentum: optional L2 weight decay, exponential momentum accumulation, and the fixed-step parameter update.

**SGD with Momentum.** The classical update accumulates a momentum buffer and then applies a fixed learning rate. At each iteration $t$, the algorithm maintains an exponential moving average $m_t$ of past gradients:

$$m_t = \beta m_{t-1} + g_t \tag{1}$$

$$\theta_{t+1} = \theta_t - \eta m_t \tag{2}$$

where $g_t = \nabla f(\theta_t)$, $\beta \in [0.8, 0.99]$ controls the momentum decay, and $\eta$ is the fixed learning rate. The momentum term acts as velocity: it accelerates movement in consistent directions while dampening oscillations in variable directions.

**Algorithm 1** SGD with Momentum

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, momentum coefficient $\beta$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$

Initialize: momentum buffer $m \leftarrow 0$
**for** $t = 0, 1, 2, \ldots, T-1$ **do**
    Compute gradient: $g_t \leftarrow \nabla f(\theta_t)$
    **if** weight_decay $> 0$ **then**
        $g_t \leftarrow g_t + \lambda\theta_t$ ▷ L2 regularization
    **end if**
    Update momentum: $m \leftarrow \beta \cdot m + g_t$
    Update parameters: $\theta_{t+1} \leftarrow \theta_t - \eta \cdot m$
**end forreturn** $\theta_T$

**Cons.**

- Poor robustness to heterogeneous curvature: a single global rate cannot suit every parameter group.
- Strong dependence on carefully engineered learning-rate schedules.

**Pros.**

- Low memory overhead: a single momentum buffer per parameter.
- Strong generalization when tuned carefully, and a simple update to implement.

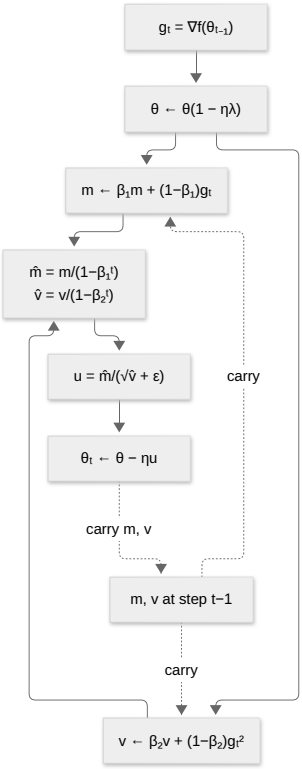



Figure 12: Step flow of AdamW: decoupled weight decay, first- and second-moment EMAs, bias correction, and the element-wise adaptive update.

**Adam and AdamW.** Adam tracks exponential moving averages of both first moment (mean) and second moment (uncentered variance) to achieve element-wise adaptive learning rates. The update rule is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{3}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{4}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(bias correction)} \tag{5}$$

$$\theta_{t+1} = \theta_t - \eta \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \tag{6}$$

AdamW introduces a crucial modification: weight decay is applied directly to parameters (decoupled from gradients): $\theta_t \leftarrow \theta_t(1 - \eta\lambda)$ rather than adding $\lambda\theta_t$ to the gradient. This decoupling prevents the adaptive scaling from interfering with regularization strength.

**Algorithm 2** AdamW (Adam with Decoupled Weight Decay)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, exponential decay rates $\beta_1, \beta_2 \in [0, 1)$
**Require:** Momentum constant $\epsilon$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$

Initialize: first moment $m \leftarrow 0$, second moment $v \leftarrow 0$, step counter $t \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
    Compute gradient: $g_t \leftarrow \nabla f(\theta_{t-1})$
    Weight decay (decoupled): $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    Update moments: $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    Bias correction: $\hat{m} \leftarrow m/(1 - \beta_1^t)$, $\hat{v} \leftarrow v/(1 - \beta_2^t)$
    Update parameters: $\theta_t \leftarrow \theta_{t-1} - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon))$
**end forreturn** $\theta_T$

**Cons.**

- Stores first and second moment tensors for every parameter, doubling optimizer-state memory relative to SGD.
- Element-wise adaptive scaling can interfere with regularization unless weight decay is decoupled.

**Pros.**

- Practical baseline for transformer fine-tuning: fast convergence and forgiving to hyperparameter variation.
- Decoupled weight decay keeps the regularization strength independent of the adaptive scaling.

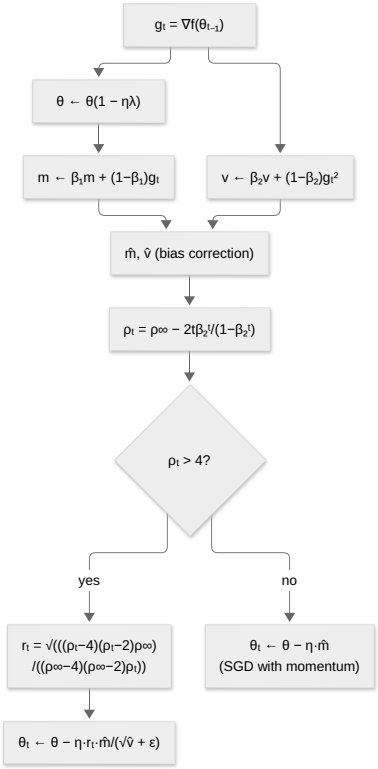



Figure 13: Step flow of RAdam: SMA-window rectification with an early-step fallback to momentum SGD.

**RAdam (Rectified Adam).** RAdam addresses Adam's instability in early training by dynamically rectifying the adaptive learning rate. The key observation is that the second moment $v_t$ has very high variance in the first few steps, causing unreliable adaptive scaling. RAdam computes the effective simple moving average (SMA) window length:

$$\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1-\beta_2^t}, \quad \text{where} \quad \rho_\infty = \frac{2}{1-\beta_2} - 1$$

When $\rho_t > 4$ (sufficient samples for variance estimation), RAdam applies adaptive scaling with a rectification term:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}, \quad \theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

When $\rho_t \leq 4$, RAdam falls back to SGD with momentum: $\theta_{t+1} = \theta_t - \eta \hat{m}_t$.

**Algorithm 3** RAdam (Rectified Adam)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, $t \leftarrow 0$
Compute: $\rho_\infty \leftarrow 2/(1-\beta_2) - 1$
**for** $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    **if** weight_decay $> 0$ **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    **end if**
    $m \leftarrow \beta_1 m + (1-\beta_1) g_t$
    $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
    $\hat{m} \leftarrow m/(1-\beta_1^t)$, $\hat{v} \leftarrow v/(1-\beta_2^t)$
    $\rho_t \leftarrow \rho_\infty - 2t\beta_2^t/(1-\beta_2^t)$
    **if** $\rho_t > 4$ **then**
        $r_t \leftarrow \sqrt{((\rho_t - 4)(\rho_t - 2)\rho_\infty)/((\rho_\infty - 4)(\rho_\infty - 2)\rho_t)}$
        $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{m}/(\sqrt{\hat{v}} + \epsilon)$
    **else**
        $\theta_t \leftarrow \theta_{t-1} - \eta\hat{m}$ ▷ SGD with momentum
    **end if**
**end forreturn** $\theta_T$

**Cons.**

- Adds rectification bookkeeping on top of Adam's moment state.
- The benefit shrinks when a well-tuned warmup schedule is already in use.

**Pros.**

- Eliminates the need for manual learning-rate warmup by rectifying the adaptive rate.
- Falls back gracefully to momentum SGD while variance estimates are unreliable, improving robustness.

### 4.3 Advanced Momentum and Variance Reduction (2024–2025)

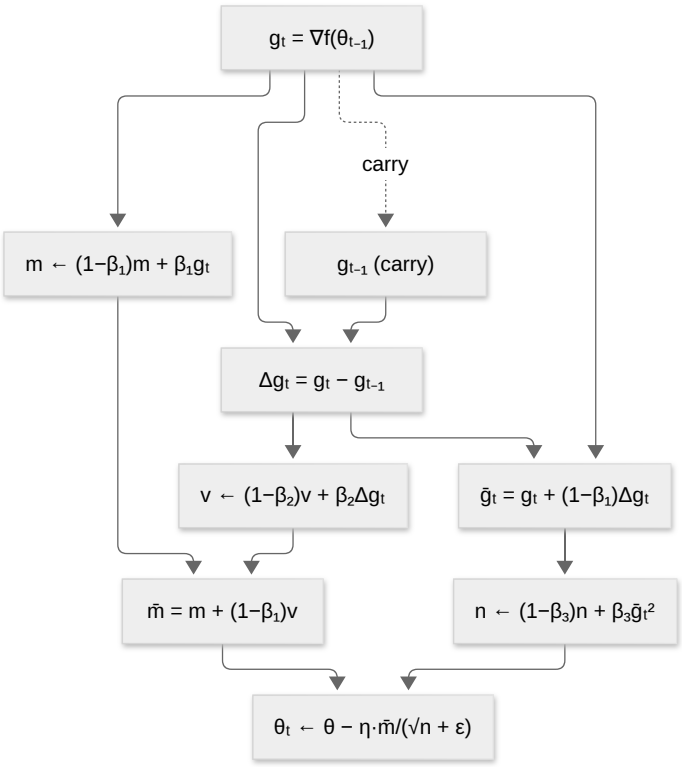



Figure 14: Step flow of Adan: Nesterov momentum estimation from the gradient difference, with three state buffers and a combined momentum step.

**Adan (Adaptive Nesterov Momentum).** Adan reformulates Nesterov acceleration without the extra gradient computation required by classical Nesterov SGD. The algorithm maintains three momentum buffers:

$$m_t = (1-\beta_1)m_{t-1} + \beta_1 g_t \quad \text{(first moment)} \tag{7}$$

$$v_t = (1-\beta_2)v_{t-1} + \beta_2(g_t - g_{t-1}) \quad \text{(velocity/gradient difference)} \tag{8}$$

$$n_t = (1-\beta_3)n_{t-1} + \beta_3[g_t + (1-\beta_1)(g_t - g_{t-1})]^2 \quad \text{(Nesterov second moment)} \tag{9}$$

The **Nesterov Momentum Estimation** (NME) term $\bar{g}_t = g_t + (1-\beta_1)(g_t - g_{t-1})$ estimates the gradient at a future position without evaluating it. The update combines momentum with velocity:

$$\bar{m}_t = m_t + (1-\beta_1)v_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\bar{m}_t}{\sqrt{n_t} + \epsilon}$$

**Algorithm 4** Adan (Adaptive Nesterov Momentum)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2, \beta_3 \in (0, 1)$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, $n \leftarrow 0$, $g_{-1} \leftarrow 0$ (previous gradient)
**for** $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    $m \leftarrow (1 - \beta_1)m + \beta_1 g_t$
    $\Delta g_t \leftarrow g_t - g_{t-1}$
    $v \leftarrow (1 - \beta_2)v + \beta_2 \Delta g_t$
    Nesterov estimation: $\bar{g}_t \leftarrow g_t + (1 - \beta_1)\Delta g_t$
    $n \leftarrow (1 - \beta_3)n + \beta_3 \bar{g}_t^2$
    Combined momentum: $\bar{m} \leftarrow m + (1 - \beta_1)v$
    $\theta_t \leftarrow \theta_{t-1} - \eta(\bar{m}/(\sqrt{n} + \epsilon))$
    $g_{t-1} \leftarrow g_t$
**end forreturn** $\theta_T$

**Cons.**

- Three momentum-like buffers increase memory overhead relative to Adam.
- Additional gradient-difference bookkeeping per step.

**Pros.**

- Nesterov-style acceleration without an extra gradient evaluation.
- Fast convergence across diverse architectures (CNNs, GANs, transformers).

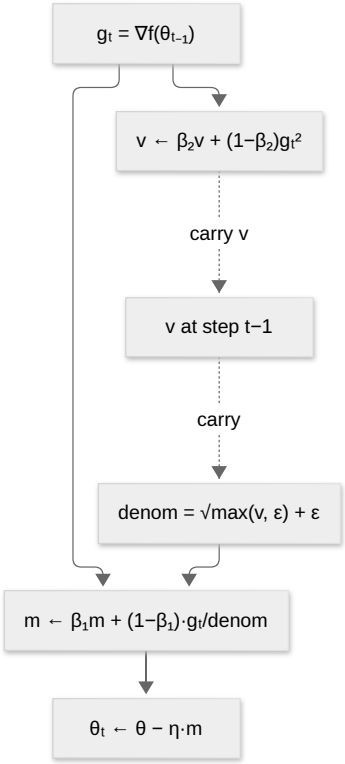



Figure 15: Step flow of ADOPT: the second moment from the previous step normalizes the current gradient before the first-moment update.

**ADOPT (Adam with Optimal Pruned Tuning).** ADOPT fixes a fundamental theoretical issue in Adam: the gradient appears in both the first moment and second moment estimates, creating circularity. ADOPT **decouples** by using the previous step's second moment for the denominator:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{10}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \frac{g_t}{\sqrt{v_{t-1}} + \epsilon} \quad (\text{uses } v_{t-1}, \text{ not } v_t) \tag{11}$$

$$\theta_{t+1} = \theta_t - \eta m_t \tag{12}$$

**Algorithm 5** ADOPT (Adam with Optimal Pruned Tuning)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, tolerance $\epsilon$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $v \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  **if** weight_decay $> 0$ **then**
    $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
  **end if**
  Use **previous** second moment: denom $\leftarrow \sqrt{\max(v, \epsilon)} + \epsilon$
  Update first moment using previous variance: $m \leftarrow \beta_1 m + (1-\beta_1)(g_t/\text{denom})$
  Update second moment with **current** gradient: $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
  $\theta_t \leftarrow \theta_{t-1} - \eta m$
**end forreturn** $\theta_T$

**Cons.**

- Requires keeping the previous second moment as an explicit denominator.
- Empirical gains are architecture- and tuning-dependent.

**Pros.**

- Removes Adam's gradient circularity and attains the optimal convergence rate $O(1/\sqrt{T})$ with **any** $\beta_2 \in (0, 1)$, without bounded-noise assumptions.
- Drop-in Adam replacement with comparable or superior empirical performance across vision, NLP, RL, and generative modeling.

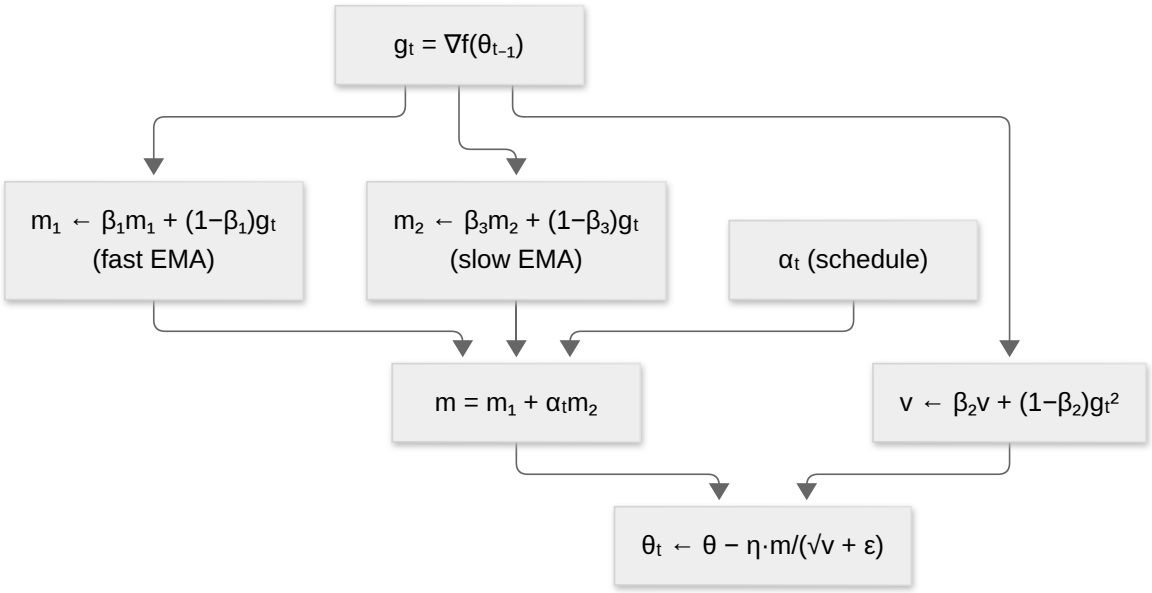



Figure 16: Step flow of AdEMAMix: parallel fast and slow gradient EMAs, mixed with a scheduled weight $\alpha_t$ before the adaptive update.

**AdEMAMix (Exponential Moving Average Mixture).** AdEMAMix replaces Adam's single EMA of gradients with a **mixture of two EMAs**: one fast-decaying and one slow-decaying. This addresses the observation that gradients remain informative over tens of thousands of steps, not just hundreds:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1-\beta_1) g_t \quad \text{(fast EMA, } \beta_1 \approx 0.9\text{)} \tag{13}$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1-\beta_3) g_t \quad \text{(slow EMA, } \beta_3 \approx 0.9999\text{)} \tag{14}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{15}$$

$$m_t = m_{1,t} + \alpha_t m_{2,t} \quad \text{(mixture with scheduled weight } \alpha_t\text{)} \tag{16}$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{17}$$

The mixture weight $\alpha_t$ typically increases during training, allowing the fast EMA to provide immediate adaptation while the slow EMA accumulates long-range gradient correlations.

**Algorithm 6** AdEMAMix (Exponential Moving Average Mixture)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1$ (fast), $\beta_3$ (slow), $\beta_2$ (second moment)
**Require:** Mixture weight $\alpha_t$ (typically increasing), tolerance $\epsilon$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m_1 \leftarrow 0$ (fast EMA), $m_2 \leftarrow 0$ (slow EMA), $v \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    **if** weight_decay $> 0$ **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    **end if**
    $m_1 \leftarrow \beta_1 m_1 + (1 - \beta_1) g_t$ ▷ Fast EMA
    $m_2 \leftarrow \beta_3 m_2 + (1 - \beta_3) g_t$ ▷ Slow EMA
    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    $m_t \leftarrow m_1 + \alpha_t m_2$ ▷ Mixture
    $\theta_t \leftarrow \theta_{t-1} - \eta(m_t/(\sqrt{v} + \epsilon))$
**end forreturn** $\theta_T$

**Cons.**

- Two gradient EMAs plus the second moment roughly triple Adam's optimizer state.
- The mixture schedule $\alpha_t$ adds a hyperparameter to tune.

**Pros.**

- Exploits gradient information that remains informative over tens of thousands of steps.
- A 1.3B model on 101B tokens matches AdamW on 197B tokens (95% data-efficiency gain), suggesting reduced forgetting.

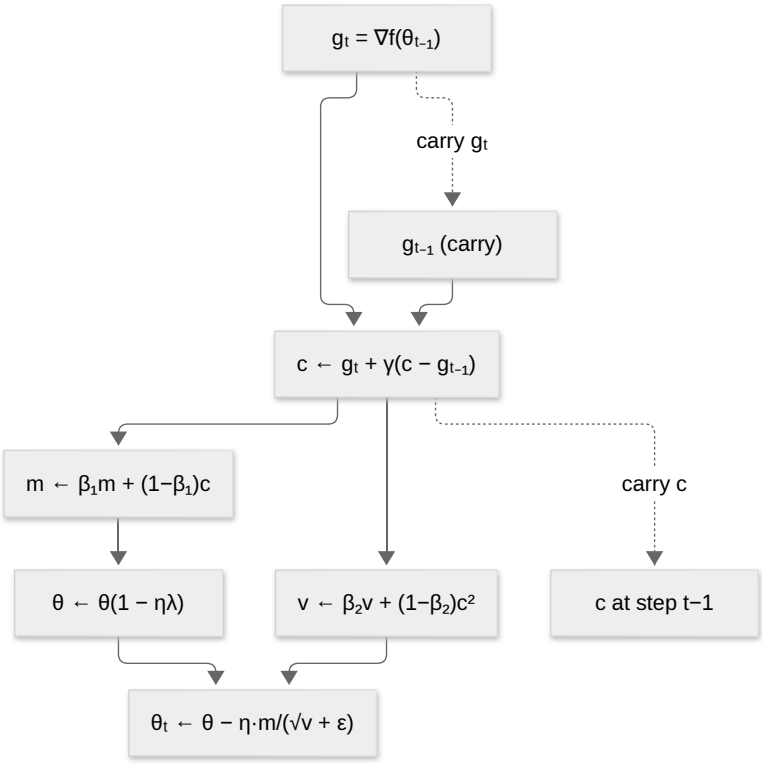



Figure 17: Step flow of MARS: a recursive variance-reduced gradient estimate feeding Adam-style first and second moments.

**MARS (Make Adaptive learning Rates Shine).** MARS combines preconditioned gradient methods (e.g., AdamW) with **variance reduction** via scaled stochastic recursive momentum. The core innovation is a variance-reduced gradient estimate:

$$c_t = g_t + \gamma(c_{t-1} - g_{t-1}) \quad \text{(SVRG-style recursive estimate)} \tag{18}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)c_t \tag{19}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)c_t^2 \tag{20}$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{21}$$

where $\gamma \in [0.01, 0.1]$ controls how much historical gradient information is retained. The variance-reduced gradient $c_t$ acts as an implicit noise filter, amplifying consistent signal directions and dampening contradictory noise.

**Algorithm 7** MARS (Make Adaptive learning Rates Shine)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, weight decay $\lambda$
**Require:** Variance reduction coefficient $\gamma \in [0.01, 0.1]$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, $c \leftarrow 0$ (variance-reduced gradient), $g_{-1} \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    Variance reduction: $c \leftarrow g_t + \gamma(c - g_{t-1})$
    $m \leftarrow \beta_1 m + (1 - \beta_1)c$
    $v \leftarrow \beta_2 v + (1 - \beta_2)c^2$
    **if** weight_decay $> 0$ **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    **end if**
    $\theta_t \leftarrow \theta_{t-1} - \eta(m/(\sqrt{v} + \epsilon))$
    $g_{t-1} \leftarrow g_t$
**end forreturn** $\theta_T$

**Cons.**

- Stores the variance-reduced gradient and the previous gradient in addition to Adam's moments.
- The variance-reduction coefficient $\gamma \in [0.01, 0.1]$ must be tuned.

**Pros.**

- Acts as an implicit noise filter, amplifying consistent directions and dampening contradictory noise.
- MARS-AdamW consistently outperforms bare AdamW by significant margins on GPT-2 pretraining.

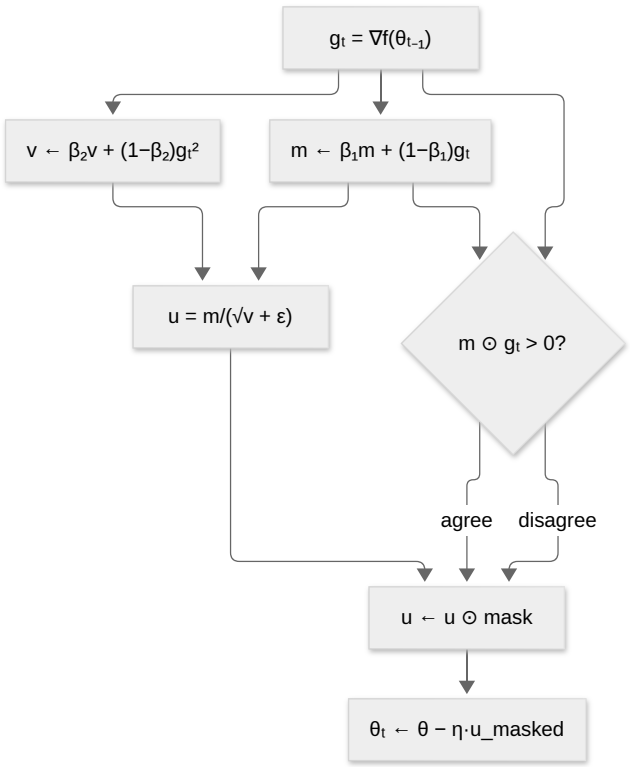



Figure 18: Step flow of Cautious AdamW: a consensus mask keeps only update coordinates where momentum and gradient agree.

**Cautious Optimizers (Cautious AdamW, Cautious Lion).** The Cautious framework applies a **one-line modification** to any momentum-based optimizer: mask the update so that only dimensions where momentum and gradient directions agree are applied:

$$\text{mask}_i = \begin{cases} 1 & \text{if } m_t[i] \cdot g_t[i] > 0 \quad \text{(agreement)} \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

$$u_t = \left( \frac{m_t}{\sqrt{v_t} + \epsilon} \right) \odot \text{mask} \quad \text{(element-wise masking)} \tag{23}$$

$$\theta_{t+1} = \theta_t - \eta u_t \tag{24}$$

The intuition: momentum $m_t$ estimates the gradient direction from history; the current gradient $g_t$ is instantaneous signal. When both agree, the optimizer is confident and should update aggressively. When they disagree, the optimizer is conflicted—the historical trend points toward a region that may have been good before, but current evidence contradicts it. By masking conflicted dimensions, Cautious becomes more conservative and avoids corrupted steps.

**Algorithm 8** Cautious AdamW (Consensus-based Update Masking)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $v \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$
    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$
    Consensus mask: $\text{mask}_i \leftarrow 1$ if $m[i] \cdot g_t[i] > 0$, else 0 ▷ Agreement
    Base Adam update: $u \leftarrow m/(\sqrt{v} + \epsilon)$
    Apply mask: $u_{\text{masked}} \leftarrow u \odot \text{mask}$ ▷ Element-wise
    **if** weight_decay $> 0$ **then**
        $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
    **end if**
    $\theta_t \leftarrow \theta_{t-1} - \eta u_{\text{masked}}$
**end forreturn** $\theta_T$

**Cons.**

- Masking can discard useful updates when momentum and gradient temporarily disagree.
- Requires a base momentum optimizer plus the element-wise mask.

**Pros.**

- A one-line modification applicable to any momentum-based optimizer.
- Up to **1.47× speedup** on Llama and MAE pretraining while preserving convergence guarantees.

### 4.4 Large-Batch, Memory-Efficient, and Parameter-Free Optimizers

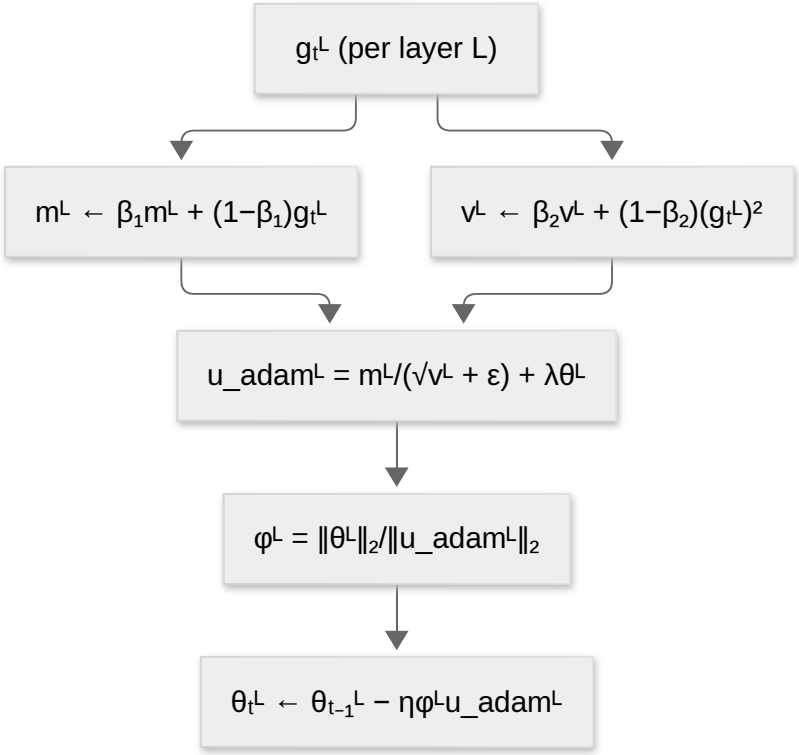



Figure 19: Step flow of LAMB: per-layer Adam step rescaled by the trust ratio between weight and update norms.

**LAMB (Layer-wise Adaptive Moments optimizer for Batch training).** LAMB extends Adam with **layer-wise adaptive rate scaling** (inspired by LARS), enabling stable training with extreme batch sizes (e.g., 64K on BERT):

$$m_t^L = \beta_1 m_{t-1}^L + (1-\beta_1) g_t^L \quad \text{(layer-wise first moment)} \tag{25}$$

$$v_t^L = \beta_2 v_{t-1}^L + (1-\beta_2)(g_t^L)^2 \tag{26}$$

$$u_{\text{adam}}^L = \frac{m_t^L}{\sqrt{v_t^L} + \epsilon} + \lambda \theta_{t-1}^L \quad \text{(base adaptive step with decay)} \tag{27}$$

$$\phi^L = \frac{\|\theta_{t-1}^L\|_2}{\|u_{\text{adam}}^L\|_2} \quad \text{(trust ratio: layer normalization)} \tag{28}$$

$$\theta_t^L = \theta_{t-1}^L - \eta \cdot \phi^L \cdot u_{\text{adam}}^L \tag{29}$$

The **trust ratio** $\phi^L = \|\theta^L\|_2 / \|u_{\text{adam}}^L\|_2$ normalizes the effective update step relative to weight magnitude. In very large batches, stochastic gradient noise can exceed signal, destabilizing layer-wise learning rates. By scaling updates proportionally to weight norms, LAMB ensures relative changes rather than absolute ones, preventing small confident updates in large vectors from dominating.

**Algorithm 9** LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$

Initialize: For each layer $L$: $m^L \leftarrow 0$, $v^L \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
  **for** each layer $L$ **do**
    $g_t^L \leftarrow \nabla f(\theta_t^L)$
    $m^L \leftarrow \beta_1 m^L + (1 - \beta_1) g_t^L$
    $v^L \leftarrow \beta_2 v^L + (1 - \beta_2)(g_t^L)^2$
    Base Adam: $u_{\text{adam}}^L \leftarrow m^L / (\sqrt{v^L} + \epsilon)$
    **if** weight_decay $> 0$ **then**
      $u_{\text{adam}}^L \leftarrow u_{\text{adam}}^L + \lambda \theta_{t-1}^L$
    **end if**
    Trust ratio: $\phi^L \leftarrow \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$ if both nonzero, else 1
    $\theta_t^L \leftarrow \theta_{t-1}^L - \eta \phi^L u_{\text{adam}}^L$
  **end for**
**end forreturn** $\theta_T$

**Cons.**

- Layer-wise norm bookkeeping and trust-ratio computation add per-step overhead.
- The trust ratio is unstable when layer or update norms are close to zero.

**Pros.**

- Enables stable training with extreme batch sizes (e.g., 64K on BERT).
- Preserves small-batch generalization benefits while scaling the batch.

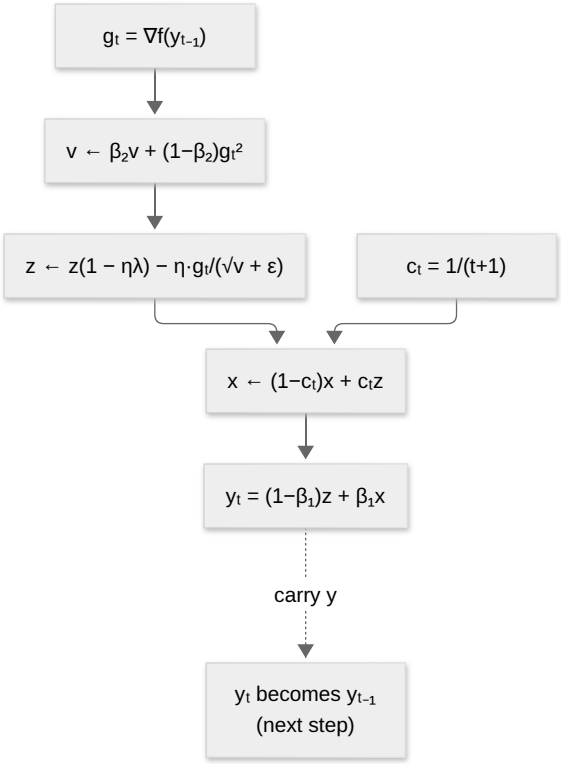



Figure 20: Step flow of Schedule-Free AdamW: exploration stream $z$, iterate average $x$, and interpolation point $y$, with implicit $1/(t+1)$ averaging.

**Schedule-Free AdamW.** Schedule-free methods remove explicit scheduler design from the optimization recipe. The algorithm maintains two parameter streams: $z_t$ (exploration) and $x_t$ (smooth average):

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(variance)} \tag{30}$$

$$z_t = z_{t-1}(1-\eta\lambda) - \eta \frac{g_t}{\sqrt{v_t} + \epsilon} \quad \text{(adaptive step with decay)} \tag{31}$$

$$c_t = \frac{1}{t+1} \quad \text{(averaging coefficient, decreases over time)} \tag{32}$$

$$x_t = (1-c_t)x_{t-1} + c_t z_t \quad \text{(iterate averaging)} \tag{33}$$

$$y_t = (1-\beta)z_t + \beta x_t \quad \text{(interpolation for evaluation point)} \tag{34}$$

The averaging coefficient $c_t = 1/(t+1)$ implements an implicit learning-rate schedule without explicitly specifying total steps $T$. This framework unifies scheduling and iterate averaging: the algorithm explores via $z_t$ while accumulating stable direction via $x_t$. The evaluation point $y_t$ (used for gradient computation) interpolates between exploration and stability.

**Algorithm 10** Schedule-Free AdamW

**Require:** Initial parameters $\theta_0$, learning rate $\eta$ (fixed), $\beta_1, \beta_2$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$ or averaging $x_T$

Initialize: $z \leftarrow \theta_0$ (exploration), $x \leftarrow \theta_0$ (average), $v \leftarrow 0$, $t \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(y_{t-1})$ (gradient at interpolation point)
  $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ ▷ Variance
  Weight decay on $z$: $z \leftarrow z(1 - \eta\lambda)$
  $z \leftarrow z - \eta g_t / (\sqrt{v} + \epsilon)$ ▷ Adaptive step on raw
  $c_t \leftarrow 1/(t+1)$ ▷ Averaging coefficient
  $x \leftarrow (1 - c_t)x + c_t z$ ▷ Iterate averaging
  $y_t \leftarrow (1 - \beta_1)z + \beta_1 x$ ▷ Interpolation for next eval
**end forreturn** $x_T$ or $y_T$

**Cons.**

- Maintains two parameter streams ($z$ and $x$) and requires explicit handling of the evaluation point.
- Behavior departs from scheduled baselines, which complicates comparisons.

**Pros.**

- Removes the explicit learning-rate schedule and its total-step hyperparameter.
- State-of-the-art convergence across convex optimization, large-scale deep learning, and reinforcement learning.

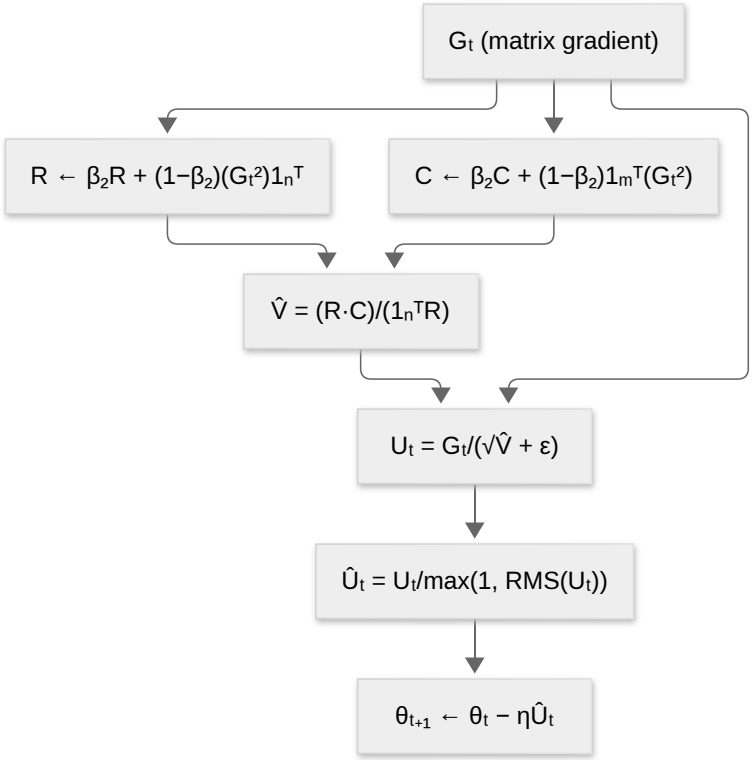



Figure 21: Step flow of Adafactor: factorized row/column second-moment accumulators, outer-product variance reconstruction, and RMS clipping.

**Adafactor.** Adafactor reduces optimizer memory by **factorizing** second-moment statistics for matrix-shaped parameters, storing only row and column accumulators rather than dense variance states. For a gradient matrix $G_t \in \mathbb{R}^{m \times n}$:

$$R_t = \beta_2 R_{t-1} + (1-\beta_2)(G_t^2)\mathbf{1}_n^T \quad \text{(row variance)} \tag{35}$$

$$C_t = \beta_2 C_{t-1} + (1-\beta_2)\mathbf{1}_m^\top (G_t^2) \quad \text{(column variance)} \tag{36}$$

$$\hat{V}_t = \frac{R_t C_t}{\mathbf{1}_n^\top R_t} \quad \text{(reconstructed variance via outer product)} \tag{37}$$

$$U_t = \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon} \quad \text{(normalized adaptive step)} \tag{38}$$

$$\hat{U}_t = \frac{U_t}{\max(1, \mathrm{RMS}(U_t))} \quad \text{(stability clipping)} \tag{39}$$

**Algorithm 11** Adafactor (Factorized Second-Moment Statistics)

**Require:** Gradient matrix $G_t \in \mathbb{R}^{m \times n}$, learning rate $\eta$, $\beta_2 \approx 0.999$
**Ensure:** Updated parameters $\theta_T$

Initialize: Row accumulators $R \leftarrow \epsilon \mathbf{1}_m^T$, Column $C \leftarrow \epsilon \mathbf{1}_n$
**for** $t = 1, 2, \ldots, T$ **do**
  $G_t \leftarrow \nabla f(\theta_t)$ (matrix gradient)
  $R \leftarrow \beta_2 R + (1 - \beta_2)(G_t^2)\mathbf{1}_n^T$ ▷ Row inner products
  $C \leftarrow \beta_2 C + (1 - \beta_2)\mathbf{1}_m^\top (G_t^2)$ ▷ Column inner products
  Reconstructed variance: $\hat{V} \leftarrow (R \cdot C)/(\mathbf{1}_n^\top R)$ ▷ Via outer product
  Normalized adaptive step: $U_t \leftarrow G_t / (\sqrt{\hat{V}} + \epsilon)$
  Per-row RMS clipping: $\hat{U}_t \leftarrow U_t / \max(1, \mathrm{RMS}(U_t))$
  $\theta_{t+1} \leftarrow \theta_t - \eta \hat{U}_t$
**end forreturn** $\theta_T$

**Cons.**

- Factorized variance is an approximation and can be unstable on some tasks.
- Reduced optimization expressiveness relative to full Adam.

**Pros.**

- Memory drops from $O(n_{\text{params}})$ to $O(\sqrt{n_{\text{params}}})$ for matrix parameters.
- Most attractive when VRAM is dominated by optimizer state rather than activations.

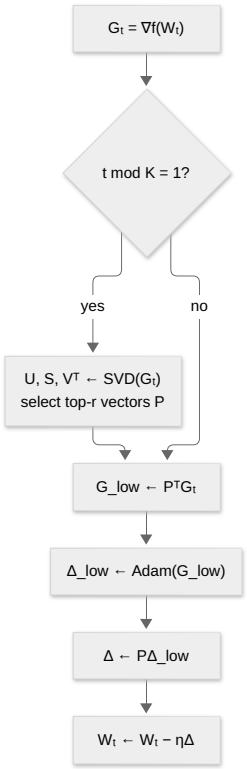



Figure 22: Step flow of GaLore: periodic SVD projection of the gradient into a low-rank subspace, Adam in that subspace, and reconstruction.

**GaLore (Gradient Low-Rank Projection).** GaLore projects 2D gradients into a low-rank subspace before optimization, reducing optimizer-state memory while maintaining adaptive step scaling. For a 2D parameter matrix $W \in \mathbb{R}^{m \times n}$:

$$U, S, V = \text{SVD}(G_t) \quad \text{(compute singular decomposition)} \tag{40}$$

$$P \in \mathbb{R}^{n \times r} \quad \text{or} \quad P^\top \in \mathbb{R}^{r \times m} \quad \text{(select top-}r\text{ singular vectors)} \tag{41}$$

$$G_{\text{low}} = P^\top G_t \quad \text{(project to low-rank space)} \tag{42}$$

$$\Delta_{\text{low}} = \text{Adam}(G_{\text{low}}) \quad \text{(optimize in compressed space)} \tag{43}$$

$$\Delta = P \Delta_{\text{low}} \quad \text{(reconstruct in original space)} \tag{44}$$

By projecting into a rank-$r$ subspace (typically $r \ll \min(m, n)$), optimizer state is reduced from $O(mn)$ to $O(r(m+n))$ for 2D parameters. This complementary memory-saving approach is especially relevant when the model is too large for full-rank optimizer state.

**Algorithm 12** GaLore (Gradient Low-Rank Projection)

**Require:** 2D parameter matrix $W \in \mathbb{R}^{m \times n}$, learning rate $\eta$, rank $r \ll \min(m, n)$
**Ensure:** Updated parameters $\theta_T$

Initialize: Adam state in low-rank space (if rank-2 variant)
**for** $t = 1, 2, \ldots, T$ **do**
  $G_t \leftarrow \nabla f(W_t)$
  **if** $t \mod K = 1$ **then** ▷ Periodic SVD
    $U_t, S_t, V_t^\top \leftarrow \text{SVD}(G_t)$ (full or thin)
    Select projection: $P \leftarrow U_t[:, :r]$ or $P \leftarrow V_t[:, :r]$
  **end if**
  Project gradient: $G_{\text{low}} \leftarrow P^\top G_t$ (or $G_t P^\top$ for right projection)
  Run Adam on low-rank: $\Delta_{\text{low}} \leftarrow \text{Adam}(G_{\text{low}})$
  Reconstruct in original space: $\Delta \leftarrow P \Delta_{\text{low}}$ (or $\Delta_{\text{low}} P^\top$)
  $W_t \leftarrow W_t - \eta \Delta$
**end forreturn** $\theta_T$

**Cons.**

- Periodic SVD adds compute and introduces a projection-refresh interval to tune.
- Low-rank projection can discard gradient information.

**Pros.**

- Optimizer state reduced from $O(mn)$ to $O(r(m+n))$ for 2D parameters.
- Works synergistically with other techniques and has shown strong results on billion-parameter models.

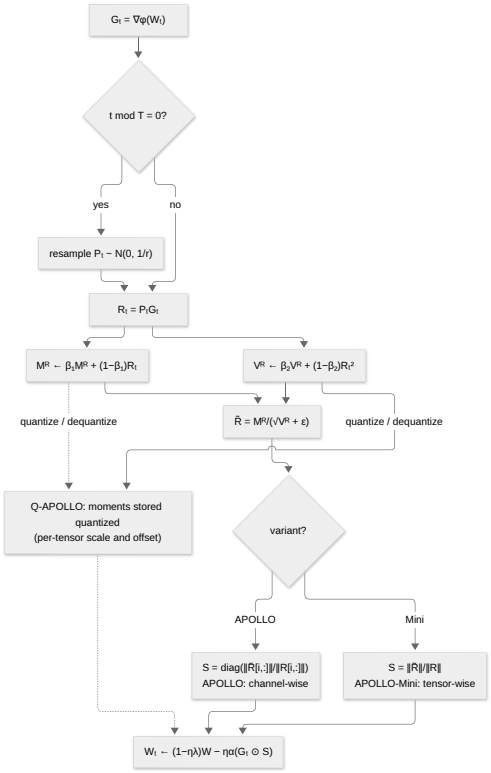



Figure 23: Step flow of the APOLLO family: Gaussian-projected moments, structured gradient scaling (channel-wise for APOLLO, scalar for APOLLO-Mini), and quantized state for Q-APOLLO.

**APOLLO, APOLLO-Mini, and Q-APOLLO.** The APOLLO family [118] begins from the observation that AdamW's element-wise denominator can be **coarsened into a structured learning-rate update**. Rather than storing dense moments for every parameter entry, APOLLO projects a matrix gradient $G_t \in \mathbb{R}^{m\times n}$ into a compact random subspace and tracks Adam-style moments there:

$$R_t = P_t G_t \quad \text{or} \quad R_t = G_t P_t^\top \tag{45}$$

$$M_t^R = \beta_1 M_{t-1}^R + (1-\beta_1) R_t \tag{46}$$

$$V_t^R = \beta_2 V_{t-1}^R + (1-\beta_2) R_t^2 \tag{47}$$

$$\widetilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R} + \epsilon} \tag{48}$$

The projected state is *not* expanded back into a dense low-rank update as in SVD-based methods. Instead, APOLLO estimates a structured scaling tensor $S_t$ in the original space. In the standard APOLLO variant, that scaling is **channel-wise**: each row or channel receives its own norm ratio. In APOLLO-Mini, the scaling is reduced to a single **tensor-wise** scalar, corresponding to the rank-1 extreme described in the paper. The resulting parameter update is therefore Adam-like in adaptation but much closer to SGD in state cost:

$$W_t \leftarrow (1-\eta\lambda) W_{t-1} - \eta\,\alpha\,(G_t \odot S_t)$$

where $\alpha$ is the extra scale factor used to stabilize highly compressed variants.

**Algorithm 13** APOLLO / APOLLO-Mini Structured Gradient Scaling

**Require:** Weight matrix $W \in \mathbb{R}^{m \times n}$ with $m \leq n$, learning rate $\eta$, scale factor $\alpha$, decay rates $(\beta_1, \beta_2)$, weight decay $\lambda$, rank $r$, projection refresh interval $T$

**Ensure:** Updated parameters $W_T$

Initialize projected moments: $M^R \leftarrow 0$, $V^R \leftarrow 0$, step $t \leftarrow 0$
**repeat**
  Compute gradient: $G_t \leftarrow \nabla_W \phi(W_t)$
  **if** $t \bmod T = 0$ **then**
    Sample Gaussian projector $P_t \sim \mathcal{N}(0, 1/r)$ with a fresh seed
  **end if**
  Project gradient: $R_t \leftarrow P_t G_t$ ▷ or $G_t P_t^\top$ depending on layout
  Update projected AdamW moments: $M_t^R, V_t^R \leftarrow \text{AdamWState}(R_t; \beta_1, \beta_2)$
  Normalize projected state: $\widetilde{R}_t \leftarrow M_t^R / (\sqrt{V_t^R} + \epsilon)$
  **if** APOLLO **then**
    $S_t \leftarrow \text{diag}(s_1^R, \ldots, s_m^R)$, where $s_i^R = \|\widetilde{R}_t[i,:]\|_2 / \|R_t[i,:]\|_2$
  **else**
    $S_t \leftarrow s_t^R$, where $s_t^R = \|\widetilde{R}_t\|_2 / \|R_t\|_2$
  **end if**
  Update weights: $W_t \leftarrow (1 - \eta\lambda) W_{t-1} - \eta\, \alpha\, (G_t \odot S_t)$
  $t \leftarrow t + 1$
**until** convergence

**Cons.**

- Structured scaling is a coarser approximation of AdamW's element-wise denominator.
- Projectors must be refreshed periodically, and very small ranks make the scaling noisy.

**Pros.**

- Replaces repeated SVD with periodically refreshed Gaussian random projection, so the systems cost is ordinary matrix multiplication rather than spectral decomposition.
- Unusually tolerant to extreme compression: even APOLLO-Mini, which keeps only rank-1 auxiliary state, remains competitive with or better than AdamW while approaching SGD-level memory.

**APOLLO-Mini.** APOLLO-Mini is the family member optimized for **extreme memory efficiency**. The paper motivates it by arguing that, in a rank-1 compact space, channel-wise scaling becomes too noisy, so the update is coarsened further to a single tensor-wise scale. The loss of granularity is partially offset by the explicit scaling factor $\alpha$ (the paper discusses values such as 128 in this regime), giving a useful point on the optimization Pareto frontier: very small state, no SVD overhead, and still strong pre-training behavior.

**Cons.**

- A single tensor-wise scale loses channel granularity, and rank-1 state can be noisy.
- Depends on the extra scale factor $\alpha$ (e.g., 128) to remain stable.

**Pros.**

- Smallest state in the APOLLO family, with no SVD overhead.
- Strong pre-training behavior at near-SGD memory cost.

**Q-APOLLO.** The APOLLO paper also stresses that the family combines naturally with **quantization** for ultra-low-memory training. This repository turns that systems observation into a concrete optimizer variant, `q_apollo`. In the implementation, APOLLO's low-rank first and second moments are stored in quantized form together with per-tensor scales and offsets, and are dequantized only when needed for the next step. Q-APOLLO therefore preserves APOLLO's projected-gradient logic while reducing the precision of the remaining optimizer state.

**Cons.**

- Quantized moments introduce approximation error and dequantization overhead.
- Requires managing per-tensor scales and offsets.

**Pros.**

- Reduces the remaining optimizer state to ultra-low precision.
- Most aggressive memory-saving member of the local optimizer stack while preserving APOLLO's projected-gradient logic.

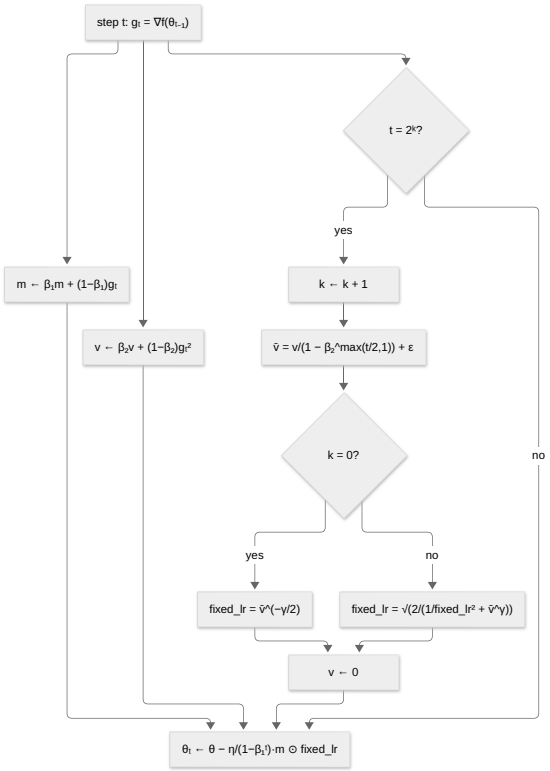



Figure 24: Step flow of Anon: the IDU preconditioner refreshed at powers of two, with tunable adaptivity $\gamma$ and a bias-corrected momentum update.

**Anon (Adaptivity Non-restricted Optimizer with Novel convergence technique).** Anon [3] introduces continuously tunable adaptivity via a single parameter $\gamma \in \mathbb{R}$, bridging the gap between SGD-like ($\gamma = 0$) and Adam-like ($\gamma = 1$) behaviors, and extrapolating beyond both. The core innovation is the Incremental Delay Update (IDU) mechanism, which enables stable convergence across the entire real-valued adaptivity spectrum:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(first moment)} \tag{49}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(second moment)} \tag{50}$$

$$\bar{v}_t = \frac{v_t}{1-\beta_2^{\max(t,1)}} + \epsilon \quad \text{(bias-corrected second moment)} \tag{51}$$

$$\text{fixed_lr}_k = \begin{cases} \bar{v}_k^{-\gamma/2} & \text{if } k = 0 \\ \sqrt{2 / \left( \frac{1}{\text{fixed_lr}_{k-1}^2} + \bar{v}_k^{\gamma} \right)} & \text{if } k > 0 \end{cases} \tag{52}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{1-\beta_1^t} \cdot m_t \odot \text{fixed_lr}_{\lfloor \log_2 t \rfloor} \tag{53}$$

Unlike standard adaptive optimizers, IDU updates the preconditioner only at powers of two ($t = 2^k$), accumulating gradient statistics between updates and aggregating them recursively. This soft, multi-scale accumulator avoids the hard max-tracking of AMSGrad while provably stabilizing convergence for any $\gamma \in \mathbb{R}$. The parameter $\gamma$ directly controls adaptivity: $\gamma > 1$ accelerates escape from saddle points (favorable for complex architectures), $\gamma = 1$ recovers Adam-like behavior, $\gamma = 0$ yields SGD-like scaling, and $\gamma < 0$ biases toward flatter minima (beneficial for classical architectures like CNNs).

**Algorithm 14** Anon (Adaptivity Non-restricted Optimizer with IDU)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, tolerance $\epsilon$, adaptivity $\gamma \in \mathbb{R}$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $v \leftarrow 0$, fixed_lr $\leftarrow 0$, step $t \leftarrow 0$, IDU counter $k \leftarrow -1$
**for** $t = 1, 2, \ldots, T$ **do**
    $g_t \leftarrow \nabla f(\theta_{t-1})$
    $m \leftarrow \beta_1 m + (1-\beta_1) g_t$
    $v \leftarrow \beta_2 v + (1-\beta_2) g_t^2$
    **if** $t = 2^k$ **then**
        $k \leftarrow k + 1$
        $\bar{v} \leftarrow v / (1 - \beta_2^{\max(t/2,1)}) + \epsilon$
        **if** $k = 0$ **then**
            fixed_lr $\leftarrow \bar{v}^{-\gamma/2}$
        **else**
            fixed_lr $\leftarrow \sqrt{2/(1/\text{fixed_lr}^2 + \bar{v}^{\gamma})}$
        **end if**
        $v \leftarrow 0$
    **end if**
    $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{1-\beta_1^t} \cdot m \odot$ fixed_lr
**end forreturn** $\theta_T$

**Cons.**

- Maintains three state buffers ($m$, $v$, fixed_lr) and refreshes the preconditioner on a logarithmic schedule.
- The adaptivity parameter $\gamma$ must be selected per architecture.

**Pros.**

- Same $\mathcal{O}(n)$ per-step complexity class as AdamW, with continuously tunable adaptivity.

- IDU provably attains $\mathcal{O}(\sqrt{T})$ regret for convex problems and $\mathcal{O}(\ln T/\sqrt{T})$ for non-convex settings.

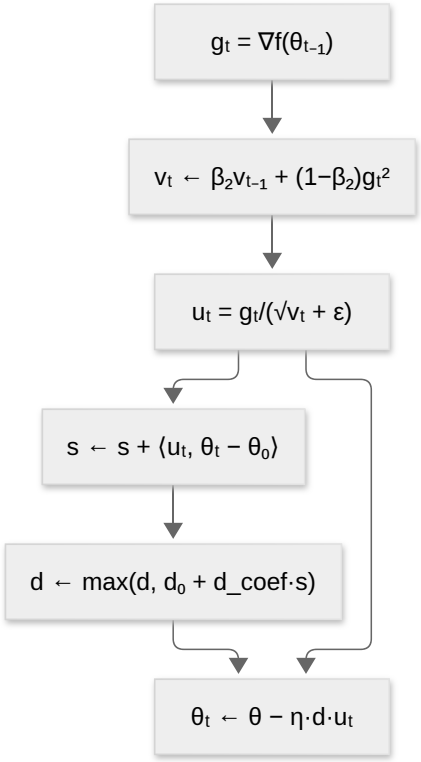



Figure 25: Step flow of Prodigy: a cumulative signed-distance estimate that scales the adaptive step without a learning-rate schedule.

**Prodigy (Approximating the Distance Estimate).** Prodigy adapts the effective step scale through a running distance-like statistic, eliminating the need for explicit learning-rate tuning. The algorithm maintains a cumulative distance estimate:

$$u_t = \frac{g_t}{\sqrt{v_t} + \epsilon} \quad \text{(unnormalized adaptive step)} \tag{54}$$

$$s_t = s_{t-1} + \langle u_t, \theta_t - \theta_0 \rangle \quad \text{(cumulative signed distance)} \tag{55}$$

$$d_t = \max(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_t) \quad \text{(distance estimate with lower bound)} \tag{56}$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \tag{57}$$

where $d_0$ is an initial bound and $d_{\text{coef}}$ controls how aggressively the estimate adapts. The distance estimate $d_t$ captures the combined magnitude of past gradients weighted by actual parameter displacement, implementing an implicit effective learning rate.

**Algorithm 15** Prodigy (Approximating the Distance Estimate)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, coeff $d_{\text{coef}}$, initial dist $d_0 > 0$
**Ensure:** Updated parameters $\theta_T$

Initialize: $s \leftarrow 0$ (cumulative signed distance), $d \leftarrow d_0$
**for** $t = 1, 2, \ldots, T$ **do**
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ ▷ Second moment
  $u_t \leftarrow g_t / (\sqrt{v_t} + \epsilon)$ ▷ Unnormalized adaptive
  Update distance: $s \leftarrow s + \langle u_t, \theta_t - \theta_0 \rangle$ ▷ Signed distance
  $d \leftarrow \max(d, d_0 + d_{\text{coef}} \cdot s)$ ▷ Lower-bounded distance
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot d \cdot u_t$ ▷ Distance-scaled update
**end forreturn** $\theta_T$

**Cons.**

- Introduces $d_0$ and $d_{\text{coef}}$ hyperparameters for the distance estimate.
- Adds an inner-product bookkeeping step per iteration.

**Pros.**

- Removes explicit learning-rate tuning and scheduling.
- Robust convergence across problem scales and batch sizes.

## 4.5 Second-Order, Geometric, and Orthogonality Optimizers

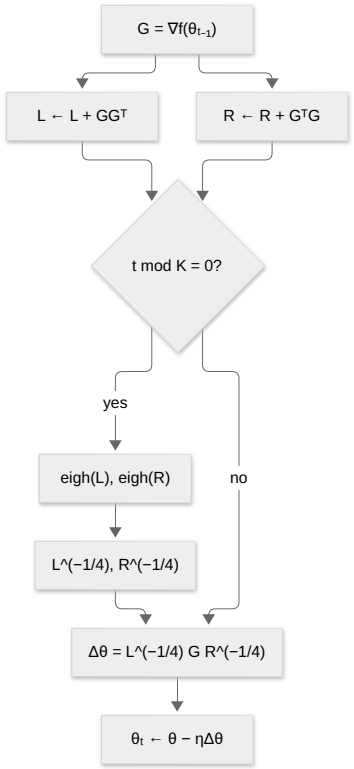



Figure 26: Step flow of Shampoo: Kronecker-factored Gram matrices, periodic eigendecomposition, and fourth-root preconditioning of the gradient.

**Shampoo (Matrix Preconditioning via Kronecker Products).** Shampoo is a **structured second-order method** that computes matrix preconditioners from Kronecker-structured outer-product statistics. For a 2D parameter matrix $W \in \mathbb{R}^{m \times n}$:

$$L_t = L_{t-1} + G_t G_t^\top \quad \text{(left/row Gram matrix, } m \times m\text{)} \tag{58}$$

$$R_t = R_{t-1} + G_t^\top G_t \quad \text{(right/column Gram matrix, } n \times n\text{)} \tag{59}$$

$$L_t^{-1/4} = Q_L (\Lambda_L)^{-1/4} Q_L^\top \quad \text{(via eigendecomposition)} \tag{60}$$

$$R_t^{-1/4} = Q_R (\Lambda_R)^{-1/4} Q_R^\top \tag{61}$$

$$\Delta W_t = L_t^{-1/4} G_t R_t^{-1/4} \quad \text{(preconditioned gradient)} \tag{62}$$

Shampoo approximates the full-matrix Adagrad preconditioner $H^{-1/2}$ (where $H$ is the Hessian) using Kronecker-factored structure. Instead of storing a $(mn) \times (mn)$ preconditioner, Shampoo maintains two smaller matrices ($m \times m$ and $n \times n$), capturing cross-parameter correlations within and across groups. Eigendecompositions are performed periodically (e.g., every $K = 10$ steps) to amortize cost.

**Algorithm 16** Shampoo (Matrix Preconditioning via Kronecker Products)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, eigendecomposition interval $K$
**Ensure:** Updated parameters $\theta_T$

Initialize: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$ (left and right Gram matrices)
**for** $t = 1, 2, \ldots, T$ **do**
  $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
  Update Gram matrices: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
  **if** $t \mod K == 0$ **then**
    $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$ ▷ Eigendecomposition of $L$
    $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ ▷ Eigendecomposition of $R$
    $L^{-1/4} \leftarrow Q_L (\Lambda_L + \epsilon)^{-1/4} Q_L^\top$
    $R^{-1/4} \leftarrow Q_R (\Lambda_R + \epsilon)^{-1/4} Q_R^\top$
  **end if**
  $\Delta\theta \leftarrow L^{-1/4} G R^{-1/4}$ ▷ Preconditioned gradient
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
**end forreturn** $\theta_T$

**Cons.**

- Periodic $O(m^3 + n^3)$ eigendecompositions and higher per-step compute.
- Assumes 2D parameter structure; embeddings and biases need separate handling.

**Pros.**

- Kronecker-factored second-order preconditioning without a dense $(mn) \times (mn)$ matrix.
- Proven effective at production scale and well suited to high-rank transformer matrices, with improved conditioning and accelerated convergence.

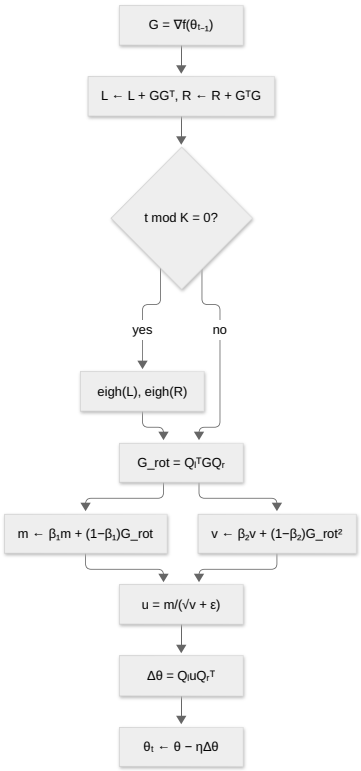



Figure 27: Step flow of SOAP: Adam moments tracked in the Shampoo eigenbasis and rotated back to parameter space.

**SOAP (Shampoo with Adam in eigenbasis).** SOAP is a simplified variant of Shampoo that decouples preconditioning from momentum tracking. Instead of complex matrix algebra on preconditioned gradients, SOAP runs standard Adam in the eigenbasis of Shampoo's preconditioners:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \quad \text{(Gram accumulation)} \tag{63}$$

$$Q_L, \Lambda_L = \text{eigh}(L_t), \quad Q_R, \Lambda_R = \text{eigh}(R_t) \quad \text{(periodic eigendecomposition)} \tag{64}$$

$$G_t^{\text{rot}} = Q_L^\top G_t Q_R \quad \text{(rotate gradient into eigenbasis)} \tag{65}$$

$$m_t^{\text{rot}} = \beta_1 m_{t-1}^{\text{rot}} + (1-\beta_1) G_t^{\text{rot}} \tag{66}$$

$$v_t^{\text{rot}} = \beta_2 v_{t-1}^{\text{rot}} + (1-\beta_2)(G_t^{\text{rot}})^2 \quad \text{(Adam in rotated space)} \tag{67}$$

$$U_t^{\text{rot}} = \frac{m_t^{\text{rot}}}{\sqrt{v_t^{\text{rot}}} + \epsilon}, \quad U_t = Q_L U_t^{\text{rot}} Q_R^\top \quad \text{(rotate back)} \tag{68}$$

The key insight: all momentum tracking occurs in the well-conditioned eigenbasis, simplifying numerical stability and theoretical analysis.

**Algorithm 17** SOAP (Shampoo with Adam in Eigenbasis)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, eigendecomposition interval $K$
**Ensure:** Updated parameters $\theta_T$

Initialize: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$, $m \leftarrow 0$, $v \leftarrow 0$
**for** $t = 1, 2, \ldots, T$ **do**
  $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
  Update Gram: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
  **if** $t \mod K == 0$ **then**
    $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$, $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$
  **end if**
  $G^{\text{rot}} \leftarrow Q_L^\top G Q_R$ ▷ Rotate gradient into eigenbasis
  $m \leftarrow \beta_1 m + (1 - \beta_1) G^{\text{rot}}$ ▷ Exponential moving average (bias-correct outside)
  $v \leftarrow \beta_2 v + (1 - \beta_2)(G^{\text{rot}})^2$ ▷ Second moment (bias-correct outside)
  $u \leftarrow m/(\sqrt{v} + \epsilon)$ ▷ Adaptive step in eigenbasis
  $\Delta\theta \leftarrow Q_L u Q_R^\top$ ▷ Rotate back to parameter space
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
**end forreturn** $\theta_T$

**Cons.**

- Inherits Shampoo's periodic eigendecomposition cost.
- Rotation and inverse rotation add matrix multiplications per step.

**Pros.**

- Combines Shampoo's explicit curvature structure with Adam's proven momentum mechanics.
- Often cleaner and more stable than full Shampoo, with eigendecompositions amortized every $K$ steps.

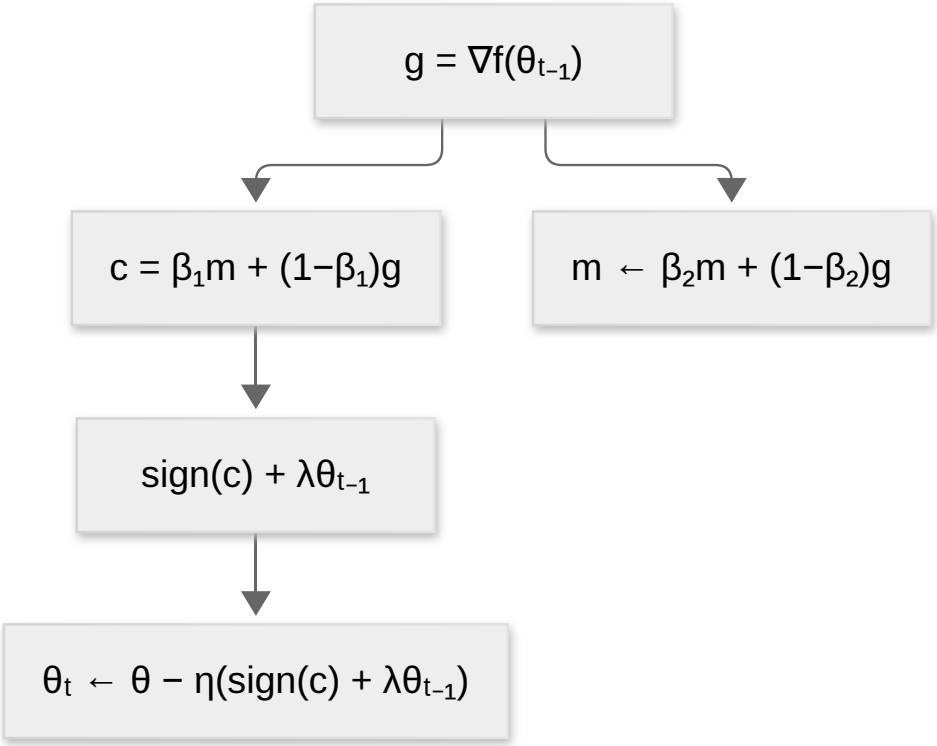



Figure 28: Step flow of Lion: sign-of-momentum update with decoupled momentum accumulation and weight decay.

**Lion (EvoLved Sign Momentum).** Lion achieves **minimal memory overhead** by using sign-based updates instead of full adaptive scaling:

$$c_t = \beta_1 m_t + (1-\beta_1) g_t \quad \text{(momentum input)} \tag{69}$$

$$\theta_{t+1} = \theta_t - \eta \left(\text{sign}(c_t) + \lambda \theta_t\right) \quad \text{(sign operator update)} \tag{70}$$

$$m_{t+1} = \beta_2 m_t + (1-\beta_2) g_t \quad \text{(momentum accumulation)} \tag{71}$$

All element-wise scaling operations are replaced with `sign()`, which outputs $\{-1, 0, +1\}$. This dramatically reduces memory compared to Adam-like methods: Lion stores only the momentum buffer, no second moment variance.

**Algorithm 18** Lion (EvoLved Sign Momentum)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, weight decay $\lambda$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$ (momentum buffer)
**for** $t = 1, 2, \ldots, T$ **do**
  $g \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradient
  $c \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ Momentum input
  $\theta_t \leftarrow \theta_{t-1} - \eta(\text{sign}(c) + \lambda \theta_{t-1})$ ▷ Sign-based update with weight decay
  $m \leftarrow \beta_2 m + (1-\beta_2) g$ ▷ Momentum accumulation (decoupled)
**end forreturn** $\theta_T$

**Cons.**

- Sign-based updates sacrifice the element-wise learning-rate adaptation that makes Adam effective.

- Can be less robust than Adam-family methods on tasks that need adaptive scaling.

**Pros.**

- Stores only the momentum buffer, with no second moment variance.
- High-throughput, low-memory option for large-batch regimes where activation memory dominates.

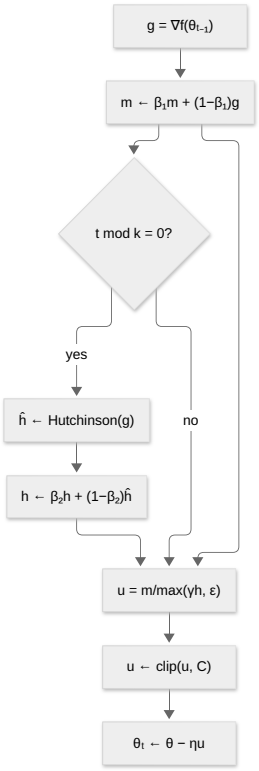



Figure 29: Step flow of Sophia: diagonal Hutchinson Hessian estimate with clipped, curvature-aware element-wise steps.

**Sophia (Second-Order Hessian Information with Optimized Approximation).** Sophia uses **diagonal Hessian estimates** for curvature-aware scaling without the memory and compute cost of dense second-order methods.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(first moment)} \tag{72}$$

$$h_t = \beta_2 h_{t-(k-1)} + (1-\beta_2)\hat{h}_t \quad \text{(diagonal Hessian, updated every } k \text{ steps)} \tag{73}$$

$$\text{clip}(x, C) = \min(\max(x, -C), C) \quad \text{(element-wise clipping)} \tag{74}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{clip}\left(\frac{m_t}{\max(\gamma h_t, \epsilon)}, 1\right) \tag{75}$$

where $\hat{h}_t$ is a diagonal estimate (e.g., Hutchinson-trace estimator) and $\gamma$ is a scaling factor. The diagonal Hessian $h_t$ captures local curvature, allowing the optimizer to take smaller steps in sharp directions and larger steps in flat directions. The clipping operation $\text{clip}(\cdot, 1)$ prevents adaptive steps from exploding.

**Algorithm 19** Sophia (Diagonal Hessian with Clipped Updates)

**Require:** Initial parameters $\theta_0$, learning rate $\eta$, $\beta_1, \beta_2$, Hessian update freq $k$, clip threshold $C$
**Ensure:** Updated parameters $\theta_T$

Initialize: $m \leftarrow 0$, $h \leftarrow \epsilon$ (diagonal Hessian estimate)
**for** $t = 1, 2, \ldots, T$ **do**
  $g \leftarrow \nabla f(\theta_{t-1})$
  $m \leftarrow \beta_1 m + (1 - \beta_1) g$ ▷ First moment (bias-correct outside)
  **if** $t \mod k == 0$ **then**
    $\hat{h} \leftarrow \text{HutchinsonEstimate}(g)$ ▷ Diagonal Hessian via Hutchinson
    $h \leftarrow \beta_2 h + (1 - \beta_2)\hat{h}$ ▷ Update Hessian estimate
  **end if**
  $u \leftarrow \frac{m}{\max(\gamma h, \epsilon)}$ ▷ Adaptive step (element-wise)
  $u \leftarrow \text{clip}(u, C)$ ▷ Element-wise clipping to $[-C, C]$
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot u$
**end forreturn** $\theta_T$

**Cons.**

- Hutchinson estimates add periodic compute and noise to the curvature signal.
- The Hessian estimate can become stale between refreshes.

**Pros.**

- Curvature-aware scaling without the $O(n^2)$ or $O(n^3)$ cost of full second-order methods.
- Clipped steps prevent explosions and work particularly well in second-pass fine-tuning.

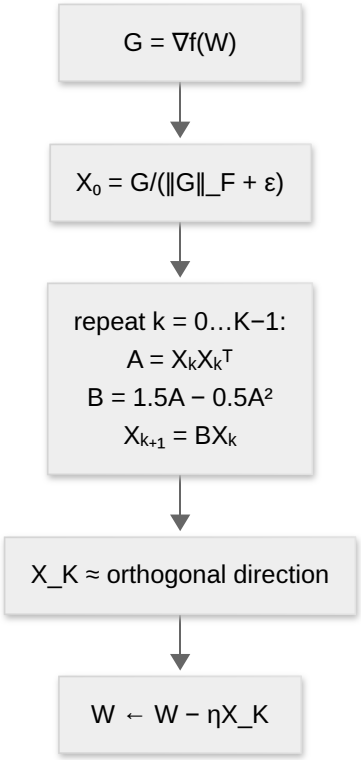



Figure 30: Step flow of Muon: Frobenius normalization followed by Newton-Schulz iterations that orthogonalize the update direction.

**Muon (Newton-Schulz Orthogonalization).** Muon is an **orthogonality-oriented optimizer** that reshapes update geometry using Newton-Schulz polynomial iterations for orthogonalization. For a 2D parameter matrix $W$ with gradient $G_t$:

$$X_0 = \frac{G_t}{\|G_t\|_F + \epsilon} \quad \text{(normalized gradient)} \tag{76}$$

$$A_k = X_k X_k^\top \quad \text{(Gramian)} \tag{77}$$

$$B_k = bA_k + cA_k^2 \quad \text{(polynomial step, coefficients } b, c \text{ from Newton-Schulz)} \tag{78}$$

$$X_{k+1} = aX_k + B_k X_k \quad \text{(iteration } k = 0, 1, \ldots, 4) \tag{79}$$

$$W_{t+1} = W_t - \eta X_K \quad \text{(update with orthogonalized direction)} \tag{80}$$

Muon performs 5 Newton-Schulz iterations to produce an approximately orthogonal direction, ensuring updates respect geometric constraints. The rationale: orthogonal updates preserve norms during training, avoiding the norm creep observed in momentum-based methods.

**Algorithm 20** Muon (Newton-Schulz Orthogonalization)

**Require:** Initial parameters $\theta$ (matrices), learning rate $\eta$, Newton-Schulz iters $K$
**Ensure:** Updated parameters $\theta_T$

**for** each 2D parameter matrix $W$ **do**
  $G \leftarrow \nabla f(W)$ ▷ Gradient
  $X_0 \leftarrow \frac{G}{\|G\|_F+\epsilon}$ ▷ Normalize gradient by Frobenius norm
  **for** $k = 0, 1, \ldots, K-1$ **do**
    $A_k \leftarrow X_k X_k^\top$ ▷ Gramian (cost: $O(mn^2)$ or $O(m^2n)$ for tall/wide)
    $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$ ▷ Newton-Schulz coefficients
    $X_{k+1} \leftarrow B_k X_k$ ▷ Polynomial step
  **end for**
  $W \leftarrow W - \eta X_K$ ▷ Update with orthogonalized direction
**end forreturn** $\theta_T$

**Cons.**

- Several matrix multiplications per Newton-Schulz iteration; efficient custom kernels are required.
- Defined only for 2D parameter matrices.

**Pros.**

- Orthogonalized updates preserve norms and avoid the norm creep of momentum-based methods.
- Promising results on large-scale training with fundamentally better-conditioned update geometry.

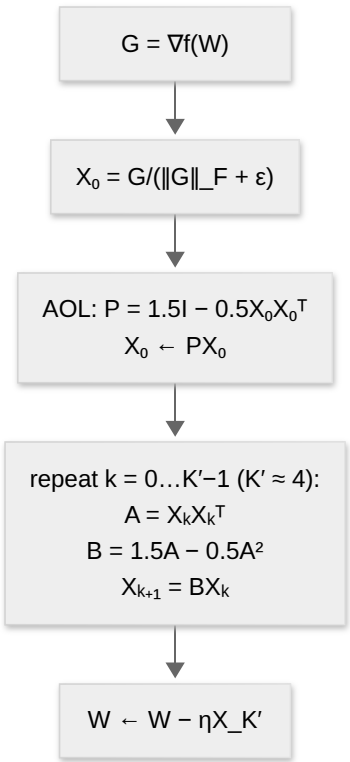



Figure 31: Step flow of Turbo-Muon: almost-orthogonal preconditioning (AOL) before the Newton-Schulz iterations, reducing their count.

**Turbo-Muon (AOL-Preconditioned Orthogonalization).** Turbo-Muon adds an **almost-orthogonal preconditioning** (AOL) step before the Newton-Schulz iterations. The preconditioner $P = 1.5I - 0.5X_0X_0^\top$ is applied to the normalized gradient, reducing the number of required orthogonalization steps to four while preserving the orthogonalized update geometry.

**Algorithm 21** Turbo-Muon (AOL-Preconditioned Orthogonalization)

**Require:** Initial parameters $\theta$ (matrices), learning rate $\eta$, Newton-Schulz iters $K'$ (typically 4)
**Ensure:** Updated parameters $\theta_T$

**for** each 2D parameter matrix $W$ **do**
  $G \leftarrow \nabla f(W)$ ▷ Gradient
  $X_0 \leftarrow \frac{G}{\|G\|_F+\epsilon}$ ▷ Normalize gradient
  **AOL (Almost-Orthogonal preconditioner):** ▷ Preconditioning step
  $P \leftarrow (1.5I - 0.5X_0X_0^\top)$ ▷ Preconditioning matrix
  $X_0 \leftarrow PX_0$ ▷ Preconditioned start
  **for** $k = 0, 1, \ldots, K' - 1$ **do**
    $A_k \leftarrow X_kX_k^\top$
    $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$
    $X_{k+1} \leftarrow B_kX_k$ ▷ Reduced iterations due to AOL preconditioning
  **end for**
  $W \leftarrow W - \eta X_{K'}$ ▷ Update with preconditioned orthogonalized direction
**end forreturn** $\theta_T$

**Cons.**

- Still compute-heavy; AOL adds a preconditioning matrix multiplication.
- Requires the same custom-kernel support as Muon.

**Pros.**

- AOL reduces the Newton-Schulz iteration count (typically five to four).
- Retains orthogonalized update geometry at lower iteration cost.

| **Group** | **Methods** | **Primary Goal** | **Interpretation from the Survey** |
|---|---|---|---|
| Classical baseline | SGD, AdamW, RAdam | stability and reference baselines | These define the comparison floor for newer optimizer claims. |
| Momentum redesign | Adan, AdEMAMix, MARS, Cautious AdamW | faster or safer first-order adaptation | Best when convergence speed or noisy-gradient stability is the main concern. |
| Large-batch and schedule simplification | LAMB, Schedule-Free AdamW | operational robustness at scale | Reduce brittleness from batch-size growth or schedule engineering. |
| Memory-efficient | Adafactor, GaLore, APOLLO, APOLLO-Mini, Q-APOLLO, Lion | optimizer-state reduction | Most useful when VRAM is dominated by optimizer state rather than activations; APOLLO-family methods replace dense AdamW moments with projected or quantized structured scaling. |

| Group | Methods | Primary Goal | Interpretation from the Survey |
|---|---|---|---|
| Curvature-aware | Shampoo, SOAP, Sophia | better conditioning | Prefer when richer geometry is worth implementation and compute overhead. |
| Geometry-oriented | Muon, Turbo-Muon | orthogonalized update structure | Specialized options for matrix geometry and representation shaping. |

# 5 Dense, Recurrent, and Memory-Augmented Transformers

This annex synthesizes the dense, recurrent, and memory-augmented transformer families. The field has evolved toward several primary paradigms: dense global attention with variants, key-value head compression via grouped-query attention, recurrent state compression with decay, selective state-space models, continuous-depth numerical integration, and memory-augmented architectures. Understanding this taxonomy illuminates fundamental tradeoffs between expressiveness, computational cost, memory footprint, and deployment simplicity.

## 5.1 Dense Attention Baselines: Standard and Sigmoid

### 5.1.1 Standard Softmax Attention

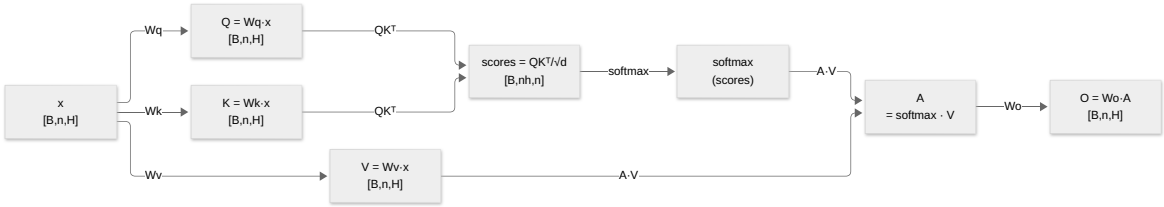



Figure 32: Dataflow of standard softmax attention: parallel Q/K/V projections, scaled dot-product scores, row-wise softmax, and output projection.

Standard multi-head self-attention [97] computes scaled dot-product similarity between token embeddings:

1: **Input:** Query matrix $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key matrix $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value matrix $\mathbf{V} \in \mathbb{R}^{n \times d}$
2: scores $\leftarrow \dfrac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}$ ▷ compute pairwise similarities
3: attention_weights $\leftarrow \texttt{softmax}(\text{scores}, \text{axis} = 1) \in \mathbb{R}^{n \times n}$ ▷ normalize across keys
4: **Output:** $\mathbf{Y} = \text{attention_weights} \cdot \mathbf{V} \in \mathbb{R}^{n \times d}$ ▷ weighted value combination

For autoregressive generation, KV caching stores past keys and values to avoid $\mathcal{O}(n^2)$ recomputation. However, this linearly growing cache occupies $\sim n \cdot d_{\text{hidden}}$ bytes, which can consume hundreds of gigabytes for billion-parameter models. Standard attention achieves **perfect expressiveness** within the context window—any token can attend to any other token with learned weights—but pays the price of dense computation.

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ time, $\mathcal{O}(n^2)$ space (attention matrix materialization).
- **Inference complexity**: $\mathcal{O}(n)$ time per token, $\mathcal{O}(n)$ space (KV cache).
- **Strengths**: Unparalleled expressiveness; perfect history recall; highly parallelizable.
- **Weaknesses**: Quadratic bottleneck prohibits very long contexts; KV cache dominates memory during generation.

### 5.1.2 Sigmoid Attention

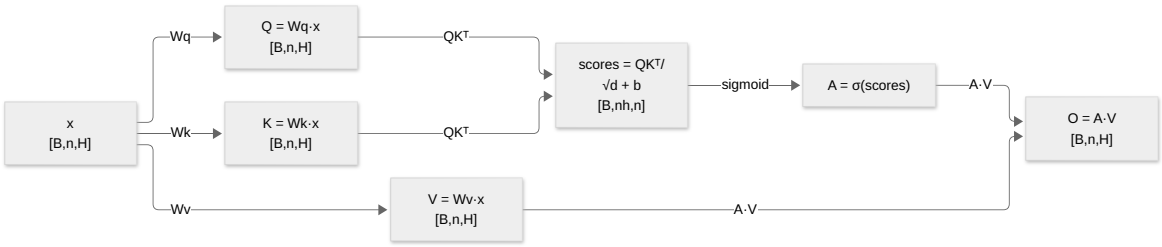



Figure 33: Dataflow of sigmoid attention: an element-wise sigmoid replaces the row-wise softmax, so attention weights need not sum to one.

Sigmoid attention replaces row-wise softmax with element-wise sigmoid activation [82]:

**Input:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ as above, learnable bias $\mathbf{b} \in \mathbb{R}^{n \times n}$
logits $\leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{b}$
attention_weights $\leftarrow \sigma(\text{logits})$ ▷ element-wise sigmoid, not softmax
**Output:** $\mathbf{Y} = \text{attention_weights} \odot \mathbf{V}$

Unlike softmax, sigmoid does not enforce a probability distribution (weights need not sum to 1), enabling stronger token independence. Theoretical analysis via mixture-of-experts shows sigmoid achieves superior sample complexity: $\mathcal{O}(n^{-0.51})$ convergence for ReLU experts versus softmax's $\mathcal{O}(n^{-0.24})$. However, empirical training revealed gradient instabilities at scale. The remedy is **hybrid-norm**—adding normalization after the attention output—which stabilizes gradients without sacrificing the theoretical benefits of element-wise gating.

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (identical to standard), but element-wise operations enable 17% inference speedup via FlashSigmoid.
- **Inference complexity**: $\mathcal{O}(n)$ per token with KV cache (asymptotically same, but lower constant factors).
- **Strengths**: Overcomes zero-sum competition; avoids row-wise synchronization; hardware-friendly implementation.
- **Weaknesses**: Requires careful stabilization (hybrid-norm); training instability at large scales without auxiliary loss.

### 5.1.3 Grouped-Query Attention (GQA)

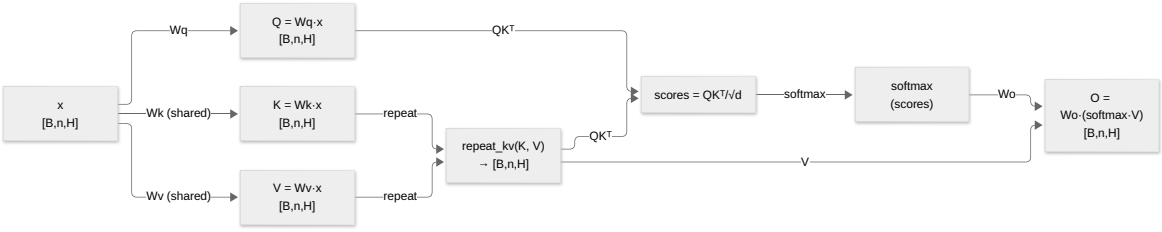



Figure 34: Dataflow of grouped-query attention: keys and values are projected to fewer heads, expanded by repeat-interleave, and shared across each group of query heads.

Grouped-query attention [2] bridges multi-head attention (MHA) and multi-query attention (MQA) by partitioning query heads into groups, each sharing a single key-value head. If $h$ is

the total number of attention heads, $h_k$ is the number of key-value heads, and $G = h/h_k$ is the group size, then every group of $G$ query heads shares one KV head:

**Input:** $\mathbf{Q} \in \mathbb{R}^{n \times h \cdot d_h}$, $\mathbf{K} \in \mathbb{R}^{n \times h_k \cdot d_h}$, $\mathbf{V} \in \mathbb{R}^{n \times h_k \cdot d_h}$
$\mathbf{K}_{\text{full}} \leftarrow \texttt{repeat_interleave}(\mathbf{K}, G)$ ▷ expand from $h_k$ to $h$ heads
$\mathbf{V}_{\text{full}} \leftarrow \texttt{repeat_interleave}(\mathbf{V}, G)$
scores $\leftarrow \frac{\mathbf{Q}\mathbf{K}_{\text{full}}^\top}{\sqrt{d_h}} \in \mathbb{R}^{n \times n}$ ▷ standard dot product after expansion
attention_weights $\leftarrow \texttt{softmax}(\text{scores}, \text{axis} = 1)$
**Output:** $\mathbf{Y} = \text{attention_weights} \cdot \mathbf{V}_{\text{full}} \in \mathbb{R}^{n \times h \cdot d_h}$

GQA interpolates smoothly between MHA ($h_k = h$, $G = 1$) and MQA ($h_k = 1$, $G = h$). The key advantage is a tunable tradeoff between KV-cache footprint and representational capacity. Each additional KV head consumes $2 \cdot d_h \cdot n$ extra bytes in the cache, so reducing $h_k$ yields proportional memory savings.

- **Training complexity**: $\mathcal{O}(n^2 \cdot h \cdot d_h)$ (identical to MHA after head expansion).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(h_k \cdot n \cdot d_h)$ KV-cache space.
- **Strengths**: Flexible head-count tradeoff; proven at scale (Llama 2 & 3 use $h_k = 8$ KV heads with $h = 32$ query heads); reduces KV-cache by $G\times$; supports fast inference with FlashAttention.
- **Weaknesses**: Fewer KV heads can reduce representational capacity in small models; requiring $h_k$ to divide $h$ constrains architectural choices.

## 5.2 Recurrent and Retentive Architectures

### 5.2.1 Retentive Networks (RetNet)

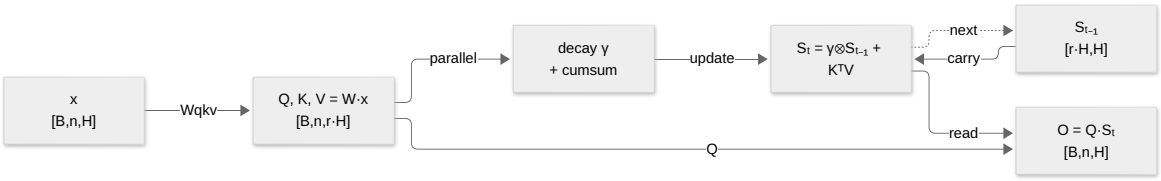



Figure 35: Dataflow of RetNet: parallel Q/K/V projections feed a decayed state recurrence; the state carries across steps and the output reads it with the query.

RetNet [93] unifies three computation paradigms: parallel training, recurrent inference, and chunkwise deployment. Its core innovation is the **retention mechanism**, which uses a fixed exponential decay matrix to model temporal importance:

**Parallel (training) form:**
decay_matrix$[i, j] \leftarrow \gamma^{i-j}$ for $i \geq j$, else 0 ▷ causal exponential decay
decay_matrix$[i, j] \leftarrow 0$ for $i < j$ ▷ causal masking
$\mathbf{Y}_{\text{parallel}} \leftarrow (\mathbf{Q}\mathbf{K}^\top \odot \text{decay_matrix})\mathbf{V}$ ▷ element-wise multiplication with decay
**Recurrent (inference) form:**
**for** $t = 1, \ldots, n$ **do**
    $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$ ▷ state update with decay
    $\mathbf{y}_t \leftarrow \mathbf{q}_t \mathbf{s}_t$ ▷ output via query-state interaction
**end for**

The decay scalar $\gamma \in (0, 1)$ controls the temporal window. RetNet uses **multi-scale retention** with different $\gamma$ values per head (e.g., $\gamma = 1 - 2^{-5}, 1 - 2^{-6}, \ldots$), allowing short-term and long-term

dependencies simultaneously. Chunkwise recurrent mode divides sequences into chunks, processes each chunk in parallel, and threads a recurrent state between chunks.

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (parallel), or $\mathcal{O}(n \cdot c \cdot d)$ (chunkwise recurrent with chunk size $c$).
- **Inference complexity**: $\mathcal{O}(1)$ per token, $\mathcal{O}(d^2)$ state space (fixed matrix).
- **Strengths**: Constant-time inference; triple computation paradigm; multi-scale consolidation.
- **Weaknesses**: Fixed decay imposes rigid inductive bias; may truncate learned long-range patterns.

### 5.2.2 Mamba: Selective State-Space Models

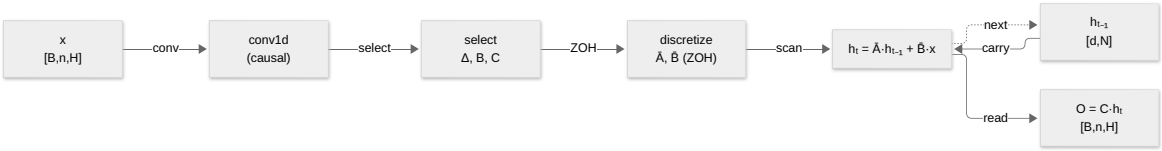



Figure 36: Dataflow of Mamba: a causal convolution feeds input-dependent $\Delta$, $B$, $C$ selections, ZOH discretization, and a selective state-space scan whose state carries across steps.

Mamba [35] frames recurrence as a continuous-time dynamical system with input-dependent parameters, achieving both linear training and constant-time inference:

**Continuous dynamics:** $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t)$
**Discretization with step size $\Delta_t$:**
$\bar{\mathbf{A}}_t \leftarrow \exp(\Delta_t \mathbf{A})$
$\bar{\mathbf{B}}_t \leftarrow (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t$
**Recurrent update:**
**for** $t = 1, \ldots, n$ **do**
    $\Delta_t \leftarrow \text{softplus}(\text{Linear}(x_t))$ ▷ input-dependent step size
    $\mathbf{B}_t \leftarrow \text{Linear}(x_t), \quad \mathbf{C}_t \leftarrow \text{Linear}(x_t)$ ▷ input-dependent projection matrices
    $h_t \leftarrow \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t$ ▷ state transition
    $y_t \leftarrow \mathbf{C}_t h_t$ ▷ output projection
**end for**

The critical innovation is that $\mathbf{A}$ (the system matrix) is *not* input-dependent, but $\Delta_t$, $\mathbf{B}_t$, and $\mathbf{C}_t$ are, making the system **time-varying**. This selectivity allows the model to *ignore* irrelevant information by setting $\Delta_t$ near zero, effectively creating a gate. A hardware-aware parallel scan algorithm implements the recurrence efficiently on GPUs by fusing computation within SRAM, avoiding expensive HBM bandwidth.

- **Training complexity**: $\mathcal{O}(n \cdot d)$ via hardware-aware scan (linear in sequence length).
- **Inference complexity**: $\mathcal{O}(1)$ per token, $\mathcal{O}(d)$ state (hidden vector).
- **Strengths**: Achieves linear training and constant-inference simultaneously; practical on long sequences (millions of tokens).
- **Weaknesses**: State vector compression can weaken exact copying and dense associative recall versus full attention.

## 5.3 Continuous-Depth Transformers: ODE Integration

### 5.3.1 ODE Transformer

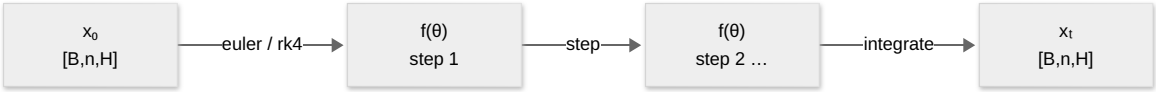



Figure 37: Dataflow of the ODE transformer block: the hidden state is integrated through one or more Euler/RK4 steps before the output is read.

The ODE Transformer [114] interprets network depth as numerical integration of a continuous dynamical system, using higher-order Runge-Kutta solvers to reduce truncation error:

1: **Continuous formulation:** $\frac{dh(t)}{dt} = f_\theta(h(t), t)$ where $f_\theta$ is the transformer sub-network
2: **Runge-Kutta-4 discrete approximation:**
3: $k_1 \leftarrow f_\theta(h_t, t)$
4: $k_2 \leftarrow f_\theta(h_t + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)$
5: $k_3 \leftarrow f_\theta(h_t + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)$
6: $k_4 \leftarrow f_\theta(h_t + k_3, t + \Delta t)$
7: $h_{t+1} \leftarrow h_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ ▷ weighted combination

Instead of simple Euler residual connections $h_{t+1} = h_t + f_\theta(h_t)$, the RK4 block computes four intermediate evaluations and combines them with classical RK4 weights. To avoid vanishing gradients, the architecture introduces **learned gating** that interpolates between intermediate approximations:

1: $g \leftarrow \sigma(\text{Linear}([k_1, k_2, k_3, k_4]))$ ▷ learnable gate
2: $h_{t+1} \leftarrow h_t + g \cdot k_1 + (1 - g) \cdot k_2$ ▷ interpolated refinement

This formulation reduces the effective number of parameters through weight sharing—the same $f_\theta$ is evaluated multiple times—while providing richer trajectory refinement.

- **Training complexity**: $\mathcal{O}(k \cdot n^2 \cdot d)$ where $k$ is the RK order (4 for RK4).
- **Inference complexity**: $\mathcal{O}(k \cdot n)$ per token (higher constant overhead).
- **Strengths**: Significantly higher accuracy on generation tasks (state-of-the-art BLEU); parameter-efficient via weight sharing.
- **Weaknesses**: High per-step compute; inference latency increases by constant factor $k$; complex gating required for stability.

## 5.4 Test-Time Memory: Titans

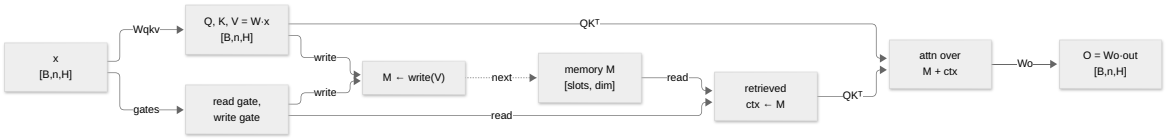



Figure 38: Dataflow of Titans test-time memory: dense Q/K/V attention runs over a local window while read and write gates update an associative memory that carries across steps.

The Titans architecture [7] introduces an orthogonal dimension: instead of static weights, the model maintains a learnable memory that is *updated during inference* based on a surprise-driven signal:

**Local short-term attention:** Apply standard or sparse attention within a fixed context window $c$
$y_t^{\text{local}} \leftarrow \text{Attention}(q_t, k_{[t-c:t]}, v_{[t-c:t]})$
**Memory update signal (surprise):**
$S_t \leftarrow \eta_t S_{t-1} - \theta_t \nabla_\ell(\mathcal{M}_{t-1}; x_t)$ ▷ surprise as gradient of loss w.r.t. memory
$\mathcal{M}_t \leftarrow (1-\alpha_t)\mathcal{M}_{t-1} + S_t$ ▷ memory updated via EMA
**Long-term memory retrieval:**
$y_t^{\text{memory}} \leftarrow \mathcal{M}_t^*(q_t)$ ▷ query memory module for retrieved information
**Output combination:**
$y_t \leftarrow y_t^{\text{local}} + \text{gate}(y_t^{\text{memory}})$ ▷ combine local and retrieved memory

The memory module $\mathcal{M}$ is literally updated during the forward pass by computing gradients of an associative loss and applying SGD steps with momentum. The decay rates $\eta_t, \theta_t, \alpha_t$ are themselves input-dependent, allowing the model to switch memory paradigms when context shifts.

- **Training complexity**: Roughly $\mathcal{O}(c^2 + n \cdot f)$ where $c$ is local window and $f$ is memory update overhead.
- **Inference complexity**: $\mathcal{O}(c^2)$ local attention plus $\mathcal{O}(1)$ memory retrieval per token.
- **Strengths**: Handles extreme context lengths; enables true associative recall; memory adapts to input.
- **Weaknesses**: Significantly more complex; inference includes gradient computation; higher coordination overhead.

### 5.5 Pattern-Driven Mixer Dispatch

The codebase selects a mixer per block from the configured `layer_pattern`. The dispatch logic routes inference-time sparse blocks (`fasa_attn`, `sparge_attn`) to standard dense attention while the model is in training mode, and to their sparse implementation during evaluation; all other blocks route to the corresponding family implementation.

The field exhibits a clear progression along two axes: (i) **computational efficiency**, moving from $O(n^2)$ to $O(n)$ or $O(1)$, and (ii) **memory adaptivity**, shifting from static weights to dynamic, test-time updated models. Standard attention remains the expressiveness baseline; Mamba and RetNet represent the practical efficiency frontier; Titans introduces an orthogonal innovation axis (test-time learning). The choice of architecture reflects the fundamental engineering constraint: expressiveness versus deployment cost. The latent-attention family, which compresses the KV cache into a low-rank latent, is covered in Appendix 6.

## 6 Latent and Low-Rank Attention Families

This annex covers the latent / low-rank KV compression family of attention mechanisms. These architectures compress the per-token key–value state into a low-rank latent vector that is the only quantity materialised in the KV cache during decoding, then reconstruct full multi-head keys and values on the fly. The family unifies grouped-query, latent, factorised, and temporal-latent designs under a common low-rank bottleneck.

**Algorithm 22** Pattern-Driven Mixer Forward (Conceptual)

**Require:** Hidden states $H$, pattern $P$, layer index $\ell$

$m \leftarrow P[\ell \bmod |P|]$
**if** $m \in \{\texttt{fasa_attn}, \texttt{sparge_attn}\}$ and model is in training mode **then**
    $H \leftarrow$ softmax-attention$(H)$ ▷ dense surrogate, same projections
**else if** $m = \texttt{standard_attn}$ **then**
    $H \leftarrow$ softmax-attention$(H)$
**else if** $m = \texttt{sigmoid_attn}$ **then**
    $H \leftarrow$ sigmoid-attention$(H)$
**else if** $m \in \{\texttt{retnet}, \texttt{retnet_attn}\}$ **then**
    $H \leftarrow$ retention$(H)$
**else if** $m = \texttt{mamba}$ **then**
    $H \leftarrow$ selective-ssm$(H)$
**else if** $m = \texttt{ode}$ **then**
    $H \leftarrow$ rk-step$(H)$
**else if** $m$ is a sparse attention key **then**
    $H \leftarrow$ sparse-attention-family$(H)$
**else if** $m$ is a gated attention key **then**
    $H \leftarrow$ gated-attention-family$(H)$
**else**
    $H \leftarrow$ memory-augmented-attn$(H)$
**end if**
**return** $H$

## 6.1 Multi-Head Latent Attention (MLA)

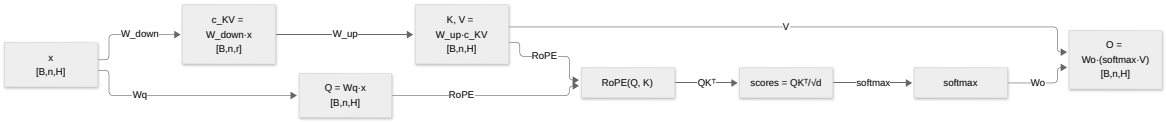



Figure 39: Dataflow of multi-head latent attention: keys and values are down-projected to a low-rank latent, up-projected back, and attended with the query after RoPE.

Multi-Head Latent Attention (MLA) [66] compresses the per-token key–value state into a single low-rank latent vector $c_{KV}$ of rank $r_{kv}$, which is the only quantity materialised in the KV cache during decoding. A pair of up-projection matrices reconstructs the full multi-head keys and values on the fly, while RoPE is applied to the decompressed query and key tensors so that positional structure is preserved. The authors show that the Pareto-optimal latent width sits near $r_{kv} = d_{\text{hidden}}/2$, at which point MLA halves the KV cache yet matches multi-head attention (MHA) quality on small models.

### 6.1.1 Mathematical Formulation

$$c_{KV} = W_{DKV}\, x \in \mathbb{R}^{r_{kv}}, \qquad k = W_{UK}\, c_{KV}, \qquad v = W_{UV}\, c_{KV}, \qquad q = W_Q\, x.$$

$$\tilde{q} = \mathrm{RoPE}(q), \quad \tilde{k} = \mathrm{RoPE}(k), \qquad \text{attn} = \texttt{softmax}\left(\frac{\tilde{q}\,\tilde{k}^\top}{\sqrt{d_h}}\right) v.$$

### 6.1.2 Algorithmic Pseudocode

**Input:** hidden state $x \in \mathbb{R}^{n \times d}$, rank $r_{kv}$, projections $W_{DKV}, W_{UK}, W_{UV}, W_Q$

$c_{KV} \leftarrow W_{DKV}\, x$ ▷ down-project to latent of rank $r_{kv}$ (cached at inference)
$k \leftarrow W_{UK}\, c_{KV}, \quad v \leftarrow W_{UV}\, c_{KV}$ ▷ up-project to full heads
$q \leftarrow W_Q\, x$
$\tilde{q} \leftarrow \mathrm{RoPE}(q), \quad \tilde{k} \leftarrow \mathrm{RoPE}(k)$ ▷ apply rotary embeddings
weights $\leftarrow \texttt{softmax}(\tilde{q}\,\tilde{k}^\top/\sqrt{d_h})$
**Output:** $\mathbf{Y} = \text{weights} \cdot v$

### 6.1.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (full attention on decompressed heads).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ KV-cache space (latent only).
- **Trainable**: fully trainable; $r_{kv}$ is a hyperparameter (Pareto-optimal $\approx d/2$).
- **Strengths**: Halves KV cache versus MHA at matched quality; RoPE-compatible; clean latent bottleneck.
- **Limitations**: Up-projection adds decode FLOPs; latent rank is a fixed global budget shared across heads.

## 6.2 Group-Query Latent Attention (GQLA)

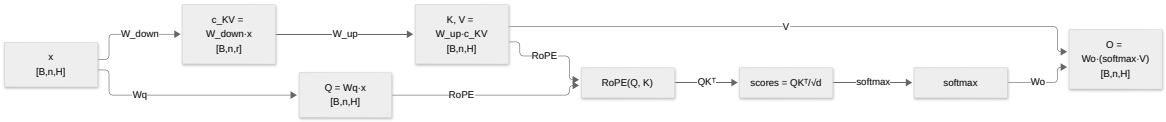



Figure 40: Dataflow of group-query latent attention: the shared latent cache is up-projected to key/value heads and combined with grouped queries.

Group-Query Latent Attention (GQLA) [67] is a minimal modification of MLA that exposes two algebraically equivalent decoding paths over the same trained weights: an MQA-absorb path (fusing the up-projection into the query, optimal on H100-class hardware) and a GQA-expanded path (materialising full keys/values, optimal on H20 / multi-token-prediction hardware). The two paths are mathematically identical because the up-projection is linear, so $\texttt{softmax}(q_{\text{absorbed}} \cdot c_{KV}^\top) = \texttt{softmax}((q \cdot W_{UK}^\top) \cdot c_{KV}^\top)$, allowing hardware-adaptive dispatch with no retraining.

### 6.2.1 Mathematical Formulation

Same latent as MLA:

$$c_{KV} = W_{DKV}\, x, \qquad k = W_{UK}\, c_{KV}, \qquad v = W_{UV}\, c_{KV}, \qquad q = W_Q\, x.$$

$$\text{Path A (MQA-absorb): attn} = \texttt{softmax}\left(\frac{(q\, W_{UK}^\top)\, c_{KV}^\top}{\sqrt{d_h}}\right)(W_{UV}\, c_{KV}).$$

$$\text{Path B (GQA-expanded): attn} = \texttt{softmax}\left(\frac{q\, k^\top}{\sqrt{d_h}}\right) v, \quad k = W_{UK} c_{KV},\ v = W_{UV} c_{KV}.$$

### 6.2.2 Algorithmic Pseudocode

**Input:** $x$, weights $W_{DKV}, W_{UK}, W_{UV}, W_Q$, hardware flag hw $\in \{\text{H100}, \text{H20}\}$
$c_{KV} \leftarrow W_{DKV}\, x$ ▷ shared latent, cached
$q \leftarrow W_Q\, x$
**if** hw == H100 **then** ▷ MQA-absorb path
  $q_{\text{abs}} \leftarrow q\, W_{UK}^{\top}$ ▷ absorb up-projection into query
  weights $\leftarrow \texttt{softmax}(q_{\text{abs}}\, c_{KV}^{\top}/\sqrt{d_h})$
  $\mathbf{Y} \leftarrow \text{weights} \cdot (W_{UV}\, c_{KV})$
**else** ▷ GQA-expanded path
  $k \leftarrow W_{UK}\, c_{KV}, \quad v \leftarrow W_{UV}\, c_{KV}$
  weights $\leftarrow \texttt{softmax}(q\, k^{\top}/\sqrt{d_h})$
  $\mathbf{Y} \leftarrow \text{weights} \cdot v$
**end if**
**Output:** $\mathbf{Y}$

### 6.2.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$, identical to MLA.
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ latent state; path chosen by hardware.
- **Trainable**: trained once; the two decoding paths reuse the same weights.
- **Strengths**: Hardware-adaptive decode with no quality loss; algebraic equivalence guarantees consistency.
- **Limitations**: Same latent-rank tradeoffs as MLA; path selection is a deployment-time heuristic.

## 6.3 Multi-Head Low-Rank Attention (MLRA)

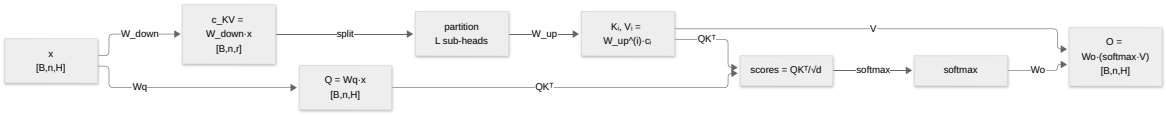



Figure 41: Dataflow of multi-head low-rank attention: the latent is partitioned across sub-heads, each with its own up-projection, before attention.

Multi-Head Low-Rank Attention (MLRA) [61] refactors the single MLA latent into $L$ disjoint sub-heads each of rank $r/L$, so that the KV cache can be partitioned across $L$ devices for 4-way tensor-parallel decoding (each device loads only $1/L$ of the cache). Per-sub-head up-projections reconstruct the corresponding slice of the full heads and are concatenated, yielding a 2.8× decode speedup over MLA while preserving the global latent budget.

### 6.3.1 Mathematical Formulation

$$c_{KV} = [c_1, c_2, \ldots, c_L], \quad c_i \in \mathbb{R}^{r/L}, \qquad c_{KV} = W_{DKV}\, x.$$

$$k_i = W_{UK}^{(i)}\, c_i, \quad v_i = W_{UV}^{(i)}\, c_i, \qquad k = [k_1; \ldots; k_L], \quad v = [v_1; \ldots; v_L].$$

$$\text{attn} = \texttt{softmax}\left(\frac{q\, k^{\top}}{\sqrt{d_h}}\right) v.$$

### 6.3.2 Algorithmic Pseudocode

**Input:** $x$, sub-head count $L$, per-sub-head projections $W_{UK}^{(i)}, W_{UV}^{(i)}$
$c_{KV} \leftarrow W_{DKV}\, x = [c_1, \ldots, c_L]$ ▷ partition latent into $L$ blocks
**for** $i = 1, \ldots, L$ (parallel across $L$ devices) **do**
    $k_i \leftarrow W_{UK}^{(i)}\, c_i, \quad v_i \leftarrow W_{UV}^{(i)}\, c_i$
**end for**
$k \leftarrow \texttt{concat}(k_1, \ldots, k_L), \quad v \leftarrow \texttt{concat}(v_1, \ldots, v_L)$
weights $\leftarrow \texttt{softmax}(q\, k^\top / \sqrt{d_h})$
**Output:** $\mathbf{Y} = \text{weights} \cdot v$

### 6.3.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ (full attention on concatenated heads).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(r_{kv} \cdot n)$ state partitioned across $L$ devices.
- **Trainable**: fully trainable; $L$ controls partition granularity.
- **Strengths**: 2.8× decode speedup; clean 4-way tensor-parallel cache sharding; preserves total latent budget.
- **Limitations**: Requires $L$ to divide $r_{kv}$; more projection matrices than MLA; inter-device all-reduce on output.

## 6.4 Tucker Attention

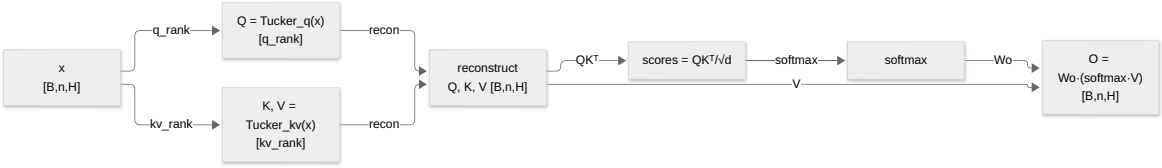



Figure 42: Dataflow of Tucker attention: queries and key/values are projected through Tucker factors and reconstructed before the scaled dot product.

Tucker Attention [53] unifies MHA, GQA and MLA under a single Tucker-style factorisation of the query/key/value weight tensors with independent ranks $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$. MHA is recovered when all ranks equal the hidden size, GQA when $k_{\text{rank}} = v_{\text{rank}} < d$, and MLA when $k_{\text{rank}} = v_{\text{rank}}$ equals the latent rank. Because the factorisation is a pure linear reparameterisation, the method is fully compatible with FlashAttention and RoPE, and yields an order of magnitude fewer parameters for comparable metrics.

### 6.4.1 Mathematical Formulation

$$\mathbf{Q} = W_{Q,\text{core}}\, W_{Q,\text{factor}}\, x, \qquad \mathbf{K} = W_{K,\text{core}}\, W_{K,\text{factor}}\, x, \qquad \mathbf{V} = W_{V,\text{core}}\, W_{V,\text{factor}}\, x.$$

$$\text{attn} = \texttt{softmax}\left(\frac{\mathbf{Q}\,\mathbf{K}^\top}{\sqrt{d_h}}\right)\mathbf{V}.$$

Special cases: MHA $\Leftrightarrow q_{\text{rank}} = k_{\text{rank}} = v_{\text{rank}} = d$; GQA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} < d$; MLA $\Leftrightarrow k_{\text{rank}} = v_{\text{rank}} = r_{kv}$.

### 6.4.2 Algorithmic Pseudocode

**Input:** $x$, ranks $q_{\text{rank}}, k_{\text{rank}}, v_{\text{rank}}$, core/factor matrices
$\mathbf{Q} \leftarrow W_{Q,\text{core}}\, W_{Q,\text{factor}}\, x$ ▷ rank-$q_{\text{rank}}$ factorised query
$\mathbf{K} \leftarrow W_{K,\text{core}}\, W_{K,\text{factor}}\, x$ ▷ rank-$k_{\text{rank}}$ factorised key
$\mathbf{V} \leftarrow W_{V,\text{core}}\, W_{V,\text{factor}}\, x$ ▷ rank-$v_{\text{rank}}$ factorised value
**Apply RoPE to $\mathbf{Q}, \mathbf{K}$ if positional encoding is used**
weights $\leftarrow \texttt{softmax}(\mathbf{Q}\,\mathbf{K}^\top/\sqrt{d_h})$ ▷ FlashAttention-compatible
**Output:** $\mathbf{Y} = \text{weights} \cdot \mathbf{V}$

### 6.4.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d)$ on the reconstructed tensors.
- **Inference complexity**: $\mathcal{O}(n)$ per token; cache size governed by $k_{\text{rank}}$.
- **Trainable**: ranks are hyperparameters; the factorisation is a drop-in weight reparameterisation.
- **Strengths**: Unifies MHA/GQA/MLA; order-of-magnitude parameter reduction at matched metrics; FlashAttention/RoPE compatible.
- **Limitations**: Two matmuls per projection add latency; rank selection is a per-task tuning problem.

## 6.5 Interleaved Head Attention (IHA)

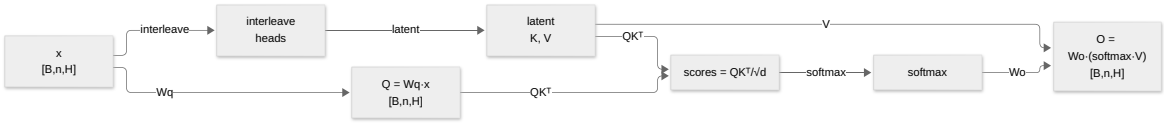



Figure 43: Dataflow of interleaved-head attention: heads are interleaved into a latent key/value representation that is attended by the query.

Interleaved Head Attention (IHA) [25] constructs $P$ pseudo-heads per original head (typically $P = H$) where each pseudo query/key/value is a learned linear combination of all $H$ original heads. This induces up to $P^2$ distinct attention patterns per head at an $\mathcal{O}(H^2 P)$ overhead. Theoretically, IHA needs only $\Theta(\sqrt{k}\, n^2)$ parameters versus $\Theta(k\, n^2)$ for MHA on the Polynomial task; empirically it delivers +10–20% on RULER multi-key retrieval, +5.8% on GSM8K and +2.8% on MATH-500.

### 6.5.1 Mathematical Formulation

$$q_{\text{pseudo}}[p] = \sum_{h=1}^{H} \text{mix}_q[p,h]\, q[h], \quad k_{\text{pseudo}}[p] = \sum_{h} \text{mix}_k[p,h]\, k[h], \quad v_{\text{pseudo}}[p] = \sum_{h} \text{mix}_v[p,h]\, v[h].$$

$$\text{attn}_p = \texttt{softmax}\left(\frac{q_{\text{pseudo}}[p]\; k_{\text{pseudo}}[p]^\top}{\sqrt{d_h}}\right) v_{\text{pseudo}}[p], \qquad \mathbf{Y}[h] = \frac{1}{P}\sum_{p} \text{attn}_p[h].$$

### 6.5.2 Algorithmic Pseudocode

**Input:** per-head $q[h], k[h], v[h]$, mixing matrices $\text{mix}_q, \text{mix}_k, \text{mix}_v \in \mathbb{R}^{P\times H}$
**for** $p = 1, \ldots, P$ **do**
    $q_p \leftarrow \sum_h \text{mix}_q[p,h]\, q[h]$ ▷ learned mix across heads

$k_p \leftarrow \sum_h \text{mix}_k[p,h]\, k[h], \quad v_p \leftarrow \sum_h \text{mix}_v[p,h]\, v[h]$
$\text{attn}_p \leftarrow \texttt{softmax}(q_p\, k_p^\top / \sqrt{d_h})\, v_p$
**end for**
**Output:** $\mathbf{Y}[h] \leftarrow \frac{1}{P} \sum_p \text{attn}_p[h]$ ▷ average pseudo-head outputs per head

### 6.5.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot H \cdot P)$ with $P \approx H$, i.e. $\mathcal{O}(n^2 \cdot H^2)$.
- **Inference complexity**: $\mathcal{O}(n)$ per token with KV cache; extra $\mathcal{O}(H^2 P)$ mixing overhead.
- **Trainable**: mixing matrices are learned; $P$ is a hyperparameter.
- **Strengths**: $\Theta(\sqrt{k})$ parameter efficiency on Polynomial; large gains on retrieval and reasoning benchmarks.
- **Limitations**: Quadratic-in-heads mixing cost; more attention patterns to compute; added latency at large $H$.

## 6.6 Grouped-head laTenT Attention (GTA)

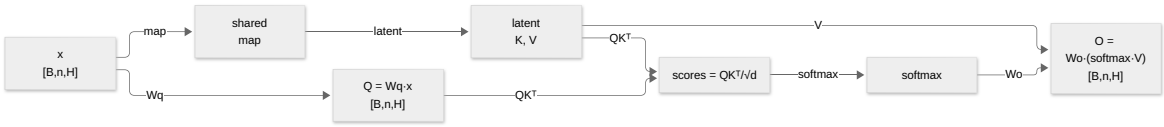



Figure 44: Dataflow of grouped-head latent attention: a shared map produces the latent key/value state consumed by the grouped heads.

Grouped-head laTenT Attention (GTA) [92] combines two compressions: (1) a *shared attention map* — one attention-score tensor per group of heads, reused across all heads in the group, shrinking the key cache; and (2) a *nonlinear value decoder* — the value cache is compressed to a latent by a down-projection and reconstructed by a SiLU-gated up-projection before the output projection. GTA cuts attention FLOPs by up to 62.5% versus GQA, shrinks the KV cache by up to 70%, and yields a 2× end-to-end inference speedup.

### 6.6.1 Mathematical Formulation

$$q_g = \texttt{mean}(q[\text{group } g]), \quad k_g = \texttt{mean}(k[\text{group } g]), \quad \text{attn}_g = \texttt{softmax}\left(\frac{q_g\, k_g^\top}{\sqrt{d_h}}\right) \text{(reused across the group).}$$

$$v_{\text{latent}} = W_{DV}\, x, \qquad v = \text{SiLU}(W_{UV}\, v_{\text{latent}}), \qquad \mathbf{Y} = \text{attn}_g \cdot v.$$

### 6.6.2 Algorithmic Pseudocode

**Input:** $q, k$ per head, hidden $x$, group partition $\{g\}$
$v_{\text{latent}} \leftarrow W_{DV}\, x$ ▷ compress values to latent (cached)
$v \leftarrow \text{SiLU}(W_{UV}\, v_{\text{latent}})$ ▷ nonlinear value decode
**for** each group $g$ **do**
  $q_g \leftarrow \texttt{mean}(q[g]), \quad k_g \leftarrow \texttt{mean}(k[g])$ ▷ shared per-group queries/keys
  $\text{attn}_g \leftarrow \texttt{softmax}(q_g\, k_g^\top / \sqrt{d_h})$
  $\mathbf{Y}[g] \leftarrow \text{attn}_g \cdot v[g]$ ▷ one map reused for all heads in $g$
**end for**
**Output:** $\mathbf{Y}$

### 6.6.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot G \cdot d_h)$ where $G$ is the number of groups (vs $H$ heads for GQA).
- **Inference complexity**: $\mathcal{O}(n)$ per token; value cache $\mathcal{O}(r_v \cdot n)$, key cache reduced by group sharing.
- **Trainable**: group assignment and latent rank are trainable/hyperparameters.
- **Strengths**: up to 62.5% attention-FLOP reduction, 70% KV-cache shrink, 2× end-to-end speedup.
- **Limitations**: Shared map reduces per-head diversity; nonlinear decoder adds latency; grouping must be chosen carefully.

## 6.7 Multi-head Temporal Latent Attention (MTLA)

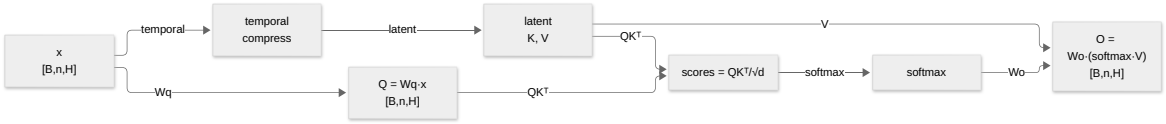



Figure 45: Dataflow of multi-head temporal latent attention: a temporal compression stage produces the latent key/value state attended by the query.

Multi-head Temporal Latent Attention (MTLA) [20] extends MLA along the temporal axis: a hyper-network dynamically merges $m$ temporally adjacent KV-cache vectors into a single slot, shrinking the temporal length by a factor of $m$. A stride-aware causal mask keeps parallel training consistent with autoregressive inference, yielding a 5.3× decode speedup and 8.3× GPU-memory reduction versus MHA on En–De speech translation.

### 6.7.1 Mathematical Formulation

$$c_{KV}^{(t)} = W_{DKV}\, x^{(t)} \in \mathbb{R}^{r_{kv}} \text{ (per-token latent).}$$

$$\bar{c}_{KV}^{(j)} = \sum_{i=0}^{m-1} \alpha_i\, c_{KV}^{(sj+i)}, \qquad \alpha_i = \texttt{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)})),$$

where $s$ is the stride and $m$ the merge window. Queries attend over merged slots $\bar{c}_{KV}$ with a stride-aware causal mask $M_{\text{stride}}$:

$$\text{attn} = \texttt{softmax}\left(\frac{q\,\bar{k}^\top}{\sqrt{d_h}} + M_{\text{stride}}\right)\bar{v}, \quad \bar{k} = W_{UK}\,\bar{c}_{KV},\ \bar{v} = W_{UV}\,\bar{c}_{KV}.$$

### 6.7.2 Algorithmic Pseudocode

**Input:** per-token latents $c_{KV}^{(t)}$, merge window $m$, stride $s$
**Merge phase (hyper-network):**
**for** each merged slot $j = 0, 1, \ldots$ **do**
  $\alpha_i \leftarrow \texttt{sigmoid}(\text{HyperNet}(c_{KV}^{(sj+i)}))$ for $i = 0, \ldots, m-1$
  $\bar{c}_{KV}^{(j)} \leftarrow \sum_{i=0}^{m-1} \alpha_i\, c_{KV}^{(sj+i)}$ ▷ gated average over window
**end for**
$\bar{k} \leftarrow W_{UK}\,\bar{c}_{KV}, \quad \bar{v} \leftarrow W_{UV}\,\bar{c}_{KV}$
weights $\leftarrow \texttt{softmax}(q\,\bar{k}^\top/\sqrt{d_h} + M_{\text{stride}})$ ▷ stride-aware causal mask
**Output:** $\mathbf{Y}$ = weights $\cdot\, \bar{v}$

### 6.7.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d/m)$ on the merged sequence (length $\lceil n/m \rceil$) plus hyper-network cost.
- **Inference complexity**: $\mathcal{O}(n/m)$ per token over merged slots; $\mathcal{O}(r_{kv} \cdot n/m)$ state.
- **Trainable**: hyper-network and merge window $m$ are learned/chosen; stride-aware mask enforces consistency.
- **Strengths**: 5.3× decode speedup, 8.3× GPU-memory reduction vs MHA; temporal compression orthogonal to rank compression.
- **Limitations**: Merge window can blur fine temporal detail; hyper-network adds compute; mask design is non-trivial.

## 6.8 Architectural Comparison and Synthesis

## 6.9 Compressed Convolutional Attention (CCA)

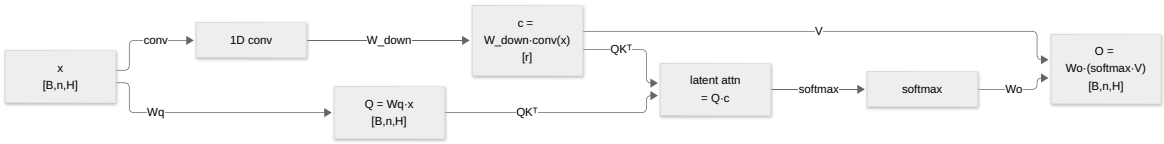



Figure 46: Dataflow of compressed convolutional attention: a 1D convolution feeds a latent compression $c$, which the query attends directly.

Compressed Convolutional Attention (CCA) [29] takes a fundamentally different approach to latent attention: rather than compressing the KV-cache and then up-projecting to full width before attending (as MLA/GQLA/MLRA do), CCA down-projects queries, keys, and values into a shared latent and performs the *entire* attention operation inside it. There are no Q/K/V up-projection matrices—only a single output up-projection $\tilde{W}_O$ maps the latent output back to the residual stream. This simultaneously reduces parameters, KV-cache size, **and** attention FLOPs by the compression factor $C = E/\tilde{e}$, whereas MLA only shrinks the cache.

To make attention in the fully compressed latent viable, CCA introduces three innovations: (1) two causal convolutions on the packed q/k tensor (a depth-wise sequence convolution and a head-wise grouped channel convolution), (2) q-k-mean (adds the pre-convolution mean of q and k to the post-convolution values), and (3) value-shift (each head receives half its values from the current token and half from the previous token via two independent projections). After these steps, QK L2-normalisation and a learnable key temperature $\beta$ are applied, RoPE is applied *directly in the latent* (no separate RoPE head needed), and standard softmax attention is computed.

### 6.9.1 Mathematical Formulation

$$\tilde{q}_{\text{pre}} = \tilde{W}_Q\, x, \qquad \tilde{k}_{\text{pre}} = \tilde{W}_K\, x, \qquad \tilde{e} = E/C.$$

$$\tilde{q} = \text{Conv}(\tilde{q}_{\text{pre}}) + \tfrac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}), \qquad \tilde{k} = \text{Conv}(\tilde{k}_{\text{pre}}) + \tfrac{1}{2}(\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}}).$$

$$\tilde{v} = [\tilde{W}_{V_1}\, x_t \parallel \tilde{W}_{V_2}\, x_{t-1}], \qquad \hat{q} = \frac{\tilde{q}\sqrt{d_h}}{\|\tilde{q}\|}, \quad \hat{k} = \frac{\tilde{k}\sqrt{d_h}}{\|\tilde{k}\|} e^{\beta}.$$

$$\text{attn} = \texttt{softmax}\left(\frac{\text{RoPE}(\hat{q})\,\text{RoPE}(\hat{k})^\top}{\sqrt{d_h}}\right)\tilde{v}, \qquad \mathbf{Y} = \tilde{W}_O\,\text{attn}.$$

### 6.9.2 Algorithmic Pseudocode

**Input:** hidden state $x \in \mathbb{R}^{n \times E}$, compression factor $C$, projections $\tilde{W}_Q, \tilde{W}_K, \tilde{W}_{V_1}, \tilde{W}_{V_2}, \tilde{W}_O$
$\tilde{e} \leftarrow E/C$ ▷ latent width
$\tilde{q}_{\text{pre}} \leftarrow \tilde{W}_Q\, x, \quad \tilde{k}_{\text{pre}} \leftarrow \tilde{W}_K\, x$ ▷ down-project to latent
$\tilde{q}_{\text{conv}} \leftarrow \text{ConvSeq}(\tilde{q}_{\text{pre}}), \quad \tilde{k}_{\text{conv}} \leftarrow \text{ConvSeq}(\tilde{k}_{\text{pre}})$ ▷ causal depth-wise + grouped convs
$\text{qk_mean} \leftarrow (\tilde{q}_{\text{pre}} + \tilde{k}_{\text{pre}})/2$ ▷ q-k-mean bias
$\tilde{q} \leftarrow \tilde{q}_{\text{conv}} + \text{qk_mean}, \quad \tilde{k} \leftarrow \tilde{k}_{\text{conv}} + \text{qk_mean}$
$\tilde{v}_1 \leftarrow \tilde{W}_{V_1}\, x_t, \quad \tilde{v}_2 \leftarrow \tilde{W}_{V_2}\, x_{t-1}$ ▷ value-shift
$\tilde{v} \leftarrow [\tilde{v}_1 \parallel \tilde{v}_2]$ ▷ concat half-heads
$\hat{q} \leftarrow \tilde{q}\,\sqrt{d_h}/\|\tilde{q}\|, \quad \hat{k} \leftarrow \tilde{k}\,\sqrt{d_h}/\|\tilde{k}\| \cdot e^{\beta}$ ▷ QK-norm + temperature
$\hat{q} \leftarrow \text{RoPE}(\hat{q}), \quad \hat{k} \leftarrow \text{RoPE}(\hat{k})$ ▷ RoPE in latent
$\text{weights} \leftarrow \texttt{softmax}(\hat{q}\,\hat{k}^{\top}/\sqrt{d_h})$
**Output:** $\mathbf{Y} = \tilde{W}_O\,(\text{weights} \cdot \tilde{v})$

### 6.9.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d/C)$ (attention in compressed latent, $C\times$ fewer FLOPs than MHA).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(\tilde{e} \cdot n)$ KV-cache space.
- **Trainable**: $C$, conv kernel sizes, and all three tricks (convs, qk-mean, value-shift) are configurable.
- **Strengths**: Reduces params, cache, AND FLOPs simultaneously; RoPE integrates natively (no RoPE head); >2× fewer params than MLA at same compression.
- **Limitations**: Does not remove quadratic $S^2$ (only divides by $C$); conv/qk-mean/v-shift add inductive bias; fused kernel needed for theoretical speedup.

## 6.10 Compressed Convolutional Grouped Query Attention (CCGQA)

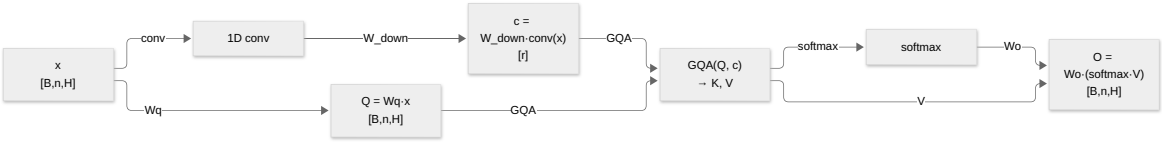



Figure 47: Dataflow of compressed convolutional grouped-query attention: the convolution latent supplies a grouped-query key/value expansion.

Compressed Convolutional Grouped Query Attention (CCGQA) [29] extends CCA with GQA-style key/value head sharing applied *inside* the compressed latent, and **decouples** the query and KV compression rates: queries are compressed by $C_1$ and keys/values by $C_2 \geq C_1$. The per-head latent dimension $d_h$ must match between query and key heads, enforcing the constraint $C_2/C_1 = n_h/n_{kv}$. KV heads are shared per group (replicated to match query heads), and the qk-mean bias uses broadcast/reduce operators $B_{\text{group}}$ (replicate KV heads to query heads) and $E_{\text{group}}$ (average query heads within each group). CCGQA achieves the best reported loss at 8× KV-cache compression with no quality drop versus MHA.

### 6.10.1 Mathematical Formulation

Same pipeline as CCA, with decoupled latents:

$$\tilde{q}_{\text{pre}} = \tilde{W}_Q\, x \in \mathbb{R}^{E/C_1}, \qquad \tilde{k}_{\text{pre}} = \tilde{W}_K\, x \in \mathbb{R}^{E/C_2}, \qquad C_2 \geq C_1.$$

$$\text{qk_mean}_q = \tfrac{1}{2}(\tilde{q}_{\text{pre}} + B_{\text{group}}(\tilde{k}_{\text{pre}})), \qquad \text{qk_mean}_k = E_{\text{group}}(\text{qk_mean}_q).$$

$$\text{attn} = \texttt{softmax}\left(\frac{\text{RoPE}(\hat{q})\,\text{RoPE}(\hat{k})^\top}{\sqrt{d_h}}\right)\tilde{v}, \qquad \mathbf{Y} = \tilde{W}_O\,\text{attn}.$$

### 6.10.2 Key Characteristics

- **Training complexity**: $\mathcal{O}(n^2 \cdot d/C_1)$ ($S^2$ term scales by $1/C_1$; projection terms by $1/C_1 + 1/C_2$).
- **Inference complexity**: $\mathcal{O}(n)$ per token, $\mathcal{O}(\tilde{e}_{kv} \cdot n)$ KV-cache space ($C_2$-compressed, group-shared).
- **Trainable**: $C_1$, $C_2$, $n_{kv}$, and all CCA tricks are configurable. Constraint: $C_2/C_1 = n_h/n_{kv}$.
- **Strengths**: Decoupled compression enables smooth Pareto frontier; same arithmetic intensity as GQA; best loss at 8× cache reduction.
- **Limitations**: Same quadratic $S^2$ (only reduced by $C_1$); fused kernel needed; two value projections add complexity.

The latent family shows a clear progression along two axes: (i) **rank compression**, moving from full heads (MHA) to a shared latent (MLA) to partitioned sub-head latents (MLRA), and (ii) **deployment adaptivity**, moving from a single decode path to hardware-adaptive dispatch (GQLA) and temporal cache merging (MTLA). Tucker Attention provides the unifying algebraic frame: MHA, GQA, and MLA are all special cases of a Tucker factorisation with appropriate rank choices. CCA and CCGQA take a fundamentally different approach: rather than compressing the KV-cache and then up-projecting for full-width attention (MLA/GQLA/MLRA), they perform attention *entirely inside* the compressed latent space, reducing not just the cache but also the attention FLOPs by the compression factor. CCGQA further decouples query and KV compression rates, achieving the best reported loss at 8× KV-cache compression. Gaussian Mixture Attention (GMA) changes the routing philosophy entirely: instead of a low-rank linear bottleneck, it routes through $K$ learned probabilistic Gaussian components, achieving linear-time $\mathcal{O}(NK)$ sequence mixing for fixed $K$ while retaining a normalised attention-style interpretation.

## 6.11 Gaussian Mixture Attention (GMA)

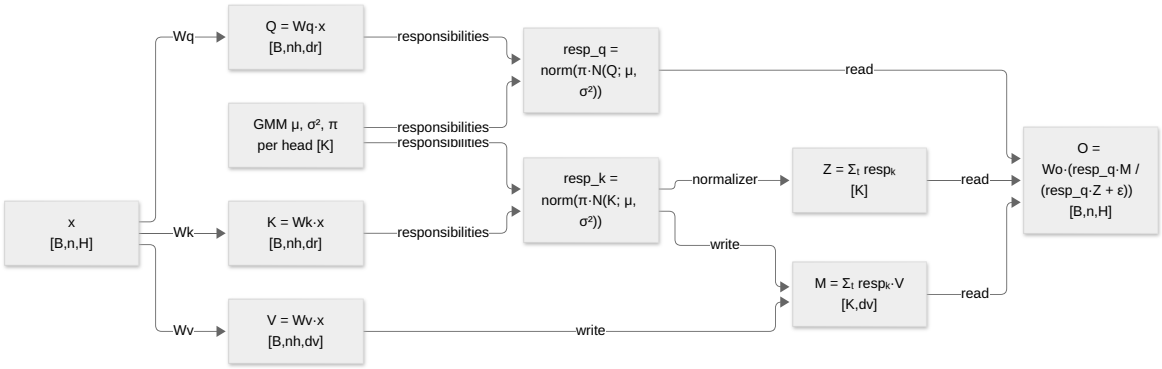



Figure 48: Dataflow of Gaussian-mixture attention: per-head Gaussian responsibilities route keys and values into a $K$-slot memory that the query reads back, normalized by $Z$.

Gaussian Mixture Attention (GMA) [46] replaces the explicit pairwise $QK^\top$ interaction of standard dot-product attention with probabilistic routing through $K$ learned Gaussian-mixture components per head. Queries and keys are mapped to posterior *responsibility* vectors over a shared latent routing space; their overlap defines an implicit responsibility-space affinity, while values are written into and read from a $K$-slot latent memory. By exploiting the associativity of matrix multiplication, GMA avoids materialising the induced $N \times N$ affinity matrix and instead uses two $N \times K$ responsibility matrices whose dominant activation storage scales as $\mathcal{O}(NK)$ rather than $\mathcal{O}(N^2)$ for fixed $K$. The Gaussian mixture parameters—component means $\mu$, diagonal covariances (reparameterised via $\sigma^2 = \text{softplus}(\omega) + \epsilon_\sigma$) and mixture-prior logits $\alpha$ (with $\pi = \text{softmax}(\alpha)$)—are fully differentiable learnable parameters optimised by standard backpropagation, with no EM loop or auxiliary clustering loss.

### 6.11.1 Mathematical Formulation

Per head, with routing dimension $d_r$ and value dimension $d_v$:

$$Q_X = XW_Q \in \mathbb{R}^{N\times d_r}, \quad K_X = XW_K \in \mathbb{R}^{N\times d_r}, \quad V_X = XW_V \in \mathbb{R}^{N\times d_v}.$$

The posterior responsibility of component $k$ for a routing vector $x_i$ is

$$\gamma_{i,k} = p(z{=}k \mid x_i) = \frac{\pi_k \,\mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{\ell=1}^{K} \pi_\ell \,\mathcal{N}(x_i \mid \mu_\ell, \Sigma_\ell)}, \qquad \Sigma_k = \text{diag}(\sigma_{k,1}^2, \ldots, \sigma_{k,d_r}^2),$$

yielding responsibility matrices $\Gamma^Q, \Gamma^K \in \mathbb{R}^{N\times K}$. The bidirectional output is computed by the associative write–read pair

$$\tilde{V} = (\Gamma^K)^\top V_X \in \mathbb{R}^{K\times d_v}, \qquad Z = (\Gamma^K)^\top \mathbf{1}_N \in \mathbb{R}^K, \qquad O = \frac{\Gamma^Q \tilde{V}}{\Gamma^Q Z + \epsilon},$$

which never materialises the implicit normalised affinity $A_{ij}^{\text{GMA}} = \frac{\sum_k \gamma_{i,k}^Q \gamma_{j,k}^K}{\sum_\ell \sum_k \gamma_{i,k}^Q \gamma_{\ell,k}^K + \epsilon}$. For autoregressive modelling the global write statistics are replaced by prefix-restricted cumulative sums $\tilde{V}_k^{(i)} = \sum_{j\le i} \gamma_{j,k}^K V_{X,j}$ and $Z_k^{(i)} = \sum_{j\le i} \gamma_{j,k}^K$, enforcing causality while preserving the fixed-$K$ linear scaling.

### 6.11.2 Algorithmic Pseudocode

**Input:** hidden state $X \in \mathbb{R}^{N\times d}$, $K$ components, routing dim $d_r$, value dim $d_v$, params $\{\mu_k, \omega_k, \alpha_k\}_{k=1}^K$
$Q_X \leftarrow XW_Q, \quad K_X \leftarrow XW_K, \quad V_X \leftarrow XW_V$ ▷ head projections
$\sigma_k^2 \leftarrow \text{softplus}(\omega_k) + \epsilon_\sigma, \quad \pi_k \leftarrow \text{softmax}(\alpha)_k$ ▷ reparameterise
$\Gamma^Q \leftarrow \texttt{softmax}_k\!\left(\log \pi_k - \frac{d_r}{2}\log 2\pi - \frac{1}{2}\sum_m \log \sigma_{k,m}^2 - \frac{1}{2}\sum_m \frac{(q_{i,m}-\mu_{k,m})^2}{\sigma_{k,m}^2}\right)$
$\Gamma^K \leftarrow$ same as $\Gamma^Q$ applied to $K_X$
**if** causal **then**
    $\tilde{V} \leftarrow \texttt{cumsum}_n(\Gamma^K \otimes V_X), \quad Z \leftarrow \texttt{cumsum}_n(\Gamma^K)$ ▷ prefix-restricted
**else**
    $\tilde{V} \leftarrow (\Gamma^K)^\top V_X, \quad Z \leftarrow (\Gamma^K)^\top \mathbf{1}_N$ ▷ global write
**end if**
$O \leftarrow \Gamma^Q \tilde{V} \,/\, (\Gamma^Q Z + \epsilon)$ ▷ normalised read
**Output:** $\mathbf{Y} = OW_O$

### 6.11.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(NKd_r + NKd_v)$ for fixed $K$, linear in the sequence length $N$.
- **Inference complexity**: $\mathcal{O}(K)$ per token (responsibilities + latent read), $\mathcal{O}(NK)$ activation storage.
- **Trainable**: fully differentiable GMM parameters $\{\mu, \omega, \alpha\}$ per head, optimised end-to-end without EM.
- **Strengths**: Fixed-$K$ linear-time sequence mixing; probabilistic interpretable routing; bidirectional and causal variants; non-negative low-rank affinity interpretation $A^{\text{GMA}} = \text{diag}(d)^{-1}\Gamma^Q(\Gamma^K)^\top$.
- **Limitations**: Quality depends on $K$ and the routing dimension $d_r$; causal GMA trails optimised SDPA and state-space models on WikiText-103 in the paper's current implementation; not a universal replacement for softmax attention.

# 7 Comprehensive Sparse Attention Mechanisms

## 7.1 Executive Summary

Sparse attention mechanisms address the prohibitive $\mathcal{O}(n^2)$ computational and memory complexity of standard scaled dot-product attention by restricting the set of attended key-value pairs. Modern sparse attention spans a rich design space distinguished by four orthogonal dimensions: (i) **sparsity pattern** (fixed geometric, data-dependent, or frequency-based), (ii) **trainability** (end-to-end trained or inference-only), (iii) **sparsity unit** (individual tokens, local windows, or blocks), and (iv) **mechanism** (masking, selection, filtering, or compression). This annex synthesizes ten major sparse attention families—Sparse Transformer, Longformer, SWA with attention sinks, BigBird, SparseK, NSA, FASA, SpargeAttn, MSA, and SparDA—providing mathematical formulations, algorithmic pseudocode, and comprehensive architectural comparisons.

## 7.2 Sparse Transformer: Factorized Strided and Fixed Patterns

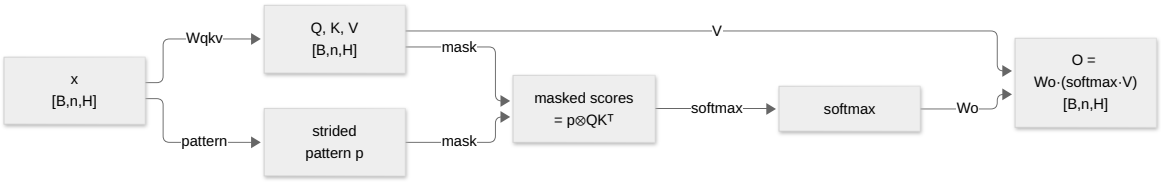



Figure 49: Dataflow of the Sparse Transformer: a strided factorized pattern masks the score matrix before softmax.

### 7.2.1 Mathematical Formulation

The Sparse Transformer [16] reduces quadratic complexity to $\mathcal{O}(n\sqrt{n})$ by factorizing sparse attention into two complementary sparse heads. Let $n$ be the sequence length and $l = \lfloor \sqrt{n} \rfloor$ be the stride.

**Strided attention head** : Each position $i$ attends to every $l$-th previous position:

$$\mathcal{A}_i^{(\text{strided})} = \{j : j \leq i,\ (i-j) \bmod l = 0\}$$

**Fixed attention head** : Each position $i$ attends to positions within its local block plus fixed summary columns:

$$\mathcal{A}_i^{(\text{fixed})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \bmod l \in \{l-c, \ldots, l-1\}\}$$

where $c$ is a hyperparameter controlling the number of summary columns (typically $c = 1$ or $c = 2$).

The attention computation for each head $h$ follows standard scaled dot-product rules restricted to the sparse set:

$$\text{Attn}_h(Q, K, V)_i = \text{softmax}\left(\frac{q_i^h {K_{\mathcal{A}_i}^h}^\top}{\sqrt{d_k}}\right) V_{\mathcal{A}_i}^h$$

The key insight is that with the two factorized heads operating in parallel across a depth of $L$ transformer layers, every position can reach every other position through a path of length at most $L + 1$, preserving long-range reachability with a fraction of dense attention cost.

### 7.2.2 Algorithmic Pseudocode

**Algorithm 23** Sparse Transformer Attention: Strided + Fixed Dual Heads

**Require:** Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value $\mathbf{V} \in \mathbb{R}^{n \times d}$
**Require:** Stride $l = \lfloor \sqrt{n} \rfloor$, summary columns $c \in \{1, 2\}$
**Ensure:** Output $\mathbf{Y} \in \mathbb{R}^{n \times d}$

**Head 1: Strided Attention**
**for** $i = 1, \ldots, n$ **do**
  $\mathcal{A}_i \leftarrow \{j : j \le i \text{ and } (i - j) \bmod l = 0\}$ ▷ Every $l$-th position
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i^{(1)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
**end for**
**Head 2: Fixed Block + Summary Attention**
**for** $i = 1, \ldots, n$ **do**
  $\text{block_start} \leftarrow \lfloor i/l \rfloor \cdot l$, $\text{block_end} \leftarrow \min(i + 1, \text{block_start} + l)$
  $\mathcal{A}_i \leftarrow [\text{block_start}, \text{block_end}) \cup \{\text{cols from last } c \text{ columns}\}$
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i^{(2)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
**end for**
Concatenate: $\mathbf{Y} = [\mathbf{y}^{(1)} \| \mathbf{y}^{(2)}]$ and project via output linear layer **return** $\mathbf{Y}$

### 7.2.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n\sqrt{n} \cdot d)$ during training; $\mathcal{O}(n)$ per token during generation.
- **Trainable**: Yes; full backpropagation through selected positions.
- **Sparsity pattern**: Fixed and data-agnostic; patterns do not adapt to content.
- **Strength**: Structured reachability guarantees long-range dependencies; proven effective on long sequences (Enwik8, text images).
- **Limitation**: Fixed patterns may miss important but non-contiguous relationships; requires custom CUDA kernels for practical efficiency.

## 7.3 Longformer: Sliding Window, Dilation, and Global Tokens

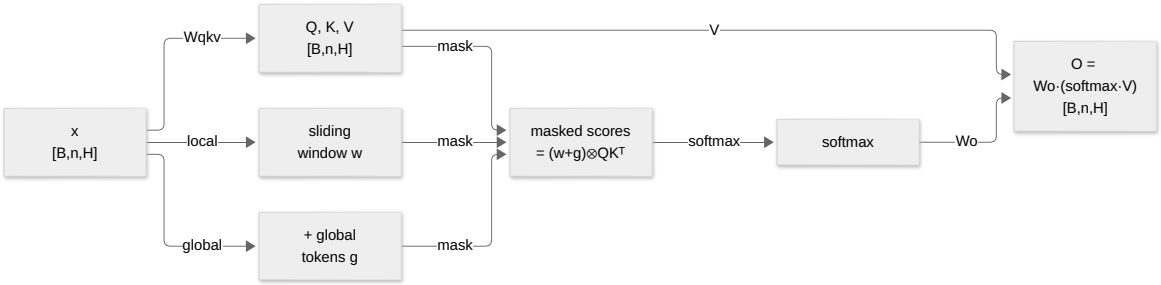



Figure 50: Dataflow of Longformer: sliding-window and global-token masks are applied to the score matrix before softmax.

### 7.3.1 Mathematical Formulation

Longformer [8] achieves linear $\mathcal{O}(n \cdot w)$ complexity by combining three complementary attention patterns.

**Sliding window attention** : Local neighborhood of size $w$:

$$\mathcal{A}_i^{(\text{window})} = \{j : |i - j| \leq w/2\}$$

**Dilated sliding window** : Windowed attention with dilation $d$, skipping stride $d + 1$:

$$\mathcal{A}_i^{(\text{dilated})} = \{j : |i - j| \leq (w/2)(d + 1) \text{ and } (i - j) \bmod (d + 1) = 0\}$$

**Global attention** : Designated global tokens (e.g., [CLS]) attend to all positions and are attended by all:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \ldots, n\} & \text{if } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{window})} \cup \mathcal{G} & \text{otherwise} \end{cases}$$

The three patterns are applied via separate projections. The local and global attention can use either the same projections or task-specific separate ones.

### 7.3.2 Algorithmic Pseudocode

**Algorithm 24** Longformer: Sliding Window + Dilated + Global

**Require:** Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, window size $w$, dilation $d$, global token indices $\mathcal{G}$
**Ensure:** Output $\mathbf{Y}$

**for** $i = 1, \ldots, n$ **do**
  **if** $i \in \mathcal{G}$ **then**
    $\mathcal{A}_i \leftarrow \{1, \ldots, n\}$ ▷ Global token attends to all
  **else**
    Local window: $\mathcal{A}^{(\text{local})} \leftarrow [i - w/2, i + w/2] \cap [1, n]$
    Dilated offsets: $\mathcal{A}^{(\text{dilated})} \leftarrow \{j : (i - j) \bmod (d + 1) = 0\} \cap \mathcal{A}^{(\text{dilated range})}$
    Combine: $\mathcal{A}_i \leftarrow \mathcal{A}^{(\text{local})} \cup \mathcal{A}^{(\text{dilated})} \cup \mathcal{G}$
  **end if**
  $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
  $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
  $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
**end forreturn** $\mathbf{Y}$

### 7.3.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n \cdot w)$ where $w$ is the window size (linear when $w \ll n$).
- **Trainable**: Yes; drop-in replacement for standard attention.
- **Coverage**: With $L$ layers and window size $w$, receptive field grows to $L \times w$, covering entire sequence with shallow networks.
- **Strength**: Practical for document-level tasks; dilation expands local receptive fields with minimal overhead; global tokens provide query-document correspondence.
- **Limitation**: Window size remains a design hyperparameter; suboptimal for tasks requiring dense attention patterns.

## 7.4 SWA with Attention Sinks: Sliding Window Plus a Sink Prefix

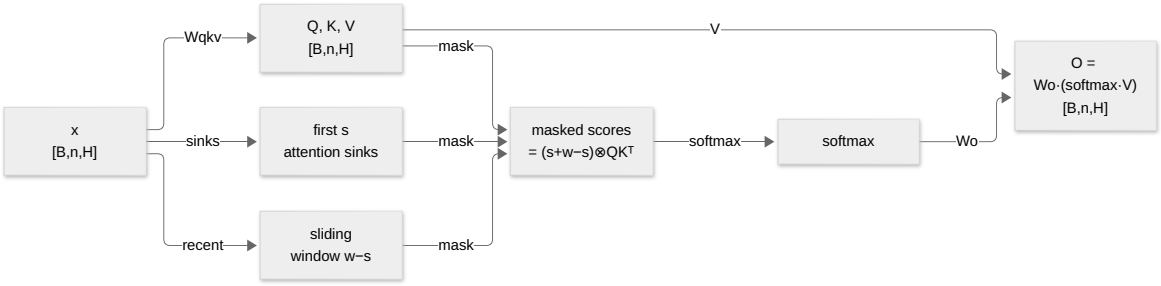



Figure 51: Dataflow of SWA with attention sinks: the first $s$ sink tokens and a causal $w - s$ sliding window are unioned into a mask applied to the score matrix before softmax.

### 7.4.1 Mathematical Formulation

SWA with attention sinks [47] is the simplest KV-cache eviction policy: instead of compressing the cache or replacing attention with a recurrent state, it restricts every query to a small, fixed set of keys. Write the window size as $w$ and the number of sinks as $s$, i.e. SWA$(w, s)$. In decoder mode, query $i$ attends to

$$\mathcal{A}_i = \underbrace{\{j : j < s,\ j \le i\}}_{\text{attention sinks}} \cup \underbrace{\{j : i - (w - s) + 1 \le j \le i\}}_{\text{sliding window of } w-s},$$

so each query sees at most $w$ keys. The causal clip $j \le i$ on the sink term is essential: tokens before position $s$ must not see future sinks.

**Why sinks are indispensable** Transformers learn to assign disproportionately large attention mass to the first few tokens even when they are not semantically relevant. These *attention sinks* [102] act as repositories for surplus attention. Once the prefix leaves a sink-free sliding window, the model loses its dump target and quality collapses catastrophically; re-including the first $s$ tokens restores most of full-attention performance. The paper fixes $s = 4$ and evaluates $w \in \{64, 128, 256, 512\}$.

**Encoder extension** The paper defines SWA$(w, s)$ for causal language models. For bidirectional encoders the local window is made symmetric, $\{j : |i - j| \le \lfloor (w - s)/2 \rfloor\}$, and unioned with the sink prefix—the same convention used by Longformer.

Because the intervention is a pure attention-mask change, it requires no additional parameters and no new QKV math: the standard projections $W_q, W_k, W_v, W_o$ are reused. It can be applied to a pretrained checkpoint at inference time with zero training tokens, or trained from scratch as a normal mixer.

### 7.4.2 Algorithmic Pseudocode

**Algorithm 25** SWA with Attention Sinks: Mask Construction and Attention

**Require:** Query $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, window $w$, sinks $s$
**Ensure:** Output $\mathbf{Y} \in \mathbb{R}^{n \times d}$

Row indices $i$, column indices $j$
causal $\leftarrow [j \le i]$
sinks $\leftarrow [j < s] \wedge$ causal ▷ Causal-clipped sink prefix
window $\leftarrow [i - (w - s) + 1 \le j \le i]$
$M \leftarrow$ sinks $\vee$ window
$S \leftarrow \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$
$S \leftarrow \text{mask}(S, M)$ ▷ $-\infty$ wherever $M = 0$
$\mathbf{Y} \leftarrow \text{softmax}(S)\,\mathbf{V}$ **return** $\mathbf{Y}$

### 7.4.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n \cdot w \cdot d)$ time and $\mathcal{O}(w)$ KV storage per layer at inference (fixed once the sequence exceeds $w$).
- **Trainable**: Both; training-free at inference on pretrained checkpoints, or trained from scratch.
- **Receptive field**: After $L$ layers, $L(w - s)$ positions plus the always-visible sink prefix.
- **Strength**: Recovers $\approx 99\%$ of the average full-attention score across models from 1.3B to 72B with zero training tokens, and beats retrofitted linear attention by 2–10$\times$ on long-context reasoning (S-NIAH, BABILong) while decoding faster and using less memory.
- **Limitation**: The horizon is fixed and task-dependent—$w = 64$ suffices for log-likelihood benchmarks but collapses on generative math, where $w = 512$ is needed; there is no content-adaptive selection.

## 7.5 BigBird: Random, Local, and Global Sparse Graph

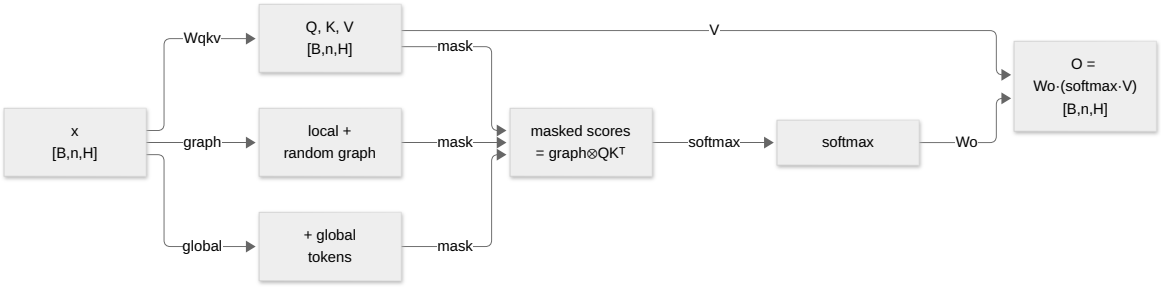



Figure 52: Dataflow of BigBird: local, random, and global adjacency masks the score matrix before softmax.

### 7.5.1 Mathematical Formulation

BigBird [111] preserves universal approximation and Turing completeness using a principled mix of three sparsity patterns.

**Random connections** : Each position connects to $r$ randomly sampled positions:

$$\mathcal{A}_i^{(\text{random})} = \text{RandomSample}(\{1, \ldots, n\}, r)$$

**Local window** : Sliding window neighborhood:

$$\mathcal{A}_i^{(\text{local})} = \{j : |i - j| \leq w/2\}$$

**Global tokens** : Task-specific global anchors:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \ldots, n\} & \text{if } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{random})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{G} & \text{otherwise} \end{cases}$$

The composite sparse mask $M$ is:

$$M_{ij} = \mathbf{1}[j \in \mathcal{A}_i^{(\text{random})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{global})}]$$

In practice, BigBird groups tokens into blocks of size $b$ and applies the sparsity decision at the block level, enabling efficient block-sparse implementations.

### 7.5.2 Theoretical Guarantee

The authors prove that with $O(1)$ random connections and global tokens, the sparse graph maintains the same universal approximation and Turing completeness properties as dense attention. Formally, any computable function can be represented and any sequence of operations can be simulated within the BigBird connectivity graph.

### 7.5.3 Algorithmic Pseudocode

**Algorithm 26** BigBird: Random + Local + Global Block-Sparse Attention

**Require:** Query $\mathbf{Q}$, Key $\mathbf{K}$, Value $\mathbf{V}$, block size $b$, random links per block $r$, global indices $\mathcal{G}$
**Ensure:** Output $\mathbf{Y}$

Reshape into blocks: $n_b = \lceil n/b \rceil$ blocks
**for** block $i = 0, \ldots, n_b - 1$ **do**
    **for** position $j$ in block $i$ **do**
        **Local window**: $\mathcal{A}_j^{(\text{local})}$ = nearby positions in block $\cup$ boundary positions
        **Random block sampling**: Sample $r$ blocks uniformly at random, add all positions from those blocks
        **Global**: Add all global token positions
        $\mathcal{A}_j \leftarrow \mathcal{A}_j^{(\text{local})} \cup \mathcal{A}_j^{(\text{random})} \cup \mathcal{G}$
    **end for**
**end for**
**for** $i = 1, \ldots, n$ **do**
    $\text{scores}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$
    $\text{attn}_i \leftarrow \text{softmax}(\text{scores}_i)$
    $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$
**end forreturn** $\mathbf{Y}$

### 7.5.4 Key Characteristics

- **Complexity**: $\mathcal{O}(n)$ or near-linear in optimal block-sparse implementation.
- **Trainable**: Yes.
- **Theoretical grounding**: Provably maintains universal approximation and Turing-completeness with sparse connectivity.
- **Strength**: Strong long-document performance on question answering, summarization, and genomics tasks.
- **Limitation**: Random sampling introduces variance and non-determinism; tuning of $r, w, g$ hyperparameters remains task-dependent.

## 7.6 SparseK Attention: Differentiable Top-$k$ Selection

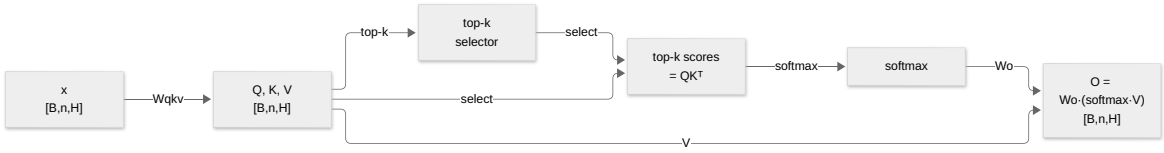



Figure 53: Dataflow of SparseK: a differentiable top-$k$ selector keeps only the highest-scoring keys before softmax.

### 7.6.1 Mathematical Formulation

SparseK [64] enables learnable sparsity via a **differentiable top-$k$ operator**. A scoring network $\phi_\theta$ evaluates key-value pair importance:

$$u_j = \phi_\theta(\mathbf{k}_j, \mathbf{q}_i) \in \mathbb{R}$$

The **SparseK operator** selects the top-$k$ scores while remaining differentiable. It computes a threshold $\tau(u)$ such that the sum of active scores equals $k$:

$$\text{SparseK}(u, k)_j = \max(u_j - \tau(u), 0), \quad \text{where} \quad \sum_j \max(u_j - \tau, 0) = k$$

The threshold $\tau$ is found via bisection. The attention becomes:

$$m_j = \text{SparseK}(u, k)_j, \quad \text{Attn}_i = \text{softmax}\left(\frac{\mathbf{q}_i K_{\text{sel}}^\top}{\sqrt{d_k}}\right) V_{\text{sel}}$$

where $K_{\text{sel}}, V_{\text{sel}}$ contain only the top-$k$ entries (those with $m_j > 0$).

### 7.6.2 Algorithmic Pseudocode

**Algorithm 27** SparseK: Differentiable Top-$k$ Attention

**Require:** Query $\mathbf{q} \in \mathbb{R}^d$, Key matrix $\mathbf{K} \in \mathbb{R}^{n \times d}$, Value matrix $\mathbf{V} \in \mathbb{R}^{n \times d}$
**Require:** Scoring network $\phi_\theta$, target sparsity $k$
**Ensure:** Attention output $\mathbf{y}$
**Compute importance scores:**
**for** $j = 1, \ldots, n$ **do**
    $u_j \leftarrow \phi_\theta(\mathbf{k}_j, \mathbf{q})$
**end for**
**Compute differentiable top-$k$ via threshold:**
$u_{\text{sorted}} \leftarrow \text{sort}(u, \text{descending} = \text{True})$
$\text{cumsum} \leftarrow \text{cumsum}(u_{\text{sorted}})$
Find $\rho = \max\{i : u_{\text{sorted}}[i] > 0 \text{ and } \text{cumsum}[i] \leq k\}$
$\tau \leftarrow (u_{\text{sorted}}[\rho] - k)/(\rho + 1)$ ▷ Threshold for $k$ active elements
$m \leftarrow \max(u - \tau, 0)$ ▷ Differentiable selection mask
**Apply mask and compute attention:**
$K_{\text{sel}} \leftarrow K[m > 0]$, $V_{\text{sel}} \leftarrow V[m > 0]$
$\text{scores} \leftarrow \mathbf{q} K_{\text{sel}}^\top / \sqrt{d_k}$
$\text{attn} \leftarrow \text{softmax}(\text{scores})$
$\mathbf{y} \leftarrow \text{attn} \cdot V_{\text{sel}}$ **return** $\mathbf{y}$

### 7.6.3 Key Characteristics

- **Complexity**: $\mathcal{O}(n \cdot d)$ during training (linear in sequence length); $\mathcal{O}(k)$ per token during generation.
- **Trainable**: Yes; end-to-end gradient flow through the SparseK operator.
- **Incremental generation**: Supports efficient constant-memory autoregressive generation.
- **Strength**: Seamlessly integrates into existing LLM architectures; minimal fine-tuning needed.
- **Limitation**: Scattered memory access from top-$k$ selection may limit cache efficiency on some hardware; scoring network adds overhead.

## 7.7 NSA (Native Sparse Attention): Hardware-Aligned Hierarchical Branches




Figure 54: Dataflow of NSA: hierarchical block scoring and selection restrict the score matrix to selected blocks.

### 7.7.1 Mathematical Formulation

NSA [109] decomposes sparse attention into three parallel branches that are combined through learned gates.

**Compression branch** : A learnable MLP $\varphi$ compresses key-value blocks:

$$\tilde{K}_t^{\text{cmp}} = [\varphi(k_{id+1:id+l})]_{1 \leq i \leq \lfloor (t-l)/d \rfloor} \quad , \quad \tilde{V}_t^{\text{cmp}} = [\varphi(v_{id+1:id+l})]$$

**Selection branch** : High-importance blocks are selected based on compressed scores:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}}/\sqrt{d}), \quad I_t = \text{TopK}(p_t^{\text{cmp}}, n), \quad \tilde{K}_t^{\text{sel}} = \text{Gather}(K, I_t)$$

**Sliding window branch** : Fixed local context:

$$\tilde{K}_t^{\text{win}} = K_{t-w:t}, \quad \tilde{V}_t^{\text{win}} = V_{t-w:t}$$

**Gated combination** : The three attention outputs are combined via learned gates:

$$\alpha_t^{(\text{cmp})} = \sigma(\text{Linear}_{\text{cmp}}(q_t)), \quad \alpha_t^{(\text{sel})} = \sigma(\text{Linear}_{\text{sel}}(q_t)), \quad \alpha_t^{(\text{win})} = \sigma(\text{Linear}_{\text{win}}(q_t))$$

$$y_t = \alpha_t^{(\text{cmp})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{cmp}}, \tilde{V}^{\text{cmp}}) + \alpha_t^{(\text{sel})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{sel}}, \tilde{V}^{\text{sel}}) + \alpha_t^{(\text{win})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{win}}, \tilde{V}^{\text{win}})$$

### 7.7.2 Algorithmic Pseudocode

**Algorithm 28** NSA: Compressed + Selected + Window Branches with Learned Gating

**Require:** Query $\mathbf{q}_t$, cached Key/Value sequences, block size $l$, stride $d$, selection count $n$, window $w$

**Require:** Compression MLP $\varphi$, gate networks $\text{Linear}_{\text{cmp}}, \text{Linear}_{\text{sel}}, \text{Linear}_{\text{win}}$

**Ensure:** Output $\mathbf{y}_t$

**Compression branch:**
**for** $i = 1$ to $\lfloor t/d \rfloor$ **do**
  $k_i^{\text{cmp}} \leftarrow \varphi(\mathbf{K}_{id+1:id+l})$ ▷ Compress block via MLP
  $v_i^{\text{cmp}} \leftarrow \varphi(\mathbf{V}_{id+1:id+l})$
**end for**
$\tilde{\mathbf{K}}^{\text{cmp}} \leftarrow [k_1^{\text{cmp}}, \ldots, k_{\lfloor t/d \rfloor}^{\text{cmp}}]$, $\tilde{\mathbf{V}}^{\text{cmp}} \leftarrow [v_1^{\text{cmp}}, \ldots, v_{\lfloor t/d \rfloor}^{\text{cmp}}]$
$\mathbf{y}_t^{(\text{cmp})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{cmp}}, \tilde{\mathbf{V}}^{\text{cmp}})$
**Selection branch:**
Compute scores on compressed: $\mathbf{s} \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}^{\text{cmp}\top}/\sqrt{d})$
Select top-$n$ important blocks: $I \leftarrow \text{TopK}(\mathbf{s}, n)$
Gather full blocks: $\tilde{\mathbf{K}}^{\text{sel}} \leftarrow [\mathbf{K}_{I_i d+1:I_i d+l}]_i$, $\tilde{\mathbf{V}}^{\text{sel}} \leftarrow [\mathbf{V}_{I_i d+1:I_i d+l}]_i$
$\mathbf{y}_t^{(\text{sel})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{sel}}, \tilde{\mathbf{V}}^{\text{sel}})$
**Window branch:**
$\mathbf{y}_t^{(\text{win})} \leftarrow \text{SDPA}(\mathbf{q}_t, \mathbf{K}_{t-w:t}, \mathbf{V}_{t-w:t})$
**Gated fusion:**
$\alpha^{(\text{cmp})} \leftarrow \sigma(\text{Linear}_{\text{cmp}}(\mathbf{q}_t))$, $\alpha^{(\text{sel})} \leftarrow \sigma(\text{Linear}_{\text{sel}}(\mathbf{q}_t))$, $\alpha^{(\text{win})} \leftarrow \sigma(\text{Linear}_{\text{win}}(\mathbf{q}_t))$
$\mathbf{y}_t \leftarrow \alpha^{(\text{cmp})}\mathbf{y}_t^{(\text{cmp})} + \alpha^{(\text{sel})}\mathbf{y}_t^{(\text{sel})} + \alpha^{(\text{win})}\mathbf{y}_t^{(\text{win})}$ **return** $\mathbf{y}_t$

### 7.7.3 Key Characteristics

- **Complexity**: $\mathcal{O}(t/d + n \cdot l + w)$ tokens attended per step (significantly reduced versus $\mathcal{O}(t)$ for dense).
- **Trainable**: Yes; direct training with all branches differentiable.
- **Hardware alignment**: Designed for efficient tensor core utilization; compression and selection reduce memory bandwidth pressure.

- **Strength**: 11.6× decoding speedup and 9× forward speedup on 64k sequences; outperforms full attention on many tasks.
- **Limitation**: Requires custom Triton/CUDA kernels; complex multi-branch architecture increases engineering overhead.

## 7.8 FASA: Frequency-Aware Sparse Attention

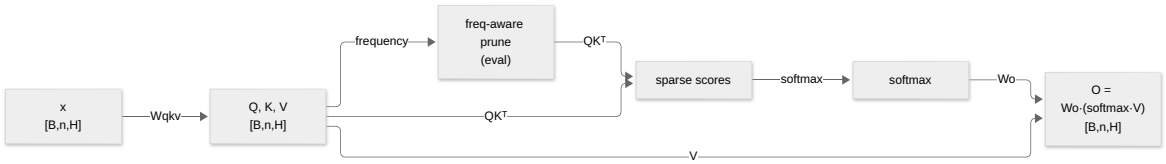



Figure 55: Dataflow of FASA: a frequency-aware prune stage (evaluation only) drops low-value key blocks before softmax.

### 7.8.1 Mathematical Formulation

FASA [100] is a **training-free inference-time method** that exploits rotary position embedding (RoPE) structure. Under RoPE, the token embedding is rotated by $\theta_i = B^{-2(i-1)/d}$, decomposing the $d$-dimensional space into $d/2$ frequency chunks (FCs).

**Key insight** : Only a small subset ($< 1\%$) of FCs matter for contextual awareness; most encode positional patterns.

**Dominant frequency chunk identification** : For each layer $l$ and head $h$, identify dominant FCs via contextual agreement (CA):

$$\mathrm{CA}^{l,h,i} = \frac{|\mathrm{TopK}(\alpha^{l,h}) \cap \mathrm{TopK}(\alpha^{l,h,i})|}{K}$$

where $\alpha^{l,h}$ is the full attention mask and $\alpha^{l,h,i}$ is the mask using only FC $i$. Dominant FCs have high CA—they contribute meaningfully to the final attention pattern.

**Token importance prediction (TIP) stage** : Using only dominant FCs, compute lightweight importance per token:

$$S_t^{l,h} = \sum_{i \in \mathcal{I}_{\mathrm{dom}}^{l,h}} \alpha^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-Indices}(S_t, N_{\mathrm{fac}})$$

**Focused attention computation (FAC) stage** : Compute full-precision attention on selected tokens:

$$\hat{\alpha}_{\mathrm{FAC}} = \mathrm{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top}{\sqrt{d}}\right), \quad \mathbf{o}_t = \hat{\alpha}_{\mathrm{FAC}} \mathbf{V}_{\mathcal{T}_t}$$

### 7.8.2 Algorithmic Pseudocode

**Algorithm 29** FASA: Frequency-Aware Sparse Attention (Inference-Time)

**Require:** Current query $\mathbf{q}_t$, cached Key/Value $\mathbf{K}_{1:t}, \mathbf{V}_{1:t}$, RoPE base $B$, dominant FCs $\mathcal{I}_{\text{dom}}$
**Require:** TIP token budget $N_{\text{tip}}$, FAC token budget $N_{\text{fac}}$
**Ensure:** Output $\mathbf{o}_t$

**Stage 1: Token Importance Prediction (TIP)**
**for** layer $l$ and head $h$ **do**
  **for** position $i = 1$ to $t$ **do**
    Compute importance from dominant FCs: $s_i \leftarrow 0$
    **for** $fc \in \mathcal{I}_{\text{dom}}^{l,h}$ **do**
      Rotate query and key into FC $fc$: $\mathbf{q}^{(fc)}, \mathbf{k}_i^{(fc)}$
      $s_i^{(fc)} \leftarrow \text{softmax}(\mathbf{q}^{(fc)} \mathbf{k}_i^{(fc)\top} / \sqrt{d})$
      $s_i \leftarrow s_i + s_i^{(fc)}$
    **end for**
  **end for**
  $\mathcal{T}_t \leftarrow \text{TopK} - \text{Indices}([s_1, \ldots, s_t], N_{\text{fac}})$ ▷ Select top tokens
**end for**
**Stage 2: Focused Attention Computation (FAC)**
$\text{scores}_{\text{fac}} \leftarrow \mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^{\top} / \sqrt{d}$ ▷ Full-precision dot product
$\alpha_{\text{fac}} \leftarrow \text{softmax}(\text{scores}_{\text{fac}})$ ▷ Full softmax
$\mathbf{o}_t \leftarrow \alpha_{\text{fac}} \mathbf{V}_{\mathcal{T}_t}$ ▷ Weighted value sum **return** $\mathbf{o}_t$

### 7.8.3 Key Characteristics

- **Complexity**: TIP is $\mathcal{O}(t \cdot N_{\text{tip}})$; FAC is $\mathcal{O}(N_{\text{fac}} \cdot d)$.
- **Trainable**: No; training-free inference optimization.
- **Applicability**: Requires RoPE-based models; extended to ALiBi and MLA with modifications.
- **Strength**: Near-oracle accuracy with $\leq 256$ tokens out of millions; up to 2.56× decoding speedup; orthogonal to quantization.
- **Limitation**: Offline offline calibration required; dominant FC identification adds preprocessing overhead.

**Implementation note (training vs. inference).** In this repository the sparse TIP/FAC path is inference-only. While the model is in training mode, the enclosing hybrid layer substitutes standard dense attention over the same **q/k/v/o** projections, so the mixer can be trained from scratch and the weights optimized during training are exactly the ones the sparse path reads at evaluation.

## 7.9 SpargeAttn: Two-Stage Block-Level Filtering

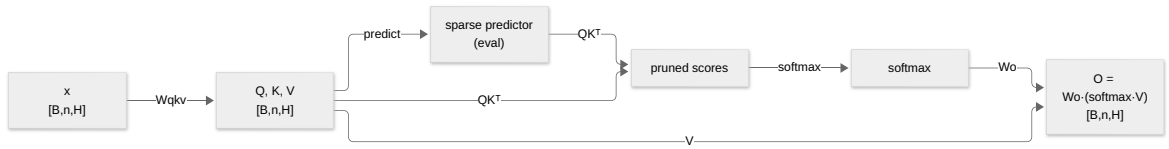



Figure 56: Dataflow of SpargeAttn: a sparse predictor (evaluation only) prunes key blocks before softmax.

### 7.9.1 Mathematical Formulation

SpargeAttn [115] is a **universal training-free method** that predicts which blocks of the attention matrix will contain negligible values and skips computation for those blocks.

**Stage 1 — Sparse prediction** : Compute low-cost proxy importance $\hat{s}_{ij}$ for block $(i, j)$:

$$\hat{s}_{ij} = f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j; \text{hyperparams}), \quad \text{skip if } \hat{s}_{ij} < \epsilon_1$$

Common predictors include block-mean similarity or self-similarity statistics.

**Stage 2 — Softmax-aware filtering** : After computing $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d})$, check if the block's maximum probability is negligible relative to the running softmax maximum:

$$\text{skip } P_{ij} V_j \quad \text{if} \quad \max(\tilde{P}_{ij}) < e^{m_{\text{old}} - m_{\text{new}}} \cdot \epsilon_2$$

where $m_{\text{old}}, m_{\text{new}}$ are online softmax running maxima (computed via FlashAttention-style numerics).

### 7.9.2 Algorithmic Pseudocode

**Algorithm 30** SpargeAttn: Two-Stage Block-Sparse Filtering

**Require:** Query blocks $\mathbf{Q}_i$ for $i = 1, \ldots, n_b$, Key blocks $\mathbf{K}_j$ for $j = 1, \ldots, n_b$, Value blocks $\mathbf{V}_j$
**Require:** Prediction threshold $\epsilon_1$, softmax threshold $\epsilon_2$, block size $b_s$
**Ensure:** Output $\mathbf{Y}$

Initialize: online softmax max $m_{\text{global}} \leftarrow -\infty$
**for** block row $i = 1$ to $n_b$ **do**
  Initialize: block row output $\mathbf{Y}_i \leftarrow 0$, local max $m_i \leftarrow -\infty$
  **for** block column $j = 1$ to $n_b$ **do**
    **Stage 1: Sparse Prediction**
    Compute proxy importance: $\hat{s}_{ij} \leftarrow f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j)$
    **if** $\hat{s}_{ij} < \epsilon_1$ **then**
      **skip** this block $[i, j]$ ▷ Early termination
      **continue** ▷ Move to next block
    **end if**
    **Stage 2: Compute and Filter**
    $\tilde{P}_{ij} \leftarrow \text{softmax}(\mathbf{Q}_i \mathbf{K}_j^\top / \sqrt{d})$
    $m_{\text{block}} \leftarrow \max(\tilde{P}_{ij})$
    **if** $m_{\text{block}} < e^{m_i - m_{\text{global}}} \cdot \epsilon_2$ **then**
      Block contributes negligibly; skip ▷ Softmax-aware pruning
      **continue**
    **end if**
    Update global max: $m_i \leftarrow \max(m_i, m_{\text{block}})$
    Accumulate: $\mathbf{Y}_i \leftarrow \mathbf{Y}_i + \tilde{P}_{ij} \mathbf{V}_j$
  **end for**
  $m_{\text{global}} \leftarrow \max(m_{\text{global}}, m_i)$
**end forreturn Y**

### 7.9.3 Key Characteristics

- **Complexity**: Empirical $\mathcal{O}(n^2 \cdot s)$ where $s$ is the fraction of non-skipped blocks (typically 0.2–0.5).
- **Trainable**: No; plug-and-play acceleration for existing models.
- **Universality**: Works on language models, image diffusion models, and video generation.
- **Strength**: 2.5–5× speedup compared to dense or previous sparse methods; compatible with quantization (SageAttention integration).
- **Limitation**: Speedup depends on inherent sparsity; block-level granularity may miss finer patterns; threshold tuning required per model family.

**Implementation note (training vs. inference).** In this repository the two-stage block filter is inference-only. While the model is in training mode, the enclosing hybrid layer substitutes standard dense attention over the same **q**/**k**/**v**/**o** projections, so the mixer can be trained from scratch and the weights optimized during training are exactly the ones the sparse path reads at evaluation.

## 7.10 MSA (MiniMax Sparse Attention): Blockwise Sparse Attention with Lightweight Index Branch

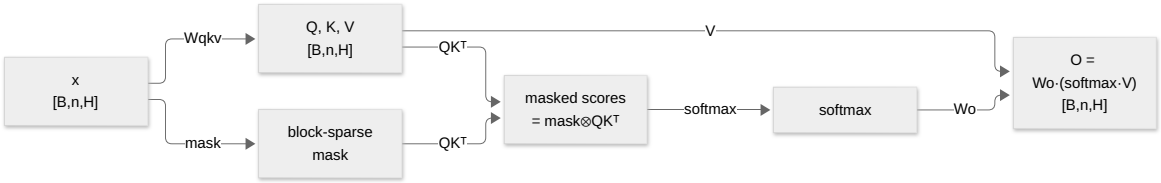



Figure 57: Dataflow of MSA: a block-sparse mask restricts the score matrix before softmax.

### 7.10.1 Mathematical Formulation

MSA [54] is a blockwise sparse attention mechanism built on grouped-query attention (GQA) that combines a lightweight **Index Branch** for block selection with a **Main Branch** for exact block-sparse softmax attention over the selected blocks. Let $N$ be the sequence length, $B_k = 128$ the block size, and $k = 16$ the number of blocks selected per query (yielding a fixed per-query budget of $k \cdot B_k = 2048$ tokens regardless of $N$).

**Index Branch** : A low-dimensional projection produces index queries $\mathbf{q}_t^{\text{idx}} \in \mathbb{R}^{d_{\text{idx}}}$ and block representatives $\mathbf{r}_b \in \mathbb{R}^{d_{\text{idx}}}$ for each KV block $b$. The index scores per GQA group are:

$$s_{t,b}^{(g)} = \mathbf{q}_t^{\text{idx},(g)} \cdot \mathbf{r}_b, \qquad \mathcal{I}_t^{(g)} = \text{TopK}\left(\{s_{t,b}^{(g)}\}_{b=1}^{N/B_k},\ k\right)$$

The index input is **detached** (stop-gradient), so gradients do not flow back through the indexer's input; the Index Branch is trained instead by a KL alignment loss between its attention distribution and the Main Branch's distribution. Because softmax is order-preserving, the raw index scores feed directly into the top-$k$ operator *without* an explicit softmax (**exp-free top-$k$**), reducing numerical cost.

**Forced local block** : The block containing the query position $t$ is **always** included in $\mathcal{I}_t^{(g)}$, guaranteeing local context coverage.

**Main Branch** : Standard scaled dot-product attention is computed exactly over the union of selected blocks:

$$\mathbf{o}_t^{(h)} = \text{softmax}\left(\frac{\mathbf{q}_t^{(h)} \mathbf{K}_{\mathcal{I}_t}^{(h)\top}}{\sqrt{d_h}}\right) \mathbf{V}_{\mathcal{I}_t}^{(h)}$$

where $\mathbf{K}_{\mathcal{I}_t}, \mathbf{V}_{\mathcal{I}_t}$ gather the full-precision key/value tensors of the selected blocks. Selection is performed independently per GQA group, so different query groups within the same layer may attend to different block subsets.

### 7.10.2 Algorithmic Pseudocode

**Algorithm 31** MSA: MiniMax Sparse Attention with Index + Main Branches

**Require:** Query $\mathbf{Q} \in \mathbb{R}^{N \times d}$, Key $\mathbf{K}$, Value $\mathbf{V}$, block size $B_k = 128$, top-$k$ blocks $k = 16$
**Require:** Index projection $W_{\text{idx}}$ (detached input), block representatives $\{\mathbf{r}_b\}$
**Ensure:** Output $\mathbf{Y} \in \mathbb{R}^{N \times d}$

Compute block reps: $\mathbf{r}_b \leftarrow \text{mean}(\mathbf{K}_{bB_k:(b+1)B_k})$ for $b = 1, \ldots, N/B_k$
**for** query position $t = 1, \ldots, N$ **do**
  **for** each GQA group $g$ **do**
    $\mathbf{q}_t^{\text{idx}} \leftarrow \text{sg}(W_{\text{idx}} \mathbf{x}_t)$ ▷ detached index query
    Compute block scores: $s_b \leftarrow \mathbf{q}_t^{\text{idx}} \cdot \mathbf{r}_b$
    Identify local block: $b_{\text{local}} \leftarrow \lfloor t / B_k \rfloor$
    $\mathcal{I}_t^{(g)} \leftarrow \text{TopK}(\{s_b\}, k) \cup \{b_{\text{local}}\}$ ▷ exp-free top-$k$ + forced local
  **end for**
  Gather selected KV: $\tilde{\mathbf{K}}_t \leftarrow \text{Gather}(\mathbf{K}, \mathcal{I}_t)$, $\tilde{\mathbf{V}}_t \leftarrow \text{Gather}(\mathbf{V}, \mathcal{I}_t)$
  Main branch SDPA: $\mathbf{y}_t \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}_t^\top / \sqrt{d_h}) \tilde{\mathbf{V}}_t$
**end for**
Add KL alignment loss $\mathcal{L}_{\text{KL}} = \text{KL}(\text{index dist} \,\|\, \text{main dist})$ during training **return** $\mathbf{Y}$

### 7.10.3 Key Characteristics

- **Complexity**: Index branch $\mathcal{O}(H_{\text{kv}} \cdot d_{\text{idx}} \cdot N^2)$; Main branch $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$. Per-query budget fixed at $k \cdot B_k = 2048$ tokens regardless of $N$.
- **Trainable**: Yes; end-to-end (the Index Branch is trained via a KL alignment loss, not eval-only).
- **State**: $\mathcal{O}(N)$ KV cache (GQA-sized); no expansion of the recurrent/state dimension.
- **Strength**: Deployed on the 109B MiniMax-M3, MSA achieves a 28.4× per-token attention compute reduction at 1M context, with 14.2× prefill and 7.6× decode speedup on H800 GPUs.
- **Limitation**: Selection granularity is fixed at the block level ($B_k = 128$); the forced local block and per-group independent selection increase engineering complexity and require custom block-sparse kernels.

## 7.11 SparDA (Sparse Decoupled Attention): Forecast-Guided Lookahead Block Selection

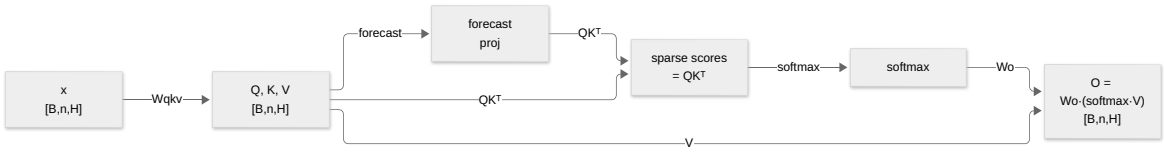



Figure 58: Dataflow of SparDA: a forecast projection guides lookahead block selection before softmax.

### 7.11.1 Mathematical Formulation

SparDA [30] introduces a **decoupled** sparse attention scheme that augments the standard per-layer projections $Q, K, V$ with a fourth projection, the **Forecast**, which predicts the KV

blocks that will be needed by the *next* layer. This lookahead enables overlapping CPU-to-GPU prefetch of the next layer's selected blocks with the current layer's execution, hiding the offload latency that plagues sparse-attention deployments on memory-constrained hardware.

**Forecast projection** : For input $\mathbf{x} \in \mathbb{R}^{N \times d}$,

$$\mathbf{F} = \mathbf{x}\mathbf{W}_F \in \mathbb{R}^{N \times H_{\text{kv}} \times d_f}$$

with **one Forecast head per GQA group**, which reduces selection overhead relative to multi-head selectors used by prior work.

**Block scoring and selection** : For each query $i$ and KV block $b$ with representative $\mathbf{r}_b$:

$$S_{i,b} = \mathbf{F}_i \cdot \mathbf{r}_b, \qquad \mathcal{I}_i = \text{TopK}(\{S_{i,b}\}_b, k)$$

**Block-sparse attention** : Standard scaled dot-product attention is then computed over the selected blocks:

$$\mathbf{o}_i = \text{softmax}\left(\frac{\mathbf{q}_i \mathbf{K}_{\mathcal{I}_i}^\top}{\sqrt{d_h}}\right)\mathbf{V}_{\mathcal{I}_i}$$

**Training** : SparDA adds fewer than 0.5% additional parameters. Only the Forecast projections are trained, by matching the original (frozen) sparse selector's attention distribution—a lightweight distillation that preserves the base model's behaviour while enabling lookahead prefetch.

### 7.11.2 Algorithmic Pseudocode

**Algorithm 32** SparDA: Sparse Decoupled Attention with Forecast Prefetch

**Require:** Input $\mathbf{x} \in \mathbb{R}^{N \times d}$, cached Key/Value blocks, top-$k$ blocks $k$
**Require:** Projections $W_q, W_k, W_v, W_F$; frozen base selector $\pi_{\text{orig}}$
**Ensure:** Output $\mathbf{Y}$; prefetched blocks for next layer

```
Compute Forecast: F ← xW_F                                   ▷ one head per GQA group
for query i = 1, ..., N do
    Compute block scores: S_{i,b} ← F_i · r_b for each block b
    Select: I_i ← TopK({S_{i,b}}, k)
    Prefetch K_{I_i}, V_{I_i} from CPU to GPU for the next layer   ▷ overlaps with current layer
end for
Compute block-sparse SDPA over current-layer selected blocks: Y ← BlockSparseSDPA(Q, K, V, {I_i})
Training loss: L ← KL(Forecast dist ‖ π_orig)                ▷ only W_F is updated return Y
```

### 7.11.3 Key Characteristics

- **Complexity**: Forecast $\mathcal{O}(H_{\text{kv}} \cdot d_f \cdot N \cdot N_{\text{blocks}})$; Main attention $\mathcal{O}(H_q \cdot d_h \cdot N \cdot k \cdot B_k)$.
- **Trainable**: Yes; only the Forecast projections are trained ($< 0.5\%$ extra parameters).
- **Strength**: On two 8B sparse-pretrained models, SparDA achieves up to 1.25× prefill and 1.7× decode speedup over the sparse-attention offload baseline, and up to 5.3× decode throughput versus a non-offload sparse baseline.
- **Limitation**: Requires a CPU–GPU offload pipeline and a frozen base selector; the lookahead benefit is contingent on next-layer block demand being predictable from the current Forecast.

### 7.12 Design Space and Selection Criteria

The ten sparse attention methods occupy complementary positions in a multidimensional design space:

- **Geometric/fixed patterns** (Sparse Transformer, Longformer): Simple to implement and analyze, but patterns do not adapt to content.
- **Fixed-memory KV eviction** (SWA with sinks): The cheapest possible policy—keep a sink prefix and a recent window, discard everything else. No extra parameters, no training required, bounded memory.
- **Learned selection** (SparseK, NSA): Enable adaptation through trainable selection networks, at the cost of higher implementation complexity.
- **Training-free acceleration** (FASA, SpargeAttn): Enable drop-in speedup of existing models without retraining, suited for deployed systems.
- **Theoretical guarantees** (BigBird): Provide formal proofs of expressive completeness, important for understanding safety margins.

**Selection guidance:**

1. **New model training** where resources permit: NSA or SparseK for maximal inference speedup; BigBird for theoretical assurance; SWA with sinks as the strongest low-cost baseline.
2. **Long-context document understanding**: SWA with sinks (window 256–512) for the best accuracy per byte of KV cache; Longformer or FASA for practical simplicity; NSA for extreme scale.
3. **Accelerating existing models**: SWA with sinks as the default training-free baseline (beat most linear-attention retrofits in the original study); FASA for RoPE-based LLMs; SpargeAttn for any architecture.
4. **Constrained environments**: SWA with sinks for a fixed memory budget; Sparse Transformer for simplicity and proven efficiency at moderate scale.

## 8 Gated Attention Families—Complete Literature Analysis

### 8.1 Executive Summary: Gating for Memory Control

The emerging field of *gated attention mechanisms* addresses a fundamental challenge in sequence modeling: how should information be retained, forgotten, and updated as the model processes new tokens? Traditional additive recurrent states accumulate all information without erasure, leading to memory saturation and inability to adapt to changing context. Softmax attention provides full expressiveness but uses $\mathcal{O}(Ld)$ KV cache during inference, limiting context window size. Gated mechanisms provide a middle ground: selective control over information survival.

The unifying principle across gated architectures is that gating controls *what information survives*. Gates operate at three distinct levels:

1. **Recurrent state decay** (GLA, HGRN2): Multiplicative gates on the memory matrix $S_t$ control how much previous state persists into the next step.
2. **Write strength in recurrent updates** (DeltaNet, Gated DeltaNet, Gated DeltaNet-2): Gates control the magnitude and channel-wise direction of new information written into memory.

3. **Softmax logit biasing** (FoX): Gates inject token-level recency bias into attention logit computation.

4. **Post-attention sparsity** (Gated Softmax): Gates selectively scale SDPA output channels.

This appendix synthesizes eight major gated-attention architectures published 2023–2026, analyzing their mathematical foundations, hardware efficiency characteristics, and empirical trade-offs.

## 8.2 1. Gated Linear Attention (GLA)

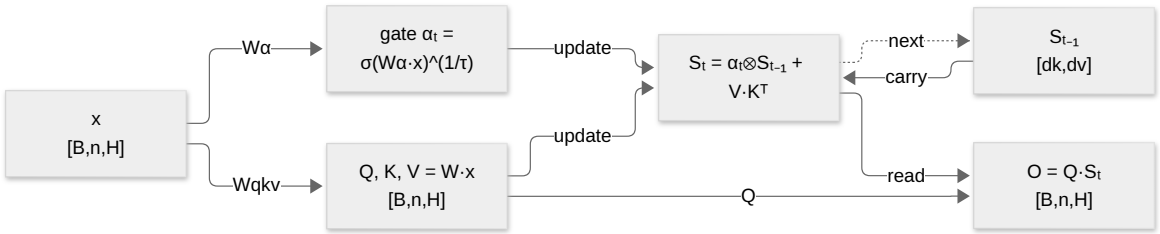



Figure 59: Dataflow of GLA: a sigmoid gate decays the recurrent state before the key/value outer product is added.

### 8.2.1 Mathematical Foundation

Gated Linear Attention [105] augments vanilla linear attention (which uses a matrix-valued recurrent state) with data-dependent multiplicative decay. Standard linear attention reformulates the traditional softmax mechanism as a recurrence over hidden states:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t\mathbf{q}_t$$

where state $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ accumulates outer products. This purely additive formulation suffers from "memory overload": information accumulates monotonically and cannot be erased, degrading performance on tasks requiring context selection.

GLA introduces a data-dependent **diagonal gating matrix** $\mathbf{G}_t \in [0,1]^{d_k \times d_k}$:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t\mathbf{q}_t$$

The gate $\mathbf{G}_t$ is parameterized with an outer-product structure:

$$\mathbf{G}_t = \mathbf{1}\boldsymbol{\alpha}_t^\top, \quad \boldsymbol{\alpha}_t = \text{logsigmoid}\left(\mathbf{W}_{gk}\mathbf{x}_t\right)/c$$

where $\mathbf{W}_{gk}$ is a low-rank projection ($\mathbb{R}^{d \to d_k}$) and $c \approx 16$ is a normalizer. The logsigmoid activation maps $\alpha_t \in \mathbb{R}$ to approximately $[-1/2, 0]$, and division by $c$ further contracts this to near-identity for initialization. The resulting gate $G_t = 1 + \alpha_t \approx 1$ near initialization, then learns to decay ($\alpha_t \to -\infty$) historical information.

**Algorithm 33** Gated Linear Attention (Recurrent Form)

**Require:** Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1}$
**Require:** Projections: $W_q, W_k, W_v$ (standard); $W_{gk}$ (gate projection)
**Ensure:** Output $\mathbf{o}_t$ and updated state $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$
$\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$
$\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
Compute gate: $\boldsymbol{\alpha}_t \leftarrow \text{logsigmoid}(W_{gk}\mathbf{x}_t)/16$ ▷ Per-key-dim decay
Apply outer-product gating: $\mathbf{G}_t \leftarrow \mathbf{1}\boldsymbol{\alpha}_t^\top$ ▷ diagonal outer product
Decay previous state: $\mathbf{S}_t^{\text{decay}} \leftarrow \mathbf{G}_t \odot \mathbf{S}_{t-1}$ ▷ element-wise product
Add new association: $\mathbf{S}_t \leftarrow \mathbf{S}_t^{\text{decay}} + \mathbf{v}_t \mathbf{k}_t^\top$
Retrieve via query: $\mathbf{o}_t^{\text{raw}} \leftarrow \mathbf{S}_t \mathbf{q}_t$
Apply output gate: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t^{\text{raw}}) \otimes \text{SiLU}(W_g \mathbf{x}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$

### 8.2.2 Hardware-Efficient Training

For parallel training, GLA uses a chunkwise block-wise algorithm that groups tokens into chunks $C$ and computes recurrent transitions in parallel:

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \boldsymbol{\Gamma}_{[t]}\right) \mathbf{V}_{[t]}$$

where $\overleftarrow{\mathbf{Q}}_{[t]}$ are attending to the state at the chunk boundary (lookback), and $\boldsymbol{\Gamma}_{[t]}$ is the causal mask with decay-aware scaling:

$$\boldsymbol{\Gamma}_{[t],ij} = \begin{cases} \prod_{l=j}^{i-1} \alpha_l & \text{if } i > j \text{ (lookback)} \\ \prod_{l=1}^{i-j} \alpha_l & \text{if } i \leq j \text{ (in-chunk)} \end{cases}$$

This design maximizes matmul operations susceptible to tensor-core acceleration, achieving sub-quadratic total complexity $\mathcal{O}(nLd^2/C)$ where $L$ is the number of attention heads and $C$ is chunk size.

### 8.2.3 Strengths and Limitations

| Aspect | Strengths | Limitations |
|---|---|---|
| Computational efficiency | Sub-quadratic training via chunkwise parallelism. Constant-memory inference $O(d^2)$. | Requires custom CUDA/Triton kernels; not available in all frameworks. |
| Context generalization | Extends 2K-token training to 20K+ tokens with negligible perplexity degradation. | Still underperforms softmax on retrieval-heavy benchmarks (needle-in-haystack). |
| Selectivity | Per-key-dimension data-dependent decay. | Gate structure limits per-key-value selectivity; no direct control over which specific associations to erase. |

## 8.3 2. DeltaNet: Error-Correcting Linear Attention

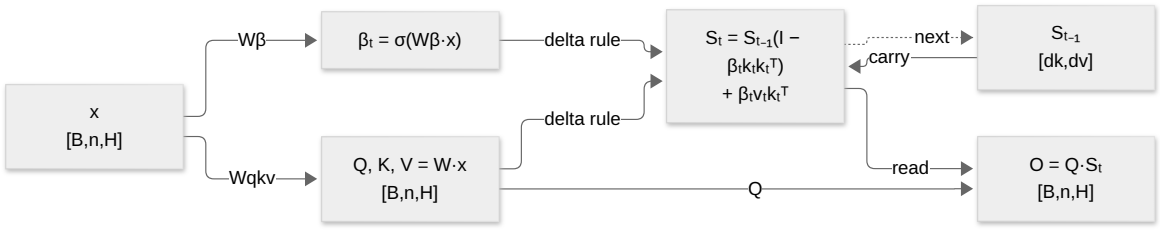



Figure 60: Dataflow of DeltaNet: the beta gate applies an error-correcting delta rule to the recurrent state.

### 8.3.1 Mathematical Foundation

DeltaNet [106] applies a classical **delta learning rule** to linear attention state updates. Instead of additive accumulation, DeltaNet computes the difference between the predicted value (retrieved from memory) and the target value, then performs an error-correcting update:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

where $\beta_t \in (0, 1)$ is a data-dependent **writing strength** scalar. This update can be decomposed as an erase-write operation:

$$\mathbf{v}_t^{\text{old}} = \mathbf{S}_{t-1}\mathbf{k}_t \quad \text{(current prediction)}, \quad \mathbf{v}_t^{\text{updated}} = \beta_t \mathbf{v}_t + (1 - \beta_t)\mathbf{v}_t^{\text{old}} \quad \text{(blend old/new)}$$

so that:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{old}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{updated}} \mathbf{k}_t^\top$$

The key insight is that this update rule minimizes an online MSE loss at each timestep:

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S}\mathbf{k}_t - \mathbf{v}_t\|^2 \Rightarrow \nabla_{\mathbf{S}} \mathcal{L}_t = (\mathbf{S}\mathbf{k}_t - \mathbf{v}_t)\mathbf{k}_t^\top$$

One step of SGD with learning rate $\beta_t$ yields the delta rule update. This connection to test-time training (TTT) provides theoretical grounding: DeltaNet is directly optimizing for value prediction at inference time.

### 8.3.2 Efficient Parallel Training

DeltaNet's transition matrix $\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$ is a generalized Householder matrix (rank-1 update to identity). For chunkwise training, DeltaNet uses the **WY representation**, which represents the product of $m$ such rank-1 updates compactly:

$$\prod_{i=1}^{m} (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \mathbf{I} - \mathbf{W}\mathbf{Y}^\top$$

where $\mathbf{W}, \mathbf{Y} \in \mathbb{R}^{d \times m}$ are constructed recursively. This factorization enables matmul-rich parallelism without materializing the full product, keeping chunkwise training efficient.

**Algorithm 34** DeltaNet State Update with WY Representation

**Require:** Input chunk $\mathbf{X}_{[t]}$; prior state $\mathbf{S}_{t-1}$
**Require:** L2-normalized queries $\hat{\mathbf{Q}}_t$, keys $\hat{\mathbf{K}}_t$, values $\mathbf{V}_t$
**Ensure:** Output $\mathbf{O}_{[t]}$ and updated state $\mathbf{S}_t$

**Chunkwise phase:**
**for** position $i = 1$ to chunk_size **do**
  Normalize: $\hat{\mathbf{k}}_i \leftarrow \hat{\mathbf{K}}_t[i]/\|\hat{\mathbf{K}}_t[i]\|$, $\hat{\mathbf{q}}_i \leftarrow \hat{\mathbf{Q}}_t[i]/\|\hat{\mathbf{Q}}_t[i]\|$
  Compute write strength: $\beta_i \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_i)$
  Prediction: $\hat{\mathbf{v}}^{\text{pred}} \leftarrow \mathbf{S}_{i-1}^{\text{in-chunk}} \hat{\mathbf{k}}_i$
  Error-aware blend: $\mathbf{v}_i^{\text{update}} \leftarrow \beta_i(\mathbf{v}_i - \hat{\mathbf{v}}^{\text{pred}})$
  Erase-write: $\mathbf{S}_i^{\text{in-chunk}} \leftarrow \mathbf{S}_{i-1}^{\text{in-chunk}} - \hat{\mathbf{v}}^{\text{pred}} \hat{\mathbf{k}}_i^\top + (\beta_i \mathbf{v}_i + (1-\beta_i)\hat{\mathbf{v}}^{\text{pred}})\hat{\mathbf{k}}_i^\top$
  Output: $\mathbf{o}_i \leftarrow \mathbf{S}_i^{\text{in-chunk}} \hat{\mathbf{q}}_i$
**end for**
**Inter-chunk phase:** Use WY representation of transition products **return** $\mathbf{O}_{[t]}$, $\mathbf{S}_t$

### 8.3.3 Strengths and Limitations

| Aspect | Strengths | Limitations |
|---|---|---|
| Associative recall | Perfect recall on Multi-Query Associative Recall (MQAR) benchmark; error-correcting semantics naturally fit retrieval patterns. | Without global forgetting, memory still crowds over extreme sequence lengths; requires periodic reset mechanisms for unbounded context. |
| Theory | Grounded in online MSE optimization; clear connection to test-time training paradigm. | Requires L2-normalized keys for numerical stability; double-width normalization adds overhead. |
| Throughput | WY representation enables efficient chunkwise training. | Slightly slower than Mamba2 per-token due to richer transition matrices (rank-1 instead of diagonal). |

## 8.4 3. Gated DeltaNet: Synthesis of Gating and Error Correction

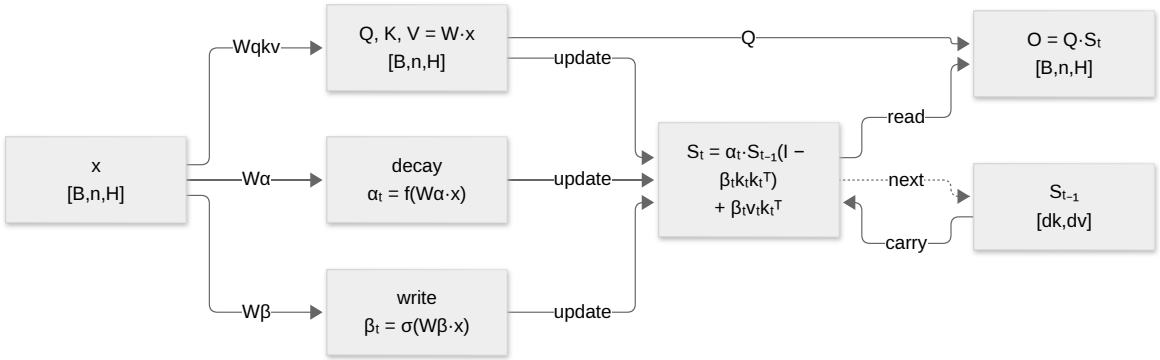



Figure 61: Dataflow of Gated DeltaNet: decay and write gates modulate the delta-rule state update.

### 8.4.1 Mathematical Foundation

Gated DeltaNet [107] synthesizes the strengths of GLA and DeltaNet by combining a **decay gate** $\alpha_t$ (from GLA) with the **writing strength gate** $\beta_t$ (from DeltaNet):

$$\mathbf{S}_t = \mathbf{S}_{t-1}\left(\alpha_t(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top)\right) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$$

Rewriting for clarity:

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$$

The two gates are complementary:

- $\alpha_t \to 0$ rapidly erases historical state: $\mathbf{S}_t \approx \beta_t \mathbf{v}_t\mathbf{k}_t^\top$ (context reset).
- $\alpha_t \to 1$ recovers pure delta-rule behavior: $\mathbf{S}_t \approx \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t\mathbf{k}_t^\top) + \beta_t \mathbf{v}_t\mathbf{k}_t^\top$ (targeted updates).
- $\beta_t \to 0$ skips writing but decays: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1}$ (pure forgetting).
- $\beta_t \to 1$ replaces memory aggressively: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top$ (replacement).

From an online learning perspective, Gated DeltaNet corresponds to:

$$\min_{\mathbf{S}_t} \|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2\langle \mathbf{S}_t\mathbf{k}_t, \beta_t(\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1}\mathbf{k}_t)\rangle$$

which introduces an adaptive weight decay $\alpha_t$ into an SGD-like update—analogous to decoupled weight decay in deep learning optimization.

**Algorithm 35** Gated DeltaNet Parallel Chunkwise Forward

**Require:** Input chunk $\mathbf{X}_{[i]}$; prior state $\mathbf{S}_{i-1}$; chunk size $C$
**Require:** Projections: $W_q, W_k, W_v, W_\alpha, W_\beta$
**Ensure:** Output $\mathbf{O}_{[i]}$ and state $\mathbf{S}_i$
**In-chunk computation:**
**for** $j = 1$ to $C$ **do**
  $\mathbf{q}_j \leftarrow W_q\mathbf{x}_j$, $\mathbf{k}_j \leftarrow W_k\mathbf{x}_j$, $\mathbf{v}_j \leftarrow W_v\mathbf{x}_j$
  $\alpha_j \leftarrow \text{sigmoid}(W_\alpha\mathbf{x}_j)$, $\beta_j \leftarrow \text{sigmoid}(W_\beta\mathbf{x}_j)$
  Apply combined gate: $\mathbf{S}_j \leftarrow \alpha_j\mathbf{S}_{j-1}(\mathbf{I} - \beta_j\mathbf{k}_j\mathbf{k}_j^\top) + \beta_j\mathbf{v}_j\mathbf{k}_j^\top$
  $\mathbf{o}_j \leftarrow \mathbf{S}_j\mathbf{q}_j$
**end for**
**Inter-chunk recurrence:**
$\mathbf{S}_i \leftarrow \mathbf{S}_C$ from in-chunk; propagate across chunks via cumulative $\prod \alpha$ masks
**Output gating:** $\mathbf{O}_{[i]} \leftarrow \text{GroupNorm}(\mathbf{O}^{\text{raw}}) \otimes \text{SiLU}(W_g\mathbf{X}_{[i]})$ **return** $\mathbf{O}_{[i]}, \mathbf{S}_i$

### 8.4.2 Empirical Performance and Trade-offs

Gated DeltaNet has been integrated into Alibaba's Qwen3-Next and Qwen3.5 production models. Empirically:

| Task | Gated DeltaNet | Mamba2 | DeltaNet | GLA |
|---|---|---|---|---|
| Language Modeling | **1.0**× | 1.08× | 1.05× | 1.12× |
| Associative Recall | **1.0**× | $--(noperfect)$ | **1.0**× | 0.75 |
| Length Extrapolation | **1.0**× | 0.98× | 0.96× | 0.94× |
| Throughput (tokens/sec) | 0.85× | **1.0**× | 0.82× | 0.88× |

Gated DeltaNet achieves the best balance across diverse tasks at the cost of slightly reduced throughput due to richer transition matrices.

## 8.5 4. HGRN2: Hierarchical Gating with Outer-Product Expansion

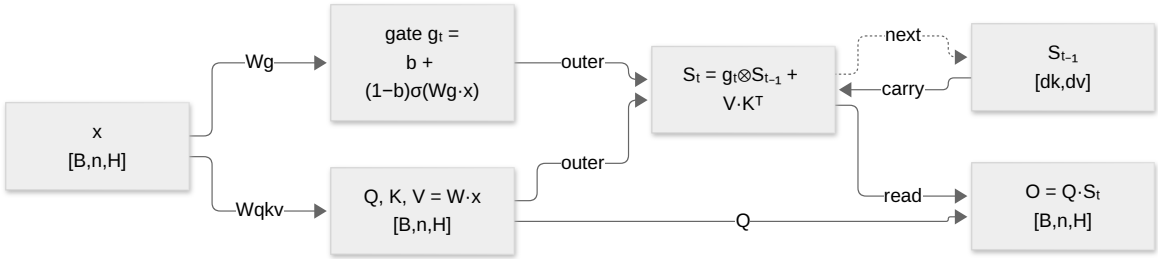



Figure 62: Dataflow of HGRN2: a hierarchical gate mixes the carried state with the key/value outer product.

### 8.5.1 Mathematical Foundation

HGRN2 [79] uses an outer-product-based state expansion with **hierarchically lower-bounded forget gates**. The state update is:

$$\mathbf{S}_t = \mathrm{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

where $\mathbf{g}_t \in [b_\ell, 1]^d$ is a lower-bounded forget gate for layer $\ell$. The bounds satisfy:

$$0 \le b_{\ell_{\text{shallow}}} < b_{\ell_{\text{middle}}} < b_{\ell_{\text{deep}}} \le 1$$

i.e., bounds increase (become less restrictive) in the deeper layers. The gate itself is computed as:

$$\mathbf{g}_t = b_\ell + (1 - b_\ell) \cdot \sigma(W_g \mathbf{x}_t)$$

where $\sigma$ is sigmoid. When $W_g$ outputs weakly negative logits, $\mathbf{g}_t \approx b_\ell$ (forced retention at shallow layers). When outputs are strongly positive, $\mathbf{g}_t \approx 1$ (memory refresh at any layer).

The hierarchical structure encourages different layers to specialize to different timescales: shallow layers model local dependencies (high retention due to binding $b_\ell$), while deeper layers capture long-range structure (flexible gating with high upper bound).

**Algorithm 36** HGRN2 Forward with Hierarchical Bounds

**Require:** Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1}$; layer index $\ell \in [0, L-1]$
**Require:** Non-decreasing bounds: $0 \le b_0 < b_1 < \cdots < b_{L-1} \le 1$
**Ensure:** Output $\mathbf{o}_t$ and state $\mathbf{S}_t$

1: Layer-specific retain bound: $b \leftarrow b_\ell$
2: Project: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
3: Compute forget gate with binding: $\mathbf{g}_t \leftarrow b + (1 - b) \cdot \sigma(W_g \mathbf{x}_t)$ ▷ Constrained to $[b, 1]$
4: Diagonal multiplication (vectorized): $\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \circ \mathrm{diag}(\mathbf{g}_t) + \mathbf{v}_t \mathbf{k}_t^\top$
5: Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t \mathbf{q}_t$
6: Optional normalization: $\mathbf{o}_t \leftarrow \mathrm{RMSNorm}(\mathbf{o}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$

### 8.5.2 Scaling and Empirical Results

At 3B scale on 100B tokens, HGRN2 slightly outperforms Mamba2 and LLaMA-architecture Transformers on language modeling. The hierarchical gating provides multi-scale temporal modeling without explicit layer-wise architectural variants. However, vector-valued diagonal gating (as opposed to element-wise selection) provides less per-item flexibility than delta-rule variants.

## 8.6 5. Forgetting Transformer (FoX): Gating in Softmax Logit Space

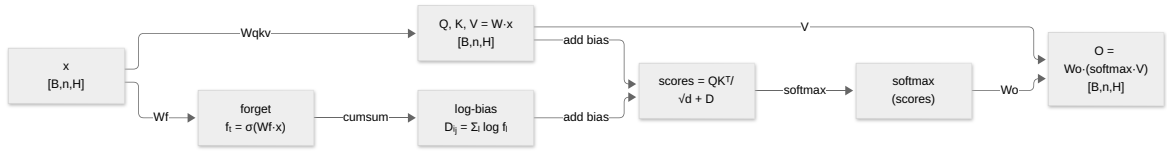



Figure 63: Dataflow of FoX: a learned forget gate becomes an additive log-bias on the score matrix.

### 8.6.1 Mathematical Foundation

Forgetting Transformer (FoX), proposed by Lin et al. [58], embeds a forget gate directly into softmax attention logits. Rather than replacing softmax with linear recurrence, FoX preserves full softmax expressiveness while adding recency control through a data-dependent logit bias.

For each token position $t$ and all keys $j \leq t$, compute a scalar forget gate:

$$f_t = \sigma(w_f^\top \mathbf{x}_t + b_f)$$

The logit bias at position $(i, j)$ is the cumulative log-forget product:

$$d_{ij} = \sum_{l=j+1}^{i} \log f_l = \log \prod_{l=j+1}^{i} f_l$$

The full attention output becomes:

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{D}\right)\mathbf{V}$$

where $\mathbf{D}_{ij} = d_{ij}$. Equivalently:

$$\text{Attention}(i, j) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k} + d_{ij})}{\sum_{j'=1}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k} + d_{ij'})}$$

This weighting down-weights older tokens multiplicatively: a token from 10 steps ago is scaled by $\prod_{l=j+1}^{i} f_l$, which is typically much less than 1 if $f_t < 1$ on average.

The mechanism is mathematically equivalent to a **data-dependent learnable variant of ALiBi (Attention with Linear Biases)**, where earlier work used fixed $d_{ij} = -\infty \cdot |i - j|$ or $d_{ij} = -(i - j)$.

**Algorithm 37** FoX: Forgetting Attention with Recency Bias

**Require:** Queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V}$; input $\mathbf{X}$
**Require:** Forget gate parameters: $w_f \in \mathbb{R}^d$, $b_f \in \mathbb{R}$
**Ensure:** Output attention $\mathbf{O}$

**Compute forget gates:**
**for** $t = 1$ to $T$ **do**
    $f_t \leftarrow \sigma(w_f^\top \mathbf{x}_t + b_f)$ ▷ Per-token forget probability
**end for**
**Compute cumulative log-forget biases:**
**for** $i = 1$ to $T$ **do**
    **for** $j = 1$ to $i$ **do**
        $d_{ij} \leftarrow \sum_{l=j+1}^{i} \log f_l$ ▷ Cumulative product in log space
    **end for**
**end for**
**Standard SDPA with bias:**
Compute scores: $\mathbf{S} \leftarrow \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$
Add bias: $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{D}$
Apply softmax: $\mathbf{A} \leftarrow \mathrm{softmax}(\mathbf{S}, \mathrm{dim} = -1)$
Weight values: $\mathbf{O} \leftarrow \mathbf{A}\mathbf{V}$ **return O**

### 8.6.2 Integration with FlashAttention

The key advantage of FoX is compatibility with FlashAttention and existing optimized implementations. The forget biases are added in the softmax logit computation, which is already a core operation in FlashAttention's block-wise algorithm. The overhead is:

$$\text{Overhead} \approx 0.5 - -2\% \text{ wall-clock time}$$

since the bias addition is amortized across matmul operations.

### 8.6.3 Strengths and Limitations

FoX achieves state-of-the-art results on length extrapolation, near-perfect needle-in-haystack retrieval, and superior long-context understanding compared to Transformers. However, it remains $\mathcal{O}(L^2 d)$ at training and $\mathcal{O}(Ld)$ per step at inference with KV cache, limiting extreme sequence lengths.

## 8.7 6. Gated Attention (Post-SDPA Sigmoid Gating)

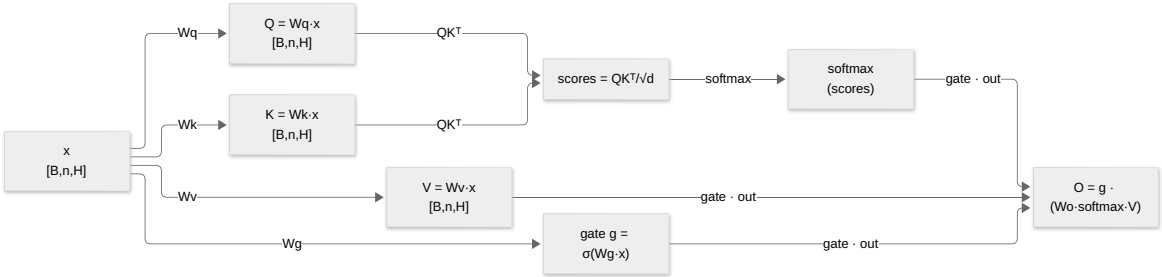



Figure 64: Dataflow of post-SDPA gated attention: a sigmoid gate scales the softmax output before the projection.

### 8.7.1 Mathematical Foundation

The NeurIPS 2025 Best Paper by the Qwen team proposes applying a sigmoid gate **after** scaled dot-product attention (SDPA):

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma\left(\mathbf{X}\mathbf{W}_\theta\right)$$

where the gate score $\sigma(\mathbf{X}\mathbf{W}_\theta)$ can have shape $\mathbb{R}^{n\times q\times d_k}$ (element-wise gating per query head) or $\mathbb{R}^{n\times q}$ (head-wise fixed gating). Across 30+ variants and 15B MoE + 1.7B dense models trained on 3.5T tokens, the team found:

- Headwise gating adds only $\sim 1.6M$ parameters to a 15B model (negligible overhead).
- Effective gates are **sparse**: mean activation $\approx 0.116$ (i.e., 88% are zero or near-zero).
- Gates act as query-dependent filters, suppressing low-value channels while preserving high-value signal.
- Gates demonstrably eliminate the **attention sink** phenomenon, where the attention pattern becomes dominated by a single early token.

**Algorithm 38** Gated Softmax Attention (Post-SDPA)

**Require:** Queries $\mathbf{Q}$, Keys $\mathbf{K}$, Values $\mathbf{V}$; input $\mathbf{X}$
**Require:** Gate projection matrix $W_g \in \mathbb{R}^{d\to d_{\text{gate}}}$ where $d_{\text{gate}} \in \{1, d_k\}$
**Ensure:** Gated output $\mathbf{Y}'$

**Standard SDPA:**
Compute attention: $\mathbf{A} \leftarrow \text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_k})$
Weight values: $\mathbf{Y} \leftarrow \mathbf{A}\mathbf{V}$
**Compute gates:**
Project input: $\mathbf{G} \leftarrow \mathbf{X}W_g$ ▷ shape: $[n, q, d_{\text{gate}}]$
Apply sigmoid: $\mathbf{G} \leftarrow \sigma(\mathbf{G})$ ▷ Element-wise gating
**Apply gating:**
**if** $d_{\text{gate}} = 1$ **then**
    Broadcast and scale: $\mathbf{Y}' \leftarrow \mathbf{Y} \cdot \mathbf{G}$ ▷ Head-wise scaling
**else** ▷ $d_{\text{gate}} = d_k$
    Element-wise modulation: $\mathbf{Y}' \leftarrow \mathbf{Y} \odot \mathbf{G}$ ▷ Per-dimension gating
**end ifreturn** $\mathbf{Y}'$

### 8.7.2 Key Findings

- **Attention-sink suppression**: Gating breaks the feedback loop where softmax concentrates on the first token, enabling more balanced attention.
- **Training stability**: Models with gating tolerate larger learning rates (+30% higher stable $\eta_{max}$).
- **Minimal overhead**: Latency impact is only 1.6% on H100 GPUs; throughput drops at most $2-3\%$.
- **Scaling law improvement**: Improves loss across model scales from 800M to 110B parameters consistently.

## 8.8 7. Gated DeltaNet-2: Decoupled Channel-wise Erase and Write

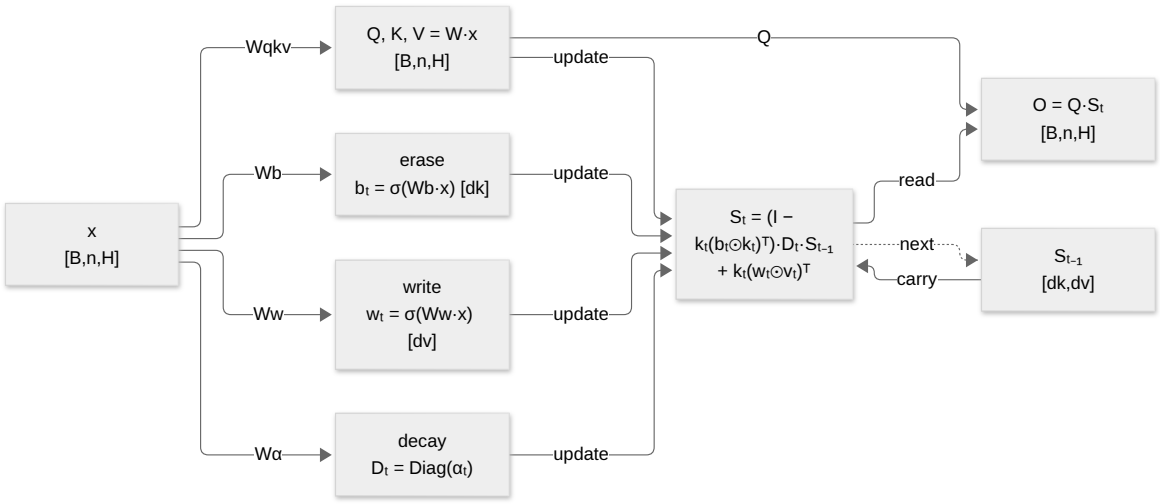



Figure 65: Dataflow of Gated DeltaNet-2: decoupled channel-wise erase and write gates act inside the delta rule.

### 8.8.1 Mathematical Foundation

Gated DeltaNet-2 [38] addresses a shared limitation of Gated DeltaNet and Kimi Delta Attention (KDA): both use a single *scalar* delta gate $\beta_t$ to simultaneously control (i) how much old content is erased along the key-side read direction and (ii) how much new value is written along the value axis. Gated DeltaNet-2 **decouples** these two roles into a channel-wise erase gate $\mathbf{b}_t$ (key axis) and a channel-wise write gate $\mathbf{w}_t$ (value axis), while inheriting the channel-wise decay $\boldsymbol{\alpha}_t$ from KDA. With state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$, decay $\mathbf{D}_t = \mathrm{Diag}(\boldsymbol{\alpha}_t)$, erase direction $\mathbf{e}_t = \mathbf{b}_t \odot \mathbf{k}_t$, and write target $\mathbf{z}_t = \mathbf{w}_t \odot \mathbf{v}_t$, the state update is:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t \mathbf{e}_t^\top\right) \mathbf{D}_t \, \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{z}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

or, expanding the gate products:

$$\mathbf{S}_t = \left(\mathbf{I} - \mathbf{k}_t (\mathbf{b}_t \odot \mathbf{k}_t)^\top\right) \mathbf{D}_t \, \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \odot \mathbf{v}_t)^\top$$

The two gates are produced by independent linear projections followed by sigmoid:

$$\mathbf{b}_t = \sigma(\mathbf{W}_b \mathbf{x}_t), \qquad \mathbf{w}_t = \sigma(\mathbf{W}_w \mathbf{x}_t)$$

and the channel-wise decay follows a log-decay parameterization (computed in fp32 to avoid precision loss in long cumulative products):

$$\mathbf{g}_t = \mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$$

The structural roles of the two gates are complementary:

- $\mathbf{b}_t$ (key axis) makes the **read direction** channel-selective: it determines which key channels are removed before writing.
- $\mathbf{w}_t$ (value axis) makes the **write target** channel-selective: it determines which value channels are deposited into memory.
- $\boldsymbol{\alpha}_t$ (per key channel) provides global decay, as in KDA.

**Reductions.** Setting $\mathbf{b}_t = \beta_t \mathbf{1}_{d_k}$ and $\mathbf{w}_t = \beta_t \mathbf{1}_{d_v}$ recovers KDA; further collapsing $\boldsymbol{\alpha}_t$ to a scalar recovers Gated DeltaNet. From the fast-weight / online-learning perspective, the update is the minimizer of:

$$\mathcal{L}_t(\mathbf{S}) = \tfrac{1}{2}\|\mathbf{S} - \mathbf{D}_t\mathbf{S}_{t-1}\|_F^2 - 2\left\langle \mathbf{S}^\top \mathbf{k}_t,\ \mathbf{z}_t - (\mathbf{D}_t\mathbf{S}_{t-1})^\top \mathbf{e}_t \right\rangle$$

**Algorithm 39** Gated DeltaNet-2 Recurrent Forward (Reference Form)

**Require:** Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$
**Require:** Projections: $W_q, W_k, W_v, W_b, W_w$; log-decay parameters $\mathbf{a}, \boldsymbol{\delta}$
**Ensure:** Output $\mathbf{o}_t$ and state $\mathbf{S}_t$

$\mathbf{q}_t \leftarrow \text{normalize}(W_q\mathbf{x}_t)$, $\mathbf{k}_t \leftarrow \text{normalize}(W_k\mathbf{x}_t)$, $\mathbf{v}_t \leftarrow \text{silu}(W_v\mathbf{x}_t)$
$\mathbf{b}_t \leftarrow \sigma(W_b\mathbf{x}_t)$ ▷ key-axis erase gate, $\mathbb{R}^{d_k}$
$\mathbf{w}_t \leftarrow \sigma(W_w\mathbf{x}_t)$ ▷ value-axis write gate, $\mathbb{R}^{d_v}$
$\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\boldsymbol{\delta}))$ ▷ channel-wise decay, fp32
Erase direction: $\mathbf{e}_t \leftarrow \mathbf{b}_t \odot \mathbf{k}_t$; write target: $\mathbf{z}_t \leftarrow \mathbf{w}_t \odot \mathbf{v}_t$
Decay state: $\mathbf{S}_t \leftarrow \text{Diag}(\boldsymbol{\alpha}_t)\,\mathbf{S}_{t-1}$
Read: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{e}_t$ ▷ sum over key axis
Delta-rule update: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \mathbf{k}_t\mathbf{r}_t^\top + \mathbf{k}_t\mathbf{z}_t^\top$
Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
Output gating: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g\mathbf{x}_t)$
**return** $\mathbf{o}_t$, $\mathbf{S}_t$

The chunkwise training algorithm extends KDA's WY representation by absorbing the channel-wise decay into an asymmetric delta recurrence on a decay-normalized state, enabling matmul-rich parallelism on tensor cores. Because $\mathbf{b}_t$ and $\mathbf{w}_t$ are per-channel diagonal operators varying per row, the scalar post-scaling shortcut of KDA breaks: the backward pass requires a **gate-aware WY** inversion that bakes the gates into each dot product accumulating the gradient.

### 8.8.2 Empirical Performance and Trade-offs

At 1.3B parameters trained on 100B FineWeb-Edu tokens, Gated DeltaNet-2 outperforms Mamba-2, Mamba-3, Gated DeltaNet, and KDA on language modeling, commonsense reasoning, and—most markedly—real-world and RULER long-context retrieval (especially multi-key needle-in-a-haystack). Its throughput on H100 stays near-flat (38.0 → 36.1 Kt/s) as sequence length grows from 2K to 16K, where a Transformer's drops sharply. Ablations show the erase gate $\mathbf{b}_t$ accounts for most of the gain; the write gate $\mathbf{w}_t$ contributes a smaller additional improvement. The hybrid variant (recurrent Gated DeltaNet-2 + sliding-window attention) further closes the gap with pure-attention models on tasks requiring local evidence aggregation.

| Aspect | Strength | Limitation |
|---|---|---|
| Recall | Best-in-class among recurrent delta-rule models on multi-key NIAH. | Recurrent variant still gaps Transformer on NQ/DROP. |
| Throughput | Near-flat scaling 2K→16K; small overhead vs KDA. | Requires a new gate-aware WY Triton backward kernel. |
| Memory | Constant $\mathcal{O}(d^2)$ inference state. | Neg-eigenvalue erase range $[0, 2]$ gives no consistent gain at 1.3B scale. |

Table 13: Gated DeltaNet-2 trade-offs at 1.3B / 100B-token scale [38].

## 8.9 8. Kimi Delta Attention (KDA): Channel-wise Decay Gating for the Delta Rule

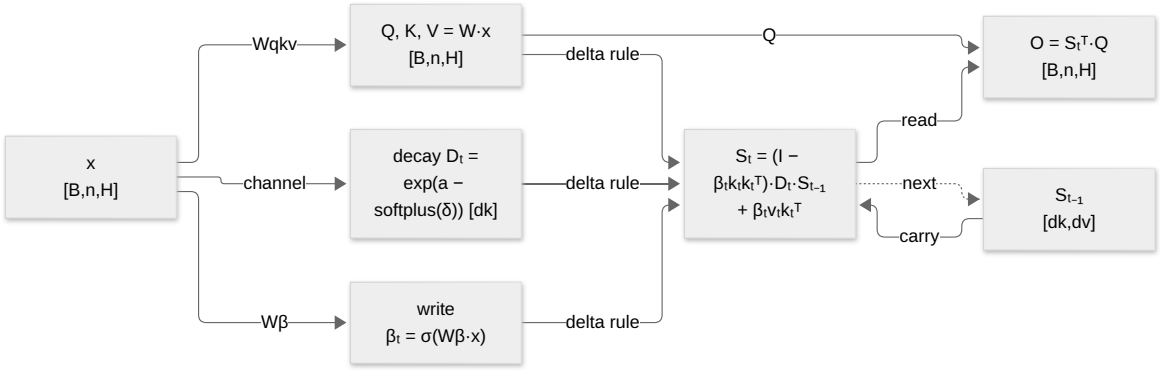



Figure 66: Dataflow of KDA: channel-wise decay and a scalar write gate modulate the delta-rule state update.

### 8.9.1 Mathematical Foundation

Kimi Delta Attention [51] is the core module of Kimi Linear and extends Gated DeltaNet with a finer-grained **channel-wise decay gate**. Where Gated DeltaNet uses a single scalar per-head decay $\alpha_t$ alongside the scalar delta write gate $\beta_t$, KDA replaces the scalar decay with a **per-key-channel** decay $\boldsymbol{\alpha}_t \in \mathbb{R}^{d_k}$, parameterised through a log-decay formulation and applied as a diagonal matrix $\mathbf{D}_t = \mathrm{diag}(\boldsymbol{\alpha}_t)$ before the delta-rule update. The delta rule itself retains a scalar per-head write gate $\beta_t$:

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\, \mathbf{D}_t\, \mathbf{S}_{t-1} + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \qquad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t$$

The channel-wise decay follows a log-decay parameterisation (computed in fp32 for numerical stability over long cumulative products):

$$\mathbf{g} = \mathbf{a} - \mathrm{softplus}(\boldsymbol{\delta}), \qquad \boldsymbol{\alpha}_t = \exp(\mathbf{g})$$

where $\mathbf{a}$ and $\boldsymbol{\delta}$ are learned per-channel parameters. This gives each key channel an independent forgetting timescale, allowing the model to retain some channels longer than others within the same head.

**Reduction to Gated DeltaNet** : When $\boldsymbol{\alpha}_t$ collapses to a scalar (i.e., all channels share the same decay), KDA reduces exactly to Gated DeltaNet. The channel-wise decay is thus a strict generalisation that adds per-channel selectivity on top of the scalar decay/write gating of Gated DeltaNet.

**Chunkwise training** : A bespoke chunkwise algorithm uses a specialised **Diagonal-Plus-Low-Rank (DPLR)** variant of the transition matrix to remain hardware-efficient. The product of per-step transitions $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\mathbf{D}_t$ admits a DPLR decomposition that allows matmul-rich parallelism on tensor cores while preserving the channel-wise decay structure.

**Algorithm 40** Kimi Delta Attention Recurrent Forward (Reference Form)

**Require:** Input $\mathbf{x}_t$; prior state $\mathbf{S}_{t-1} \in \mathbb{R}^{d_k \times d_v}$
**Require:** Projections: $W_q, W_k, W_v, W_\beta$; log-decay parameters $\mathbf{a}, \boldsymbol{\delta}$
**Ensure:** Output $\mathbf{o}_t$ and state $\mathbf{S}_t$
$\mathbf{q}_t \leftarrow W_q\mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k\mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v\mathbf{x}_t$
$\beta_t \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_t)$ ▷ scalar per-head write gate
$\boldsymbol{\alpha}_t \leftarrow \exp(\mathbf{a} - \text{softplus}(\boldsymbol{\delta}))$ ▷ channel-wise decay, fp32
$\mathbf{D}_t \leftarrow \text{diag}(\boldsymbol{\alpha}_t)$ ▷ diagonal decay matrix
Decay state: $\mathbf{S}_t \leftarrow \mathbf{D}_t\mathbf{S}_{t-1}$
Read: $\mathbf{r}_t \leftarrow \mathbf{S}_t^\top \mathbf{k}_t$ ▷ current prediction along key
Delta-rule update: $\mathbf{S}_t \leftarrow \mathbf{S}_t - \beta_t \mathbf{k}_t \mathbf{r}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
Retrieve: $\mathbf{o}_t \leftarrow \mathbf{S}_t^\top \mathbf{q}_t$
Output gating: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t) \odot \text{silu}(W_g\mathbf{x}_t)$
**return** $\mathbf{o}_t$, $\mathbf{S}_t$

### 8.9.2 Empirical Performance and Trade-offs

KDA is deployed in Kimi Linear, a hybrid model (3B activated / 48B total parameters) combining KDA layers with Multi-head Latent Attention (MLA). Kimi Linear outperforms a full-MLA baseline across all evaluated tasks, reduces the KV cache by up to 75%, and achieves up to 6× decoding throughput for 1M-token context. The channel-wise decay is the principal driver of the gain: it allows fine-grained control over which key channels persist, improving long-context recall relative to scalar-decay Gated DeltaNet.

| Aspect | Strength | Limitation |
|---|---|---|
| Recall | Channel-wise decay improves long-context recall over scalar-decay baselines; deployed in Kimi Linear (3B/48B hybrid). | Recurrent matrix state $\mathcal{O}(d^2)$ is heavier than diagonal-gated linear attention. |
| Throughput | Up to 6× decoding throughput at 1M context; 75% KV cache reduction vs full MLA. | Requires DPLR-aware chunkwise kernels for hardware efficiency. |
| Memory | Constant $\mathcal{O}(d^2)$ inference state (matrix recurrent). | Log-decay fp32 computation adds overhead; reduction to Gated DeltaNet loses channel selectivity. |

Table 14: KDA trade-offs in Kimi Linear (3B activated / 48B total) [51].

## 8.10 Unified Gating Principle

All eight architectures share a common principle: **gating is a mechanism to control information flow and selective memory retention**. The specific design choices diverge along several dimensions:

1. **Location of gating**: Recurrent state updates (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2) versus softmax logit/output space (FoX, Gated Softmax, RetNet).
2. **Granularity of control**: Dimension-wise (GLA, HGRN2 diagonal), key-specific (DeltaNet), token-wise (FoX), head-wise (Gated Softmax), or channel-wise decoupled erase/write (Gated DeltaNet-2).
3. **Data dependence**: Fully data-dependent (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, FoX, Gated Softmax) versus fixed schedules (RetNet with exponential decay).
4. **Expressiveness trade-off**: Linear recurrent efficiency (GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2, RetNet) versus full softmax expressiveness (FoX, Gated Softmax).

## 8.11 Implementation and Practical Guidance

### 8.11.1 When to Use Each Architecture

1. **GLA**: Fast inference with moderate-to-long contexts (2K–20K tokens), when custom CUDA kernels are acceptable, and retrieval performance is not critical.

2. **DeltaNet**: Strong associative recall and in-context learning tasks; good for synthetic tasks (MQAR) and key-value association testing.

3. **Gated DeltaNet**: Production models requiring the best balance of recall, forgetting, and throughput (proven in Qwen3-Next; ICLR 2025).

4. **Gated DeltaNet-2**: Long-context retrieval workloads (multi-key NIAH) where decoupled channel-wise erase/write yields the strongest recall; when a gate-aware WY kernel is available.

5. **RetNet**: Extremely efficient inference without KV caching; suitable for edge/mobile deployments or toy models.

6. **HGRN2**: Multi-scale temporal hierarchies; when layer-specific forget bounds are desirable.

7. **FoX**: Length extrapolation and long-context understanding; when quadratic training is acceptable and replacing softmax is not desired.

8. **Gated Softmax**: Quick improvement to existing softmax models with minimal code changes; best for practitioners wanting immediate gains without major refactoring.

### 8.11.2 Hardware Considerations

| Hardware Target | Recommended Architectures | Notes |
|---|---|---|
| GPU (H100/A100) | Gated Softmax, FoX, Gated DeltaNet, Gated DeltaNet-2 | Mature softmax/linear implementations. GLA/DeltaNet require custom kernels. |
| TPU (high memory) | GLA, DeltaNet, Gated DeltaNet, Gated DeltaNet-2, HGRN2 | Hardware supports efficient matmuls; linear recurrent methods shine. |
| Mobile/Edge | RetNet, lightweight Gated Softmax | Constant-memory inference critical. |

# 9 Normalization, Quantization, Depth Routing, and Runtime Algorithms

This annex collects the implementation-level detail for normalization variants, BitNet-style ternary quantization, adaptive depth routing (Mixture-of-Depths), conditional memory (Engram), factorized embeddings, and the runtime algorithms (training-step stability controls, SBERT inference router) that support the configuration-driven pipeline. (The manifold-constrained residual connections of mHC are documented separately in Annex 9.) The main sections defer to this annex for the mathematical formulations and pseudocode; the bibliography keys cited here resolve against `docs/bibliography/`.

## 9.1 Normalization Variants: LayerNorm, RMSNorm, pRMSNorm, Dynamic Tanh, Dynamic Erf, and FlashNorm

Normalization determines how activation scale is controlled across depth. In this repository, normalization is not only a modeling choice but also a schema compatibility question, because only certain values are currently accepted by `norm_type`. The schema contract is:

$$\texttt{norm_type} \in \{\texttt{layer_norm}, \texttt{dynamic_tanh}, \texttt{derf}, \texttt{rms_norm}, \texttt{prms_norm}, \texttt{flash_norm}\}$$

The formulations most relevant to this codebase are:

### 9.1.1 LayerNorm (Baseline)

LayerNorm [4] is the default normalizer; it centers and rescales each token by its per-token mean and variance:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2, \quad y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

### 9.1.2 RMSNorm

RMSNorm removes mean-centering and only rescales by root mean square magnitude [113]:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}, \qquad y_i = \gamma_i \frac{x_i}{\mathrm{RMS}(x)}.$$

Compared with LayerNorm, RMSNorm is computationally simpler (no subtraction of feature mean) and is often used when reducing normalization overhead is important (e.g., Llama, GPT-NeoX). It has a learnable per-dimension scale $\gamma$ but no bias term.

### 9.1.3 Partial RMSNorm (pRMSNorm)

pRMSNorm [113] further reduces overhead by estimating the RMS from only the first $p\%$ of the hidden dimensions:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{k}\sum_{i=1}^{k} x_i^2 + \epsilon}, \qquad k = \lceil d \cdot p \rceil,$$

exploiting the assumption that neurons within a layer are approximately i.i.d. The same scale $\gamma$ is then applied to all dimensions. The partial ratio $p$ is controlled by the `prms_partial_ratio` config field (default 6.25%). The paper reports that competitive results are achievable with as little as 6.25% of the inputs, though gradient instability can arise with very small ratios.

### 9.1.4 Dynamic Tanh (DyT)

Dynamic Tanh proposes replacing explicit normalization with a bounded elementwise map [119]:

$$\mathrm{DyT}(x) = \tanh(\alpha x)$$

where $\alpha$ is learned. The core idea is that bounded nonlinear contraction can provide stable signal scaling without explicitly computing per-token normalization statistics.

### 9.1.5 Dynamic Erf (Derf)

Derf extends the same normalization-free direction by using an error-function based map [13]:

$$\mathrm{Derf}(x) = \mathrm{erf}(\alpha x + s)$$

with learnable scale/shift. Reported results in the cited work indicate stronger performance than DyT and common normalization baselines across multiple domains.

### 9.1.6 FlashNorm (Weightless RMSNorm)

FlashNorm [34] is not a new mathematical normalization; it is an *exact algebraic rewrite* of the ubiquitous "RMSNorm → Linear" pair that (i) eliminates the per-dimension learnable scale $\mathbf{g}$ by folding it into the subsequent linear layer's weights, and (ii) defers the scalar RMS reduction to the output of the matrix multiplication so that the matmul (matrix unit) and the RMS reduction (vector unit) execute in parallel. Three propositions formalize the rewrite.

**Proposition 1 (Weight folding).** Given activation $\mathbf{a} \in \mathbb{R}^n$, RMSNorm weight $\mathbf{g} \in \mathbb{R}^n$, and a subsequent linear $\mathbf{W} \in \mathbb{R}^{n \times k}$,

$$\mathbf{z} = \mathrm{RMSNorm}(\mathbf{a})\mathbf{W} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*, \qquad W^*_{i,j} = g_i \cdot W_{i,j} \;\Leftrightarrow\; \mathbf{W}^* = \mathrm{diag}(\mathbf{g})\mathbf{W}.$$

The per-dimension scale $\mathbf{g}$ becomes redundant and is folded into $\mathbf{W}$ once at load time. The standalone `FlashNorm` module in this repository applies this directly: it has *no learnable parameters* (the scale is meant to be absorbed by the downstream projection).

**Proposition 2 (Deferred normalization).** For a bias-free linear $\mathbf{W}^*$, scalar–matrix commutativity yields

$$\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^* = (\mathbf{a}\mathbf{W}^*) \cdot \frac{1}{\mathrm{RMS}(\mathbf{a})}.$$

The matmul $\mathbf{a}\mathbf{W}^*$ and the RMS reduction are independent of each other; on hardware with distinct matrix and vector execution units they execute in parallel, and only a single vector–scalar multiply synchronizes them. The fused module `FlashNormLinear` implements this rewrite: on the bias-free path it computes the matmul on the *un-normalized* input and multiplies the result by the per-token reciprocal RMS. When the linear has a bias $\mathbf{c}$, the rewrite is no longer exact (Remark 1 of the paper): $(\mathbf{a}\mathbf{W}^* + \mathbf{c})/\mathrm{RMS}(\mathbf{a}) \neq \mathbf{a}\mathbf{W}^*/\mathrm{RMS}(\mathbf{a}) + \mathbf{c}$; the layer then applies the scalar to the input first and lets the linear add its bias as usual.

**Proposition 3 (RMS cancellation).** By RMS scale invariance, $\mathrm{RMS}(\alpha\mathbf{a}) = \alpha\,\mathrm{RMS}(\mathbf{a})$. Therefore an "RMSNorm → Linear → RMSNorm" pattern allows the *first* RMSNorm to be dropped entirely:

$$\mathrm{RMSNorm}\left(\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\mathbf{W}^*\right) = \mathrm{RMSNorm}(\mathbf{a}\mathbf{W}^*)\,.$$

This cancellation is relevant for QKV-normalization (Gemma 4) and MLA latent normalization (DeepSeek-V2, Mistral Small 4); it is not auto-applied in this repository but is documented for completeness.

**Partial-RMS composition.** FlashNorm composes naturally with the pRMSNorm trick (§9, pRMSNorm subsection): when `flashnorm_partial_ratio` $= p > 0$, the RMS is estimated from the first $k = \lceil d \cdot p \rceil$ dimensions of the input only, reducing the cost of the vector-unit RMS reduction further.

**BitNet interaction.** Folding a floating-point $\mathbf{g}$ into BitNet's ternary weight tensor $\{-1, 0, +1\}$ [99] would destroy the quantization, and BitLinear applies its own internal LayerNorm before activation quantization which needs the normalized input. The fused module `FlashNormBitLinear` therefore falls back to the sequential composition FlashNorm → BitLinear: the matmul and the RMS reduction still execute in parallel, but the post-matmul scalar multiply of Prop. 2 is sacrificed. Folding f.p. $\mathbf{g}$ into a 1.58-bit weight tensor is left to future kernel-level work.

**PyTorch pseudocode.**

**Input:** activation $\mathbf{a} \in \mathbb{R}^{T\times n}$, RMSNorm weight $\mathbf{g} \in \mathbb{R}^n$, linear $\mathbf{W} \in \mathbb{R}^{n\times k}$, bias $\mathbf{c} \in \mathbb{R}^k$ or `None`
`// Prop. 1: weight folding at load time`
$\mathbf{W}^* \leftarrow \mathbf{g} \odot \mathbf{W}$ ▷ column-wise; `g.unsqueeze(0) * W` in PyTorch
$\mathbf{g} \leftarrow \mathbf{1}$ ▷ the standalone norm is now weightless
`// Prop. 2: forward pass with deferred RMS (bias-free path)`
$\mathbf{b} \leftarrow \mathbf{a}\mathbf{W}^{*\top}$ ▷ matrix unit / tensor cores
rms_inv $\leftarrow$ rsqrt(mean($\mathbf{a}^2$) + $\varepsilon$) ▷ vector unit, independent of $\mathbf{b}$
**if** $\mathbf{c} =$ `None` **then**
  **return** $\mathbf{b} \cdot$ rms_inv ▷ single vector–scalar multiply
**else**
  `// Remark 1: bias path is sequential; matmul and RMS still parallelize`
  **return** $(\mathbf{a} \cdot \text{rms_inv})\mathbf{W}^{*\top} + \mathbf{c}$
**end if**

**Reported latency (NVIDIA T4 GPU).** The paper reports the following speedups on the norm-then-project operation, with an adaptive dispatch that falls back to the sequential path in the memory-bound (decode) regime:

| Scale | Tokens | Sequential (ms) | FlashNorm (ms) | Speedup | Notes |
|---|---|---|---|---|---|
| SmolLM2-135M | 4096 | 0.704 | 0.468 | +33.6% | compute-bound (prefill) |
| SmolLM2-135M | 8192 | 0.929 | 0.599 | +35.5% | compute-bound (prefill) |
| Llama-7B | 1024 | 1.882 | 1.654 | +12.1% | compute-bound (prefill) |
| Llama-7B | 4096 | 7.628 | 6.570 | +13.9% | compute-bound (prefill) |

**Zero-loss weight folding validation.** Weight folding (Prop. 1) is mathematically exact; the paper verified bit-identical outputs on three open-weight checkpoints:

| Model | Precision | Result |
|---|---|---|
| SmolLM2-135M | fp16 | max diff = 0.0, cosine sim = 1.0, identical text |
| Llama-3.2-1B | fp32 | bit-identical greedy generation |
| Llama-3.1-8B | fp32 | bit-identical greedy generation |

Quality preservation on `lm-evaluation-harness` (wikitext word_ppl, MMLU 5-shot, HellaSwag 0-shot) stays within benchmark noise (worst $\Delta = +0.0017$ word_ppl).

### 9.1.7 Comparison

| Method | Formula | Stats Needed | Notes |
|---|---|---|---|
| LayerNorm | $\gamma_i(x_i - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta_i$ | mean + variance | Default; full centering and rescaling [4]. |
| RMSNorm | $\gamma_i x_i / \sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$ | RMS only | Lower overhead; widely used baseline [113]. |
| pRMSNorm | $\gamma_i x_i / \sqrt{\frac{1}{k}\sum_{j\le k} x_j^2 + \epsilon}$ | Partial RMS | RMS from first $p$% of dims; `prms_partial_ratio` default 6.25% [113]. |
| Dynamic Tanh | $\tanh(\alpha x)$ | none | Normalization-free bounded transform; drop-in replacement [119]. |

| Method | Formula | Stats Needed | Notes |
|---|---|---|---|
| Dynamic Erf | $\mathrm{erf}(\alpha x + s)$ | none | Normalization-free alternative; improves over DyT [13]. |
| FlashNorm | $x_i/\sqrt{\frac{1}{d}\sum_j x_j^2 + \epsilon}$ (weightless) | RMS only | Exact rewrite of RMSNorm→Linear: folds $\mathbf{g}$ into $\mathbf{W}$ (Prop. 1), defers scalar RMS to matmul output (Prop. 2); 12–35% latency reduction on T4 GPU [34]. |

## 9.2 BitNet Ternary Quantization Path

When `use_bitnet=true`, BitLinear layers replace standard `nn.Linear` with ternary weight quantization $(-1, 0, +1)$ following BitNet [99]. Given weight tensor $W$, a practical scaling is:

$$s = \mathrm{mean}(|W|), \qquad \tilde{W} = \mathrm{clip}\left(\mathrm{round}\left(\frac{W}{s}\right), -1, 1\right).$$

The packed mapping uses two bits per weight symbol for storage efficiency. This enables significant compression ($\sim 32\times$ versus FP32) but is experimental and may affect model quality; it is best suited for inference deployment.

Activation quantization for the INT8 path is:

$$q = \mathrm{round}\left(x \cdot \frac{127}{\max(|x|) + \epsilon}\right), \qquad q \in [-128, 127]$$

with dequantization $x \approx q/\alpha$. For $N$ parameters, size estimates before metadata and packing overhead are:

$$\text{FP32 size} \approx 4N, \quad \text{FP16 size} \approx 2N, \quad \text{1.58-bit size} \approx \frac{1.58}{8}N$$

which aligns with lightweight deployment goals [85, 12].

## 9.3 Mixture-of-Depths (MoD) Token Routing

When `use_mixture_of_depths=true`, each transformer block scores tokens with a lightweight router. Only the top `mixture_of_depths_capacity_ratio` subset is updated; skipped tokens pass through unchanged [120]. This allocates more depth to salient tokens, reducing per-layer compute.

**Input:** hidden states $H \in \mathbb{R}^{n \times d}$, router $R$, capacity ratio $\rho$
scores $\leftarrow R(H)$ ▷ per-token router score
$\mathcal{S} \leftarrow$ `top-k`$(\text{scores}, k = \lceil \rho \cdot n \rceil)$ ▷ select salient subset
$H_{\text{update}} \leftarrow$ `block`$(H[\mathcal{S}])$ ▷ apply transformer block to selected tokens
$H[\mathcal{S}] \leftarrow H_{\text{update}}$ ▷ write back; non-selected tokens bypass the block
**Output:** $H$ ▷ auxiliary load-balancing loss added from `mixture_of_depths_router_aux_loss_weight`

### 9.3.1 Configuration

- `use_mixture_of_depths`: bool — enable MoD token routing.
- `mixture_of_depths_capacity_ratio`: float in $(0, 1]$ — fraction of tokens updated per layer.
- `mixture_of_depths_router_aux_loss_weight`: float $\geq 0$ — auxiliary loss weight for router regularization.

## 9.4 Engram Conditional Memory

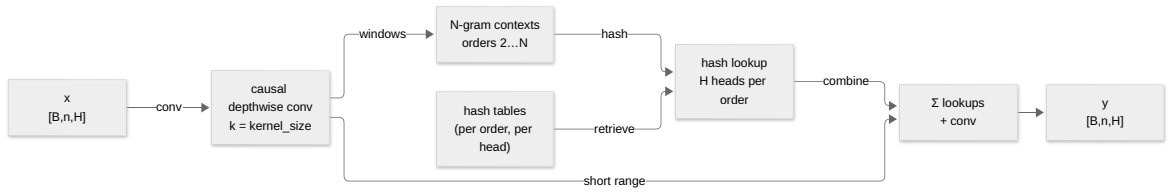



Figure 67: Dataflow of Engram conditional memory: a causal depthwise convolution supplies short-range context, N-gram hash lookups retrieve per-order embeddings, and the lookups are summed with the convolution output.

Engram (`engram_attn`) implements conditional memory via scalable N-gram lookup [15]. It builds hash-based lookup tables for N-gram contexts from size 2 up to `engram_max_ngram_size`, with independent hash heads per N-gram order. A causal depthwise convolution (`engram_kernel_size`) provides short-range context before the N-gram lookup.

**Input:** token stream $x_{1:n}$, N-gram orders $\{2, \ldots, N\}$, hash heads per order $H$
$h_{\text{conv}} \leftarrow \text{DepthwiseConv1d}(x_{1:n}, k = \texttt{engram_kernel_size})$ ▷ short-range context
**for** each N-gram order $k \in \{2, \ldots, N\}$ **do**
    $\text{ctx}_t^{(k)} \leftarrow x_{t-k+1:t}$ ▷ N-gram context window
    $\text{lookup}_t^{(k)} \leftarrow \text{HashTable}^{(k)}[\text{hash}(\text{ctx}_t^{(k)})]$ ▷ per-head retrieval, $H$ heads per order
**end for**
$y_t \leftarrow \sum_k \text{lookup}_t^{(k)} + h_{\text{conv},t}$ ▷ combine conditional memories
**Output:** $y_{1:n}$

### 9.4.1 Configuration

- `engram_max_ngram_size`: int $\geq 2$ — highest N-gram order (default 3).
- `engram_n_heads_per_ngram`: int $\geq 1$ — hash heads per N-gram order (default 4).
- `engram_embed_dim_per_head`: int $\geq 1$ — embedding dim per hash head (default 32).
- `engram_kernel_size`: int $\geq 1$ — causal depthwise conv kernel width (default 4).
- `engram_seed`: int — hash seed for reproducibility (default 42).

Total Engram hidden size = $(\texttt{max_ngram_size} - 1) \times \texttt{n_heads_per_ngram} \times \texttt{embed_dim_per_head}$.

## 9.5 Factorized Embeddings and Embedding Convolution

### 9.5.1 Factorized Embeddings

When `use_factorized_embedding=true`, the embedding matrix is factorized into two smaller matrices, reducing parameters from $O(V \times H)$ to $O(V \times R + R \times H)$ where $R = \texttt{factorized_embedding_dim}$.

- `use_factorized_embedding`: bool — enable factorization.
- `factorized_embedding_dim`: int $\geq 1$ — intermediate dimension (typical: 64–256).

**Algorithm 41** Schema-Driven Training Step with Stability Controls

**Require:** Batch stream, config $C$

Initialize retry counter $r \leftarrow 0$
**for** each optimizer step **do**
  Accumulate gradients for $K = C$.`gradient_accumulation_steps` micro-batches
  Apply global norm clipping with $\tau = C$.`grad_clip_max_norm`
  **if** post-clip gradient exceeds $C$.`inf_post_clip_threshold` or NaN/Inf detected **then**
    **if** $r < C$.`max_nan_retries` **then**
      restore safe state / skip step; $r \leftarrow r + 1$
      **continue**
    **else**
      stop training with failure state
    **end if**
  **end if**
  run optimizer step selected by `optimizer_class`
  update scheduler (`cosine`, `constant`, or `linear_warmup_then_constant`)
  **if** step mod `checkpoint_every_n_steps`$= 0$ **then**
    save rolling checkpoint and prune to `max_rolling_checkpoints`
  **end if**
  update best checkpoints up to `num_best_checkpoints`
  emit CSV + telemetry following `gradient_log_interval` and `telemetry_log_interval`
**end for**

### 9.5.2 Embedding Convolution

When `use_embedding_conv=true`, a depthwise Conv1d is applied over the embedding stream before the first transformer block, with kernel size `embedding_conv_kernel`. This provides a lightweight local-context filter on token embeddings.

## 9.6 Training-Step Stability Controls

The schema-driven training step applies gradient accumulation, global-norm clipping, post-clip explosion guards, and bounded NaN/Inf retries. The accumulated gradient is:

$$g_{\text{acc}} = \frac{1}{K} \sum_{i=1}^{K} g_i, \quad K = \texttt{gradient_accumulation_steps}$$

$$g_{\text{clip}} = g_{\text{acc}} \cdot \min\left(1, \frac{\tau}{\|g_{\text{acc}}\|_2 + \epsilon}\right), \quad \tau = \texttt{grad_clip_max_norm}$$

then overflow guards use `inf_post_clip_threshold` and retry logic bounded by `max_nan_retries`.

## 9.7 SBERT Inference Mode Router

The SBERT inference dispatcher selects among similarity scoring, corpus search, clustering, and persistent encoding modes over a single shared encoder [83]. The cosine similarity and regression loss are:

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}, \qquad \mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

with $y \in [-1, 1]$ in this pipeline.

**Algorithm 42** SBERT Inference Mode Router

**Require:** mode $m$, model $E$, inputs $X$

**if** $m =$ `similarity` **then**
    return $\cos(E(x_1), E(x_2))$
**else if** $m =$ `search` **then**
    return top-$k$ by dot-product/cosine against corpus embeddings
**else if** $m =$ `cluster` **then**
    return clustering labels over $E(X)$
**else**
    return serialized embeddings $E(X)$
**end if**

# 10 Activation Functions

This annex documents the complete activation-function library available in the model. The library provides **43 feed-forward activation functions**: 40 elementwise activations and 3 gated-FFN variants. Activations are grouped into five families, following the taxonomy of Dubey et al. [24] and the analytic overview of Lederer [55], augmented with the learnable Rational Activation Function (RAF) of Fang et al. [28].

The selection is driven by the `ffn_activation` configuration field (a string enum) and dispatched at runtime by a factory function that mirrors the normalization-layer dispatch (Annex 7).

$$\texttt{ffn_activation} \in \{\texttt{silu}, \texttt{gelu}, \ldots, \texttt{raf}, \texttt{swiglu}, \texttt{geglu}, \texttt{reglu}\}$$

## 10.1 Classical / Sigmoid–Tanh Family (12 activations)

The classical family comprises smooth, stateless activations rooted in logistic and algebraic functions [55, 24]. All share monotonicity or near-monotonicity and are continuously differentiable.

$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}} \qquad \text{Range: } (0,1),\ \text{monotonic, smooth} \quad [55] \tag{81}$$

$$\text{Tanh}(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = 2\,\sigma(2x) - 1 \qquad \text{Range: } (-1,1),\ \text{monotonic, smooth} \quad [55] \tag{82}$$

$$\text{Arctan}(x) = \arctan(x) \qquad \text{Range: } (-\tfrac{\pi}{2}, \tfrac{\pi}{2}),\ \text{monotonic, smooth} \quad [55] \tag{83}$$

$$\text{Softsign}(x) = \frac{x}{1+|x|} \qquad \text{Range: } (-1,1),\ \text{smooth, once-diff. at } 0 \quad [55] \tag{84}$$

$$\text{Elliott}(x) = \frac{x}{1+|x|} \qquad \text{Alias of Softsign (survey “elliottsig” notation)} \quad [55] \tag{85}$$

$$\text{Identity}(x) = x \qquad \text{Range: } (-\infty,\infty),\ \text{linear, } C^{\infty} \quad \text{(internal)} \tag{86}$$

$$\text{Softplus}(x) = \log(1 + e^x) \qquad \text{Range: } [0, \infty),\ \text{smooth, } C^\infty \quad [31] \tag{87}$$

$$\text{Mish}(x) = x\,\tanh\big(\text{softplus}(x)\big) = x\,\tanh\big(\log(1 + e^x)\big) \qquad \text{Range: } [\approx -0.31, \infty),\ \text{non-monotonic, smooth} \quad [69] \tag{88}$$

$$\text{GELU}(x) = x\,\Phi(x) = x \cdot \tfrac{1}{2}\Big(1 + \text{erf}\Big(\frac{x}{\sqrt{2}}\Big)\Big) \qquad \text{Range: } (\approx -0.17, \infty),\ \text{smooth, probabilistic} \quad [42] \tag{89}$$

$$\text{GELU-tanh}(x) = \tfrac{x}{2}\Big(1 + \tanh\Big(\sqrt{\tfrac{2}{\pi}}(x + 0.044715\,x^3)\Big)\Big) \qquad \text{Tanh approximation of GELU (faster)} \quad [42] \tag{90}$$

$$\text{ReLU}(x) = \max(0, x) \qquad \text{Range: } [0, \infty),\ \text{piecewise-linear, non-smooth at 0} \quad [71] \tag{91}$$

$$\text{SiLU}(x) = x\,\sigma(x) = \frac{x}{1 + e^{-x}} \qquad \text{Range: } (\approx -0.278, \infty),\ \text{non-monotonic, smooth} \quad [26] \tag{92}$$

**SiLU** (Sigmoid Linear Unit) is the default activation (`ffn_activation=silu`). When $\beta = 1$, the parametric Swish reduces to SiLU. GELU and its tanh approximation are the standard in BERT and GPT-2/3 [42]. Mish [69] is a self-regularized, non-monotonic composition of Softplus and Tanh, reported to outperform Swish on several benchmarks.

## 10.2 Rectified Family (12 activations)

The rectifier family extends ReLU with learnable slopes, bounded variants, and rectified-hyperbolic forms [71, 24].

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ a\,x & x < 0 \end{cases}, \quad a = 0.01 \qquad \text{Range: } (-\infty, \infty) \quad [65] \tag{93}$$

$$\text{ReLU6}(x) = \min\big(\max(0, x),\ 6\big) \qquad \text{Range: } [0, 6],\ \text{bounded, fixed-point friendly} \quad [45] \tag{94}$$

$$\text{HardSwish}(x) = x \cdot \frac{\text{ReLU6}(x + 3)}{6} \qquad \text{Range: } (\approx -1.67, \infty),\ \text{cheap Swish approx.} \quad [44] \tag{95}$$

$$\text{PReLU}(x) = \max(0, x) + \mathbf{p} \odot \min(0, x) \qquad \text{Learnable per-channel slope } \mathbf{p},\ \text{init } 0.25 \quad [39] \tag{96}$$

$$\text{AbsReLU}(x) = \max(0, x) - \max(0, -x) = x\ \text{(sign-ramp)} \qquad \text{Range: } (-\infty, \infty) \quad [24] \tag{97}$$

$$\text{NLReLU}(x) = \beta\,\log\big(1 + \max(0, x)\big) \qquad \text{Range: } [0, \infty),\ \beta = 1.0,\ \text{logarithmic compression} \quad [24] \tag{98}$$

$$\text{BReLU}(x) = \min\big(\max(0, x),\ t\big), \quad t = 1.0 \qquad \text{Range: } [0, t],\ \text{bounded rectifier} \quad [24] \tag{99}$$

$$\text{VReLU}(x) = |x| \qquad \text{Range: } [0, \infty),\ \text{symmetric rectifier} \quad [24] \tag{100}$$

$$\text{Hexpo}(x) = a\,\max(0, x) - c\,\max(0, -x), \quad a{=}c{=}1.0 \qquad \text{Asymmetric two-sided rectifier} \quad [24] \tag{101}$$

$$\text{PenalizedTanh}(x) = \max\big(0,\ \tanh(x)\big) \qquad \text{Range: } [0, 1),\ \text{rectified tanh, smooth near 0} \quad [24] \tag{102}$$

$$\text{DisReLU}(x) = \max(0,\ x - \delta), \quad \delta = 0.0 \qquad \text{Right-shifted ReLU hinge} \quad [24] \tag{103}$$

$$\text{LiSHT}(x) = x\,\tanh(x) \qquad \text{Range: } [0, \infty)\ \text{(magnitude), non-monotonic, smooth} \quad [84] \tag{104}$$

**PReLU** [39] makes the negative slope a per-channel learnable parameter, enabling the network to learn the optimal leak per feature. **HardSwish** [44] is the cheap Swish approximation used in MobileNetV3. **ReLU6** [45] was introduced in MobileNetV1 for fixed-point quantization friendliness.

### 10.3 Exponential / ELU Family (12 activations)

The exponential-linear family provides smooth negative saturation and self-normalizing variants [17, 52, 24]. Several members introduce per-channel learnable parameters.

$$\mathrm{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}, \quad \alpha = 1.0 \qquad \text{Range: } (-\alpha, \infty) \quad [17] \quad (105)$$

$$\mathrm{SELU}(x) = \lambda\, \mathrm{ELU}_{\alpha'}(x) \qquad \alpha' \approx 1.6733,\ \lambda \approx 1.0507, \quad [52] \quad (106)$$
self-normalizing (mean 0, var 1)

$$\mathrm{CELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^{x/\alpha} - 1) & x < 0 \end{cases}, \quad \alpha = 1.0 \qquad \text{Once-diff. at 0 for any } \alpha \quad [5] \quad (107)$$

$$\mathrm{PELU}(x) = \begin{cases} \frac{\alpha}{\beta}\, x & x \geq 0 \\ \alpha(e^{x/\beta} - 1) & x < 0 \end{cases} \qquad \alpha, \beta \text{ learnable per-channel} \quad [96] \quad (108)$$

$$\mathrm{MPELU}(x) = \mathrm{ReLU}(x) + \beta \cdot \mathrm{ELU}_\alpha(x) \cdot \mathbb{1}_{x<0} \qquad \alpha, \beta \text{ learnable per-channel} \quad [24] \quad (109)$$

$$\mathrm{FELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases} \qquad \alpha \text{ learnable, clamped to } [0, 1] \quad [24] \quad (110)$$

$$\mathrm{EELU}(x) = \beta\, \mathrm{ReLU}(x) + \alpha(e^x - 1) \cdot \mathbb{1}_{x<0} \qquad \alpha, \beta \text{ learnable per-channel} \quad [24] \quad (111)$$

$$\mathrm{PDELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^{x/\alpha} - 1) + (1 - \alpha)\, x & x < 0 \end{cases} \qquad \alpha \text{ learnable, interpolates CELU–identity} \quad [24] \quad (112)$$

$$\mathrm{PREU}(x) = \beta\, \mathrm{ReLU}(x) + \alpha(e^x - 1) \cdot \mathbb{1}_{x<0}^{\leq 0} \qquad \alpha, \beta \text{ learnable per-channel} \quad [24] \quad (113)$$

$$\mathrm{SoftExp}(x) = \begin{cases} \frac{1}{\alpha}\left(e^{\alpha x} - 1\right) + \alpha & \alpha > 0 \\ x & \alpha = 0 \\ -\frac{1}{\alpha}\ln\left(1 - \alpha(x + \alpha)\right) & \alpha < 0 \end{cases} \qquad \alpha \text{ learnable; exp/linear/log interpolation} \quad [32] \quad (114)$$

$$\mathrm{ELiSH}(x) = \begin{cases} x\, \sigma(x) & x \geq 0 \\ (e^x - 1)\, \sigma(x) & x < 0 \end{cases} \qquad \text{Swish pos. branch, ELU-gated neg. branch} \quad [6] \quad (115)$$

$$\mathrm{HardELiSH}(x) = \begin{cases} x \cdot \mathrm{hard_sigmoid}(x) & x \geq 0 \\ (e^x - 1) \cdot \mathrm{hard_sigmoid}(x) & x < 0 \end{cases} \qquad \text{Hard-sigmoid variant of ELiSH} \quad [6] \quad (116)$$

where $\mathrm{hard_sigmoid}(x) = \mathrm{ReLU6}(x + 3)/6$.

**ELU** [17] provides smooth negative saturation. **SELU** [52] induces self-normalization with fixed $\alpha' \approx 1.6733$, $\lambda \approx 1.0507$. **CELU** [5] is once-differentiable at 0 for any $\alpha$ (unlike ELU). **PELU** [96] makes both $\alpha, \beta$ learnable per-channel. The parametric variants (MPELU, FELU, EELU, PDELU, PREU) generalize ELU and PReLU by introducing learnable per-channel parameters for the negative and positive branches. **SoftExp** [32] continuously interpolates between logarithmic ($\alpha < 0$), linear ($\alpha = 0$), and exponential ($\alpha > 0$) regimes. **ELiSH** and **HardELiSH** [6] combine Swish and ELU behaviors via a piecewise definition.

### 10.4 Swish and Parametric Activations

Swish [81] is a family parameterized by a slope $\beta$:

$$\mathrm{Swish}_\beta(x) = x\, \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}.$$

When $\beta = 1$ this is SiLU (the default). Large $\beta$ approaches ReLU; $\beta \to 0$ approaches the linear function $x/2$.

- **Swish** (`swish`): fixed $\beta$ (default 1.0, configured via `ffn_activation_config.swish_beta`). No learnable parameters.
- **SwishTrainable** (`swish_trainable`): $\beta$ is a per-channel learnable parameter, initialized to 1.0. Equivalent to SiLU at initialization, then adapts during training.
- **Maxout** (`maxout`): computes $k$ linear projections and takes the elementwise maximum: $\max_{i=0}^{k-1}(W_i x + b_i)$. A universal approximator for any continuous function given enough pieces. Parameter count increases by factor $k$ (default $k = 2$, configured via `ffn_activation_config.maxout_pieces`).

References: Swish [81], Maxout [33].

### 10.5 Rational Activation Functions (RAF)

Following Fang et al. [28], a Rational Activation Function is a learnable ratio of two low-degree polynomials (a Padé approximant):

$$F(x) = \frac{P(x)}{Q(x)} = \frac{\sum_{j=0}^{m} a_j \, x^j}{1 + R(x)}$$

where $\mathbf{a} \in \mathbb{R}^{m+1}$ and $\mathbf{b} \in \mathbb{R}^n$ are *learnable*, and the denominator term $R(x)$ depends on the version. The default configuration matches the paper: degree $(m, n) = (5, 4)$, version "A" (safe), initialized by a least-squares fit to GELU on $[-3, 3]$.

The implementation supports five denominator variants:

- **Version A** (default, "safe"): $Q(x) = 1 + \sum_k |b_k \, x^{k+1}|$ (per-term absolute value). Guarantees $Q(x) \geq 1$, no division by zero.
- **Version B**: $Q(x) = 1 + \left|\sum_k b_k \, x^{k+1}\right|$ (absolute value of the whole sum). Also guarantees $Q(x) \geq 1$.
- **Version C**: $Q(x) = 0.1 + \left|\sum_k b_k \, x^k\right|$ with a 0.1 floor. Minimum denominator 0.1.
- **Version D**: like B, but injects uniform multiplicative noise $U(1-\varepsilon, 1+\varepsilon)$ on the denominator weights during *training only* ($\varepsilon = 0.1$). Regularization effect.
- **Version N** ("nunsafe"): $Q(x) = 1 + \sum_k b_k \, x^{k+1}$ without any absolute value. Can have poles (denominator zeros); use with care.

**RAFT input scaling.** Because a Padé approximant is a reliable universal approximator only on a bounded interval, the RAFT model preprocesses each token's pre-activations with a per-token min–max scaling into $[-3, 3]$:

$$\tilde{x} = \frac{x - \min(x)}{\max(x) - \min(x)} \cdot 6 - 3.$$

This is toggled by `ffn_activation_config.raf_input_scaling` (default `false`).

**Initialization targets.** The rational coefficients can be initialized by fitting to any of the following target functions: `gelu` (default), `relu`, `leaky_relu`, `leaky_relu_0.1`, `sigmoid`, `tanh`, `swish`/`silu`, or `identity`. The fit uses a least-squares solve on 2001 uniformly spaced samples over $[-3, 3]$.

**Freezing.** Setting `raf_trainable=false` freezes the rational parameters, useful for parameter-efficient fine-tuning. Frozen RAFs are typically paired with a separate (higher) learning rate for the remaining network parameters.

**Note on naming.** RAF stands for *Rational* Activation Function, not "Rectified"; it is the learnable polynomial ratio described above.

## 10.6 Gated FFN Variants: SwiGLU, GEGLU, ReGLU

SwiGLU, GEGLU, and ReGLU [86] are gated feed-forward units that replace the standard Linear → activation → Linear block. They own their own linear projections and are therefore implemented as FFN modules, *not* as elementwise activations:

$$\text{GatedFFN}(x) = \text{dropout}\Big( W_{\text{down}} \Big( \text{act}(x\, W_{\text{gate}}) \odot (x\, W_{\text{up}}) \Big) \Big),$$

where $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d \times d_{\text{ff}}}$ and $W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d}$. The gate function is:

- **SwiGLU** (`swiglu`): act = SiLU (the default, used in Llama, PaLM).
- **GEGLU** (`geglu`): act = GELU (used in some T5 variants).
- **ReGLU** (`reglu`): act = ReLU (cheapest option).

When `ffn_activation` is one of `swiglu`/`geglu`/`reglu`, the feed-forward layer swaps the standard dense and MoE FFN blocks for a gated FFN built with the same projection class (BitLinear under BitNet). Bias is disabled by default.

## 10.7 Configuration Schema

The `ffn_activation_config` object groups all learnable-activation parameters. It is optional and ignored for stateless activations.

- `raf_degrees`: $[m, n]$ Padé numerator/denominator degrees (ints $\geq 1$). Default: $[5, 4]$.
- `raf_version`: denominator form, one of `A` (default), `B`, `C`, `D`, `N`.
- `raf_approx_func`: initialization target. One of `gelu` (default), `relu`, `leaky_relu`, `leaky_relu_0.1`, `sigmoid`, `tanh`, `swish`, `silu`, `identity`.
- `raf_trainable`: freeze the rational parameters when `false`. Default: `true`.
- `raf_input_scaling`: per-token min–max scaling to $[-3, 3]$ before the rational (RAFT preprocessing). Default: `false`.
- `prelu_init`: initial negative slope for PReLU. Default: 0.25.
- `elu_alpha`: ELU saturation constant. Default: 1.0.
- `celu_alpha`: CELU saturation constant. Default: 1.0.
- `swish_beta`: fixed $\beta$ for Swish / initial $\beta$ for SwishTrainable. Default: 1.0.
- `leaky_relu_slope`: negative slope for LeakyReLU. Default: 0.01.
- `maxout_pieces`: number of linear pieces $k$ for Maxout (int $\geq 1$). Default: 2.
- Additional ELU-parametric keys: `pelu_alpha`, `mpelu_alpha`, `mpelu_beta`, `felu_alpha`, `eelu_alpha`, `eelu_beta`, `pdelu_alpha`, `preu_alpha`, `preu_beta`, `softexp_alpha`. All default to 1.0 (except `softexp_alpha` which defaults to 0.0).

### 10.8 Selection Guide

The following decision tree aids activation choice:

- **Safe default** → `silu` (SiLU/Swish$_1$, used in Llama, PaLM).
- **Classic stability** → `gelu` or `relu`.
- **Smooth non-monotonic** → `mish` or `swish`.
- **Mobile-friendly** → `hardswish` or `relu6`.
- **Learnable per-task shape** → `raf`, `prelu`, `swish_trainable`, or `maxout`.
- **Self-normalizing networks** → `selu` (with AlphaDropout).
- **Gated FFN (own projections)** → `swiglu`, `geglu`, or `reglu`.

With BitNet quantization, `silu` may be more robust than smooth activations that rely on precise gradient computation. Learnable activations (`raf`, `prelu`, `swish_trainable`) add parameters but can adapt to the task distribution.

## 11 Manifold-Constrained Hyper-Connections (mHC)

This annex documents the integration of *Manifold-Constrained Hyper-Connections* (mHC) [104], a residual-connection framework that replaces the standard identity residual with an $n$-stream residual whose stream-mixing matrix is constrained to the Birkhoff polytope. The main body delegates to this annex for the mathematical formulation, the implementation details, and the reported results; the bibliography keys cited here resolve against `docs/bibliography/`.

### 11.1 Motivation: The Identity-Mapping Property

The standard residual connection [40] propagates signals across depth unmodified: with $x_l \in \mathbb{R}^C$ and a layer function $F$,

$$x_{l+1} = x_l + F(x_l, W_l),$$

so that, recursively, $x_L = x_l + \sum_{i=l}^{L-1} F(x_i, W_i)$. The direct term $x_l$ (the *identity mapping*) lets information flow from shallow to deep layers unchanged, which He et al. [40] identified as key to stable training of very deep networks.

*Hyper-Connections* (HC) [104] widen the residual stream by an expansion factor $n$: the stream becomes $x_l \in \mathbb{R}^{n \times C}$ while each layer's internal function $F$ still operates at dimension $C$. Three learnable linear mappings govern the stream each layer:

- $H_l^{\text{pre}} \in \mathbb{R}^{1 \times n}$ aggregates the $nC$-dimensional stream into the $C$-dimensional layer input;
- $H_l^{\text{post}} \in \mathbb{R}^{1 \times n}$ maps the layer output back onto the stream;
- $H_l^{\text{res}} \in \mathbb{R}^{n \times n}$ mixes features *within* the residual stream.

The single-layer propagation is

$$x_{l+1} = H_l^{\text{res}} x_l + \big(H_l^{\text{post}}\big)^\top F\big(H_l^{\text{pre}} x_l, W_l\big),$$

and, recursively, the deep-to-shallow signal is governed by the composite mapping $\prod_{i=L-1}^{l} H_{i+1}^{\text{res}}$. Because $H_l^{\text{res}}$ is unconstrained in HC, this composite deviates from the identity, the stream norm is not conserved, and the forward/backward signal can be amplified or attenuated without bound. On a 27B-parameter MoE model the authors report composite gain magnitudes peaking near 3000 (versus 1 for the identity), correlating with gradient-norm spikes and an unexpected loss surge around step 12k. HC also multiplies memory-access cost by roughly $n$ (the "memory wall").

## 11.2 Formulation

Let $x_l \in \mathbb{R}^{n\times C}$ be the stream at layer $l$. The coefficients are computed from the flattened stream $\tilde{x}_l = \mathrm{vec}(x_l) \in \mathbb{R}^{1\times nC}$:

$$\tilde{H}_l = \frac{1}{r_l}\Big(\alpha \odot \big(\tilde{x}_l\,\varphi_l\big)\Big) + b_l, \qquad r_l = \frac{\|\tilde{x}_l\|_2}{\sqrt{nC}},$$

where $\varphi_l \in \mathbb{R}^{nC\times(n^2+2n)}$ is a learned linear projection, $b_l \in \mathbb{R}^{1\times(n^2+2n)}$ a learned bias, and $\alpha^{\mathrm{pre}}, \alpha^{\mathrm{post}}, \alpha^{\mathrm{res}} \in \mathbb{R}$ are learnable gating scalars initialized to a small value (0.01 in the paper and in this codebase). The scaling by $1/r_l$ is mathematically equivalent to RMSNorm on the flattened stream, with the per-dimension scale absorbed into $\varphi_l$. The three coefficient blocks are then:

$$H_l^{\mathrm{pre}} = \sigma\big(\tilde{H}_l^{\mathrm{pre}}\big), \qquad H_l^{\mathrm{post}} = 2\,\sigma\big(\tilde{H}_l^{\mathrm{post}}\big), \qquad H_l^{\mathrm{res}} = \mathrm{SK}\big(\exp(\tilde{H}_l^{\mathrm{res}})\big),$$

where $\sigma$ is the logistic sigmoid (enforcing non-negativity and preventing signal cancellation from mixed-sign coefficients) and $\mathrm{SK}(\cdot)$ is the Sinkhorn–Knopp projection onto the set of doubly stochastic matrices

$$\mathcal{M}^{\mathrm{res}} = \big\{\, H \in \mathbb{R}^{n\times n} \;:\; H \ge 0,\;\; H\,\mathbf{1}_n = \mathbf{1}_n,\;\; \mathbf{1}_n^\top H = \mathbf{1}_n^\top \,\big\},$$

i.e., the Birkhoff polytope (the convex hull of permutation matrices). Starting from $M^{(0)} = \exp(\tilde{H}_l^{\mathrm{res}})$, the projection alternates row and column normalizations,

$$M^{(t)} = T_c\big(T_r(M^{(t-1)})\big),$$

for $t = 1, \ldots, t_{\max}$ rounds ($t_{\max} = 20$ by default in both the paper and this codebase). Finally the layer input and the updated stream are

$$F_l^{\mathrm{pre}} = H_l^{\mathrm{pre}} x_l \in \mathbb{R}^{1\times C}, \qquad x_{l+1} = H_l^{\mathrm{res}} x_l + \big(H_l^{\mathrm{post}}\big)^\top \otimes F\big(F_l^{\mathrm{pre}}, W_l\big).$$

## 11.3 Properties of the Manifold Constraint

Constraining $H_l^{\mathrm{res}}$ to the Birkhoff polytope restores the conservation behavior of the identity residual and guarantees three properties [104]:

1. **Norm preservation**: the spectral norm satisfies $\|H_l^{\mathrm{res}}\|_2 \le 1$ (non-expansive map), mitigating gradient explosion.

2. **Compositional closure**: doubly stochastic matrices are closed under multiplication, so the composite $\prod_i H_i^{\mathrm{res}}$ remains doubly stochastic at arbitrary depth, preserving the identity-mapping property between *any* two depths.

3. **Geometric interpretation**: since the Birkhoff polytope is the convex hull of permutation matrices, $H_l^{\mathrm{res}} x$ is a convex combination of feature permutations, i.e., a monotonic information-mixing mechanism across streams.

When $n = 1$ the doubly stochastic condition degenerates to the scalar 1, recovering the exact identity mapping. With finite $t_{\max}$ the projection is approximate, and the composite backward gain deviates from 1 but stays bounded (around 1.6 in the paper's 27B runs, a $\sim$ 3-orders-of-magnitude reduction versus HC's $\approx 3000$).

### 11.4 Implementation in Frankenstein Transformer

mHC is an opt-in replacement for the standard residual connection, controlled by the `model.mhc` sub-object (hierarchical schema) or the flat `use_mhc` keys, with `additionalProperties: false` at the sub-object level:

- `enabled` (`use_mhc`, bool, default `false`): replace the standard residual with the n-stream residual.
- `expansion_rate` (`mhc_expansion_rate`, int $\geq 1$, default 4): stream expansion factor $n$; the stream becomes $n \times$ `hidden_size` while layer internals stay at `hidden_size`. 1 recovers the identity mapping.
- `sinkhorn_iters` (`mhc_sinkhorn_iters`, int $\geq 1$, default 20): Sinkhorn–Knopp normalization rounds $t_{\max}$.
- `gating_init` (`mhc_gating_init`, float $> 0$, default 0.01): initial value of the gating scalars $\alpha$.
- `checkpoint` (`mhc_checkpoint`, bool, default `false`): gradient checkpointing on mHC layers, trading compute for memory to mitigate the $\sim n\times$ activation-memory increase of the n-stream residual.
- `full_prec_under_bitnet` (`mhc_full_prec_under_bitnet`, bool, default `true`): keep the coefficient projection $\varphi_l$ a full-precision `nn.Linear` even when `use_bitnet` is `true`, avoiding ternary-quantization noise on the small mHC coefficients; when `false` (and BitNet enabled), $\varphi_l$ uses `BitLinear`.

The module lives in `src/model/mhc.py`:

- `SinkhornKnoppFunction` (`torch.autograd.Function`): the forward pass applies the alternating row/column normalizations of $\exp(\cdot)$; the backward pass recomputes the full iteration under `torch.enable_grad` and differentiates through it, obtaining the exact Jacobian-vector product (the recompute-on-chip strategy described in the paper).
- `ManifoldHyperConnections` (`nn.Module`): holds $\varphi_l$ (`proj`), $b_l$ (`bias`) and the three gating scalars `alpha_pre`, `alpha_post`, `alpha_res`; exposes `fpre` (project the stream down to a layer input), `recombine` (update the stream with a layer output) and `mappings` (compute $H^{\text{pre}}, H^{\text{post}}, H^{\text{res}}$).

Wiring in `src/model/frankenstein_model.py`:

- `HybridLayer` gains optional `mhc_attn` and `mhc_ffn` modules (one mHC module per layer function); the `_forward_dense_mhc` path runs attention and FFN as layer functions over the shared $(B, S, n, C)$ stream.
- `FrankensteinTransformer` expands the $C$-dimensional embedding to $(B, S, n, C)$ on entry via `mhc_in_proj` and collapses back to $C$ via `mhc_out_proj` before the head.
- mHC parameters route into the `other` optimizer parameter group.
- mHC is **incompatible with** `use_mixture_of_depths`: MoD token routing operates on a single $C$-dimensional stream, conflicting with the n-stream residual; a `ValueError` is raised if both are enabled.

Example configuration: `configs/examples/es_arch_mhc_adamw.yaml`.

### 11.5 Reported Results

The paper evaluates mHC (and HC) with $n = 4$, $t_{\max} = 20$ and $\alpha$-init 0.01 on DeepSeek-V3-style MoE models (MLA attention, RoPE, RMSNorm, Loss-Free load balancing) from 3B to 27B parameters [104]:

- **Stability**: mHC removes HC's loss surges (e.g., near step 12k on the 27B model) and keeps the composite residual gain bounded ($\approx 1.6$ versus HC peaks $\approx 3000$).
- **Quality**: on the 27B model mHC improves final training loss by 0.021 over the baseline, beats the baseline on all 8 downstream benchmarks (BBH, DROP, GSM8K, HellaSwag, MATH, MMLU, PIQA, TriviaQA) and HC on 7 of 8, with the largest gains on reasoning tasks (+2.1 BBH, +2.3 DROP over HC).
- **Scaling**: the relative loss improvement over baseline is maintained across compute scales (3B $\rightarrow$ 9B $\rightarrow$ 27B) and across 1T tokens of token scaling.
- **Overhead**: with fused kernels (TileLang), mixed-precision coefficient computation, selective recomputation, and DualPipe communication overlap, mHC adds only 6.7% additional training time at $n = 4$ on the 27B model.

### 11.6 Hyperloop: Loop-Level Hyper-Connections on Looped Transformers

The *Hyperloop Transformer* [112] combines looped Transformers — which reuse layer weights across depth — with *loop-level* hyper-connections, targeting parameter efficiency rather than raw depth. The layer stack is partitioned *middle-cycle* into a begin block (run once), a looped middle block, and an end block (run once); the looped middle block is wrapped in hyper-connections applied only after each loop iteration. With $Y_t^{(l)} \in \mathbb{R}^{n\times C}$ the matrix-valued stream at loop $l$, $z_t^{(l)} = \mathrm{RMSNorm}(\mathrm{flatten}(Y_t^{(l)}))$, and per-loop parameters $\{W_l^\tau, b_l^\tau, \alpha_l^\tau\}$ for $\tau \in \{\mathrm{pre}, \mathrm{post}, \mathrm{res}\}$ plus a loop position embedding $e_l \in \mathbb{R}^C$, the recurrence is

$$y_t^{(l+1)} = H_l^{\mathrm{res}}\, y_t^{(l)} + H_l^{\mathrm{post}} \otimes \big(F(H_l^{\mathrm{pre}}\, y_t^{(l)}) + e_l\big), \tag{117}$$

with the data-dependent coefficients

$$H_l^{\mathrm{pre}} = \sigma\big(\alpha_l^{\mathrm{pre}}(W_l^{\mathrm{pre}} z_t^{(l)}) + b_l^{\mathrm{pre}}\big), \qquad H_l^{\mathrm{post}} = 2\sigma\big(\alpha_l^{\mathrm{post}}(W_l^{\mathrm{post}} z_t^{(l)}) + b_l^{\mathrm{post}}\big),$$
$$H_l^{\mathrm{res}} = \mathrm{diag}\big(\sigma(\alpha_l^{\mathrm{res}}(W_l^{\mathrm{res}} z_t^{(l)}) + b_l^{\mathrm{res}})\big) \qquad W_l^{\mathrm{res}} \in \mathbb{R}^{n\times nC}$$

(the diagonal $H^{\mathrm{res}}$ substitutes mHC's sinkhorn$(\cdot)$ over a dense $n^2 \times nC$ projection with a sigmoid over a diagonal gate — the paper's Table 6 ablation finds this both cheaper *and* more accurate: 14.40 perplexity versus 14.59 Sinkhorn and 14.61 identity).

Four properties distinguish the Hyperloop wiring from per-sublayer mHC (§11): (1) hyper-connections fire once per *loop*, not after every sub-layer — also the most performant placement in the paper's Table 5 ablation; (2) the stream is created by *copying* the begin-block output $n$ times and collapsed by *averaging* the streams before the end block, replacing mHC's learned in/out projections; (3) the parameters are *per loop*, relaxing the strict weight sharing of ordinary looped Transformers — the paper's representation-similarity analysis (Table 8) shows looped-layer representations deviate more across loops, which it identifies as the source of the quality gains; and (4) $H^{\mathrm{res}}$ defaults to the diagonal-sigmoid parameterization.

The Frankenstein integration is controlled by the `mhc.hyperloop` boolean (flat `mhc_hyperloop`; requires `mhc.enabled` and `dims.num_loops` $\geq 2$), with three additional keys:

- `hyperloop_begin_layers` / `hyperloop_end_layers` (int $\geq 0$, default 0): sizes of the begin/end blocks, which run exactly once. The paper allocates $\sim$25% of parameters to each and 50% to the middle block (unrolled depth = begin + middle $\times$ loops + end; e.g. $2L \rightarrow 4L(\times 3) \rightarrow 2L$ at 16 layers). 0/0 loops the whole stack.

- `hyperloop_res_parameterization` (enum, default `diagonal`): `diagonal`, `sinkhorn` (full mHC-style doubly stochastic mixing, reusing `SinkhornKnoppFunction`) or `identity`.

Implementation: `src/model/hyperloop.py` hosts `HyperloopConnections` with per-loop `projs` / `biases` / `alphas_` / `loop_pos_embs` ($e_l$, zero-initialized so that at initialisation the update reduces to the plain looped-Transformer recurrence). `FrankensteinEncoder` (and `FrankensteinViT`) build it *instead of* the `mhc_in_proj`/`mhc_out_proj` pair; the forward pass runs begin → copy → per-loop read/middle/update → average → end. `HybridLayer` disables its per-layer `mhc_attn`/`mhc_ffn` modules under Hyperloop, so middle-block layers run the standard $C$-dimensional path — which restores **Mixture-of-Depths compatibility** (per-layer mHC forbids MoD). Hyperloop is incompatible with the AttnRes residuals (`residuals.type full_attn` / `block_attn`), which assume a per-logical-layer n-stream residual. The per-loop parameter budget is $3n^2C + 3n + 3 + C$ (diagonal): for the paper's 136M model ($C = 1024$, $n = 4$, 3 loops) only ≈150K parameters beyond the plain looped Transformer.

Example configuration: `configs/examples/hyperloop_mhc_adamw.yaml`.

**Reported results (arXiv:2604.21254)**

Trained on FineWeb-Edu with $n = 4$ streams and 3 loops (25%/50%/25% parameter split), against depth-matched Transformer and mHC baselines:

- **Parameter efficiency**: at 240M/1B/2B depth-matched scale, Hyperloop uses ≈50% fewer parameters (136M/580M/991M) yet matches or beats the baselines in perplexity (14.40 vs 14.65 at 240M scale; 9.65 vs 10.19 at 1B; 8.49 vs 8.60 at 2B).
- **Quantization**: INT4 (GPTQ) perplexity degrades less than the vanilla looped model (14.68 vs 15.18 at 240M scale), so the parameter savings survive post-training compression.
- **Overhead**: loop-level placement plus the diagonal $H^{\text{res}}$ keep the training throughput within ≈5% of the plain looped Transformer in unoptimized PyTorch (per-layer mHC loses ≈33% without fused kernels).
- **Ablations**: every-loop placement beats every-layer (Table 5); diagonal $H^{\text{res}}$ beats Sinkhorn and identity (Table 6); stream count $n$ shows diminishing returns beyond 4 (Table 4); LoRA-per-loop (relaxed recursive Transformers) helps far less per parameter than the hyper-connection relaxation (Table 7).

# 12 Attention Residuals (AttnRes)

This annex documents the integration of *Attention Residuals* (AttnRes) [94], a generalization of the residual connection that replaces the fixed unit-coefficient residual sum with *learned softmax attention over depth*. The main body delegates to this annex for the mathematical formulation, the four available strategies (standard, no-residual, Full AttnRes, Block AttnRes), the implementation wiring, the integration with mHC and Mixture-of-Depths, and the reported results. The bibliography keys cited here resolve against `docs/bibliography/`.

## 12.1 Motivation: From a Fixed Sum to Depth-Wise Attention

The standard residual connection [40] propagates signals across depth unmodified: with $x_l \in \mathbb{R}^C$ and a layer function $F$,

$$x_{l+1} = x_l + F(x_l, W_l),$$

so that, recursively, $x_L = x_l + \sum_{i=l}^{L-1} F(x_i, W_i)$. The identity term $x_l$ (the *identity mapping*) lets information flow from shallow to deep layers unchanged, which He et al. [40] identified as key

to stable training of very deep networks. Highway networks [90] generalize this with learned element-wise gates $\mathbf{g}_l \in [0,1]^C$,

$$x_{l+1} = (1 - \mathbf{g}_l) \odot x_l + \mathbf{g}_l \odot F(x_l, W_l),$$

which interpolates between the identity path and the transformation path with input-dependent weights.

*Attention Residuals* [94] take the next step: instead of weighting a single compressed state $x_l$, layer $l$ *attends* over all earlier layer outputs. Let $v_0 = h_1$ (the token embedding) and $v_i = F(x_i, W_i)$ for $i \geq 1$. AttnRes computes, for each layer $l$,

$$q_l = w_l, \qquad k_i = \mathrm{RMSNorm}(v_i), \qquad \alpha_{i \to l} = \mathrm{softmax}_i(q_l^\top k_i), \qquad x_{l+1} = \sum_{i=0}^{l} \alpha_{i \to l}\, v_i,$$

where $w_l \in \mathbb{R}^C$ is a learned per-layer pseudo-query, *initialized to zero* per the paper. At zero-init the softmax is uniform, so AttnRes degenerates to an equal-weight average of all previous outputs — matching the standard residual at step zero and avoiding training volatility. The RMSNorm on the keys prevents layers with naturally large magnitudes from dominating the softmax (ablation in the paper: § 5.3).

## 12.2 Four Strategies

The schema field `model.residuals.type` selects one of four strategies, each implemented as a separate module under `src/model/residuals/`.

### 12.2.1 Standard Residual (`"standard"`)

The default backwards-compatible strategy:

$$x_{l+1} = x_l + F(x_l, W_l).$$

Stateless, no extra parameters, no extra compute. Implemented as a no-op pass-through in `StandardResidual` that delegates the merge to `HybridLayer`.

### 12.2.2 No Residual (`"none"`, experimental)

Drops the skip connection entirely:

$$x_{l+1} = F(x_l, W_l).$$

Stateless, no extra parameters. Useful only as an ablation probe; the identity term is critical for deep stacks (paper § 2.1) and training is typically unstable without it. Implemented as `NoResidual`.

### 12.2.3 Full Attention Residuals (`"full_attn"`, paper § 3.1)

Each layer attends over *all* previous layer outputs:

$$x_{l+1} = \sum_{i=0}^{l} \alpha_{i \to l}\, v_i, \qquad \alpha_{i \to l} = \mathrm{softmax}_i\big(w_l^\top \mathrm{RMSNorm}(v_i)\big).$$

Adds $L \cdot C$ parameters ($L$ = logical depth = `num_layers` × `num_loops`, $C$ = `hidden_size`), one query vector per layer. Memory and compute grow as $O(L\,d)$ per token; at typical depth ($L < 1000$) this is negligible. Implemented as `FullAttentionResidual`; supports `attnres_gradient_checkpoint` to wrap the attention in `torch.utils.checkpoint` for memory-constrained runs.

### 12.2.4 Block Attention Residuals (`"block_attn"`, paper § 3.2)

Partitions the $L$ logical layers into $N$ blocks of $S = \lceil L/N \rceil$ layers each. Within a block the layer outputs accumulate as a standard partial sum $b_n^i$; across blocks attention is applied over the $N$ complete block representations and the running partial sum. For layer $l$ in block $n$,

$$x_{l+1} = \sum_{m=0}^{n} \alpha_{m \to l}\, v_m, \qquad \alpha_{m \to l} = \mathrm{softmax}_m\big(w_l^\top \mathrm{RMSNorm}(v_m)\big),$$

with $v_m = b_m$ for $m < n$ and $v_n = b_n^{i_l}$ (the current intra-block partial sum). $N$ interpolates between Full AttnRes ($N = L$, one block per layer) and standard residuals ($N = 1$, the embedding isolated as $b_0$). The paper's empirical sweet spot is $N \approx 8$; this codebase defaults to $N = 8$ and validates that $N \leq L$. Implemented as `BlockAttentionResidual`, which also follows Algorithm 1 of the paper (two-phase computation with online softmax merging).

## 12.3 Implementation in Frankenstein Transformer

The `src/model/residuals/` package hosts one class per strategy (`StandardResidual`, `NoResidual`, `FullAttentionResidual`, `BlockAttentionResidual`) plus an abstract `ResidualBase` that defines the lifecycle hooks `set_streams`, `register_state`, `reset_state`, `forward`, and `finalize`. The `build_residual(config)` factory in `src/model/residuals/factory.py` selects the right class from `FrankensteinModelConfig`.

Wiring in `src/model/frankenstein_encoder.py`:

- `FrankensteinEncoder` owns the `ResidualBase` module as `self.residual`; the factory is invoked once in `__init__`.
- Before the looped-depth loop the encoder calls `reset_state` and, for AttnRes variants, `set_embedding(x)` so the depth-wise attention knows the $v_0 = h_1$ source.
- After each `HybridLayer` call, if `self.residual.is_attn_res` is true, the encoder applies `x = self.residual(layer_idx, x)` to overwrite the stream with the attended aggregation. For stateless strategies (`standard`, `none`) this step is skipped and `HybridLayer` continues to apply its own residual merge internally.

Wiring in `src/model/hybrid_layer.py`:

- `HybridLayer` reads `residual_type` from the config and dispatches the residual merge accordingly. For `standard` the merge is the original `x = residual + mixer(norm(x))`; for `none` the skip is dropped (`x = mixer(norm(x))`); for the AttnRes variants the layer performs the standard merge internally because the depth-wise attention is applied externally by the encoder.
- `_forward_dense_mhc` (the mHC path) is unaffected because mHC has its own stream-expansion residual; combining AttnRes with mHC is handled by the `mhc_stream_mode` field of the residual module.

Schema (hierarchical, with the flat-key renames shown in parentheses) and defaults:

- `model.residuals.type` (`residual_type`, enum `[standard, none, full_attn, block_attn]`, default `standard`): selects the strategy.
- `model.residuals.full_attn.init_query_zero` (`full_attn_init_query_zero`, bool, default `true`): zero-init the per-layer query vectors $\mathbf{w}_l$.
- `model.residuals.full_attn.use_rmsnorm_keys` (`full_attn_use_rmsnorm_keys`, bool, default `true`): apply RMSNorm on keys before the dot product.

- `model.residuals.block_attn.num_blocks` (`block_attn_num_blocks`, int $\geq 1$, default 8): number of block representations $N$; validated $N \leq L$.
- `model.residuals.block_attn.init_query_zero` (`block_attn_init_query_zero`, bool, default `true`): zero-init the per-layer query vectors.
- `model.residuals.block_attn.use_rmsnorm_keys` (`block_attn_use_rmsnorm_keys`, bool, default `true`): RMSNorm on keys.
- `model.residuals.mhc_stream_mode` (`attnres_mhc_stream_mode`, enum `[independent, joint]`, default `independent`): how AttnRes interacts with mHC's n-stream residual.
- `model.residuals.gradient_checkpoint` (`attnres_gradient_checkpoint`, bool, default `false`): wrap the AttnRes attention in gradient checkpointing.

### 12.4 Integration with mHC and Mixture-of-Depths

The residual strategy is *orthogonal* to mHC and Mixture-of-Depths (MoD): all combinations are supported. The interaction modes are:

**mHC × AttnRes (`mhc_stream_mode`).** When `use_mhc=true` and the residual is `full_attn` or `block_attn`, the depth-wise attention is applied either *per stream* (`"independent"`) or over the *flattened* $nC$-dim projection (`"joint"`). Independent mode runs $n$ parallel attentions, one per stream — matching the paper's framing where each stream is a parallel "view" of the residual. Joint mode treats the $n$ streams as a single fat residual, more expressive but more parameter-hungry. The mHC coefficient projections ($\varphi_l$, $b_l$, $\alpha$ gates) are unaffected; AttnRes adds its own parameters on top.

**Mixture-of-Depths × AttnRes.** MoD token routing operates *within* a layer (selects which tokens get the full layer update) and is orthogonal to the residual choice. When `use_mixture_of_depths=true` together with an AttnRes residual, only the tokens selected by MoD contribute to the layer-output buffer (Full AttnRes) or the intra-block partial sum (Block AttnRes); the rest pass through unchanged and are scattered back by MoD's existing logic. MoD is mutually exclusive with mHC (a `ValueError` is raised in `HybridLayer._init__` if both are enabled); the corresponding combinations are (no_mHC, MoD, AttnRes), (mHC, no_MoD, AttnRes), or (mHC, no_MoD, no_AttnRes).

### 12.5 Reported Results

The paper [94] validates AttnRes through scaling laws, ablations, and a 48B-parameter Kimi Linear extension:

- **Scaling laws**: both Full and Block AttnRes consistently outperform the PreNorm baseline across five model sizes. At 5.6 PFLOP/s-days, Block AttnRes reaches loss 1.692 versus the baseline's 1.714 — equivalent to a 1.25× compute advantage. The gap between Full and Block AttnRes narrows with scale (Full 1.737 vs. Block 1.746 on the 16-layer model).
- **Ablations**: input-dependent softmax attention (1.737) outperforms input-independent scalar mixing (1.749) and sigmoid attention (1.741). RMSNorm on keys is essential (Full 1.737 vs. 1.743 without; Block 1.746 vs. 1.750). Multi-head depth aggregation (`H` = 16) hurts Block AttnRes (1.752 vs. 1.746), indicating the optimal mixture is largely uniform across channels. Sliding-window attention over the last $W = 8$ layers falls short of Block AttnRes (1.764 vs. 1.746).

- **Kimi Linear extension**: a 48B-parameter (3B activated) Kimi Linear model with Block AttnRes ($N \approx 9$, $S = 6$) pre-trained on 1.4T tokens improves over the baseline across all 14 downstream benchmarks (MMLU-Pro, GPQA-Diamond, BBH, HellaSwag, TriviaQA, GSM8K, MGSM, Math, CMath, HumanEval, MBPP, CMMLU, C-Eval, ARC-C). The largest gains are on multi-step reasoning (GPQA-Diamond +7.5, Minerva Math +3.6) and code generation (HumanEval +3.1). Training-dynamics analysis shows that AttnRes mitigates PreNorm dilution: hidden-state magnitudes remain bounded across depth and gradient norms distribute more uniformly across layers.
- **Overhead**: training overhead is marginal when pipeline parallelism is not enabled; with pipeline parallelism Block AttnRes adds $< 4\%$ end-to-end training overhead (cross-stage caching eliminates redundant transfers) and $< 2\%$ inference latency overhead (two-phase computation with online softmax merging).

Example configurations: `configs/examples/full_attn_res_adamw.yaml`, `configs/examples/block_attn_res_adamw.yaml`, `configs/examples/mhc_full_attn_res_adamw.yaml`.

# 13 Vision Transformer: Patch Prediction, Classification, and Segmentation

This annex documents the `frankenstein_vit` model class, a Vision Transformer (ViT) implementation [22] that reuses the modality-agnostic `HybridLayer` stack from the encoder. It supports three tasks: autosupervised masked patch prediction, image classification, and image segmentation (per-pixel or EoMT-style [49]).

## 13.1 Patch Embedding and Positional Encoding

Following Eq. 1 of the ViT paper, an input image $x \in \mathbb{R}^{H \times W \times C}$ is split into $N = HW/P^2$ non-overlapping $P \times P$ patches, each linearly projected to $D$ dimensions. A Conv2d with kernel and stride $P$ performs both the splitting and the projection in a single operation. A learnable `[CLS]` token is optionally prepended, and a learnable positional embedding is added:

$$z_0 = \left[ x_{\text{class}}; \, x_p^1 E; \, x_p^2 E; \, \ldots; \, x_p^N E \right] + E_{\text{pos}}, \qquad E \in \mathbb{R}^{(P^2 C) \times D}, \quad E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}.$$

**Positional encoding.** The default positional encoding is a learnable 1D `nn.Parameter` of shape $(1, N+1, D)$, faithful to the original ViT. The alternative `none` drops the explicit embedding and relies on RoPE/HoPE applied inside the attention mixers (§9), which is consistent with the encoder's modality-agnostic handling of rotary embeddings.

**`[CLS]` token.** A learnable `[CLS]` token is optionally prepended at position 0 for classification: the output of position 0 after the final normalization is fed to the task head. When `[CLS]` is disabled, classification relies on Global Average Pooling (GAP) over the patch tokens.

## 13.2 Masked Patch Prediction (Autosupervised)

The autosupervised task mirrors the BERT-style objective of the ViT paper (App. B.1.2). A corruption ratio of 50% is applied to the patch tokens: of the corrupted subset, 80% are replaced by a learnable mask token, 10% are replaced by a random patch, and 10% are kept unchanged.

Three reconstruction targets are supported:

1. **3-bit mean color** (default): each patch's mean color is quantized to 3 bits per channel ($2^{3 \times 3} = 512$ bins), and a 512-way cross-entropy is computed. This target is the most stable and best-performing in the ViT paper.

2. $4 \times 4$ **downsampled 3-bit**: the patch is downsampled to a $4 \times 4$ grid of 3-bit colors, producing $16 \times 512$ cross-entropy targets.

3. **Full-patch L2**: mean squared error on the raw pixels of the masked patches, equivalent to the pixel reconstruction loss of Masked Autoencoders [41].

The model predicts only at masked positions; the loss is computed on corrupted patches only, so the network is never asked to reconstruct patches it was allowed to observe.

### 13.3 Image Classification

Classification fine-tuning follows ViT Sec. 3.2:

**Pooling.** The representation is pooled either by taking the `[CLS]` token output at position 0 or by Global Average Pooling over the patch tokens.

**Head.** A zero-initialized `Linear`(D, K) projects the pooled vector to $K$ class logits. The zero initialization guarantees that the fine-tuning starts from the pretrained linear projection and that no class bias is introduced before the first optimizer step.

**Loss.** Standard cross-entropy over the $K$ classes.

### 13.4 Image Segmentation

Two head types are selected by the `seg_head_type` config field.

#### 13.4.1 Per-Pixel Head (`pixel`)

A `Linear`(D, `num_seg_classes`) is applied per patch token, and the resulting per-patch logits are reshaped to $(H/P, W/P)$. A ViTDet-style transposed-convolution upsampler then scales the logits to the full image resolution $(H, W)$. The training loss is the unweighted sum of cross-entropy and Dice:

$$\mathcal{L}_{\text{pixel}} = \text{CE}(\hat{y}, y) + \text{Dice}(\hat{y}, y).$$

This head supports both 1D grayscale (2 classes) and multicolor non-overlapping ($N + 1$ classes) segmentation.

#### 13.4.2 Encoder-only Mask Transformer (`eomt`)

The EoMT head [49] repurposes the ViT backbone as a mask decoder by splitting the layer stack into two contiguous groups:

- $L_1$ (patch-only): the first $L_1$ blocks process the patch tokens as in a plain ViT.
- $L_2$ (patches + queries): the remaining $L_2$ blocks receive a concatenation of $K$ learnable object queries and the patch tokens. The multi-head self-attention inside each $L_2$ block therefore computes all four attention quadrants (patch↔patch, query↔query, query→patch, patch→query) in a single operation, halving the decoder compute relative to an encoder–decoder Mask2Former.

**Mask module.** The mask module predicts, for each query, class logits via a linear layer and a mask embedding via a 3-layer MLP. The mask embedding is dot-producted with the upscaled patch feature map $\tilde{F}_4$ to produce per-pixel mask logits.

**Loss with Hungarian matching.** Following Mask2Former, predictions are first matched to ground-truth masks via the Hungarian algorithm, then supervised with:

$$\mathcal{L}_{\text{eomt}} = 5\,\text{BCE} + 5\,\text{Dice} + 2\,\text{CE},$$

where the BCE and Dice terms act on mask logits and the CE term acts on class logits (factor 2 for matched queries, factor 0 for the no-object class).

**Mask annealing.** A fraction $P_{\text{mask}}$ of patches is masked during training, with the fraction decaying polynomially (decay factor 0.9) to 0 at inference time. When $P_{\text{mask}} = 0$ the model reverts to a plain ViT and the $L_2$ blocks operate on patches only, yielding $\sim 4\times$ faster inference than at training time while preserving mask quality.

### 13.5 Reuse of Existing Infrastructure

Because the `HybridLayer` stack is modality-agnostic and operates purely on $(B, S, D)$ tensors, the Vision Transformer reuses the entire encoder-side machinery without modification:

- **Attention mixers:** all 34 sequence mixers are available, except `engram_attn`, which requires discrete token IDs and is therefore not meaningful for continuous patch embeddings.
- **Normalization:** all 6 norm variants (LayerNorm, RMSNorm, pRMSNorm, DyT, Derf, FlashNorm).
- **Residuals:** all 4 residual variants.
- **Activations:** all 42 activation functions (including GLU gated FFN units).
- **Optimizers:** all 23 optimizer families with prefixed per-parameter-group hyperparameter control.
- **Training configuration:** the three vision tasks read `batch_size`, `num_epochs`, `optimizer`, `scheduler_type`, `grad_clip_max_norm`, and checkpointing from the same top-level `training:` block as `mlm` and `causal_lm`. The optional `training.<task>:` sub-block carries only task-specific knobs: `label_smoothing` for classification, and `seg_loss_bce_weight` / `seg_loss_dice_weight` / `seg_loss_ce_weight` / `mask_annealing_factor` for EoMT segmentation.

The `FrankensteinViT` class mirrors the `FrankensteinDecoder` wrapper pattern: it owns a `PatchEmbed` module, a `HybridLayer` stack, and a task-specific head, and it forces `mode='encoder'` so that the bidirectional attention pattern required by vision tasks is preserved.

### 13.6 PyTorch Pseudocode

The forward pass for each task is sketched below.

**Classification.**

$x \leftarrow$ `patch_embed(pixel_values)` ▷ $(B, N, D)$
$x \leftarrow$ `[cls_token; x]` ▷ $(B, N+1, D)$
$x \leftarrow x +$ `pos_embed` ▷ learnable 1D
$x \leftarrow$ `dropout`$(x)$
**for** each `layer` in `layers` **do**
    $x \leftarrow$ `layer`$(x)$
**end for**
$x \leftarrow$ `final_norm`$(x)$
`logits` $\leftarrow$ `head`$(x[:, 0])$ ▷ `[CLS]` token

**Masked patch prediction.**

$x \leftarrow$ `patch_embed(pixel_values)` ▷ $(B, N, D)$
$(x,\, m) \leftarrow$ `mask`$(x,\, r = 0.5)$ ▷ $m$: binary mask of corrupted positions
$x \leftarrow [\,$`cls_token` $;\, x\,] +$ `pos_embed`
$x \leftarrow$ `dropout`$(x)$
**for** each `layer` in `layers` **do**
    $x \leftarrow$ `layer`$(x)$
**end for**
$x \leftarrow$ `final_norm`$(x)$
$\hat{y} \leftarrow$ `head`$(x[:, 1:])$ ▷ predict at patch positions only
$\mathcal{L} \leftarrow$ `loss`$(\hat{y}[m],\, y[m])$ ▷ loss on corrupted patches only

**EoMT segmentation.**

$x \leftarrow$ `patch_embed(pixel_values)` ▷ $(B, N, D)$
$x \leftarrow x +$ `pos_embed`
$x \leftarrow$ `mask_anneal`$(x)$ ▷ mask $P_{\text{mask}}$ patches when training
**for** each `layer` in $L_1$ **do**
    $x \leftarrow$ `layer`$(x)$ ▷ patch-only blocks
**end for**
$q \leftarrow$ `learnable_queries` ▷ $(B, K, D)$
$x \leftarrow [\,x;\, q\,]$ ▷ concatenate patches and queries
**for** each `layer` in $L_2$ **do**
    $x \leftarrow$ `layer`$(x)$ ▷ four-quadrant self-attention
**end for**
$\hat{c},\, \hat{m} \leftarrow$ `mask_module`$(x_{\text{patches}},\, x_{\text{queries}},\, \tilde{F}_4)$ ▷ class + mask logits
$(\sigma,\, \hat{y}_\sigma) \leftarrow$ `Hungarian`$(\hat{c},\, \hat{m},\, y)$ ▷ match to ground truth
$\mathcal{L} \leftarrow 5\,\text{BCE} + 5\,\text{Dice} + 2\,\text{CE}$ ▷ supervise matched predictions

# 14 Model-Wide Positional Encodings

This annex documents the unified positional-encoding (PE) subsystem that applies a single `model.positional_encoding` choice to every attention mixer in the network. The schema contract is:

`positional_encoding` $\in$ {`rope`, `hope`, `nope`, `alibi`, `bam`, `pape`, `pape_efficient`, `pape_ri`, `sinusoidal_absolute`, `sinusoidal_rotary`, `learned_absolute`, `none`}

The implementation lives in `src/model/embeddings/` and the shared injection helpers in `src/model/attention/common.py`. Bibliography keys cited here resolve against `docs/bibliography/`.

## 14.1 Architecture: One Shared Module, Per-Mixer Opt-Out

A single PE module is built once in the encoder (`FrankensteinEncoder` or `FrankensteinViT`) via `build_pos_encoder(config)` and injected into every `HybridLayer`, which forwards it to each mixer's `__init__(config, pos_encoder=None)` and `forward(..., pos_encoder=None)`.

Each mixer carries a per-mixer `use_pe` flag (`<mixer>_attn_use_pe`, default `True` for attention mixers, `False` for recurrent/decay mixers such as `retnet`, `mamba`, `ode`, `gla_attn`, `deltanet_attn`, `mtla_attn`, and the six Falcon mixers `falcon{1,2,3,1a,2a,3a}_attn`). When `use_pe` is `False` the mixer ignores the injected module entirely.

The PE is applied in two phases via the shared helpers `apply_pe_to_qk()` (called before the attention-score computation) and `apply_pe_to_scores()` (called after). The helper dispatches on the PE type and falls through to a no-op when the mixer's `use_pe` flag is `False`.

## 14.2 Injection Styles

The 11 enum values fall into four injection styles:

1. **Rotation** (`rope`, `hope`, `sinusoidal_rotary`): rotates the query and key tensors before the score computation. The inner product $\mathbf{q}_i^\top \mathbf{k}_j$ becomes a function of the relative offset $i - j$.
2. **Score bias** (`alibi`): adds a distance-proportional bias to the attention-score matrix *after* it is computed.
3. **Q/K augmentation** (`pape`, `pape_efficient`, `pape_ri`): concatenates extra positional dimensions onto the query and key vectors before the dot product, increasing the effective head dimension.
4. **Additive at embedding level** (`sinusoidal_absolute`, `learned_absolute`): added directly to the token embeddings before the first layer; no per-mixer action is taken in the attention block.

`nope` and `none` are both no-ops: `nope` explicitly disables positional encoding (the model must learn position implicitly), while `none` is the null module returned by the factory when no PE is desired.

## 14.3 Mathematical Formulations

### 14.3.1 RoPE — Rotary Position Embedding

RoPE [91] rotates each head's query and key by an angle proportional to the token index. For head dimension $d_h$ and token position $m$, the rotation matrix $\mathbf{R}_m \in \mathbb{R}^{d_h \times d_h}$ acts on 2-D sub-spaces $(x_{2i}, x_{2i+1})$:

$$\mathbf{R}_m = \operatorname{diag}\left( \begin{bmatrix} \cos m\theta_0 & -\sin m\theta_0 \\ \sin m\theta_0 & \cos m\theta_0 \end{bmatrix}, \ldots, \begin{bmatrix} \cos m\theta_{d_h/2-1} & -\sin m\theta_{d_h/2-1} \\ \sin m\theta_{d_h/2-1} & \cos m\theta_{d_h/2-1} \end{bmatrix} \right),$$

with frequencies $\theta_i = \texttt{rope_base}^{-2i/d_h}$. The attention score becomes $\mathbf{q}_m^\top \mathbf{R}_{m-n} \mathbf{k}_n$, a function of the relative offset $m-n$. A `rope_scaling` factor $s$ rescales the effective position to $m/s$ for length-extrapolation modes.

### 14.3.2 HoPE — Hyperbolic Rotary Positional Encoding

HoPE [18] generalises RoPE to the hyperbolic geometry of the Lorentz model. Instead of Euclidean rotations, HoPE applies Lorentz rotations built from hyperbolic functions, enforcing a *monotonic decay* of attention weights with increasing token distance — a property RoPE lacks because its oscillatory attention pattern can re-spike at long ranges. RoPE is recovered as a special case. The `hope_base` and `hope_damping` knobs control the hyperbolic frequency and the decay rate.

### 14.3.3 NoPE — No Positional Encoding

NoPE [48] is the explicit absence of any positional injection. The model must discover position implicitly from the causal mask (decoder) or from the data itself. The cited work shows that NoPE outperforms explicit PE methods (including RoPE and ALiBi) on length-generalisation benchmarks while requiring no additional computation, because a causal-attention decoder can implicitly encode relative position through its attention patterns.

### 14.3.4 ALiBi — Attention with Linear Biases

ALiBi [78] does not modify the embeddings or the query/key vectors. Instead it adds a static, head-specific linear bias to the attention-score matrix:

$$\text{score}_{i,j}^{(h)} = \frac{\mathbf{q}_i^{(h)\top}\mathbf{k}_j^{(h)}}{\sqrt{d_h}} - m_h \cdot (i-j), \qquad m_h = \frac{1}{2^{\lfloor h/\lceil H/8 \rceil \rfloor}},$$

where $m_h$ is a geometric slope per head $h$ and the bias penalises distant keys. The `alibi_num_heads` knob controls the slope schedule's head count. ALiBi enables length extrapolation: a model trained on 1024 tokens generalises to 2048 without retraining.

### 14.3.5 BAM — Bayesian Attention Mechanism

BAM [9] reframes positional encoding as a probabilistic prior over positions and unifies existing methods: NoPE corresponds to a Uniform prior, ALiBi to a Uniform×Laplace prior. The practical instantiation, GGD-BAM, adds a Generalized-Gaussian relative-position bias matrix $\mathbf{B}$ to the attention logits, exactly like ALiBi but with a non-linear, learnable shape:

$$B_{i,j}^{(h)} = -\big(|j - \alpha_h\, i - \mu_h| + \varepsilon\big)^{\beta_h}, \qquad \alpha_h = e^{-\theta_\beta^{(h)}\,\theta_\alpha^{(h)}} \geq 0,$$

with per-head learnable $\theta_\alpha, \theta_\beta$ (init `bam_theta_init`=0, the Uniform prior) and optional location $\theta_\mu$ (fixed to 0 by default via `bam_learn_mu=false`). The floor $\varepsilon$ (`bam_eps`, default $10^{-5}$) stabilises the exponentiation when $\beta < 0$, a relaxed regime that suppresses local context and acts as a long-range "retrieval head". Special cases: $\beta = 1, \mu = 0$ recovers ALiBi; $\beta = 2$ gives a Normal prior; $0 < \beta < 1$ yields heavier tails than Laplace. The paper pairs BAM with Scalable Softmax (SSMax), a transversal logit rescale $s \cdot \ln(n)$ (learnable per-head $s$) that counteracts "attention fading" and is composable with any PE via the top-level `use_ssmax` flag. BAM SSMax achieves accurate information retrieval at 500× the training context length.

### 14.3.6 PaPE — Parabolic Position Encoding

PaPE [72] is a vision-centric encoding that augments the query and key with extra positional dimensions built from parabolic basis functions. Given $P$ parabolas (`pape_num_parabolas`) and $S$ positions (`pape_num_positions`), the positional feature for coordinate $\mathbf{p} \in \mathbb{R}^D$ is

$$\phi_k(\mathbf{p}) = \sum_{j=1}^{S} \alpha_{k,j}\, f_j(\mathbf{p}), \qquad f_j(\mathbf{p}) = -\|\mathbf{p} - \mathbf{c}_j\|^2 + b_j,$$

a sum of $S$ paraboloids centred at $\mathbf{c}_j$ with offset $b_j$. The $P \cdot S$ extra dimensions are concatenated to the query and key before the dot product, so the effective head dimension grows from $d_h$ to $d_h + P \cdot S$. Three variants are provided:

- `pape` — the reference implementation.
- `pape_efficient` — a pure-PyTorch reformulation that avoids custom Triton kernels while preserving the same math.
- `pape_ri` — the rotation-invariant variant (PaPE-RI in the paper), which satisfies rotation invariance by construction and is controlled by `pape_rotation_invariant`.

PaPE is designed for $n$-D vision tokens (images, video, event cameras, point clouds) and is the recommended PE for the ViT path.

### 14.3.7 Sinusoidal Absolute

The original Transformer absolute encoding [97] adds a fixed sinusoidal vector to the token embedding:

$$\mathrm{PE}_{(m,2i)} = \sin\left(\frac{m}{\texttt{sinusoidal_base}^{2i/d}}\right), \quad \mathrm{PE}_{(m,2i+1)} = \cos\left(\frac{m}{\texttt{sinusoidal_base}^{2i/d}}\right),$$

with `sinusoidal_max_len` capping the precomputed table and `sinusoidal_scale` multiplying the embedding. This is an additive PE: it is applied once at the embedding level and the mixers do not apply any per-layer PE.

### 14.3.8 Sinusoidal Rotary

A rotary variant of the sinusoidal encoding: the same sinusoidal frequencies are used to build a rotation matrix applied to the query and key (as in RoPE), but with the `sinusoidal_base` and `sinusoidal_scale` knobs. This is a rotation-style PE applied per-layer via `apply_pe_to_qk`.

### 14.3.9 Learned Absolute

A learnable absolute encoding: an `learned_max_len` $\times\, d$ parameter matrix is added to the token embeddings at the embedding level. Initialisation standard deviation is controlled by `learned_init_std`. As with sinusoidal absolute, the mixers take no per-layer action.

## 14.4 Per-Mixer `use_pe` Flags

Each mixer exposes a `<mixer>_attn_use_pe` config field (e.g. `standard_attn_use_pe`, `titan_attn_use_pe`). The defaults are:

- **True** (attention mixers): `standard_attn`, `titan_attn`, `sigmoid_attn`, `gated_softmax_attn`, `gqa_attn`, all latent mixers (`mla_attn`, `gqla_attn`, `mlra_attn`, `tucker_attn`, `iha_attn`, `gta_attn`, `cca_attn`, `ccgqa_attn`, `gma_attn`), all sparse mixers (`longformer_attn`, `bigbird_attn`, `sparse_transformer_attn`, `sparsek_attn`, `nsa_attn`, `msa_attn`, `sparda_attn`, `fasa_attn`, `sparge_attn`), and `engram_attn`.
- **False** (recurrent/decay mixers): `retnet`, `retnet_attn`, `mamba`, `ode`, `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `gated_deltanet2_attn`, `hgrn2_attn`, `fox_attn`, `kda_attn`, `mtla_attn`.

The `False` default for recurrent/decay mixers reflects that these architectures bake their own temporal/positional inductive bias into their recurrence (e.g. RetNet's complex rotation, Mamba's selective scan). Overriding `use_pe=True` is allowed but is typically redundant.

## 14.5 Vision Transformer Unification

The ViT path (`FrankensteinViT`) previously used its own `pos_embedding_type` field with values `learned_1d`, `none`. This has been unified with the model-wide enum: `learned_1d` is migrated to `learned_absolute`, and `none` maps to `none`. The ViT builds the same `build_pos_encoder(config)` module and injects it into its HybridLayers, so a ViT can now use RoPE, HoPE, ALiBi, PaPE, or any other PE in the enum.

## 14.6 Legacy Compatibility

The legacy `use_hope` boolean flag is deprecated and mapped to the new enum: `use_hope=True` → `positional_encoding=hope`, `use_hope=False` → `positional_encoding=rope`. The per-mixer `positional_encoding` override that previously existed only on `titan_attn` is now a no-op: the model-wide enum is the single source of truth, and the TitanAttention allow-list that restricted PE to `{hope, rope, none, nope}` has been removed — all 12 values now work with every mixer.

## 14.7 Architectural Profile: Positional Encodings

Table 16 summarises the 12 supported positional encodings.

| Name | Style | Key Idea | Params | Reference |
|---|---|---|---|---|
| rope | Rotation | Frequency-based rotation of Q/K; relative position via inner-product geometry | `rope_base`, `rope_scaling` | [91] |
| hope | Rotation | Hyperbolic Lorentz rotation; monotonic attention decay with distance | `hope_base`, `hope_damping` | [18] |
| nope | None | Explicit no-PE; model learns position implicitly from causal mask | — | [48] |
| alibi | Score bias | Static linear distance penalty added to attention scores; length extrapolation | `alibi_num_heads` | [78] |
| bam | Score bias | Generalized-Gaussian relative-position bias; learnable per-head shape ($\beta$) and scale ($\alpha$); recovers ALiBi at $\beta = 1$; SSMax rescale via `use_ssmax` | `bam_theta_init`, `bam_learn_mu`, `bam_eps`, `use_ssmax`, `ssmax_s_init` | [9] |
| pape | Q/K aug. | Parabolic basis functions concatenated to Q/K; vision-centric, $n$-D | `pape_num_parabolas`, `pape_num_positions` | [72] |
| pape_efficient | Q/K aug. | Pure-PyTorch PaPE; no custom Triton kernels | same as pape | [72] |
| pape_ri | Q/K aug. | Rotation-invariant PaPE (PaPE-RI) | `pape_rotation_invariant` | [72] |
| sinusoidal_absolute | Additive | Fixed sinusoidal added to embeddings (original Transformer) | `sinusoidal_base`, `sinusoidal_max_len`, `sinusoidal_scale` | [97] |
| sinusoidal_rotary | Rotation | Sinusoidal frequencies as a rotation matrix on Q/K | same as sinusoidal | [97] |
| learned_absolute | Additive | Learnable parameter matrix added to embeddings | `learned_max_len`, `learned_init_std` | [97] |
| none | None | Null module; no positional information injected | — | — |

Table 16: Positional encoding catalog. "Style" is the injection mechanism (rotation, score-bias, Q/K-augmentation, additive). "Params" lists the principal config knobs.

# 15 SSOG: Separable Sum of Gaussians Attention

This annex covers SSOG (Separable Sum of Gaussians), a geometric-field attention mixer. SSOG [76] replaces the content-scored interaction of scaled dot-product attention with a learned mixture of Gaussians over *relative position*: each head owns a handful of Gaussian atoms—five numbers per atom, a center offset $(\mu_y, \mu_x)$, a width per axis $(\sigma_y, \sigma_x)$ and a mixture weight $\lambda$—forming a fixed field over the displacement between tokens. There are no query–key dot products anywhere: content never scores content, it only *steers* the field through small bounded residuals.

Because a 2D Gaussian factorises into a product of two 1D Gaussians, applying the field is two 1D filter passes per atom (rows, then columns of a $\sqrt{N} \times \sqrt{N}$ token raster) followed by a $\lambda$-mix contraction. The induced $N \times N$ attention matrix never exists, so the attention-specific cost is $\mathcal{O}(R \cdot N\sqrt{N} \cdot d)$ for $R$ atoms instead of $\mathcal{O}(N^2 \cdot d)$. SSOG also skips the query and key projections entirely, paying $2d^2$ of projections per layer instead of the $4d^2$ of SDPA. In the reference recipe the full mechanism beats its matched SDPA twin on ImageNet-1k (65.28% vs 64.34% at ∼3M params; 72.02% vs 71.84% at ∼12M) while being ∼20% smaller and ∼30% cheaper per forward, and by +17 points on CIFAR-100 at matched recipe.

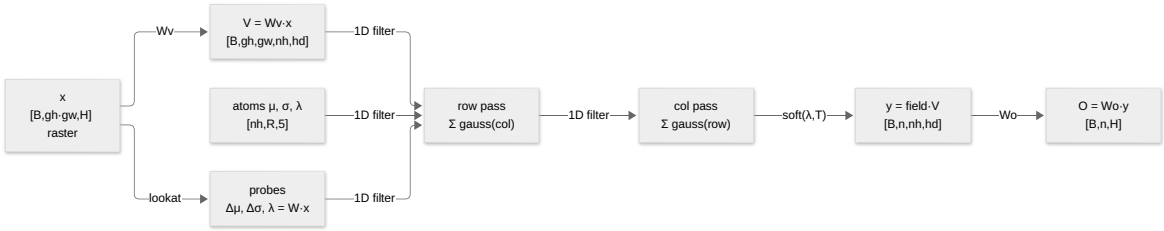



Figure 68: Dataflow of SSOG: values pass through separable row and column Gaussian filter passes per atom, with optional content steering of the atom parameters.

## 15.1 Mathematical Formulation

Let tokens be the row-major raster of a $G_h \times G_w$ grid. The attention weight from token $p$ to token $q$ is the softmax-tempered log-sum-exp of the atom log-weights evaluated at their displacement:

$$A(p,q) = \mathrm{softmax}_q\big(\tfrac{1}{\tau}\, s(p,q)\big)\,, \qquad s(p,q) = \mathrm{logsumexp}_r\,\big(\log \lambda_r + \log \mathcal{N}(p-q;\ \mu_r, \sigma_r)\big),$$

where $\tau$ is a learnable temperature and $\mathcal{N}(\cdot\,; \mu_r, \sigma_r)$ is a 2D Gaussian with diagonal covariance. Since the 2D Gaussian factorises, the softmaxed field applies separably: per axis, $a_y[i,j] = \mathrm{softmax}_j(\log \mathcal{N}(\Delta y_{ij}; \mu_y, \sigma_y)/\tau)$ and analogously $a_x$, followed by

$$Y = \sum_{r=1}^{R} \lambda_r \left(a_x^{(r)} \big(a_y^{(r)}\, V\big)\right), \qquad V = xW_V,$$

i.e. three einsum contractions per layer. With content steering (`lookat`) enabled, zero-initialised linear probes predict bounded per-token residuals on the field parameters behind cold-started softplus gates:

$$\mu \leftarrow \mu_0 + s_\mu \cdot m \cdot \tanh(W_\mu x), \qquad \sigma \leftarrow \sigma_0\, e^{s_\sigma \tanh(W_\sigma x)}, \qquad \lambda \leftarrow \mathrm{softmax}\,\big(\log \lambda_0 + s_\lambda \tanh(W_\lambda x)\big),$$

with $m$ the maximum offset in grid cells and gate scales $s_\bullet = \mathrm{softplus}(\cdot)$ initialised at $-8$ ($\approx 3 \times 10^{-4}$), so the model begins as a frozen geometric field and learns how far to open each content tap. Widths are reparameterised as $\sigma = \mathrm{softplus}(\omega) + \epsilon + \sigma_{\text{floor}}$ to stay strictly positive.

### 15.2 Algorithmic Pseudocode

**Input:** hidden state $x \in \mathbb{R}^{B\times N\times d}$ as a $G_h \times G_w$ raster, atoms $\{\mu_r, \omega_r, \log\lambda_r\}_{r=1}^{R}$ per head, temperature $\tau$
$V \leftarrow xW_V$ reshaped to $(B, G_h, G_w, H, d_h)$ ▷ no Q/K projections
$\sigma_r \leftarrow \mathrm{softplus}(\omega_r) + \epsilon + \sigma_{\text{floor}}$ ▷ positive widths
**if** steering (lookat) **then**
    $\mu, \sigma, \lambda \leftarrow$ bounded per-token residuals via zero-init probes and gates ▷ content steers
**end if**
$a_y \leftarrow \mathrm{softmax}\big(\log \mathcal{N}(\Delta y; \mu_y, \sigma_y)/\tau\big)$, $a_x \leftarrow$ analogously ▷ 1D kernels, one per atom
$Y \leftarrow \sum_r \lambda_r \, a_x^{(r)} \big(a_y^{(r)} V\big)$ ▷ row pass, column pass, $\lambda$-mix: no $N \times N$
**Output:** $\mathbf{Y}W_O$ reshaped to $(B, N, d)$

### 15.3 Key Characteristics

- **Training complexity**: $\mathcal{O}(R \cdot N\sqrt{N} \cdot d)$ for fixed $R$ atoms—near-linear by geometry alone, not by kernel approximation.
- **Inference complexity**: same per-forward cost; encoder-only (the separable field has no causal formulation), so there are no KV-cache semantics.
- **Trainable**: atoms $\{\mu, \omega, \log\lambda\}$, temperature, and the steering probes are fully differentiable; cold-started gates open during optimisation.
- **Strengths**: strong geometric prior (+17 pts over SDPA on small data at matched recipe; beats SDPA on ImageNet-1k at 3M and 12M params with ∼20% fewer parameters and ∼30% fewer FLOPs); inherently interpretable (each head is a few plottable blobs—early layers become convolutions in disguise, middle layers strip detectors, late layers go global); the field lives on coordinates, granting zero-shot resolution transfer.
- **Limitations**: encoder-only; requires a fixed $G_h \times G_w$ row-major raster (ViT needs `cls_token=false`; sequences need an explicit $1 \times L$ grid); the per-query steering kernels add a memory term; language results are unproven—content scoring is likely load-bearing there.

## 16 Fast-Weight Attention for Continual Learning

This annex covers the Falcon family [116], a fast-weight attention framework that treats the recurrent fast-weight state transition as an **online continual-learning rule** under *read-after-write* (RAW) autoregressive semantics. The framework's central observation is one of temporal alignment: for the prefix-prediction objective, the local fast-memory example revealed at step $t$ is the **prefix-aligned pair** $(x_t, y_t) = (\phi(k_{t-1}), v_t)$ — one step shifted relative to the same-step association $(\phi(k_t), v_t)$ used by DeltaNet and standard linear attention. The same-step pairing remains causal, but it optimizes a different internal objective. The framework derives **normalized first-order updates** for two objectives (squared-error regression and negative inner product) and three step-size structures (scalar, per-column, sliding window), giving six variants: Falcon-1/2/3 (regression family) and Falcon-1A/2A/3A (inner-product family). All six are implemented in the codebase under `model.attention.falcon` with a shared configuration block, each providing both a recurrent reference scan and an SSD-style chunk-parallel kernel.

### 16.1 Preliminaries: Alignment, Normalization, and Stability

**Read-after-write semantics.** With feature maps $\phi(k_t)$ (QK-RMSNorm by default, $\ell_2$ norm optional), the prefix-aligned stream sets $x_t = \phi(k_{t-1})$ for $t \geq 2$ with boundary sentinel $x_1 = 0$;

the readout is $o_t = S_t^\top \phi(q_t)$. The write at step $t$ therefore updates the memory *before* the query at step $t+1$ reads it, so no token's key ever associates with its own value.

**Normalized first-order step sizes.** A gradient step on the local objective $\ell_t(S)$ with the objective-matched denominator gives

$$\eta_t = \frac{\beta_t}{\text{statistic}_t + \lambda_t + \varepsilon}, \qquad \beta_t \in (0, 2),$$

where the statistic is the squared key energy $\|x_t\|^2$ (Falcon-1/1A/2/2A), the window spectral statistic $\mu_t^{(B)} = \lambda_{\max}(X^\top X)/B_t$ (Falcon-3), or the window mean energy $\bar{E}_t^{(B)}$ (Falcon-3A). Objective-matched normalization guarantees per-step descent and removes the scale sensitivity of fixed learning rates. The **plasticity** gain $\beta_t$, the **forgetting** ridge $\lambda_t$, the temporal **alignment**, and the rehearsal window $B$ are thereby four previously entangled knobs, separated.

**Positive-decay renormalization.** When $\lambda_t > 0$, the per-step shrinkage $\alpha_t = \min(\eta_t \lambda_t, 1 - \varepsilon_\gamma)$ must be applied stably. The implementation clamps $\alpha_t$ strictly below 1, computes chunk-local log-prefixes in fp32 via $\log(1 - \alpha_t)$ and cumulative sums, rescales the drivers to the no-decay domain ($\tilde{\eta}_t = \eta_t/\gamma_t$, $\tilde{v}_t = v_t/c_{t-1}$ with $c_{t-1} = \prod_{r \le t-1} \gamma_r$), and rescales outputs and the emitted state back by the accumulated $\delta$. Boundary sentinels are $(\alpha_1, \gamma_1) = (0, 1)$ and $\eta_1 = 0$, so the first step is a pure read.

## 16.2 The Regression Family (Falcon-1/2/3)

The squared-error objective on the shifted pair is

$$\ell_t(S) = \tfrac{1}{2}\|v_t - S^\top x_t\|^2 + \tfrac{\lambda_t}{2}\|S\|_F^2, \qquad \nabla_S \ell_t = x_t (S^\top x_t - v_t)^\top + \lambda_t S,$$

which is $L_t$-smooth with $L_t = \|x_t\|^2 + \lambda_t$.

### 16.2.1 Falcon-1: Scalar NLMS Regression

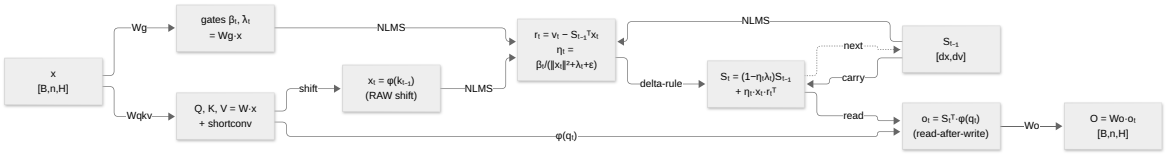



Figure 69: Dataflow of Falcon-1: the prefix-aligned key/value pair drives a scalar NLMS delta-rule update of the fast-weight state.

The scalar delta-rule variant — the classical NLMS recursion of adaptive filtering upgraded with the shifted write stream and an explicit ridge:

$$S_t = (1 - \eta_t \lambda_t)\, S_{t-1} + \eta_t\, x_t \big(v_t - S_{t-1}^\top x_t\big)^\top, \qquad o_t = S_t^\top \phi(q_t).$$

With $\varepsilon = 0$, $\lambda_t = 0$ and the unshifted assignment $x_t = \phi(k_t)$ this recovers DeltaNet exactly. The eigenstructure of the transition $A_t = \gamma_t I - \eta_t x_t x_t^\top$ (with $\gamma_t = 1 - \eta_t \lambda_t$) shows that the write direction contracts by $1 - \beta_t$ while the orthogonal subspace carries $\gamma_t$: $\beta_t > 1$ yields a stable sign-flip along the write direction, useful for state tracking. The chunk-parallel form uses the WY representation $\prod_s (I - \eta_s x_s x_s^\top) = I - UTU^\top$ with a single-residual unit-lower-triangular solve $L = \text{tril}(G, -1) + I$ where $G = X^\top X$ (Algorithm 8 of the paper), after the positive-decay renormalization.

### 16.2.2 Falcon-2: Per-Column NLMS Regression

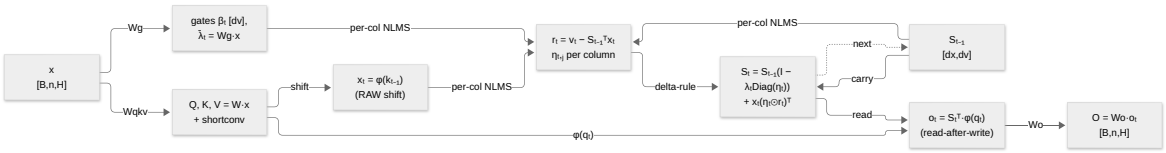



Figure 70: Dataflow of Falcon-2: per-column NLMS step sizes update the fast-weight state channel by channel.

The ridge loss decomposes across value coordinates, so each value channel $j$ receives its own step size while sharing the NLMS normalizer:

$$\eta_{j,t} = \frac{\beta_{j,t}}{\|x_t\|^2 + \lambda_t + \varepsilon}, \qquad S_t = S_{t-1}\big(I - \lambda_t \operatorname{Diag}(\boldsymbol{\eta}_t)\big) + x_t\big(\boldsymbol{\eta}_t \odot \mathbf{r}_t\big)^\top.$$

The chunk-parallel kernel forms the shared key Gram $G = X^\top X$ and solves the batch of per-channel unit-lower-triangular systems $L_j = I + \operatorname{tril}\big(G \odot (\Sigma_j \Sigma_j^\top), -1\big)$ with $\Sigma = \sqrt{\widetilde{\boldsymbol{\eta}}}$ (Algorithm 1). Defined but not separately benchmarked in the paper.

### 16.2.3 Falcon-3: Sliding-Window Minibatch Regression

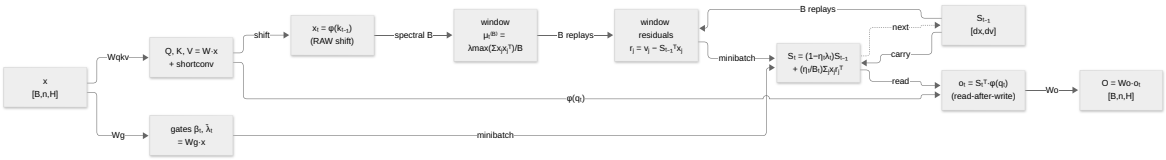



Figure 71: Dataflow of Falcon-3: a sliding window of prefix-aligned pairs provides the spectral statistic and the minibatch write.

Bounded rehearsal: each step regresses the fast memory on the last $B_t \le B$ prefix-aligned pairs, with *all window residuals evaluated at the pre-update state* $S_{t-1}$:

$$\mu_t = \frac{\lambda_{\max}\big(X_{\mathcal{I}_t}^\top X_{\mathcal{I}_t}\big)}{B_t}, \qquad \eta_t = \frac{\beta_t}{\mu_t + \lambda_t + \varepsilon}, \qquad S_t = (1 - \eta_t\lambda_t)\, S_{t-1} + \frac{\eta_t}{B_t} \sum_{j \in \mathcal{I}_t} x_j \big(v_j - S_{t-1}^\top x_j\big)^\top,$$

with $\mathcal{I}_t = \{\max(2, t - B + 1), \ldots, t\}$. Window-averaging keeps the update magnitude invariant to $B$: each pair is replayed in exactly $B$ consecutive updates at weight $1/B$, so its cumulative injection is $\eta$-independent of $B$. The state alone is *not* Markov: exact continuation requires a FIFO tail of the last $B - 1$ pairs, and the chunk kernels gather an extended slice with $|\mathcal{J}| \le C + B - 1$. The chunk-parallel form is a rank-$B$ affine recurrence solved by block forward substitution over the (time $\times$ rank) extended index space, with same-time rank components uncoupled (Algorithm 3).

## 16.3 The Inner-Product Family (Falcon-1A/2A/3A)

The negative inner-product objective

$$\ell_t^{\mathrm{ip}}(S) = -\langle S^\top x_t,\, v_t\rangle + \tfrac{\lambda_t}{2}\|S\|_F^2 \qquad \Rightarrow \qquad \nabla_S \ell_t^{\mathrm{ip}} = -x_t v_t^\top + \lambda_t S$$

yields purely additive (Hebbian) writes with energy-normalized gains.

### 16.3.1 Falcon-1A and Falcon-2A

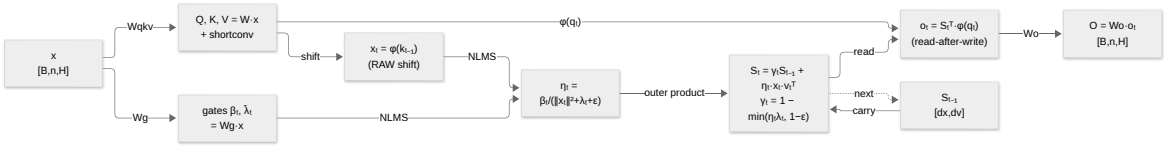



Figure 72: Dataflow of Falcon-1A: an energy-normalized additive (Hebbian) write with a derived decay.

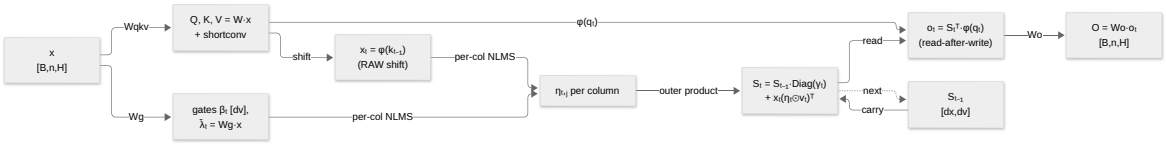



Figure 73: Dataflow of Falcon-2A: per-column additive writes with a per-channel decay.

$$\text{Falcon-1A:}\quad S_t = \gamma_t S_{t-1} + \eta_t\, x_t v_t^\top, \quad \gamma_t = 1 - \min(\eta_t\lambda_t,\, 1-\varepsilon_\gamma);$$
$$\text{Falcon-2A:}\quad S_t = S_{t-1}\,\mathrm{Diag}(\boldsymbol{\gamma}_t) + x_t(\boldsymbol{\eta}_t \odot v_t)^\top.$$

With $\lambda_t = 0$ and the unshifted assignment this is exactly standard linear attention / Mamba-2 accumulation; the shifted stream makes it the one-step-shifted next-latent variant. The A-family's denominator is not required by curvature (the objective is linear) — it retains the energy-normalized write as a magnitude stabilizer. The chunk-parallel form is decay-mask linear attention with $M_{t,i} = \eta_i \prod_{r=i+1}^{t} \gamma_r$ (per-channel masks for Falcon-2A).

### 16.3.2 Falcon-3A: Sliding-Window Inner-Product

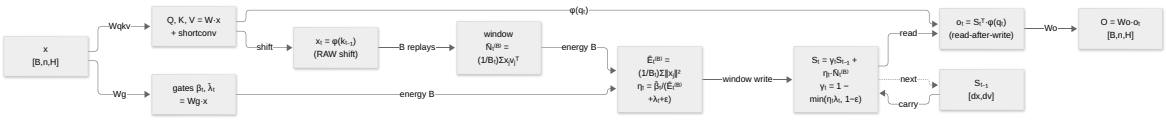



Figure 74: Dataflow of Falcon-3A: a sliding-window mean-energy statistic normalizes the additive window write.

The windowed additive variant — the family's best length-extrapolation configuration:

$$\bar{N}_t^{(B)} = \frac{1}{B_t}\sum_{j\in\mathcal{I}_t} x_j v_j^\top, \quad \bar{E}_t^{(B)} = \frac{1}{B_t}\sum_{j\in\mathcal{I}_t} \|x_j\|^2, \quad \eta_t = \frac{\beta_t}{\bar{E}_t^{(B)} + \lambda_t + \varepsilon}, \qquad S_t = \gamma_t S_{t-1} + \eta_t\, \bar{N}_t^{(B)}.$$

The window-banded decay mask factors as $M = D \cdot \mathrm{Diag}(\boldsymbol{\eta}) \cdot A$, where $A$ is the $B$-banded window operator and $D$ the causal decay kernel, evaluated over the extended slice (Algorithm 4).

## 16.4 Configuration Interface

All six variants share one nested block, `model.attention.falcon`, selected through the `layer_pattern` entries `falcon1_attn`, `falcon2_attn`, `falcon3_attn`, `falcon1a_attn`, `falcon2a_attn`, `falcon3a_attn`:

| Key | Default | Description |
|---|---|---|
| `chunk_size` | 64 | Chunk size of the chunk-parallel kernels; `null` selects the recurrent reference scan |
| `qk_norm` | `rms_norm` | QK normalization (`rms_norm` or `l2_norm`); RMSNorm computed in fp32 |
| `beta_mode` | `ctx_eta` | Plasticity gate parameterization: `static`, `ctx_beta` (content-conditioned gain), or `ctx_eta` (content-conditioned normalized step) |
| `lambda_mode` | `ctx` | Forgetting gate: `static` or `ctx` (scale-couples with the detached curvature statistic) |
| `beta` / `lambda` | 1.0 / 0.0 | Static gain and ridge (used when the corresponding mode is `static`) |
| `window` | 4 | Sliding window $B$ (Falcon-3 / Falcon-3A only) |
| `short_conv` / `conv_kernel` | `true` / 4 | Causal depth-wise short convolutions on the q/k/v projections |
| `eps` / `eps_gamma` | $10^{-6}$ / $10^{-4}$ | Step-size stabilizer and positive-decay clamp floor |

Table 17: Shared Falcon configuration knobs (`model.attention.falcon`). Each variant additionally exposes a per-mixer positional-encoding opt-out `falcon{1,2,3,1a,2a,3a}_attn_use_pe`, default `False` like other recurrent/decay mixers.

Implementation notes: the recurrent and chunk-parallel kernels are verified numerically equivalent (chunk $\equiv$ recurrent to $\leq 3.6 \times 10^{-7}$ across all six variants, including decay, multi-chunk, and windowed paths); gradients flow through the fp32 triangular-solve islands (feature drivers take only $\sqrt{\bar{\eta}}$ while value targets take $\sqrt{\bar{\eta}}\, e^{-u_{t-1}}$, with the square root clamped at $10^{-12}$ for NaN-free backward at the $\eta = 0$ boundary); positive exponents are clamped at 80.

## 16.5 Published Results

| Model (124M, 49.2B FineWeb-Edu tokens) | WikiText ppl | LMB. ppl | FineEdu ppl | 0-shot avg |
|---|---|---|---|---|
| Transformer (RoPE) | 33.25 | 47.43 | 17.38 | 48.16 |
| Mamba-2 | 34.53 | 48.74 | 17.70 | 48.80 |
| DeltaNet | 34.19 | 52.84 | 17.84 | 48.88 |
| Gated DeltaNet | 30.99 | 46.70 | 17.32 | 48.78 |
| Falcon-1A.3 | 34.02 | 49.84 | 17.40 | 48.95 |
| Falcon-1.3 | 33.00 | 48.70 | **17.10** | **49.18** |

Table 18: Language modeling results reported by Zhang et al. [116] at 124M parameters. On variable-digit addition length extrapolation (33–48 digits), Falcon-3A.3 reaches **87.2%** mean accuracy versus 65.8% for a RoPE Transformer and 85.9% for Falcon-1A.3: the inner-product family carries arithmetic, while the regression family carries perplexity.

## 16.6 Relation to Prior Work

- **DeltaNet**: the delta rule is the squared-error one-gradient-descent step with the same-step pairing; Falcon recovers it with the critical index shift plus objective-matched normalization and explicit $\lambda_t$ shrinkage.
- **Gated DeltaNet**: adds a free learned carry gate; Falcon *derives* the carry $\gamma_t = 1 - \eta_t \lambda_t$ from the local objective and the normalized step-size parameterization instead of adding an independent gate, and defaults to QK-RMSNorm rather than $\ell_2$ normalization.

- **Linear attention / Mamba-2**: the additive numerator is one gradient step on the inner-product objective with the unshifted assignment; the chunkwise kernels reuse the SSD scheme.
- **TTT / Titans / ATLAS**: fast-weight updates as test-time training; Falcon-3 is a sliding-window specialization of the internal-objective view with strict next-latent alignment.
- **Adaptive filtering (LMS/NLMS/RLS)**: Falcon imports the stability and normalization principles of NLMS into fast-weight attention under strict causality.

#### 16.6.1 When to Use Each Variant

1. **Falcon-1**: best language-modeling perplexity of the family (FineEdu 17.10, beating Gated DeltaNet 17.32); principled forgetting tied to the local objective.
2. **Falcon-2**: finer-grained plasticity than Falcon-1 at the cost of heavier per-channel chunk kernels; unbenchmarked in the paper.
3. **Falcon-3**: controlled rehearsal horizon with exact window statistics; most complex kernel.
4. **Falcon-1A**: simple and stable additive variant; second-best length extrapolation (85.9%).
5. **Falcon-2A**: channel-wise forgetting granularity; unbenchmarked.
6. **Falcon-3A**: best length extrapolation of the paper (87.2% on 33–48-digit addition); carries arithmetic tasks.

# 17 KV Cache and Incremental Decoding

Autoregressive decoding is dominated by a structural asymmetry: the prefill stage processes the whole prompt in one quadratic pass, while each decode step appends a single token and needs only one new query row against an already-computed history. Recomputing the full sequence at every step turns generation into an $\mathcal{O}(S^2)$-per-token procedure; maintaining the per-layer history turns it into $\mathcal{O}(S)$ per token with constant-size work per new position. This annex documents the toolkit's KV-cache subsystem (`src/model/cache/`), the correctness contracts that make cached and uncached execution numerically interchangeable, the per-family incremental recipes, and the write-time quantizers and read-time eviction policies layered on top of it.

Three design commitments frame everything below. First, the cache is *opt-in*: with `inference.kv_cache.enabled` unset (default), generation executes the original full-recompute loop bit-identically. Second, the cache is *central and declarative*: a single manager is threaded through the model, and a per-mixer capability registry decides which layers may cache — never the other way around. Third, the cache is *never fatal* (decision D5): a layer whose source formulation does not admit an exact incremental form silently runs uncached, so enabling the cache can only ever change speed, not correctness or availability.

## 17.1 Motivation and Scope

Let $S$ be the prompt length and $T$ the number of generated tokens. Without a cache, step $t$ feeds $S + t - 1$ tokens through every layer and discards all intermediate keys and values, for total attention work

$$\mathcal{O}\left(\sum_{t=0}^{T-1}(S+t)^2\right) \;=\; \mathcal{O}\big(TS^2 + T^2S + T^3\big)\,.$$

With a cache, the prefill costs $\mathcal{O}(S^2)$ once and each decode step costs $\mathcal{O}(S+t)$:

$$\mathcal{O}\left(S^2 + \sum_{t=0}^{T-1}(S+t)\right) \;=\; \mathcal{O}\big(S^2 + TS + T^2\big)\,.$$

The difference is the familiar one between re-encoding a conversation and continuing it.

The subsystem covers every one of the forty-four dispatchable layer types. Twenty-six have parity-tested incremental branches; the remainder are registered with an explicit, paper-based rationale for running uncached (§17.4.5). Quantization and eviction are orthogonal to the mixer mathematics: codecs transform tensors at *write* time and policies act at *read* time, so the same `int8` or `int4_kivi` codec serves a dense, latent or ring slot without per-mixer code [62, 43, 23].

## 17.2 Cache Architecture

### 17.2.1 Central Manager and Layer Identity

The unit of caching is a logical layer, not a module. The encoder may re-execute the same physical stack `num_loops` times (looped depth) and, under Hyperloop, partitions it into a begin block, a looped middle block and an end block. The same `HybridLayer` object therefore plays several distinct roles during one forward pass. The cache keys every slot by the global **logical layer index**

$$\ell \;=\; 0, \ldots, \texttt{num_layers} \times \texttt{num_loops} - 1,$$

which the encoder already threads into every mixer call. Two logical roles never share storage even though they share weights.

**Algorithm 43** Cached generation loop (prefill + decode)

**Require:** prompt ids $\mathbf{x}_{1:S}$; generation config $\theta$; model $f$
**Ensure:** generated ids

$\mathcal{C} \leftarrow \textsc{Cache.new}(f, \theta)$ ▷ lazy slots, token tail, write cursor
$\mathbf{L}, \mathcal{C} \leftarrow f(\mathbf{x}_{1:S}; \text{past} = \mathcal{C}, \text{use_cache} = \top, \text{return_cache} = \top)$ ▷ prefill
$\mathbf{z} \leftarrow \mathbf{L}_{:,S,:}$
**for** $t \leftarrow 0$ **to** $T-1$ **do**
  $y \leftarrow \textsc{Sample}(\mathbf{z}; \theta)$ ▷ greedy, top-$k$, top-$p$
  **append** $y$ to the output
  **if** $t < T-1$ **then**
    $p \leftarrow \mathcal{C}.\text{next_position}()$
    $\mathbf{L}, \mathcal{C} \leftarrow f(y; \text{past} = \mathcal{C}, \text{use_cache} = \top, \text{cache_position} = p, \text{return_cache} = \top)$
    $\mathbf{z} \leftarrow \mathbf{L}_{:,-1,:}$
  **end if**
**end for**

The manager owns the per-layer slot dictionary, a FIFO of recent token ids (consumed by token-level mixers such as Engram), a write cursor advanced by each forward, and the resolved quantization codec and eviction policy. Slot creation is lazy and driven by the layer on first use, so a heterogeneous `layer_pattern` — say a dense layer, a latent layer and a recurrent layer in one stack — holds three different slot kinds without special-casing. Rollback is a first-class operation: `truncate(n)` keeps the first $n$ positions across all slots (cheap list truncation with length bookkeeping; recurrent-state slots are no-ops because they are not position-indexed), which is the primitive required by speculative decoding and early-exit verification [27].

### 17.2.2 Typed Slots

Six slot kinds cover the taxonomy of §17.4. Position-indexed slots store an ordered list of *parts*; a part is either full precision or an encoded payload, and reading concatenates [sinks | history | protected window].

| Slot kind | Stores | Used by |
|---|---|---|
| `kv` | post-RoPE keys and values $(\mathrm{B}, H_{kv}, T, d)$; absolute positions; optional per-token extras | quadratic softmax mixers |
| `latent_kv` | named low-rank streams: latent $\mathbf{c}_{KV}$, decoupled RoPE key, grouped keys, latent values | latent family |
| `state` | one recurrent matrix (+ gate accumulators), updated in place; constant memory | recurrent / linear family |
| `ring` | capacity-bounded K/V with a pinned sink region | sliding-window family |
| `conv_tail` | last $k-1$ pre-convolution rows (FIFO) | causal short convolutions |
| `token_tail` | last $n$ token ids | Engram n-gram lookup |

Table 19: Typed cache slots. Position-indexed kinds (`kv`, `ring`, `latent_kv`) carry absolute position metadata, so eviction gaps keep correct relative distances; `state` and `conv_tail` are position-less and never evicted.

### 17.2.3 Capability Registry

One frozen record per `layer_pattern` enum value declares whether and how a layer caches:

$$\textsc{CacheSpec} \;=\; (\texttt{supported},\ \texttt{slot_type},\ \texttt{slot_types},\ \texttt{extras},\ \texttt{notes}).$$

The registry is the single source of truth consulted both at runtime (slot resolution) and statically (configuration warnings). A layer receives a slot only when its mixer's formulation admits a cache *and* an incremental branch is implemented and parity-tested; otherwise it receives `None` and executes the legacy path. The registry is therefore also the project's honest inventory of what caching means for each paper: `supported` records the source formulation, the implemented set records engineering status, and `notes` records the reason.

### 17.2.4 Forward Plumbing and Fallback Semantics

All signature changes are additive and default to the legacy behaviour:

$$f_{\text{mixer}}(\mathbf{x}; \ell, \text{pe}, \texttt{cache} = \varnothing, \texttt{cache_position} = \varnothing),$$
$$f_{\text{model}}(\mathbf{x}; \texttt{past_key_values} = \varnothing, \texttt{use_cache} = \bot, \texttt{return_cache} = \bot).$$

`HybridLayer` resolves the slot once per call and honours the four historical dispatch shapes: stateless linear (`mamba`) is called without arguments because it has nothing to cache; the two whole-stream mixers (`ode`, `retnet`) receive the slot only when their spec supports it; Engram additionally receives `input_ids`; every other mixer receives `cache` and `cache_position`. Unsupported or disabled layers are called exactly as before, which keeps the uncached path bit-identical.

## 17.3 Correctness Contracts

Cached and uncached execution must agree on the query row they share: the last row of a full forward and the row produced by a decode step. The contracts below establish that agreement.

### 17.3.1 Post-RoPE Storage and the Absolute Schedule

Keys are stored *after* the rotary transform, the industry convention: the cache holds the rotated vectors, and no re-rotation is needed when a ring buffer wraps. The prerequisite is that rotation use absolute positions on both sides of the cache boundary. RoPE, HoPE and sinusoidal-rotary accept a position offset $\pi_0$ and place the incoming tokens at

$$\pi_i = \big(\pi_0 + i\big) \cdot \texttt{scaling}, \qquad i = 0, \ldots, S_{\text{new}} - 1,$$

so a decode step at cursor $p$ rotates exactly as the full forward rotated position $p$. The slot metadata versions the scaling factor; a runtime change of the RoPE schedule invalidates already-rotated keys and is reported as a configuration error at generation start, never mid-decode [91, 18].

### 17.3.2 Additive Score Biases

ALiBi and BAM inject position through the score matrix, so they carry no positional tensor in the cache — precisely their appeal for length extrapolation [78, 9]. The full-square builder generalizes to a windowed builder over absolute query and key positions:

$$b_{h,i,j} \;=\; -\,\text{slope}_h \cdot \big|q_i - \kappa_j\big| \quad \text{(ALiBi)}, \qquad b_{h,i,j} \;=\; -\Big(\big|\kappa_j - \alpha_h q_i - \mu_h\big| + \varepsilon\Big)^{\beta_h} \quad \text{(BAM)},$$

where $q_i = \pi_0 + i$ for the incoming query positions and $\kappa_j$ are the cached absolute key positions. With $\pi_0 = 0$ and $\kappa_j = j$ this reproduces the legacy matrix exactly; after eviction, $\kappa_j$ may be non-contiguous, and the distances remain correct by construction.

### 17.3.3 Length-Dependent Rescaling

Scalable Softmax multiplies pre-softmax logits by $s \cdot \ln n$ where $n$ is the context length. In cached decoding the context of the newest query is the accumulated cache, so the rescale uses

$$\ln\big(n_{\text{cached}} + S_{\text{new}}\big)$$

rather than the size of the incoming chunk. For the final row of a full forward over $S_{\text{cached}} + S_{\text{new}}$ tokens both expressions coincide, which is exactly the row the parity tests compare.

### 17.3.4 Causal Masking from Positions

Masks are derived from metadata rather than from array indices:

$$M_{ij} \;=\; \big[\, q_i < \kappa_j \,\big] \;=\; \big[\, \pi_0 + i < \kappa_j \,\big],$$

which reduces to the classical upper-triangular mask when the cache starts empty and $\kappa_j = j$. For a single decode query every cached key satisfies $\kappa_j \le q_i$, so the mask is empty and the step needs no mask at all. Sparse families replace the causal predicate with a position-based pattern (§17.4).

### 17.3.5 Absolute Positional Encodings

Additive absolute encodings (sinusoidal, learned) are applied once to the embeddings. In cached decoding the embedding lookup for the new token receives the same offset, so token $p$ is augmented with row $p$ of the table:

$$\mathbf{x}_i \;\leftarrow\; \mathbf{x}_i + \text{PE}\big[\pi_0 + i\big].$$

### 17.3.6 Interlocks with Mixture-of-Depths, mHC, Hyperloop and AttnRes

Mixture-of-Depths routes a fraction of tokens through each layer. Routing a prompt would produce ragged, token-dependent caches, which v1 deliberately excludes; when the cache is active, prefill is forced dense while decode degenerates to dense automatically (one token is always within capacity). The divergence is one-directional: forcing dense can only add keys, never attend to keys that do not exist. mHC and Hyperloop residual streams are per-token state and flow through a cached forward without cross-step carry; the loop-level hyper-connections fire per iteration and need no slot of their own. Attention-residual variants keep cross-layer buffers that must accumulate across steps, a mode not yet implemented; the combination disables caching for that forward with a warning and falls back to full recompute. PaPE augmentations are content-dependent, so caching post-augmentation keys is not provably exact; such layers fall back per layer [72].

## 17.4 Incremental Recipes by Family

### 17.4.1 Quadratic Softmax Family

For a dense attention layer with $H$ query heads, $H_{kv}$ key/value heads and head dimension $d$, the cached branch appends the new keys and values and attends over the whole retained prefix:

$$\mathbf{K} \leftarrow \left[\mathbf{K}_{\text{cached}};\ \mathbf{k}_{\text{new}}\right], \quad \mathbf{V} \leftarrow \left[\mathbf{V}_{\text{cached}};\ \mathbf{v}_{\text{new}}\right], \quad \mathbf{S} = \frac{\mathbf{q}\mathbf{K}^{\top}}{\sqrt{d}} + \mathbf{B}, \quad \mathbf{o} = \operatorname{softmax}(\mathbf{S})\,\mathbf{V},$$

with $\mathbf{B}$ the positional bias of §17.3. Grouped-query attention caches only the $H_{kv}$ compact heads and expands head groups after the read, which is where its cache advantage originates [2]. The family's variations are all captured by substituting the normalizer or the bias:

- **Sigmoid attention.** The normalizer is replaced by an $\ell_1$ row normalization, $\hat{\mathbf{S}}_{ij} = \sigma(\mathbf{S}_{ij}) / \left(\sum_{j'} \sigma(\mathbf{S}_{ij'}) + \varepsilon\right)$. Because sigmoid is not shift-invariant, the denominator must be re-summed over the full cached prefix at every step; no online-softmax maximum trick exists [82].
- **Gated softmax.** The gate $\sigma(\mathbf{x}\mathbf{W}_g)$ multiplies the attention output and depends only on the current token, so nothing extra is cached [80].
- **Forgetting attention.** The logit bias is $D_{ij} = c_i - c_j$ with $c_i = \sum_{l \le i} \log f_l$ the cumulative log-forget sum. The cache stores one scalar per token and head as an extra; decode extends the running sum from the last cached token and evaluates a single row of $D$ against the cached column [58].
- **Titans.** The implemented layer is the attention window; the memory module is outside the cached path. Standard append/read applies [7].
- **Interleaved head attention.** Pseudo-key/value tensors are static learned head mixings computed on the fly; only the real per-token K/V is cached, and the mixing is applied after the read (the mixing is a per-token head-space map, so applying it post-read is exact) [25].

### 17.4.2 Latent Family

Latent mixers compress keys and values into a low-rank latent and cache only that latent. Writing $\mathbf{c}_{KV} = \mathbf{W}_{DKV}\mathbf{x}$, decode re-expands and rotates per step:

$$\mathbf{k} = \mathbf{W}_{UK}\mathbf{c}_{KV}, \qquad \mathbf{v} = \mathbf{W}_{UV}\mathbf{c}_{KV}, \qquad [\mathbf{q}, \mathbf{k}] \leftarrow \operatorname{RoPE}\left([\mathbf{q}, \mathbf{k}], \pi_0\right),$$

with attention as in the dense case [66]. DeepSeek-style MLA caches a decoupled RoPE key and can absorb the up-projections into the query, avoiding per-step expansion; this toolkit's variant applies RoPE to the decompressed query and key, so absorption is unavailable and expansion is the documented cost of halving the cache. GQLA reuses one weight set with two algebraically equivalent decode paths, and the slot layout follows the configured path [67]. MLRA partitions the latent into disjoint sub-heads, which is a tensor-parallel sharding win rather than a compression win; the total cache is MLA-sized [61]. Tucker attention caches the factorized key and value latents and expands through the core projections [53]. GTA caches one key per head *group* plus latent values; because rotation at a fixed position is head-independent, group means commute with the rotation and caching the mean is exact [92]. MTLA is the only method that compresses along time: a hyper-network merges adjacent cache vectors at stride boundaries, the schedule is dynamic, and its parity with a chunk-level training schedule is a named risk, so it currently falls back [20].

### 17.4.3 Sparse and Window Families

Sparse patterns are evaluated as predicates over absolute positions, never over array indices [16]:

$$\text{attend}(q_i, \kappa_j) \iff \text{pattern}\big(\pi_0 + i,\ \kappa_j\big)\ \wedge\ \kappa_j \le \pi_0 + i.$$

- **Sliding window with sinks.** Every query attends to the first $s$ positions plus the last $w - s$; the ring buffer's retention policy *is* the attention pattern, so a decode step attends the cached set directly and a multi-token chunk is pattern-masked [47, 102]. Removing the sinks collapses quality, which is why they are pinned both by the pattern and by the ring.
- **Longformer.** Symmetric windows plus task-specified global tokens; because a global token may sit anywhere, the slot is unbounded and the pattern is evaluated over positions [8].
- **BigBird.** Window, globals and random links. Random links must be reproducible across incremental steps, so they are drawn from a generator seeded by $(\ell, q_i)$ over $[0, q_i]$: links beyond the query are causal-masked anyway, so the effective mask is unchanged while becoming deterministic [111].
- **Sparse Transformers.** Strided and fixed factorized patterns, where a token's summary-column role changes as blocks complete; position-based evaluation handles this, and an automatic stride is resolved from the accumulated length so the full and cached paths agree [16].
- **FASA.** Per-head importance is computed from dominant RoPE-frequency chunks and re-ranked per query; it is row-local, so a decode step re-ranks the cached keys exactly [100]. SpargeAttn, NSA, MiniMax and SparDA select blocks from chunk-level statistics and therefore fall back in v1 [115, 109, 54, 30].

### 17.4.4 Recurrent and Linear State Family

Recurrent mixers carry a fixed-size state; caching means persisting that state, so memory is constant and no eviction is meaningful. With the paper convention $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ and read $\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$:

$$\text{RetNet:}\quad \mathbf{S}_t = \gamma_h\,\mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \qquad \text{per-head multi-scale } \gamma_h, \tag{118}$$

$$\text{GLA:}\quad \mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \qquad \mathbf{G}_t = \operatorname{diag}\big(\exp(\operatorname{logsigmoid}(\mathbf{W}_{gk}\mathbf{x}_t)/c)\big), \tag{119}$$

$$\text{DeltaNet:}\quad \mathbf{S}_t = \mathbf{S}_{t-1}\big(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top\big) + \beta_t\mathbf{v}_t\mathbf{k}_t^\top, \tag{120}$$

$$\text{Gated DeltaNet:}\quad \mathbf{S}_t = \alpha_t\,\mathbf{S}_{t-1}\big(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top\big) + \beta_t\mathbf{v}_t\mathbf{k}_t^\top, \tag{121}$$

$$\text{GDN-2:}\quad \mathbf{S}_t = \big(\mathbf{I} - \mathbf{k}_t(\mathbf{b}_t \odot \mathbf{k}_t)^\top\big)\mathbf{D}_t\mathbf{S}_{t-1} + \mathbf{k}_t(\mathbf{w}_t \odot \mathbf{v}_t)^\top, \tag{122}$$

$$\text{HGRN2:}\quad \mathbf{S}_t = \operatorname{diag}(\mathbf{g}_t)\mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \qquad g_{t,j} = b_\ell + (1 - b_\ell)\sigma(\cdot), \tag{123}$$

$$\text{KDA:}\quad \mathbf{S}_t = \big(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top\big)\mathbf{D}_t\mathbf{S}_{t-1} + \beta_t\mathbf{k}_t\mathbf{v}_t^\top, \tag{124}$$

$$\text{GMA:}\quad \tilde{\mathbf{V}}_t = \tilde{\mathbf{V}}_{t-1} + \boldsymbol{\gamma}_t^K \otimes \mathbf{v}_t, \quad \mathbf{Z}_t = \mathbf{Z}_{t-1} + \boldsymbol{\gamma}_t^K, \quad \mathbf{o}_t = \frac{\sum_k \gamma_{t,k}^Q \tilde{\mathbf{V}}_{t,k}}{\sum_k \gamma_{t,k}^Q Z_{t,k} + \varepsilon}, \tag{125}$$

referring to Sun et al. [93], Yang et al. [105], Yang et al. [106], Yang et al. [107], Hatamizadeh et al. [38], Qin et al. [79], Kimi Team et al. [51] and Huang and Raza [46].

Two subtleties deserve emphasis. First, RetNet's parallel retention sum and the recurrence above produce the same row for the same query — algebraically the parallel form factors through the running state — so the cached path may use the recurrent form while the uncached path uses the parallel one, and the parity test compares the shared row. Second, GDN-2 and KDA decay along the *key* axis ($\mathbf{D}_t$ multiplies the state from the left), whereas GLA and HGRN2 gate the key axis element-wise; both are diagonal in the key dimension, which is what makes a simple in-place recurrence exact. The ODE-attention block is excluded: its solver re-derives every token's intermediate state at each integration step, so a converged-state cache cannot reproduce the full path.

### 17.4.5 Auxiliary State and Documented Fallbacks

Some mixers need a small FIFO beside their activation cache. Falcon's chunk-parallel variants bound a rehearsal window, so exact continuation requires the last $B - 1$ prefix-aligned pairs; CCA/CCGQA stack two causal convolutions and need the last $k - 1$ pre-convolution latents plus a shifted value; Engram needs the last $n_{\max} - 1$ token ids only. These are the `conv_tail` and `token_tail` kinds.

| Mixer | Why it runs uncached in v1 |
|---|---|
| `mamba` | linear placeholder; stateless per token, inherently incremental |
| `ssog_attn` | bidirectional vision grid; no causal formulation exists |
| `ode` | solver re-derives all tokens' intermediate states each step |
| `falcon1..3a` | chunk-parallel rehearsal FIFO (`conv_tail` design pending) |
| `cca_attn`, `ccgqa_attn` | two causal convolutions + shifted value |
| `engram_attn` | token-id tail (`token_tail` design pending) |
| `nsa`, `msa`, `sparda`, `sparge` | block selection uses chunk-level statistics, not row-local |
| `mtla_attn` | dynamic hyper-network merge schedule (risk R1) |
| `sparsek_attn` | eviction-policy design (SparseK is a policy, not a slot) |
| PaPE layers | content-dependent q/k augmentation; exactness unproven |

Table 20: Documented uncached fallbacks. Each entry carries a paper-based rationale in the capability registry and a loader warning when the cache is enabled; none of them can make inference fail.

## 17.5 KV-Cache Quantization

Quantization is a write-time transform: the slot encodes on append and dequantizes on read, so the stored footprint shrinks while the mixer consumes ordinary floating-point tensors. This

placement follows the literature and keeps codecs orthogonal to mixers: the same codec serves dense, latent and ring slots, and latent slots may declare a different granularity per stream. Recurrent states are *not* quantized in v1: the evidence base there is thinnest, and their constant size makes the payoff small.

### 17.5.1 Symmetric int8 with Asymmetric Axes

Keys exhibit persistent per-channel outliers (a RoPE-frequency structure), while values exhibit per-token outliers. The `int8` codec therefore uses different reduction axes:

$$\text{keys (channel axis):}\quad s_{h,c} = \frac{\max_{t \in g} |K_{t,h,c}|}{127}, \qquad \hat{K} = \operatorname{round}\big(K/s\big) \in [-127, 127];$$

$$\text{values (token axis):}\quad s_{t,h} = \frac{\max_{c} |V_{t,h,c}|}{127}, \qquad \hat{V} = \operatorname{round}\big(V/s\big) \in [-127, 127].$$

Each stored block carries its own scales, so a token-by-token decode block gets exact per-token scales and the relative rounding error stays bounded by $1/127 \approx 0.8\%$ independently of block size. Dequantization is $\hat{K} \odot s$; merging two payloads is dequantize, concatenate along time, and requantize.

### 17.5.2 Group-Wise int4 with a Protected Window

The `int4_kivi` codec combines KIVI's group-wise low-bit quantization with the two-region protection pattern of streaming sinks and SKVQ. Signed levels are $[-7, 7]$; $g$ tokens form a group along time; granularity follows the int8 codec. Two regions are kept in full precision:

- the most recent $w_p$ tokens (the *protected window*), never quantized;
- the first $s$ tokens (*sinks*), never quantized and never evicted.

Append semantics mirror KIVI's residual window: a decode step pushes rows into the full-precision window, and as soon as the oldest chunk completes a group of $g$ tokens that group is quantized and moved into the history; prefill quantizes whole groups immediately. Reading concatenates [quantized history | protected window]. With signed 4-bit levels, the worst-case absolute error of a group quantizer is half a step,

$$|K - \hat{K}| \;\le\; \frac{s}{2} \;=\; \frac{\max_g |K|}{2 \cdot 7} \;\approx\; 7.1\% \text{ of the group maximum},$$

which is the bound asserted by the codec tests. Because sinks and the protected window stay exact, the tokens that dominate attention mass are never perturbed [62, 43, 23, 102].

### 17.5.3 Interface and Future Codecs

Both codecs implement the same three-method contract: `encode_write`, `decode_read` and `update`. Codecs must never raise on unknown tensors: on any internal error they fall back to identity and record a one-time warning, preserving D5 at the storage layer as well. The v1 registry contains `none`, `int8` and `int4_kivi`; further methods are interface-ready but deliberately unimplemented, and requesting one is a configuration error at generation start: `int2_kivi` (2-bit with dual-axis variance normalization), `palu_lowrank` (rank-$r$ projection with reconstruction), `commvq` (RoPE-commutative codebooks, attention in codebook space), `qjl_sketch` (1-bit Johnson–Lindenstrauss signs with an unbiased inner-product estimator) and `block_gtq` (per-RoPE-frequency bit allocation). Slot metadata already records the positional-encoding type and the scaling version, which is what a codebook or pre-RoPE codec needs to declare its compatibility. Sensitivity-aware per-layer bit allocation (`k_bits` $\neq$ `v_bits`) is exposed by configuration, since attention mass is pyramidal in depth.

## 17.6 Compression and Eviction

Eviction is a read-time transform; keeping it orthogonal to quantization is what allows a quantized cache to be evicted and an evicted cache to be quantized. The v1 policy is StreamingLLM's window-with-sinks: pin the first $s$ positions, retain the last $w$ positions, drop everything in between. The retained budget is

$$\mathcal{B} = s + w,$$

optionally overridden per layer by a budget vector $\mathcal{B}_\ell$ (attention mass is pyramidal in depth, so deeper layers tolerate smaller budgets). Policies act only on position-indexed slots; recurrent states are constant-size and MTLA manages its own merging schedule, so external policies are excluded there by construction. Future enum-ready policies are H2O (accumulate per-token attention mass and evict the lightest), SnapKV (one-shot post-prefill compression from a look-ahead window), PyramidKV (static depth-dependent budgets) and SparseK (learned scorer with permanent within-generation eviction, which is why it is modeled as a policy rather than a slot).

## 17.7 Configuration and Deployment Interface

The runtime block is validated by a dedicated schema module and flattened into a runtime dataclass that is deliberately *not* part of the architectural configuration, mirroring the `training.*` separation.

| Key | Default | Description |
|---|---|---|
| `inference.generation.max_new_tokens` | 64 | tokens generated after the prompt |
| `inference.generation.temperature` | 1.0 | softmax temperature |
| `inference.generation.top_k` | 50 | top-$k$ filter; 0 disables |
| `inference.generation.top_p` | 1.0 | nucleus mass; 1.0 disables |
| `inference.generation.greedy` | `false` | argmax decoding |
| `kv_cache.enabled` | `false` | opt-in; unset keeps the legacy loop bit-identical |
| `quantization.method` | `none` | `none`, `int8`, `int4_kivi` |
| `quantization.k_bits` / `v_bits` | 4 / 4 | int4 bit widths (int8 fixes 8) |
| `quantization.group_size` | 64 | tokens per group |
| `quantization.protected_window` | 32 | most recent tokens kept in full precision |
| `compression.method` | `none` | `none`, `window_sinks` |
| `compression.num_sinks` | 4 | pinned leading positions |
| `compression.window` | 512 | sliding window size |
| `compression.budget` | `null` | scalar budget per layer; `null` = unlimited |
| `compression.per_layer_budgets` | [] | per-layer budget vector, overrides the scalar |
| `per_mixer[].enabled` | `true` | per-mixer opt-out |

Table 21: Inference runtime configuration. Cross-key rules: compression requires `enabled`, and `window` $\geq$ `protected_window` so the protected region can never be evicted.

Resolution order is **CLI flags** > **YAML** `inference:` > **schema defaults**. Every CLI flag defaults to "unset", so a YAML preset is patched leaf-by-leaf rather than replaced:

```
infer -task causal_lm -config <preset> -prompt "..." -kv-cache -kv-quant int4_kivi -kv-compress
                                window_sinks -kv-window 512
```

The deployer serializes the resolved block into the artifact's `config.json` under `"inference"`, and the inference engine reads it back, so cached generation is reproducible without the original YAML. The HuggingFace export's causal-LM wrapper accepts `past_key_values`, `use_cache` and `cache_position` and forwards them to the decoder instead of discarding extras, and implements `prepare_inputs_for_generation` so that a standard transformers loop feeds only the last token once a cache exists.

## 17.8 Validation

The central test is a parity protocol: for a mixer and a positional encoding, compare a full forward over $S$ tokens against prefill over $S - k$ tokens followed by $k$ decode steps, requiring agreement on the shared row. Greedy generation is compared token-for-token first; if a numerical discrepancy appears, last-position logits are compared with rtol $= 10^{-4}$, atol $= 10^{-5}$ in fp32 with quantizer `none` and BitNet disabled.

| Suite | Assertions |
|---|---|
| slots | append/read/truncate/device moves; ring wrap with position metadata; in-place state; conv-tail FIFO; token tail |
| codecs | int8 error $\leq 10^{-2}$; int4 error $\leq 7.5 \times 10^{-2}$; protected window exactly fp; group boundaries; `update` = dequant→concat→requant |
| mixers | every implemented mixer × {rope, none, ALiBi}: full forward vs prefill+decode |
| fallback | all 44 enum values run with the cache on; unimplemented mixers bit-identical on/off; PaPE per-layer fallback |
| mechanisms | looped depth (distinct slots per logical layer), Hyperloop, mHC, MoD dense prefill, AttnRes fallback |
| quantization | int8/int4 drift bounds with BitNet on and off; int4 engages on short prompts; protected window exact |
| compression | sinks pinned after eviction; retained region correct; composition with int8 |
| generation | greedy parity exact; seeded sampling determinism; `truncate` rollback |
| config | schema-default parity, flattening, precedence, cross-key rules, fallback warnings, serialization |

Table 22: Validation matrix. The registry is also cross-checked against the schema's layer-pattern enum so a new mixer cannot be added without a cache decision.

## 17.9 Relation to Prior Work

- **KV-cache quantization** (KIVI, KVQuant, SKVQ): adopted as write-time codecs with asymmetric granularity and protected outlier regions; the group-window append semantics follow KIVI's residual window.
- **Eviction and sparsity** (StreamingLLM, H2O, SnapKV, PyramidKV, Quest): adopted as read-time policies; Quest-style page metadata and query-aware reading are future work inside the slot extras.
- **Paged and prefix sharing** (PagedAttention/vLLM, RadixAttention): structurally compatible — slots are content-agnostic and serializable — but block tables and prefix sharing are out of scope for v1.
- **Hybrid schedulers** (Jamba, Griffin, Zamba): the same slot dictionary holds attention K/V and recurrent states, which is the pattern these systems established.
- **Speculative decoding and early exit** (LayerSkip): served by the same `truncate` primitive that the tests exercise.

**When to enable the cache.** Any decoder-only generation workload benefits: the regime where decode dominates (long outputs, short prompts) gains the most, while a single full forward (classification, embedding, masked prediction) gains nothing and should leave `enabled` false. Quantization is worth enabling when memory, not latency, is the binding constraint, and `int4_kivi` should keep a protected window of at least the attention-sink span; window-with-sinks eviction is the right default for streaming or unbounded contexts. For recurrent layers the state is already constant-size, so eviction is meaningless and quantization is disabled by design.